

INFOSYS INSTEP

Agentic AI Procurement Agent on Beckn Protocol

System Documentation

Eduardo Garcia Diaz · Jose Emiliano Carrillo Barreiro · Cristian Montiel Garcia

Infosys InStep · July 2026

Contents

1	Procurement Agent	14
1.1	Quick Start	14
1.2	Documentation Index	15
2	Project Overview — Agentic AI Procurement Agent on Beckn Protocol	17
2.1	1. Problem Statement	17
2.2	2. Solution Summary	18
2.3	3. Target Users	18
2.4	4. Scope and Constraints	20
	2.4.1 Programme scope	20
	2.4.2 What is in scope	20
	2.4.3 What is deferred	21
2.5	5. Key Design Principles	21
	2.5.1 Async-first throughout	21
	2.5.2 Data sovereignty via local LLMs	21
	2.5.3 Event-driven with Redis Pub/Sub for async Beckn discovery	21
	2.5.4 Anti-corruption layer on the domain model	22
	2.5.5 Vendor-neutral ERP via structural typing	22
2.6	6. Technology Choices and Rationale	22
2.7	7. Project Phases	24
2.8	8. Known Limitations	27
3	Architecture	28
3.1	1. Architectural Pattern	28
3.2	2. Component Map	29
	3.2.1 Port Reference	29
3.3	3. Beckn Protocol Integration	31
	3.3.1 3.1 ONIX Adapter Pair	31
	3.3.2 3.2 ED25519 Signing	32
3.4	4. Async Discovery Decoupling (ADR-0001)	32
	3.4.1 The Deadlock Problem	32
	3.4.2 The Solution	32
3.5	5. IntentParser 3-Stage Pipeline	33
3.6	6. LangGraph State Machines	33
	3.6.1 6.1 Orchestrator Pipeline	35
	3.6.2 6.2 Negotiation Engine LangGraph	35
3.7	7. Data Stores	38
	3.7.1 PostgreSQL 16 + pgvector	38
	3.7.2 Redis 7	38
3.8	8. Architectural Decisions Summary	39

3.9	9. Component Reference	40
3.9.1	Dependency Overview	41
3.9.2	IntentParser (:8001, local)	41
3.9.3	orchestrator (:8004)	42
3.9.4	beckn-bap-client (:8002)	43
3.9.5	data-normalizer (:8006)	43
3.9.6	erp-adapter (:8007)	44
3.9.7	erp-mock (:8008)	44
3.9.8	negotiation_engine (:18004 host / :8004 container)	44
3.9.9	comparative-scoring (:8003)	45
3.9.10	ComparativeAndScoreing (MLOps)	45
3.9.11	discovery_engine	46
3.9.12	catalog-normalizer (:8005)	46
3.9.13	mcp-sidecar (:3000, local)	46
3.9.14	sim-bpp (:3002)	47
3.9.15	intention-parser (Docker wrapper)	47
3.9.16	frontend_demo_gateway (:8015)	47
3.9.17	analytics (:8009)	48
3.9.18	notification-dispatcher (:8010)	48
3.9.19	onix-bap (:8081, Go)	48
3.9.20	onix-bpp (:8082, Go)	49
3.9.21	frontend (Next.js, :3000)	49
3.9.22	shared/ (cross-service models)	49
3.9.23	Dependency Matrix	50
4	System Design	52
4.1	1. Service Boundaries and Responsibilities	52
4.1.1	1.1 Service Responsibility Map	52
4.1.2	1.2 Why the Orchestrator Is the Single Frontend Entry Point	54
4.1.3	1.3 Why data-normalizer Is the Sole PostgreSQL Writer	54
4.2	2. Communication Patterns	55
4.2.1	2.1 Redis Pub/Sub Channels	56
4.2.2	2.2 Beckn Async Discovery — Why <code>asyncio.create_task</code> , Not <code>await</code>	56
4.3	3. The Anti-Corruption Layer	56
4.3.1	3.1 Canonical Field Conventions	57
4.3.2	3.2 Why These Invariants Exist	57
4.3.3	3.3 Model Routing and the Docker Override	58
4.4	4. Error Handling Philosophy	58
4.4.1	4.1 Never-Throw MCP Contract	58
4.4.2	4.2 Fail-Closed ERP Budget Gate	59
4.4.3	4.3 <code>asyncio.gather</code> Channel Isolation in Notifications	60
4.4.4	4.4 Always-200 Discovery Engine	60
4.4.5	4.5 Negotiation Engine Three-Layer Guardrails	60
4.5	5. Data Consistency	61
4.5.1	5.1 Strong Consistency — ERP Outbox	61
4.5.2	5.2 Fire-and-Forget Writes — Eventual Consistency with Silent Failure Risk	61
4.5.3	5.3 In-Memory Session State	62
4.6	6. Scalability Design and Current Limits	62
4.6.1	6.1 Current State (Phases 1–3)	62

4.7	7. Architecture Decision Records	63
4.7.1	ADR-0001: Redis Pub/Sub for Async Beckn Discovery	63
4.7.2	pgvector Instead of Qdrant	64
4.7.3	Local LLMs (qwen3:8b / qwen3:1.7b) Instead of GPT-4o	65
4.7.4	Outbox Pattern for ERP PO Push Instead of Kafka	66
4.7.5	DeDi Registry Bypass (targetType: url)	67
5	Data Flow Reference	69
5.1	1. Main Procurement Happy Path	69
5.1.1	Steps	70
5.2	2. Beckn Async Discovery Flow (ADR-0001)	71
5.2.1	The deadlock problem	71
5.2.2	The solution — Redis Pub/Sub on per-transaction channels	72
5.2.3	Full sequence (MCP sidecar path)	72
5.2.4	Dual unblock path	72
5.2.5	Timeout behaviour	73
5.3	3. Compare / Commit Two-Phase Flow	73
5.3.1	Phase 1 — Compare	73
5.3.2	Phase 2 — Commit	74
5.3.3	Approval routing thresholds	74
5.4	4. Automated Negotiation Flow	75
5.4.1	Routing chain	75
5.4.2	Steps	75
5.4.3	LangGraph node graph	77
5.4.4	Category discount profiles	77
5.5	5. ERP Integration Flow	78
5.5.1	5a. Budget Gate (synchronous, ≤ 800 ms)	78
5.5.2	5b. PO Push (asynchronous outbox)	78
5.6	6. Agent Memory Learning Flywheel	81
5.6.1	Write path (triggered after /confirm)	81
5.6.2	Read path (triggered before scoring)	83
5.6.3	Scoring delta formula	83
5.7	7. Notification Dispatch Flow	85
5.7.1	Order lifecycle (sim-bpp auto-advance)	85
5.7.2	Notification routing rules	85
5.7.3	Fan-out steps	85
5.8	Known Data Flow Anomalies	86
6	Installation Guide	89
6.1	1. Prerequisites	89
6.2	2. Clone and Prepare the Repository	90
6.3	3. Python Environment Setup	91
6.4	4. Pull LLM Models with Ollama	91
6.5	5. Environment Variables	92
6.6	6. Database Initialization	93
6.6.1	6.1 Start a PostgreSQL container with pgvector	93
6.6.2	6.2 Apply all migrations	94
6.6.3	6.3 Verify the schema	95
6.7	7. Start the Docker Stack	95

6.8	8. Start Local Services	96
6.8.1	Terminal 1 — IntentParser	96
6.8.2	Terminal 2 — mcp-sidecar	97
6.9	9. Smoke Test	98
6.10	10. Troubleshooting Installation	99
7	Configuration Guide	101
7.1	1. Configuration Philosophy	101
7.2	2. Configuration Files	102
7.2.1	How It Works	102
7.2.2	File Map	102
7.2.3	Root .env Template	103
7.2.4	The Two os.getenv() Exceptions	104
7.3	3. Complete Environment Variable Reference	104
7.3.1	orchestrator (:8004, host ports 8000 and 8004)	104
7.3.2	IntentParser (:8001, runs locally or as intention-parser container)	106
7.3.3	beckn-bap-client (:8002)	107
7.3.4	mcp-sidecar (:3000, runs locally only)	108
7.3.5	erp-adapter (:8007)	109
7.3.6	data-normalizer (:8006)	111
7.3.7	sim-bpp (:3002)	111
7.3.8	frontend (:3000)	112
7.3.9	Secrets That Must Be Rotated Before Production	113
7.4	4. ONIX Routing Configuration	114
7.4.1	File Map	114
7.4.2	Pipeline Stages	114
7.4.3	targetType: url — DeDi Registry Bypass	114
7.4.4	Schema Validator Pin	115
7.4.5	ED25519 Key Rotation	115
7.5	5. LLM Configuration	115
7.5.1	Switching Models	117
7.5.2	ANTHROPIC_API_KEY — Stage 3 Broadening Fallback	117
7.5.3	negotiation_engine LLM (:8004)	117
7.6	6. ERP Configuration	118
7.6.1	Vendor Selection	118
7.6.2	Switching from Mock to SAP	118
7.6.3	Budget Gate Flags	119
7.6.4	HMAC Dual-Secret Rotation	119
7.7	7. Beckn Network Configuration	121
7.7.1	Pointing the Stack at a Different Beckn Network	121
7.7.2	Testnet vs Production Keys	121
7.8	8. sim-bpp Catalog Configuration	122
7.8.1	Catalog Hot-Reload	122
7.8.2	Automated Lifecycle Progression	122
7.9	9. Adding a New Configuration Variable	123
7.9.1	Step 1 — Add to the module-level config.py	123
7.9.2	Step 2 — Add to .env.example with a safe placeholder	123
7.9.3	Step 3 — Add to docker-compose.yml if a different value is needed in Docker	124
7.9.4	Step 4 — Document in this file	124

7.9.5	Step 5 — Document in ENVIRONMENT.md	124
8	Database Reference	125
8.1	1. Overview	125
8.2	2. Schema by Domain	126
8.2.1	2.1 Core Procurement	126
8.2.1.1	users (migration 01_users.sql)	126
8.2.1.2	procurement_requests (migration 03_procurement_requests.sql, extended by 22b)	127
8.2.1.3	parsed_intents (migration 04_parsed_intents.sql)	127
8.2.1.4	beckn_intents (migration 05_beckn_intents.sql)	129
8.2.1.5	bpp (migration 02_bpp.sql)	130
8.2.1.6	discovery_queries (migration 06_discovery_queries.sql)	130
8.2.1.7	catalog_cache (migration 07_catalog_cache.sql)	131
8.2.1.8	seller_offerings (migrations 08_seller_offerings.sql + 21_order_detail_fidelity.sql)	131
8.2.2	2.2 Scoring and Approval	132
8.2.2.1	scored_offers (migration 09_scored_offers.sql)	132
8.2.2.2	negotiation_outcomes (migration 10_negotiation_outcomes.sql)	133
8.2.2.3	approval_decisions (migration 11_approval_decisions.sql)	133
8.2.2.4	purchase_orders (migrations 12_purchase_orders.sql + 21_order_detail_fidelity.sql)	134
8.2.3	2.3 ERP Synchronisation	135
8.2.3.1	erp_sync_records (migrations 13_erp_sync_records.sql + 19_erp_enum_extensions.sql + 19b_erp_sync_records_outbox.sql)	135
8.2.4	2.4 Audit and Observability	137
8.2.4.1	audit_trail_events (migration 14_audit_trail_events.sql)	137
8.2.4.2	model_governance_records (migration 16_model_governance_records.sql)	137
8.2.5	2.5 AI Memory	138
8.2.5.1	agent_memory_vectors (migrations 15_agent_memory_vectors.sql + 22_agent_memory_vector_dim.sql)	138
8.2.5.2	bpp_catalog_semantic_cache (migration 18_bpp_catalog_semantic_cache.sql)	138
8.2.6	2.6 Negotiation Engine (negotiation database, port 55432)	140
8.3	3. ENUMs	140
8.4	4. Indexes	144
8.4.1	4.1 Named B-tree / Partial Indexes (17_indexes.sql)	144
8.4.2	4.2 Migration-Added Indexes	146
8.5	5. Running Migrations	147
8.5.1	Applying migrations	147
8.5.2	Adding a new migration	147
8.5.3	Verification	148
8.6	6. pgvector Similarity Search	148
8.6.1	6.1 Agent Memory (agent_memory_vectors)	148
8.6.2	6.2 BPP Catalog Semantic Cache (bpp_catalog_semantic_cache)	150
8.7	7. Redis Schema	151
8.7.1	7.1 Beckn Discovery Pub/Sub	151
8.7.2	7.2 Negotiation Engine Channel	151
8.7.3	7.3 ONIX Adapter Response Cache	152
8.7.4	7.4 Redis Connection Reference	153

9	Security	154
9.1	1. Authentication Model	154
9.1.1	1.1 Frontend Authentication	154
9.1.2	1.2 Internal Service Authentication	156
9.1.3	1.3 ERP Webhook Authentication (HMAC-SHA256)	156
9.1.4	1.4 Claude OpenAI Proxy Authentication	157
9.2	2. Beckn Protocol Security	159
9.2.1	2.1 ED25519 Request Signing	159
9.2.2	2.2 Schema Validator Pin at Commit d43ec30d	159
9.2.3	2.3 Testnet Key Material	160
9.2.4	2.4 DeDi Registry Bypass	160
9.3	3. Data Sovereignty	161
9.3.1	3.1 Claude Stage-3 Broadening Fallback	161
9.3.2	3.2 Claude OpenAI Proxy Loopback Binding	162
9.4	4. Input Validation	162
9.4.1	4.1 BecknIntent Field Validation	162
9.4.2	4.2 ONIX Schema Validation	162
9.4.3	4.3 MCP Sidecar Never-Throw Contract	163
9.4.4	4.4 Database Error Middleware	163
9.4.5	4.5 ERP Budget Gate Fail-Closed	163
9.4.6	4.6 Negotiation Engine Guardrails	165
9.5	5. Secrets Management	165
9.5.1	5.1 Env-Var Pattern	165
9.5.2	5.2 .env Files	165
9.5.3	5.3 Secrets That Must Be Rotated Before Any Production Deployment	165
9.6	6. Known Security Gaps	166
9.7	7. Security Checklist for New Features	168
10	Deployment Guide	170
10.1	1. Current Deployment: Docker Compose (Phases 1–3)	170
10.1.1	1.1 Stack Overview	170
10.1.2	1.2 Container Reference	171
10.1.3	1.3 Key Bind Mounts	174
10.1.4	1.4 Common Operations	175
10.1.5	1.5 Deployment Topology Diagram	176
10.2	2. Services That Run Outside Docker	176
10.2.1	2.1 IntentParser API (:8001)	176
10.2.2	2.2 MCP Sidecar (:3000)	177
10.2.3	2.3 Claude OpenAI Proxy (:8012)	178
10.3	3. MLOps Sub-Stack (ComparativeAndScoreing)	180
10.4	4. Applying Database Migrations	181
10.4.1	4.1 First-Time Setup	182
10.4.2	4.2 Adding New Migrations	182
10.4.3	4.3 setup_database.py Flags	182
10.4.4	4.4 Verify Schema	183
10.4.5	4.5 Key Migrations	183
10.4.6	4.6 Root .env Template	184
10.5	5. Monitoring and Observability	184
10.5.1	5.1 Service Health Endpoints	184

10.5.2	5.2 erp-adapter Prometheus Metrics	185
10.5.3	5.3 Log Streaming	185
10.5.4	5.4 Phase 4: Prometheus + Grafana (Planned)	186
10.6	Appendix: Orphaned Service — discovery_engine	186
11	Testing Guide	187
11.1	1. Test Framework	187
11.1.1	asyncio_mode = auto — Critical Rule	187
11.1.2	HTTP Mocking	188
11.1.3	asyncpg Mocking (DataNormalizer unit layer)	188
11.2	2. Test Organization	188
11.3	3. Test Markers	194
11.4	4. Running Tests	194
11.4.1	IntentParser — unit only (no infrastructure)	195
11.4.2	IntentParser — full suite (requires Ollama at localhost:11434)	195
11.4.3	IntentParser Stage 3 async pipeline — live mode (requires PostgreSQL + pgvector + Ollama + MCP sidecar)	195
11.4.4	Bap-1 — unit only (no infrastructure; all HTTP mocked with aioresponses)	195
11.4.5	Bap-1 — Ollama integration tests only (requires qwen3:8b pulled)	195
11.4.6	Database schema (requires PostgreSQL 16 + pgvector with procurement_agent DB applied)	195
11.4.7	DataNormalizer unit layer (no infrastructure; asyncpg mocked)	196
11.4.8	data-normalizer integration tests (requires procurement_agent_test DB or Docker)	196
11.4.9	Phase 3 memory performance tests only	196
11.4.10	Phase 3 audit trail tests only	196
11.4.11	negotiation_engine (no infrastructure; uses MemorySaver)	196
11.4.12	erp-adapter contract test (standalone script — not pytest-discoverable)	196
11.4.13	erp-adapter smoke tests (requires Docker stack with erp-mock and PostgreSQL)	197
11.4.14	MLOps stack (requires docker-compose.mlops.yaml)	197
11.4.15	Bap-1 smoke test (end-to-end with ONIX Docker stack)	197
11.5	5. Writing New Tests	198
11.5.1	Async test functions — no decorator	198
11.5.2	Async fixtures	198
11.5.3	Integration marker for infra-dependent tests	198
11.5.4	Config via environment variables	199
11.5.5	asyncpg mock pattern (DataNormalizer unit tests)	199
11.5.6	aioresponses pattern (Bap-1 tests)	200
11.5.7	LangGraph in-memory checkpointing (negotiation_engine tests)	200
11.6	6. Phase 3 Acceptance Test Suites	200
11.6.1	6.1 Agent Memory and Learning (phase3_agent_memory_learning.md)	201
11.6.2	6.2 Audit Trail System (phase3_audit_trail_system.md)	202
11.7	7. Coverage Assessment	203
11.7.1	Well-tested areas	203
11.7.2	Known coverage gaps	203
11.7.3	Infrastructure requirements matrix	204
11.8	8. End-to-End Smoke Test	205
11.8.1	Prerequisites	205
11.8.2	Smoke request	206
11.8.3	Expected healthy response shape	206

11.8.4	Distinguishing partial failure	207
12	Development Guide	208
12.1	Table of Contents	208
12.2	1. Code Conventions	208
12.2.1	1.1 Async-First	208
12.2.2	1.2 Module-Level Config	209
12.2.3	1.3 Pydantic v2	209
12.2.4	1.4 Numbered SQL Migrations	210
12.2.5	1.5 Bind-Mounted Libraries	211
12.2.6	1.6 Library vs Service Boundary	211
12.2.7	1.7 Test Conventions	211
12.3	2. Things NOT to Do	212
12.3.1	2.1 Never POST Directly to a BPP	212
12.3.2	2.2 Never Include the Action Name in ONIX Routing URLs	213
12.3.3	2.3 Never Use Contract.status.code = “CONFIRMED”	213
12.3.4	2.4 Never Await the /discover POST in the MCP Sidecar	213
12.3.5	2.5 Never Use time.sleep() to Flush asyncio.create_task Side Effects	213
12.3.6	2.6 Never Reuse a transaction_id	214
12.3.7	2.7 Never Put Billing or Fulfillment Inline in Contract	214
12.4	3. Repository Layout Quick-Reference	214
12.4.1	Port Map	215
12.5	4. Local Development Workflow	216
12.5.1	4.1 Prerequisites	216
12.5.2	4.2 The Three-Terminal Pattern	216
12.5.3	4.3 Starting a Subset of the Docker Stack	217
12.5.4	4.4 Tailing Logs	217
12.5.5	4.5 Hot-Reloading sim-bpp Catalog	218
12.5.6	4.6 Running Tests	218
12.5.7	4.7 End-to-End Smoke Test	218
12.6	5. Adding a New Service	219
12.6.1	Step 1: Create the service directory	219
12.6.2	Step 2: Write the Dockerfile	219
12.6.3	Step 3: Add to docker-compose.yml	219
12.6.4	Step 4: Add /health endpoint	220
12.6.5	Step 5: Update documentation	220
12.7	6. Adding a New Database Table	220
12.7.1	Step 1: Create the migration file	220
12.7.2	Step 2: Apply the migration	221
12.7.3	Step 3: Add repository class	221
12.7.4	Step 4: Add integration test	221
12.7.5	ENUM types — important gotcha	221
12.8	7. Working with the IntentParser Pipeline	222
12.8.1	7.1 Pipeline Stages	222
12.8.2	7.2 Complexity Routing	222
12.8.3	7.3 Adding a New Intent Type	222
12.8.4	7.4 Modifying BecknIntent Fields	224
12.8.5	7.5 Stage 3 Validation Thresholds	224
12.8.6	7.6 Recovery Flow	224

12.9	8. Working with LangGraph (Negotiation Engine)	225
12.9.1	8.1 Graph Overview	225
12.9.2	8.2 Adding a New Node	225
12.9.3	8.3 Guardrail Layers	225
12.9.4	8.4 HITL Interrupt and Resume	227
12.9.5	8.5 Orchestrator Routes Negotiation via demo-gateway	227
12.10.9	9. Error Handling Patterns	227
12.10.1	9.1 Never-Throw Contract — MCP Tool Handlers	227
12.10.2	9.2 Fire-and-Forget for Non-Critical Writes	228
12.10.3	9.3 asyncio.gather with return_exceptions=True for Fan-Out	228
12.10.4	9.4 ERP Budget Gate — Fail-Closed vs Fail-Open	228
12.10.5	9.5 Database Error Middleware	229
12.11	10. Known Problems and Workarounds	229
12.11.1	10.1 Database ENUM Types Contain Spec-Era Values	229
12.11.2	10.2 Docker Container Collapses Complexity Routing to qwen3:1.7b	230
12.11.3	10.3 catalog-normalizer LLM Fallback Uses Ollama, Not OpenAI	230
12.11.4	10.4 demo-gateway Port Mismatch	231
12.11.5	10.5 Frontend Requires Live Keycloak — No Stub Credentials	231
12.11.6	10.6 negotiate Audit Events Are Never Written	231
12.11.7	10.7 discovery_engine Is Orphaned	231
12.11.8	10.8 recovery.py Stubs Never Notify or Write to DB	232
12.12	See Also	232
13	Contributing Guide	233
13.1	1. Git Workflow	233
13.1.1	1.1 Branch Naming	233
13.1.2	1.2 Commit Message Format	233
13.1.3	1.3 PR Process	234
13.2	2. Pre-Submission Checklist	234
13.2.1	Async and Python conventions	234
13.2.2	Configuration	235
13.2.3	Database	235
13.2.4	Tests	235
13.2.5	Beckn routing	235
13.2.6	Documentation	236
13.2.7	Wire-shape	236
13.3	3. Documentation Standards	236
13.3.1	3.1 New Service	236
13.3.2	3.2 Shared Library Changes	237
13.3.3	3.3 Significant Features	237
13.4	4. Code Style	237
13.4.1	4.1 Python	237
13.4.2	4.2 TypeScript (Frontend)	238
13.4.3	4.3 SQL Migrations	238
13.5	5. Reviewing Others' PRs	238
13.5.1	Async patterns	238
13.5.2	Error handling	239
13.5.3	Configuration discipline	239
13.5.4	Database migrations	239

13.5.5	Beckn traffic routing	239
13.5.6	Shared library consumers	240
14	Troubleshooting Guide	241
14.1	1. Service Health Check	241
14.1.1	1.1 Health Endpoints	241
14.1.2	1.2 Check All Docker Services at Once	242
14.2	2. Common Issues	243
14.2.1	2.1 Natural Language Parsing	243
14.2.1.1	Stage 3 returns "found": false for every query	243
14.2.1.2	qwen3 model error / LLM call timeout	244
14.2.1.3	BecknIntent fields are null after Stage 2	245
14.2.1.4	Stage 3 recovery flow does not notify the buyer or trigger an RFQ	245
14.2.2	2.2 Beckn Discovery	246
14.2.2.1	on_discover never arrives — timeout after REDIS_RESULT_TIMEOUT	246
14.2.2.2	ONIX routing returns 404 on Beckn actions	246
14.2.2.3	Schema validation failure at ONIX — Beckn message rejected	247
14.2.2.4	sim-bpp returns zero results for multi-noun queries	248
14.2.3	2.3 Database	248
14.2.3.1	Migration fails with FK violation	248
14.2.3.2	pgvector extension missing	248
14.2.3.3	agent_memory_vectors inserts fail with "invalid input value for enum"	249
14.2.3.4	agent_memory_vectors table empty after confirmed orders	250
14.2.4	2.4 ERP Integration	250
14.2.4.1	Budget check always denies	250
14.2.4.2	Circuit breaker is OPEN	251
14.2.4.3	PO stuck in outbox / DLQ	251
14.2.4.4	ERP cancellation silently drops for SAP / Oracle	252
14.2.5	2.5 Frontend	252
14.2.5.1	Analytics page shows no data / blank dashboard	252
14.2.5.2	WebSocket order tracking not updating	253
14.2.5.3	Keycloak login fails / frontend refuses to start	253
14.2.6	2.6 Negotiation Engine	254
14.2.6.1	Negotiation connection refused — http://localhost:8012	254
14.2.6.2	Negotiation stuck waiting for on_select	255
14.2.6.3	LangGraph checkpoint error on negotiation-engine startup	255
14.3	3. Known Limitations (Expected Behavior)	256
14.4	4. Useful Debug Commands	257
14.4.1	4.1 Docker Logs and Containers	257
14.4.2	4.2 Redis	258
14.4.3	4.3 PostgreSQL	258
14.4.4	4.4 ERP / Mock Control	259
14.4.5	4.5 IntentParser End-to-End Smoke Test	260
14.4.6	4.6 Beckn Protocol / ONIX	260
14.4.7	4.7 MLOps / Comparative Scoring	261
15	API Reference	262
15.1	Service Port Map	262

15.2	Shared Data Models	263
15.2.1	BecknIntent	263
15.2.2	DiscoverOffering	264
15.3	orchestrator (:8004)	264
15.3.1	Pipeline Entry Points	265
15.3.2	Session and Order Tracking	265
15.3.3	WebSocket	266
15.3.4	Admin, Approvals, Analytics	266
15.3.5	Inbound Webhook	266
15.3.6	POST /run	267
15.3.7	POST /compare	269
15.3.8	POST /commit	269
15.3.9	POST /run/{run_id}/decide	270
15.3.10	POST /approvals/{request_id}/decide	271
15.3.11	GET /status/{txn_id}/{order_id}	272
15.3.12	orchestrator Environment Variables	272
15.4	IntentParser (:8001)	273
15.4.1	Endpoints	273
15.4.2	POST /parse	274
15.4.3	POST /parse/batch	275
15.4.4	POST /parse/full	275
15.4.5	IntentParser Environment Variables	276
15.5	beckn-bap-client (:8002)	277
15.5.1	Orchestrator-Facing Routes	277
15.5.2	ONIX Callback Receivers	278
15.5.3	POST /discover	278
15.5.4	POST /select	279
15.5.5	POST /init	280
15.5.6	POST /confirm	281
15.5.7	POST /status	282
15.5.8	POST /on_discover	283
15.5.9	Beckn Lifecycle Sequence	283
15.5.10	beckn-bap-client Environment Variables	283
15.6	data-normalizer (:8006)	284
15.6.1	Pipeline Write Path	284
15.6.2	Order Retrieval	286
15.6.3	Audit Trail	286
15.6.4	Agent Memory	286
15.6.5	Admin and Approvals	287
15.6.6	Liveness	287
15.6.7	Key Normalizations Applied on Write	287
15.6.8	FK Chain for purchase_order Creation	287
15.6.9	Error Handling	288
15.6.10	Configuration	288
15.7	erp-adapter (:8007)	289
15.7.1	All Endpoints	289
15.7.2	Authentication	290
15.7.3	POST /api/v1/budget/check	290

15.7.4	POST /api/v1/po/sync	291
15.7.5	POST /api/v1/policy/evaluate	291
15.7.6	Outbox Pattern	292
15.7.7	Circuit Breakers	292
15.7.8	erp-adapter Environment Variables	292
15.8	Remaining Services — Compact Reference	293
15.8.1	catalog-normalizer (:8005)	293
15.8.2	mcp-sidecar (:3000, SSE transport)	294
15.8.3	sim-bpp (:3002)	296
15.8.4	analytics (:8009)	297
15.8.5	comparative-scoring (:8003)	298
15.8.6	negotiation-engine (:8004 container / :18004 host)	299
15.8.7	discovery-engine (:8006) — Orphaned Service	300
15.8.8	erp-mock (:8008)	301
15.8.9	frontend-demo-gateway (:8015)	302
15.9	notification-dispatcher	303
15.9.1	Kafka Consumer	304
15.9.2	Routing Rules	304
15.9.3	Output Channel Formats	304
15.9.4	Configuration	305
15.10	Authentication Summary	305

16 Glossary — Agentic AI Procurement Agent on Beckn Protocol 307

16.1	A	307
16.2	B	308
16.3	C	309
16.4	D	310
16.5	E	310
16.6	F	311
16.7	H	311
16.8	I	312
16.9	K	312
16.10	L	312
16.11	M	312
16.12	N	313
16.13	O	313
16.14	P	314
16.15	Q	315
16.16	R	315
16.17	S	316
16.18	T	316

1 Procurement Agent

The Procurement Agent is an event-driven agentic AI system built for corporate buyers. It accepts natural-language purchase requests — such as "300 meters Cat6 UTP cable Mumbai 5 days" — classifies intent, extracts structured procurement parameters via a three-stage LLM pipeline, discovers matching suppliers on a Beckn Protocol v2.0.0 network, scores and ranks their offers using an ML model with agent memory, optionally negotiates price autonomously via a LangGraph state machine, obtains ERP budget approval, and drives the full Beckn lifecycle (discover → select → init → confirm → status). Every decision is persisted to an auditable PostgreSQL database with 7-year retention. The buyer interacts through a Next.js 13 dashboard without writing code or issuing protocol calls directly.

1.1 Quick Start

1. **Provision the database.** Install PostgreSQL 16 with the pgvector extension natively on the host, then run `cd database && python setup_database.py` to apply the 24 numbered SQL migrations. See [Installation Guide](#) for required environment variables and the exact Conda environment setup.
2. **Start the Docker stack.** Bring up all 18 containerised services:

```
docker compose up -d
```

Verify Beckn routing is live by checking `docker compose logs -f becn-bap-client`.

3. **Start the three host-side processes.** These are not Dockerised and must be started manually in the `conda activate infosys_project` environment:

```
# MCP sidecar – required for IntentParser Stage 3 validation
cd services/mcp-sidecar
BAP_API_KEY="any-value" uvicorn server:app --port 3000

# Frontend
cd frontend && npm run dev # http://localhost:3000
```

```
# Claude OpenAI proxy – required for negotiation demo flows
export CLAUDE_PROXY_KEY=your-key
uvicorn services.claude_openai_proxy.main:app --host 0.0.0.0 --port 8012
```

Open <http://localhost:3000> and authenticate via the Phase Two Keycloak tenant. See [Installation Guide](#) for all required environment variables.

1.2 Documentation Index

Document	Purpose
PROJECT_OVERVIEW.md	Problem statement, solution summary, target users, project scope, design principles, tech-choice rationale, and known limitations
ARCHITECTURE.md	Detailed architecture: component map, ADR-0001 Redis Pub/Sub, IntentParser pipeline, LangGraph state machines, and full component reference (§9)
SYSTEM_DESIGN.md	Service boundaries, design decisions, anti-corruption layer, error handling philosophy, and five full ADRs
DATA_FLOW.md	Seven major data flows with Mermaid sequence diagrams: NL→Beckn, discovery, scoring, negotiation, ERP, audit, memory
INSTALLATION.md	Step-by-step setup: Conda environment, PostgreSQL native install, Docker Compose, host-side processes, required environment variables
CONFIGURATION.md	Complete environment variable reference, ONIX routing YAMLS, LLM model selection, ERP vendor wiring, and Beckn network parameters
DATABASE.md	Database schema: ER diagrams, 24 SQL migrations, vector tables, Redis channels, known schema deviations
DEPLOYMENT.md	Docker Compose deployment guide: container reference, bind mounts, common operations, database migrations, observability

Document	Purpose
SECURITY.md	Authentication model, Beckn ED25519 signing, HMAC webhook rotation, input validation, secrets management, known security gaps
TESTING.md	Test framework, test suites by scope, infrastructure requirements, coverage gaps
DEVELOPMENT_GUIDE.md	Coding conventions, async patterns, adding services and DB tables, IntentParser and LangGraph internals, known problems and workarounds
CONTRIBUTING.md	Git workflow, branch naming, pre-submission checklist, documentation standards, PR review criteria
TROUBLESHOOTING.md	Diagnostic runbooks: NL parsing, Beckn discovery, database, ERP integration, frontend, negotiation engine
API_REFERENCE.md	HTTP API surface: all endpoints, request/response schemas, and error codes for every service
GLOSSARY.md	Alphabetical definitions of all domain terms, architectural concepts, and database identifiers

2 Project Overview — Agentic AI Procurement Agent on Beckn Protocol

Self-contained reading. This document describes what the system is, who uses it, how it works at a high level, and why the core technology choices were made. For the low-level architecture see [Architecture](#). For API contracts see [API Reference](#). For service-by-service details see the individual `services/*/README.md` files.

2.1 1. Problem Statement

Enterprise procurement is a high-latency, high-friction process. Buyers enter RFQs into closed platforms (SAP Ariba, Coupa, Oracle Procurement Cloud), wait days for supplier responses, manually compare quotes, escalate approvals through email chains, and re-key approved orders into ERP systems. The entire cycle — from identifying a need to receiving a confirmed purchase order — typically takes 3–30 business days for non-catalogue purchases.

The core problem has two dimensions:

1. **Protocol lock-in.** Each procurement platform requires bilateral EDI or API integration agreements with every supplier. Small or new suppliers cannot participate without significant integration cost. Buyers are therefore restricted to a supplier roster determined by past IT investment, not current market quality or price.
2. **Manual translation work.** Procurement professionals spend a disproportionate share of their time reformatting natural-language needs into structured RFQ forms, chasing approval signatures, and reconciling purchase orders against ERP budget records. This is automatable work that does not require human judgment.

A Beckn-native procurement agent addresses both dimensions. Beckn Protocol v2.0.0 is an open, interoperable commerce protocol (similar in intent to SMTP for email) that allows any buyer application (BAP) to discover offers from any supplier network (BPP) without bilateral agreements. Combining Beckn with an

LLM-driven intent parser and an automated orchestration layer eliminates the manual translation work and opens procurement to any Beckn-registered supplier, not just contracted ones.

2.2 2. Solution Summary

The system accepts a natural-language purchase request — for example, "300 meters Cat6 UTP cable Mumbai 5 days" — and drives the full procurement lifecycle to a confirmed purchase order without manual data-entry. The five-step flow is:

Every decision — intent parse, discover results, score rationale, approval decision, Beckn message payloads — is persisted to a SOX 404-compliant audit trail with 7-year retention.

2.3 3. Target Users

User	Role	Interaction mode
Corporate buyer (routine)	Procurement officer sourcing standard office supplies, IT consumables, raw materials	Next.js web dashboard; types a natural-language request; reviews ranked options in advisory or hitl mode
IT buyer (complex)	IT manager sourcing servers, networking equipment, software licences	Same dashboard; additional CPO benchmarking panel comparing contracted price vs live Beckn market
System administrator	Infosys ops engineer or client IT team	Docker Compose stack; environment variable configuration; ERP connector setup
Procurement approver	Line manager or CFO	Email / Slack approval notification; approval workflow UI in the dashboard
Infosys InStep internship team	Three engineers (Eduardo Garcia Diaz, Cristian Montiel Garcia, Jose Emiliano Carrillo Barreiro) building the reference implementation	Direct API access; local Docker Compose stack; pytest suite

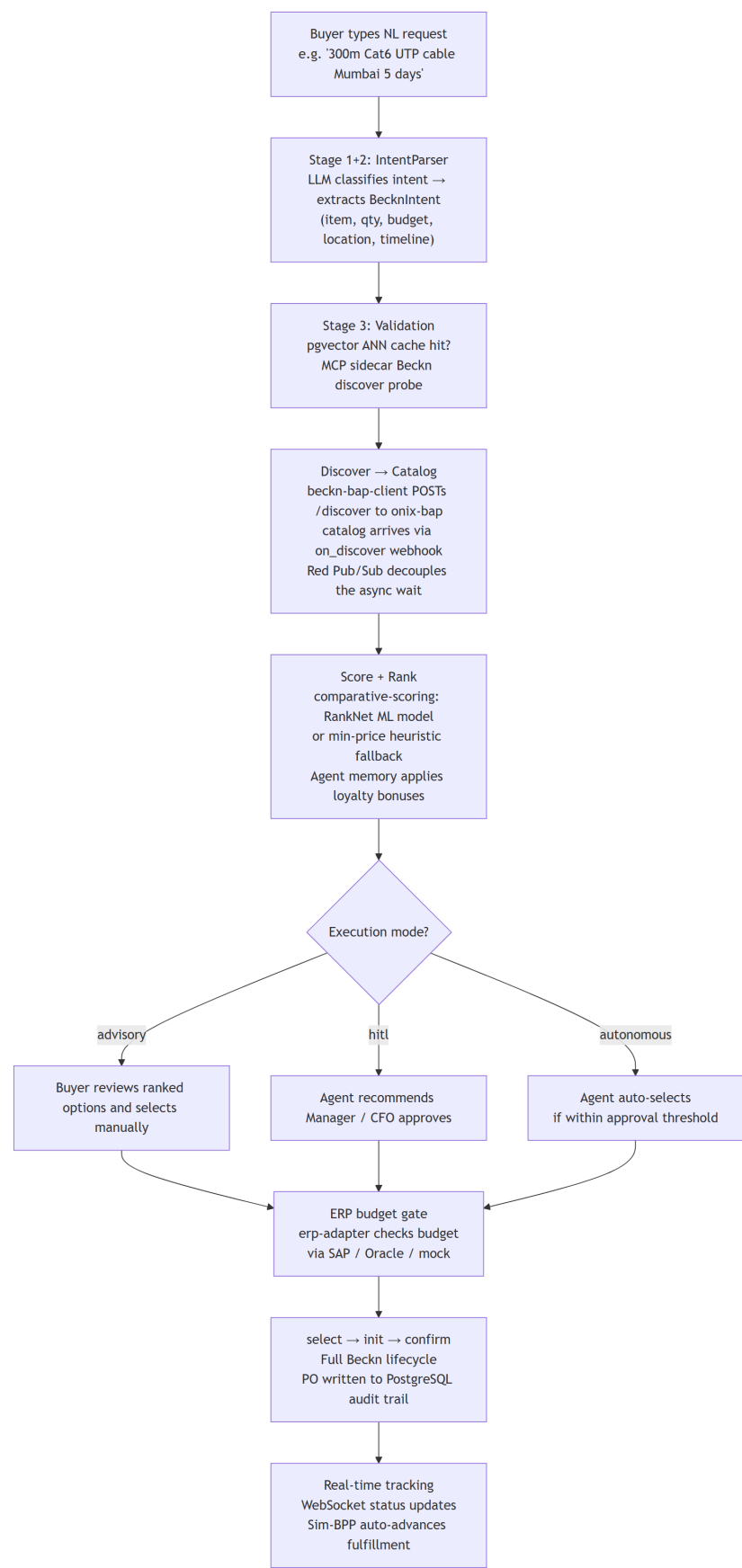


Figure 2.1: PROJECT_OVERVIEW diagram 1

The primary demo context is the Infosys InStep internship programme. The system is the reference implementation that Infosys intends to productise for enterprise procurement clients targeting the \$9.5 B global enterprise procurement software market.

2.4 4. Scope and Constraints

2.4.1 Programme scope

This is a **16-week Infosys InStep internship project** (April–July 2026). It delivers a functionally complete reference implementation across three phases, with a fourth hardening phase explicitly deferred. It is **not a production system**. All deployment runs on a single developer workstation using Docker Compose. No cloud infrastructure, no Kubernetes cluster, and no live Beckn network registration are part of the current scope.

2.4.2 What is in scope

- Full Beckn Protocol v2.0.0 lifecycle: `discover` → `select` → `init` → `confirm` → `status`
- Three-stage NL intent parsing pipeline (LLM classify → LLM extract → hybrid pgvector ANN + MCP sidecar validation)
- Six-service microservices pipeline on the Docker Compose stack
- ML-based comparative scoring (RankNet) with heuristic fallback
- Agent memory with time-decayed supplier loyalty bonuses
- SOX 404-ready audit trail (9 event types, 7-year retention)
- ERP integration: budget gate + PO push outbox, SAP + Oracle + mock surfaces, dual-HMAC webhook rotation, per-vendor circuit breakers
- Autonomous LangGraph price negotiation engine with three-layer guardrails
- RBAC-based approval workflow (advisory / hitl / autonomous modes)
- Real-time order tracking via WebSocket and sim-bpp auto-advance lifecycle
- Analytics dashboard: spend metrics, cycle-time KPIs, CPO benchmarking
- Notification fan-out: Kafka → Slack Block Kit, Teams Adaptive Card, SMTP email

2.4.3 What is deferred

Kubernetes / Helm deployment, CI/CD pipeline (GitHub Actions), OWASP pen test, integration test coverage $\geq 80\%$, Kafka as primary event bus for audit and ERP, LangSmith LLM tracing, Splunk/SIEM sink, retention enforcement job, and Kong API gateway. See Section 7 for the full phase breakdown.

2.5 5. Key Design Principles

2.5.1 Async-first throughout

Every service and every new feature ships as a coroutine. The orchestrator pipeline calls downstream microservices concurrently using `aiothttp` sessions. Non-critical side-effects — audit trail writes, agent memory persistence, ERP outbox rows — are fire-and-forget `asyncio.create_task(...)` calls that never block the critical response path. The sync wrapper in `IntentParser.core.parse_request` exists only for legacy test callers and explicitly disables Stage 3. Using `time.sleep()` to “wait for tasks” is explicitly prohibited in `CLAUDE.md` because it blocks the event loop and prevents tasks from running; `await asyncio.sleep(0)` must be used instead.

2.5.2 Data sovereignty via local LLMs

The original spec called for GPT-4o and OpenAI embeddings, which would route procurement request content (item names, quantities, budgets, vendor names) through a third-party API with associated billing and data-egress risks. All primary inference runs locally: `qwen3:8b` and `qwen3:1.7b` via Ollama for intent classification and extraction; `all-MiniLM-L6-v2` (sentence-transformers) and `BAAI/bge-small-en-v1.5` (fastembed ONNX) for embeddings. Claude Sonnet 4.6 is used only as an opt-in last-resort fallback for Stage 3 query broadening, requiring an explicit `ANTHROPIC_API_KEY`, and is never on the primary pipeline path.

2.5.3 Event-driven with Redis Pub/Sub for async Beckn discovery

Beckn v2.0.0 discovery is inherently asynchronous: `POST /discover` returns only an ACK, and the actual catalog arrives later via an `on_discover` webhook. A naive blocking wait would deadlock the `asyncio` event loop. ADR-0001 solves this with Redis Pub/Sub: the MCP sidecar subscribes to a per-transaction channel (`beckn_results:{transaction_id}`) before firing the discover request as

`asyncio.create_task`, then awaits the Redis message. The `beckn-bap-client` publishes the catalog to the same channel when the webhook arrives. This decouples the wait without introducing new infrastructure, since Redis is already required for ONIX’s internal cache.

2.5.4 Anti-corruption layer on the domain model

`shared/models.BecknIntent` is the canonical representation of a buyer’s intent extracted from natural language. Its types are strictly defined: `delivery_timeline` is `int` hours (not ISO 8601 strings), `location_coordinates` is "`lat,lon`" decimal (not city names), `budget_constraints` is a typed `{max, min}` struct (not raw strings). This ACL prevents NL ambiguities from propagating into Beckn protocol messages and ERP records.

2.5.5 Vendor-neutral ERP via structural typing

The ERP adapter defines `ERPAdapter` as a `Python typing.Protocol` with six methods. SAP S/4HANA, Oracle ERP Cloud, and a mock stub each implement the protocol independently. Selecting the active adapter is a runtime configuration choice (`ERP_VENDORS` environment variable). Adding a new ERP vendor (e.g., Microsoft Dynamics) requires only implementing the six-method Protocol and registering it in the factory — no changes to the orchestrator, budget-gate logic, or webhook routes.

2.6 6. Technology Choices and Rationale

Choice	Alternative considered	Why this was chosen
qwen3:8b / qwen3:1.7b via local Ollama (primary LLM)	GPT-4o / GPT-4o-mini (original spec)	Zero per-call cost; offline operation; procurement data never leaves the developer workstation; <code>instructor Mode.JSON</code> retry loop handles qwen3 JSON non-conformance; quality sufficient for structured <code>BecknIntent</code> extraction

Choice	Alternative considered	Why this was chosen
Claude Sonnet 4.6 (opt-in, Stage 3 only)	General LLM fallback (original spec)	Scope-limited to the <code>broaden_procurement_query()</code> recovery path triggered only on ANN cache miss + item not found; not on the hot path; requires explicit <code>ANTHROPIC_API_KEY</code> opt-in
pgvector in PostgreSQL 16 (vector store)	Qdrant self-hosted + pgvector mirror (original spec)	Pilot corpus < 100 K records where pgvector HNSW cosine is computationally equivalent to Qdrant HNSW; eliminates a separate container, StatefulSet, and backup strategy; vector writes are atomic with FK-dependent relational writes in the same transaction
PostgreSQL outbox pattern (erp_sync_records with FOR UPDATE SKIP LOCKED)	Apache Kafka broker (original spec) for ERP PO push	Kafka not yet deployed in Phases 1–3; outbox guarantees atomicity between PO write and sync record creation in a single database transaction — a Kafka publish after a DB commit can fail, leaving systems out of sync; <code>kafka_offset</code> column is pre-wired as a migration path to Kafka in Phase 4
Redis Pub/Sub (beckn_results:{transaction_id})	HTTP polling, in-process <code>asyncio.Queue</code> , direct callback injection	Redis was already a required dependency; Pub/Sub breaks the asyncio deadlock with no new infrastructure; channel name encodes <code>transaction_id</code> for natural isolation; observable live via <code>redis-cli subscribe</code>

Choice	Alternative considered	Why this was chosen
Docker Compose (18 services, becn_network bridge)	Kubernetes EKS/AKS/GKE + Helm + ArgoCD (original spec)	Phases 1–3 are development and pilot phases; Kubernetes adds cluster management overhead with no pilot-phase value; docker compose up -d is a one-command startup; service code is cluster-agnostic via env-variable-driven URLs; Kubernetes is the Phase 4 production-readiness target
sim-bpp (custom Node.js BPP simulator, port 3002)	fidedocker/sandbox-2.0 (original spec)	sandbox-2.0 had a fixed, immutable catalog; sim-bpp hot-reloads catalog.json, supports AND-token catalog matching to prevent false positives, and drives an auto-advance fulfillment lifecycle (ACCEPTED → DELIVERED) enabling real-time notification and WebSocket tests
ONIX adapter pinned at commit d43ec30d	Latest fidedocker/onix-adapter upstream	A \$ref resolution bug introduced after d43ec30d breaks SignatureHeader / AckSignatureHeader validation, causing 100% of Beckn transactions to fail; pinning preserves correct ED25519 signing at zero cost
384-dim local embedding models (all-MiniLM-L6-v2, BAAI/bge-small-en-v1.5)	OpenAI text-embedding-3-large (3072 dims, original spec)	Zero per-call cost; no data egress; sub-100 ms ONNX inference after warm-up; 384-dim HNSW indexes build faster and consume less memory; quality sufficient for < 100 K structured procurement records

2.7 7. Project Phases

Phase	Weeks	Focus	Status	Key deliverables
1	1–4	Foundation and Protocol Integration	Complete	ONIX Go adapter; core Beckn API flows (discover + select + init + confirm + status); NL IntentParser (qwen3 via Ollama + instructor); LangGraph ReAct agent framework; PostgreSQL schema; shared domain models (BecknIntent, DiscoverOffering)
2	5–8	Core Intelligence and Transaction Flow	Complete	Six-service microservices pipeline (beckn-bap-client, catalog-normalizer, data-normalizer, comparative-scoring, orchestrator, analytics); IntentParser Stage 3 (pgvector ANN + MCP sidecar); ADR-0001 Redis Pub/Sub async discovery; sim-bpp replacing sandbox-2.0; RankNet MLOps stack (optional); ERP adapter (budget gate + outbox); Next.js frontend scaffold

Phase	Weeks	Focus	Status	Key deliverables
3	9–12	Advanced Intelligence and Enterprise Features	Complete	Agent memory with time-decayed loyalty bonuses (pgvector RAG); audit trail; RBAC + three-tier approval workflow; LangGraph autonomous negotiation engine with three-layer guardrails; real-time order tracking (WebSocket + sim-bpp auto-advance + Kafka); notification fan-out (Slack / Teams / email); analytics dashboard; Keycloak OIDC authentication
4	13–16	Hardening and Production Readiness	In progress	Kubernetes + Helm chart + ArgoCD GitOps; CI/CD pipeline (GitHub Actions lint → test → build → Helm); OWASP Top 10 pen test; integration test coverage >= 80%; eval suite accuracy >= 85%; Kafka as primary event bus for audit and ERP; LangSmith LLM tracing; Splunk/SIEM sink; retention enforcement job; Kong API gateway; mTLS to SAP/Oracle

2.8 8. Known Limitations

As of July 2026, the system is a functionally complete pilot that demonstrates the full procurement lifecycle end-to-end but has several gaps before it is production-grade. It runs on a single developer workstation with no horizontal scaling, no rolling deploys, and no automated health-based restarts. Authentication requires a live Phase Two Keycloak cloud tenant (`euc1 . au th . ac`) and cannot run in a fully offline environment.

3 Architecture

Audience: Engineers joining the project. Read this document first; it is the system map. For per-service APIs and contracts see [Components](#). For full ADR rationale see [System Design](#). For data schemas see [Database](#).

3.1 1. Architectural Pattern

The system is an **event-driven microservices monorepo** that translates natural-language purchase requests into validated Beckn Protocol v2.0.0 transactions.

Two Python services run outside Docker on the developer host (IntentParser on :8001, mcp-sidecar on :3000). Eighteen containers run on a bridged Docker network called `beckn_network`. Services communicate over HTTP and a Redis Pub/Sub broker.

Why this pattern fits Beckn’s async nature. Beckn Protocol v2.0.0 defines discovery as inherently asynchronous: `POST /discover` returns only an ACK; the catalog arrives minutes later via an `on_discover` webhook. Holding an HTTP connection open to wait for that callback blocks the asyncio event loop and creates a deadlock. Redis Pub/Sub on per-transaction channels (`beckn_results:{transaction_id}`) decouples the fire-and-forget outbound probe from the inbound callback, eliminating the deadlock while keeping the microservices boundary intact. This is the foundational architectural decision — formalised in [ADR-0001](#) and also described in [System Design](#).

The remaining major deviations from the original spec (pgvector replacing Qdrant, qwen3 replacing GPT-4o, sim-bpp replacing sandbox-2.0, PostgreSQL outbox replacing Kafka for ERP PO push, Docker Compose replacing Kubernetes for Phases 1–3) are documented with full rationale in [System Design](#).

3.2 2. Component Map

All containers and local services, grouped by logical layer.

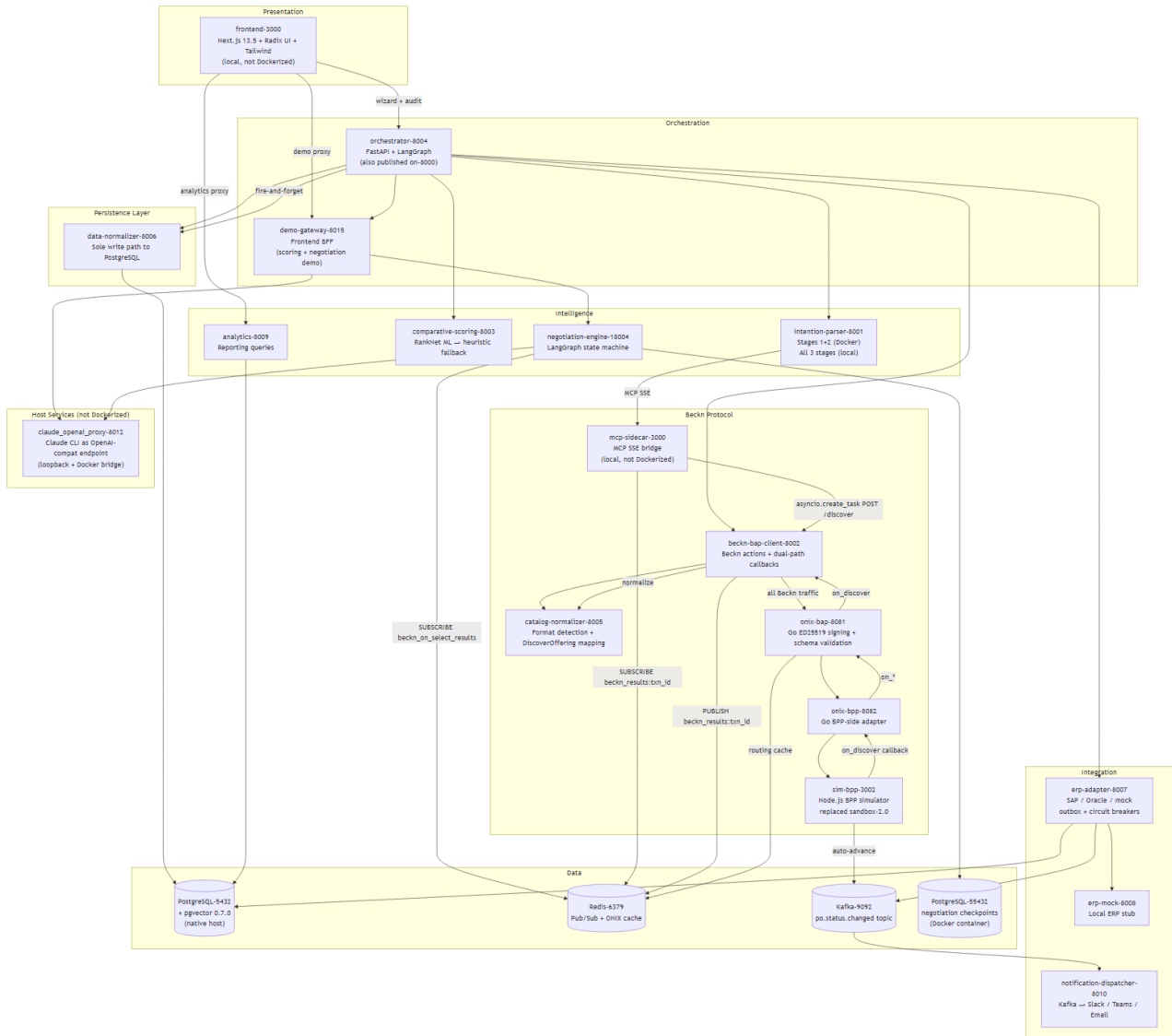


Figure 3.1: ARCHITECTURE diagram 1

3.2.1 Port Reference

Service	Host port	Container port	Run mode
frontend	3000	—	local
mcp-sidecar	3000	—	local

Service	Host port	Container port	Run mode
IntentParser	8001	—	local
intention-parser (Docker)	8001	8001	Docker
beckn-bap-client	8002	8002	Docker
comparative-scoring	8003	8003	Docker
orchestrator	8000, 8004	8004	Docker
catalog-normalizer	8005	8005	Docker
data-normalizer	8006	8006	Docker
erp-adapter	8007	8007	Docker
erp-mock	8008	8008	Docker
analytics	8009	8009	Docker
notification-dispatcher	—	8010	Docker (no host port)
claude_openai_proxy	8012	—	host process
demo-gateway	8015	8015	Docker
sim-bpp	3002	3002	Docker
onix-bap	8081	8081	Docker
onix-bpp	8082	8082	Docker
negotiation-engine	18004	8004	Docker
PostgreSQL (main)	5432	—	native host
PostgreSQL (negotiation)	55432	5432	Docker
Redis	6379	6379	Docker
Kafka	9092	9092	Docker

Port collision notes. The orchestrator container maps both :8000 and :8004 on the host — :8000 matches the Next.js frontend BAP_URL default and :8004 is used for direct testing. The negotiation-engine uses :18004 to avoid colliding with orchestrator's :8004.

3.3 Beckn Protocol Integration

3.3.1 3.1 ONIX Adapter Pair

All Beckn traffic passes through two Go adapters built from the `fidedocker/onix-adapter` image. **No service ever POSTs directly to a BPP.** This rule is enforced by tests that assert `select_url` always contains `caller` in the path.

Adapter	Host port	Role	Signing direction
<code>onix-bap</code>	8081	BAP-side	Signs outbound; validates inbound signatures
<code>onix-bpp</code>	8082	BPP-side	Mirrors <code>onix-bap</code> for the BPP direction

Six YAML files in `config/` wire the routing. Each file configures one of: `BAPCaller` (outbound from BAP), `BAPReceiver` (inbound callbacks to BAP), `BPPCaller` (outbound from BPP), `BPPReceiver` (inbound requests to BPP).

Critical routing rules:

- Never include the action name in a target URL.** ONIX appends it automatically. `http://beckn-bap-client:8002+actionon_discover` resolves to `http://beckn-bap-client:8002/on_discover`. Including `/on_discover` in the target produces a double-path 404.
- on_discover uses split routing.** It routes to `http://beckn-bap-client:8002` directly (not via the generic `/bap/receiver` base). All other `on_*` callbacks route to `http://beckn-bap-client:8002/bap/receiver`. This is required for the Redis Pub/Sub path — see [Section 4](#).
- DeDi registry is bypassed.** All routing YAMLs use `targetType: url` instead of `bap/bpp`, enabling a complete Beckn lifecycle entirely within the Docker network without an external registry. Switching to a live Beckn network requires only changing `targetType` and `registryUrl` in the YAMLs — no code changes.
- ONIX schema validator is pinned to commit d43ec30d.** A later commit introduced a `$ref` resolution bug in `SignatureHeader` that breaks ED25519 signature validation on every signed message. Do not upgrade the ONIX image without running a full end-to-end signing test across `discover` → `select` → `init` → `confirm` → `status`.

3.3.2 3.2 ED25519 Signing

onix-bap signs every outbound Beckn message using ED25519. The private key material is loaded at container start from the volume-mounted `config/` directory. The schema validator compiled into the `.so` plugin enforces Beckn v2 wire shapes on both request and response. See [System Design](#) for the key-rotation procedure.

3.4 4. Async Discovery Decoupling (ADR-0001)

3.4.1 The Deadlock Problem

IntentParser Stage 3 calls the mcp-sidecar tool `search_bpp_catalog`, which must fire a `POST /discover` request and return a synchronous catalog result. Beckn v2.0.0 returns only an ACK from `/discover`; the catalog arrives later via the `on_discover` webhook. If the mcp-sidecar awaits the discover POST while the `on_discover` callback must be processed by the same `asyncio` event loop, the loop is blocked and the callback can never arrive — a deadlock.

3.4.2 The Solution

Redis Pub/Sub on `beckn_results:{transaction_id}`. The subscriber is opened **before** the discover request is fired. The discover POST is dispatched as `asyncio.create_task(...)` — never `await`. When `beckn-bap-client` receives the `on_discover` callback, it publishes the normalized catalog payload to the Redis channel. The mcp-sidecar's subscriber unblocks and returns the catalog to IntentParser Stage 3.

Timeout behaviour. If no Redis message arrives within `REDIS_RESULT_TIMEOUT` (default 15 s), the mcp-sidecar returns `{"found": false, "items": [], "probe_latency_ms": 15000}`. `MCP_BAP_TIMEOUT` (default 8 s) is only a safety valve for the HTTP fire-and-forget task, not the end-to-end deadline. If Redis is unavailable, `beckn-bap-client` logs a warning and falls through to `CallbackCollector` only; the mcp-sidecar times out and returns `found=false`.

The anti-pattern. `await`-ing the `/discover` POST inside the mcp-sidecar reintroduces the deadlock. Always use `asyncio.create_task(...)`. This is an explicit hard rule in `CLAUDE.md`.

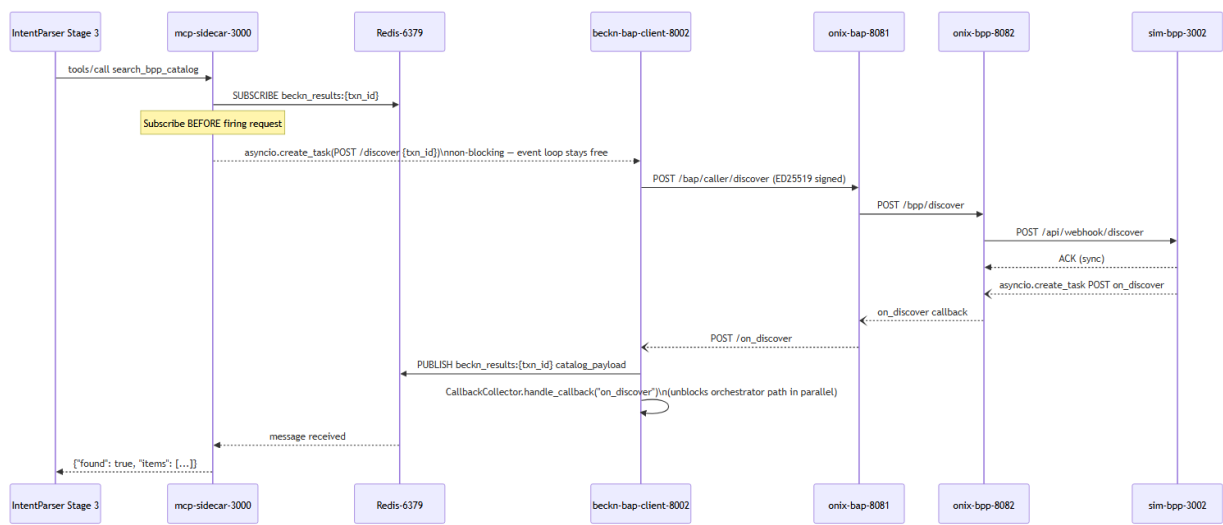


Figure 3.2: ARCHITECTURE diagram 2

3.5 5. IntentParser 3-Stage Pipeline

Natural language is processed in three sequential stages. Stages 1 and 2 always execute. Stage 3 runs only on the /parse/full endpoint and only when the mcp-sidecar is reachable. The Docker-deployed intention-parser container exposes only Stages 1 and 2.

Canonical field encodings enforced by BecknIntent (defined in shared/models.py):

Field	Encoding	Example
delivery_timeline	int hours	72 (not "P3D")
location_coordinates	"lat,lon" decimal string	"12.9716,77.5946"
budget_constraints	BudgetConstraints(max, min) typed object	{"max": 200.0, "min": 0.0}

Docker override. docker-compose.yml sets both COMPLEX_MODEL=qwen3:1.7b and SIMPLE_MODEL=qwen3:1.7b for the intention-parser container, collapsing the complexity routing. The qwen3:8b model is only used when IntentParser runs locally outside Docker.

3.6 6. LangGraph State Machines

Two independent LangGraph StateGraph machines drive the orchestration and negotiation flows.

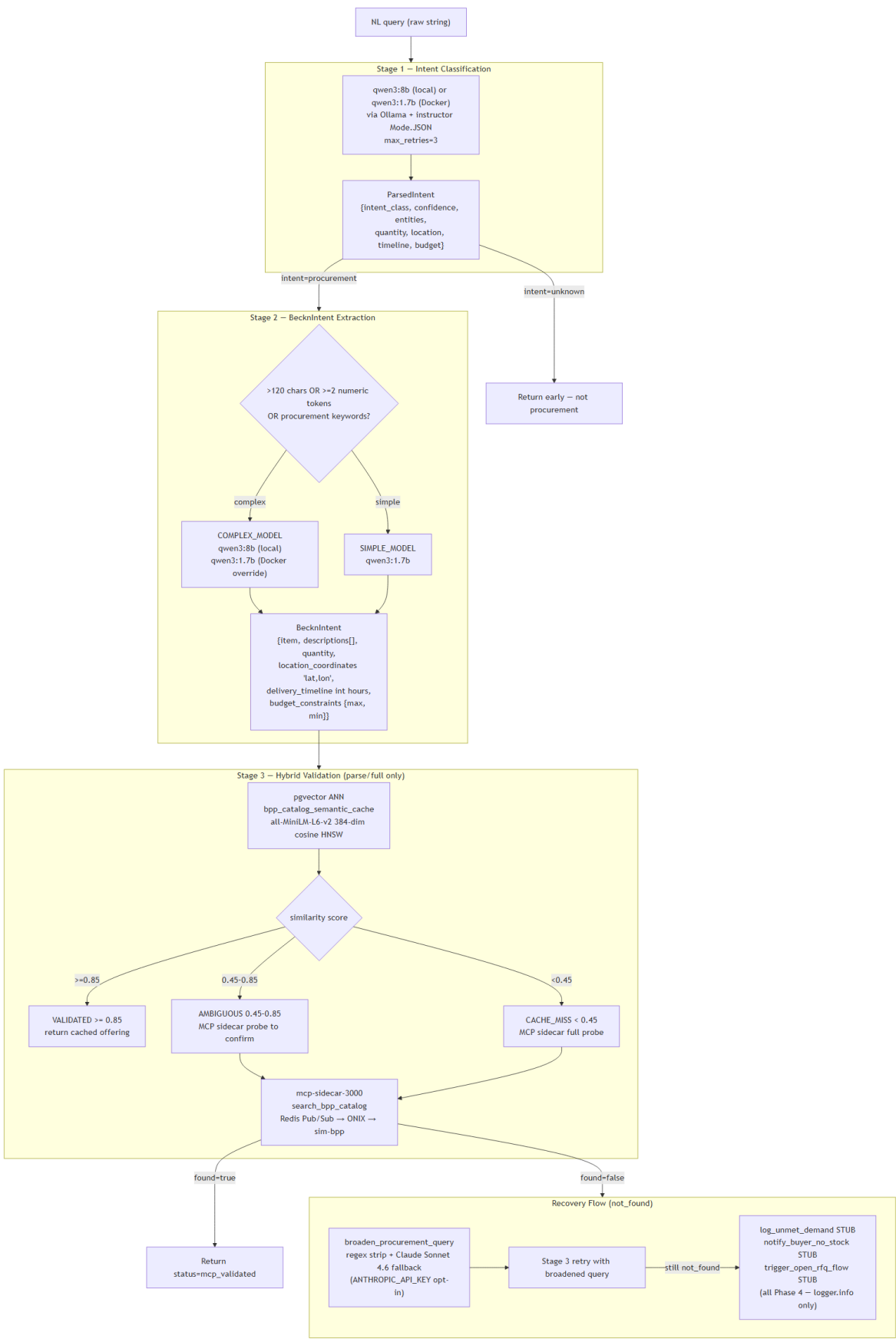


Figure 3.3: ARCHITECTURE diagram 3

3.6.1 6.1 Orchestrator Pipeline

The orchestrator drives procurement from NL query to confirmed Beckn order. Two execution paths exist: `/run` (full 4-step pipeline including autonomous negotiation) and `/compare + /commit` (two-phase, no autonomous negotiation).

Approval tiers: - `amount <= requester.approval_threshold` → auto-approved - `amount > requester.threshold AND <= approver.threshold` → manager approval - `amount > approver.threshold` → CFO approval - `is_emergency = TRUE` → CFO with 60-minute deadline

Autonomous negotiation path. When `decision.requires_negotiation` is true on `/run`, the orchestrator calls `DEMO_GATEWAY_URL/api/demo/negotiate` (default `http://localhost:8015`). Note: the `frontend_demo_gateway` service README incorrectly documents port 8005; the `docker-compose.yml` and `orchestrator` env var both use port 8015. The `/compare + /commit` two-phase flow does not trigger autonomous negotiation.

3.6.2 6.2 Negotiation Engine LangGraph

A LangGraph StateGraph with five category discount profiles and three-layer guardrails. Maximum discount is 20%, enforced independently at all three layers. The engine is always reached via demo-gateway, never called directly by the orchestrator.

Checkpointing. When `NEGOTIATION_POSTGRES_DSN` is set, `AsyncPostgresSaver` writes durable LangGraph checkpoints to the negotiation PostgreSQL container (port 55432). Otherwise `MemorySaver` is used and state is lost on container restart. The negotiation database (container `postgres` on port 55432) is entirely separate from the main `procurement_agent` database on port 5432.

Async supplier resume. `OnSelectListener` subscribes to Redis channel `beckn_on_select_results`. When demo-gateway drives `SupplierAgent` (Claude Sonnet 4.6 via `claude_openai_proxy :8012`) to generate a counter-offer and publishes the result to that Redis channel, `OnSelectListener` calls `graph.ainvoke(Command(resume=payload))` to unpark the waiting graph node.

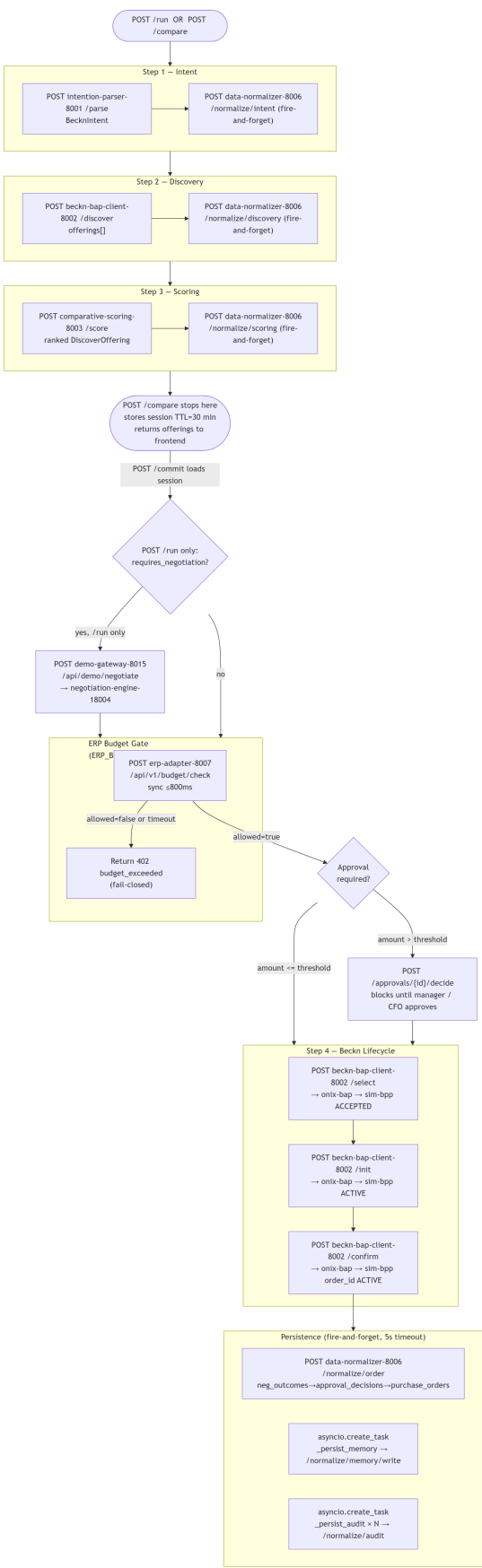


Figure 3.4: ARCHITECTURE diagram 4

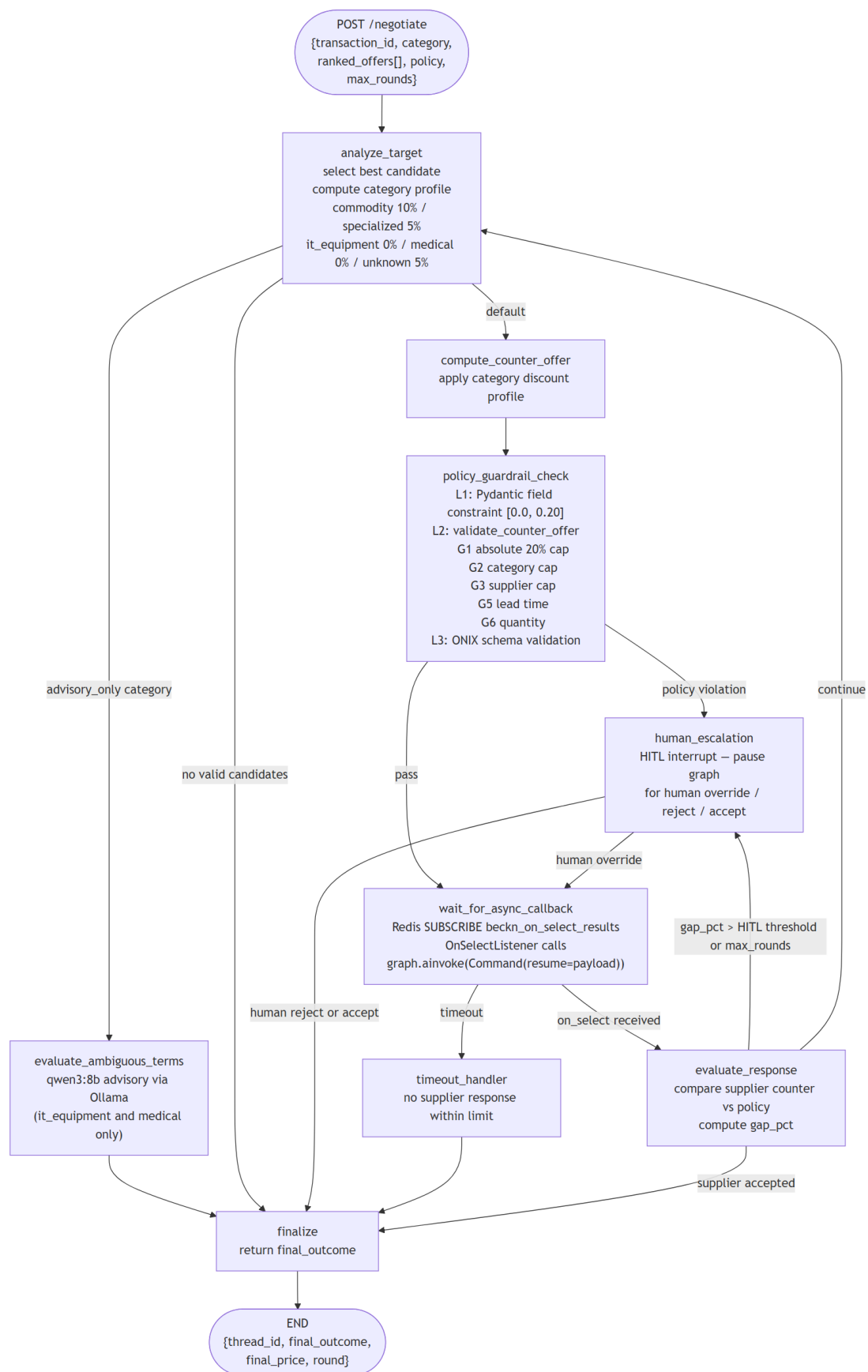


Figure 3.5: ARCHITECTURE diagram 5

3.7 7. Data Stores

3.7.1 PostgreSQL 16 + pgvector

The main procurement database runs natively on the developer host (port 5432) and is reached by containers via `host.docker.internal:5432`. A separate negotiation database runs inside the Docker stack (port 55432) for LangGraph checkpoints only.

The schema consists of 24 numbered SQL migration files in `database/sql/`. Files are applied in lexicographic order; the numeric prefix encodes the FK dependency chain. Never renumber existing files.

`data-normalizer :8006` is the **sole write path** for all production data. No other service writes directly to the main PostgreSQL database. Vector workloads use two HNSW cosine indexes:

Table	Dimensions	Model	Use
bpp_catalog_semantic_cache	384	all-MiniLM-L6-v2 (sentence-transformers)	IntentParser Stage 3 ANN validation (similarity thresholds: validated ≥ 0.85 , ambiguous 0.45–0.85, cache miss < 0.45)
agent_memory_vectors	384	BAAI/bge-small-en-v1.5 (fastembed ONNX)	Orchestrator supplier memory; time-decayed loyalty bonus $\Sigma [0.03 \times \exp(-\text{days} \times \ln(2)/90)]$ capped at ± 0.10 ; retrieval threshold 0.75

Both models replaced the spec-era OpenAI `text-embedding-3-large` (3072-dim) to eliminate per-call cost and data egress. See [System Design](#) for the full rationale.

3.7.2 Redis 7

Redis serves three distinct purposes, each with its own key namespace:

Purpose	Key pattern	Producer	Consumer
Async Beckn discovery (ADR-0001)	<code>beckn_results:{transaction_id}</code>	<code>bap_client/on_discover</code> handler	<code>mcp-sidecar/bap_client.py</code>

Purpose	Key pattern	Producer	Consumer
Negotiation async resume	beckn_on_select_resu	demo-gateway after SupplierAgent call	negotiation-engine OnSelectListener
ONIX routing cache	internal ONIX keys	onix-bap / onix-bpp	onix-bap / onix-bpp

Redis is a **hard runtime dependency** for IntentParser Stage 3. If Redis is unavailable, the mcp-sidecar returns `{"found": false, "items": []}` after `REDIS_RESULT_TIMEOUT` (default 15 s) and the orchestrator path is unaffected (it uses `CallbackCollector` directly, which does not require Redis).

Audit records carry a `retention_until = NOW() + 7 years` field (SOX 404 / IT Act 2000). The ERP outbox uses `erp_sync_records` with `FOR UPDATE SKIP LOCKED` for safe concurrent worker execution and exponential backoff at 5, 30, 120, 600, 3600 seconds.

3.8 8. Architectural Decisions Summary

The eight most impactful decisions made during Phases 1–3. Full ADR rationale is in [System Design](#).

Decision	What was chosen	One-line rationale
ADR-0001 — Redis Pub/Sub for async discovery	Per-transaction <code>beckn_results:{txn_id}</code> channel; <code>asyncio.create_task</code> probe	Breaks the asyncio event-loop deadlock that arises when <code>on_discover</code> callback arrives on the same event loop that is blocking for it
pgvector instead of Qdrant	pgvector 0.7.0 HNSW cosine at 384 dims; no Qdrant container	Pilot corpus < 100K records; pgvector HNSW is computationally equivalent at this scale and eliminates a separate stateful service
Local LLMs instead of GPT-4o	qwen3:8b (complex) and qwen3:1.7b (simple) via Ollama; Claude Sonnet 4.6 as opt-in broadening fallback only	Zero per-call cost, offline capability, and full data sovereignty for procurement content

Decision	What was chosen	One-line rationale
PostgreSQL outbox instead of Kafka for ERP push	erp_sync_records table, FOR UPDATE SKIP LOCKED worker, kafka_offset placeholder column	PO write and outbox row are atomic in a single PostgreSQL transaction; eliminates Kafka broker dependency for Phases 1–3 while keeping a migration path
DeDi registry bypass (targetType: url)	All four ONIX routing YAMLS use targetType: url	Full Beckn lifecycle testable offline inside the Docker bridge network; switching to a live network requires only YAML changes
ONIX schema validator pinned to d43ec30d	Image pinned; do not upgrade without an end-to-end signing test	A post-d43ec30d commit introduced a \$ref resolution bug in SignatureHeader that breaks ED25519 validation on every signed message
Vendor-neutral ERPAdapter typing.Protocol	One Protocol, three implementations (SAP / Oracle / mock), factory by ERP_VENDORS env var	New ERP vendors require only a new six-method implementation class; no changes to orchestrator, budget-gate logic, or webhook routes
Docker Compose instead of Kubernetes (Phases 1–3)	18-service docker-compose.yml on becn_network; Kubernetes is Phase 4 scope	Single-command startup, sub-minute cold start, and sufficient isolation to prove the full Beckn lifecycle without cluster management overhead

3.9 9. Component Reference

The system consists of 21 named components across five logical layers. Each entry describes the component’s role, run mode, responsibilities, and notable implementation details including known bugs and deviations from documentation. For environment variable details see [Configuration](#). For endpoint shapes see [API Reference](#).

Note on claude_openai_proxy (:8012). A host-only loopback service wraps the Claude Code CLI as an OpenAI-compatible endpoint. It is not in docker-compose.yml, but negotiation_engine and frontend_demo_gateway both depend on it at runtime via http://host.docker.internal:8012/v1. It must be started manually (uvicorn services.claude_openai_proxy.main:app --

host 0.0.0.0 --port 8012) before those services can function.

3.9.1 Dependency Overview

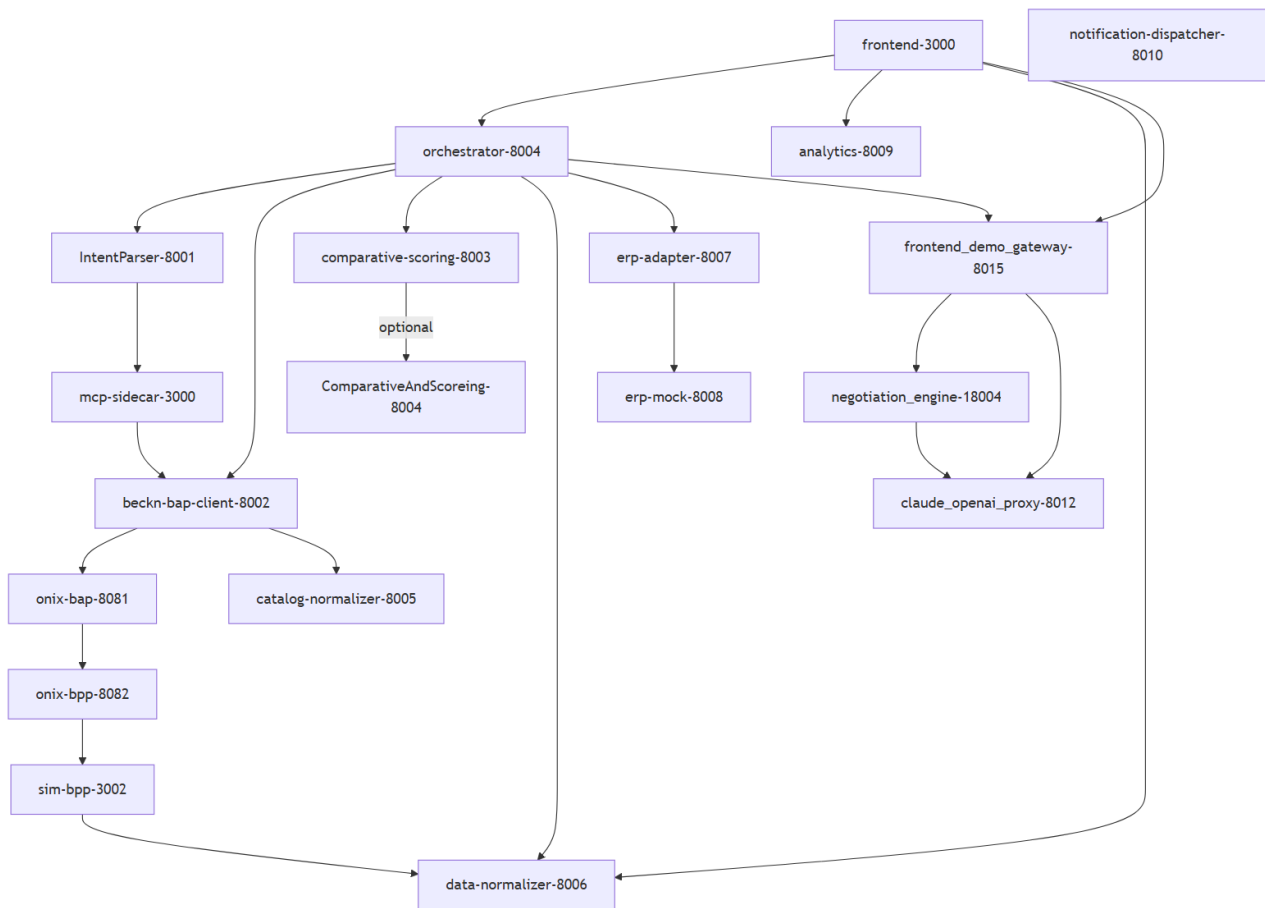


Figure 3.6: ARCHITECTURE diagram 6

3.9.2 IntentParser (:8001, local)

- **Role:** Three-stage NL-to-BecknIntent pipeline that classifies buyer intent, extracts structured procurement parameters, and validates them against live BPP catalogs.
- **Run mode:** Local process (not Dockerized at this path); `uvicorn api:app --port 8001 --reload` inside `conda activate infosys_project`. A Docker wrapper exists as `intention-parser`.
- **Language / framework:** Python 3.11, FastAPI, instructor + Ollama, sentence-transformers, asyncpg + pgvector.
- **Key responsibilities:**

- Stage 1: LLM intent classification (qwen3:8b or qwen3:1.7b) — returns ParsedIntent with confidence score.
 - Stage 2: Structured BecknIntent extraction via instructor; complexity-routes to COMPLEX_MODEL (queries >120 chars, ≥2 numeric tokens, or procurement keywords) vs SIMPLE_MODEL.
 - Stage3 (/parse/ full only): Hybrid pgvector ANN cosine search against bpp_catalog_semantic_cache followed by MCP sidecar probe on cache miss; thresholds VALIDATED ≥ 0.85 / AMBIGUOUS 0.45–0.85 / CACHE_MISS < 0.45.
 - Recoveryflow: broaden_procurement_query → Stage3 retry → stub handlers (log_unmet_demand, notify_buyer_no_stock, trigger_open_rfq_flow) — all three stubs are no-op logger calls as of Phase 3.
 - Enforces canonical BecknIntent encodings: delivery_timeline as int hours, location_coordinates as "lat,lon" decimal, budget_constraints as typed {max, min}.
- **Upstream dependencies:** Ollama (qwen3:8b and qwen3:1.7b), PostgreSQL + pgvector (bpp_catalog_semantic_cache), mcp-sidecar :3000 (Stage 3 only), Claude Sonnet 4.6 via ANTHROPIC_API_KEY (opt-in broadening fallback).
 - **Notable behavior:** In Docker (intention-parser container) both models are overridden to qwen3:1.7b, collapsing two-tier routing. Similarity thresholds are named constants in config.py and are not env-configurable.

3.9.3 orchestrator (:8004)

- **Role:** Central pipeline state machine that drives the full procurement lifecycle from NL query to confirmed Beckn order.
- **Run mode:** Docker container; host ports 8000 and 8004 both map to container port 8004 (dual mapping for frontend compatibility).
- **Language / framework:** Python 3.11, FastAPI, LangGraph, aiohttp.
- **Key responsibilities:**
 - 4-step pipeline: Step 1 intent parsing → Step 2 Beckn discovery → Step 3 comparative scoring → Step 4 select/init/confirm.
 - Two-phase POST /compare + POST /commit flow alongside single-shot POST /run.
 - Approval workflow: auto-approval below requester.approval_threshold; manager/CFO routing above.
 - Persists audit trail at 13 call sites via fire-and-forget asyncio.create_task — never raises on failure.

- Routes autonomous negotiation through `DEMO_GATEWAY_URL/api/demo/negotiate`.
- **Upstream dependencies:** `intention-parser:8001`, `beckn-bap-client:8002`, `comparative-scoring:8003`, `data-normalizer:8006`, `erp-adapter:8007`, `analytics:8009`, `frontend_demo_gateway:8015`.
- **Notable behavior:** All in-memory state is lost on container restart. `ERP_BUDGET_CHECK_REQUIRED=false` in the default stack (fail-open) — must be `true` in production. The `/compare+/commit` path does not invoke negotiation.

3.9.4 beckn-bap-client (:8002)

- **Role:** Beckn Protocol v2.0.0 BAP client that translates orchestrator/MCP sidecar requests into signed Beckn messages and collects asynchronous callbacks.
- **Run mode:** Docker container; host port 8002 → container port 8002.
- **Language / framework:** Python 3.11, FastAPI, aiohttp.
- **Key responsibilities:**
 - Sends all five Beckn actions to `onix-bap:8081` (direct BPP POST is prohibited).
 - Receives `POST /on_discover`: publishes to `beckn_results:{txn_id}` Redis channel AND enqueues to `CallbackCollector` — dual-path implements ADR-0001.
 - Handles `POST /bap/receiver/{action}` for all other `on_*` callbacks.
 - Delegates `on_discover` normalization to `catalog-normalizer:8005`.
- **Notable behavior:** `REDIS_URL` must be a real env var (not just in `.env`) — read via `os.getenv()` in `handler.py`, bypassing the Pydantic Settings model.

3.9.5 data-normalizer (:8006)

- **Role:** Sole write path into PostgreSQL for the entire stack; also serves audit trail read and agent memory endpoints.
- **Run mode:** Docker container; host port 8006 → container port 8006.
- **Language / framework:** Python 3.11, FastAPI, `asyncpg`, `sentence-transformers` (all-MiniLM-L6-v2).
- **Key responsibilities:**
 - Receives persistence payloads from all other services; normalizes field types on write.
 - Builds the full FK chain on `POST /normalize/order`: `negotiation_outcomes` → `approval_decisions` → `purchase_orders`.
 - Manages agent memory: embeds content with all-MiniLM-L6-v2 (384-dim) and stores in `agent_memory_vectors`.

- **Notable behavior:** Memory embedding failures are silently skipped. all-MiniLM-L6-v2 downloads from Hugging Face on first startup (~10–30 s cold start).

3.9.6 erp-adapter (:8007)

- **Role:** Vendor-neutral ERP integration layer: synchronous budget gate, async PO push with outbox retry, and inbound HMAC-verified vendor webhooks.
- **Run mode:** Docker container; host port 8007 → container port 8007.
- **Language / framework:** Python 3.11, FastAPI, asyncpg, pybreaker.
- **Key responsibilities:**
 - POST /api/v1/budget/check — synchronous gate (≤ 800 ms); fail-closed when ERP_BUDGET_CHECK_REQUIRED=true.
 - POST /api/v1/po/sync — PostgreSQL outbox with FOR UPDATE SKIP LOCKED, per-vendor circuit breakers, exponential backoff (5/30/120/600/3600 s).
 - POST /api/v1/webhooks/{vendor}/po-status — zero-downtime HMAC rotation via primary + _NEXT secondary secret.
 - Adapter selected at startup via ERP_VENDORS env var (SAP / Oracle / mock).
- **Notable behavior:** ERP_BUDGET_CHECK_REQUIRED=false in the default stack — must be true in production.

3.9.7 erp-mock (:8008)

- **Role:** In-memory local ERP stub simulating SAP S/4HANA and Oracle ERP Cloud surfaces for development.
- **Run mode:** Docker container; host port 8008 → container port 8008. All state is non-durable.
- **Key responsibilities:**
 - Six named scenarios selectable via MOCK_SCENARIO env var or X-Mock-Scenario header.
 - Emits HMAC-signed webhook callbacks to erp-adapter after WEBHOOK_DELAY_SECONDS on every PO creation.
- **Notable behavior:** The Oracle HMAC mock webhook emitter currently only signs with SAP_WEBHOOK_HMAC_SECRET.

3.9.8 negotiation_engine (:18004 host / :8004 container)

- **Role:** Automated price negotiation LangGraph state machine with policy-bounded counter-offers and three-layer guardrails.

- **Run mode:** Docker container; host port **18004** (avoids collision with orchestrator at 8004).
- **Language / framework:** Python 3.11, FastAPI, LangGraph.
- **Key responsibilities:**
 - Hard 20% maximum discount cap enforced at three independent layers: Pydantic (L1), policy shield (L2), ONIX schema (L3).
 - Per-category discount profiles: commodity 10%, specialized 5%, it_equipment/medical 0% advisory-only.
 - OnSelectListener subscribes to Redis becn_on_select_results to resume parked LangGraph states.
 - Checkpoints to PostgreSQL via AsyncPostgresSaver when NEGOTIATION_POSTGRES_DSN is set; falls back to MemorySaver.
- **Upstream dependencies:** Redis :6379, PostgreSQL (separate negotiation DB port 55432), claude_openai_proxy :8012.
- **Notable behavior:** The orchestrator never calls negotiation_engine directly — all paths go through frontend_demo_gateway.

3.9.9 comparative-scoring (:8003)

- **Role:** Thin scoring adapter forwarding offering lists to the Phase 2 ML RankNet backend with cheapest-wins heuristic fallback.
- **Run mode:** Docker container; host port 8003 → container port 8003.
- **Key responsibilities:**
 - POST /score — tries {PREDICTION_API_URL}/score (8 s timeout); falls back to min-price heuristic.
- **Notable behavior:** SCORING_FALLBACK_ENABLED=true by default — always functional without the MLOps stack.

3.9.10 ComparativeAndScoreing (MLOps)

- **Role:** Phase 2 MLOps stack providing a RankNet/LambdaRank learning-to-rank model; separate docker-compose.mlops.yaml.
- **Run mode:** Not in the main docker-compose.yaml. prediction-api host port 8004, MLflow UI on port 5000.
- **Key responsibilities:**

- `nn.Linear(3,1)` forward pass on `[x_price, x_speed, x_risk]` (all min-max scaled within session).
- Training: 200 epochs SGD/AdamW; auto-promotes to MLflow Staging if `NDCG@5 ≥ 0.85`. Never auto-promotes to Production.
- **Notable behavior:** Production promotion is always manual via the MLflow CLI.

3.9.11 discovery_engine

- **Role:** Multi-network Beckn discovery fan-out service with per-network circuit breakers and geographic deduplication.
- **Run mode: Orphaned** — code is complete at `services/discovery_engine/` but there is no entry in `docker-compose.yml` and no other service calls it.
- **Notable behavior:** The operational discovery path is entirely inside `services/beckn-bap-client/`. Do not extend `discovery_engine` expecting it to affect runtime behaviour.

3.9.12 catalog-normalizer (:8005)

- **Role:** Beckn catalog normalization bridge mapping raw `on_discover` payload variants to `DiscoverOffering` objects.
- **Run mode:** Docker container; host port 8005 → container port 8005.
- **Key responsibilities:**
 - `POST /normalize` — detects format variant (1-5) and maps to `[DiscoverOffering]`.
 - Variants 1-4: deterministic rule-based mappers. Variant 5: LLM fallback via instructor + Ollama.
- **Notable behavior:** The service README incorrectly states `OPENAI_API_KEY` is required — the implementation reads `OLLAMA_URL` and `NORMALIZER_MODEL`. `OPENAI_API_KEY` is never read.

3.9.13 mcp-sidecar (:3000, local)

- **Role:** MCP SSE bridge exposing `search_bpp_catalog` to IntentParser Stage 3; implements the subscriber side of ADR-0001.
- **Run mode:** Local process (not Dockerized); `BAP_API_KEY="<any-string>"` `uvicorn server:app --port 3000`.
- **Key responsibilities:**

- Pre-subscribes to `beckn_results:{txn_id}` on Redis, fires `POST becn-bap-client/discover` as `asyncio.create_task` (never `await`), waits up to `REDIS_RESULT_TIMEOUT` (default 15 s).
- All failure paths return `{"found": false, "items": [], "probe_latency_ms": elapsed}` — never a JSON-RPC error.
- **Notable behavior:** The discover POST **must** use `asyncio.create_task(...)`, never `await`. `REDIS_URL` and `REDIS_RESULT_TIMEOUT` must be real env vars (read via `os.getenv()`, not Pydantic Settings).

3.9.14 sim-bpp (:3002)

- **Role:** Local Beckn BPP simulator (Node.js) replacing `fidedocker/sandbox-2.0`; hot-reloadable catalog, async callbacks.
- **Run mode:** Docker container; host port 3002 → container port 3002.
- **Key responsibilities:**
 - AND-token matching against `catalog.json` (9 providers, 31 items); re-read on every request.
 - Auto-advance lifecycle: `ACCEPTED` → `PACKED` → `SHIPPED` → `OUT_FOR_DELIVERY` → `DELIVERED`; PATCHes data-normalizer and publishes to Kafka per transition.
- **Notable behavior:** `SIM_BPP_AUTO_ADVANCE` defaults to `false` in code but is `true` in `docker-compose.yml`.

3.9.15 intention-parser (Docker wrapper)

- **Role:** Thin Docker container wrapping the `IntentParser` package; exposes Stages 1 and 2 only (Stage 3 disabled in Docker context).
- **Run mode:** Docker container; host port 8001 → container port 8001.
- **Notable behavior:** Both models overridden to `qwen3:1.7b` in `docker-compose.yml` — `qwen3:8b` is never invoked from Docker.

3.9.16 frontend_demo_gateway (:8015)

- **Role:** Demo BFF bridging the Next.js frontend to the PyTorch scoring model and the LangGraph negotiation engine.
- **Run mode:** Docker container; host port **8015**. (The service's own README incorrectly states port 8005.)
- **Key responsibilities:**

- POST /api/demo/negotiate+GET /api/demo/negotiate/{thread_id} — negotiation lifecycle.
- POST /api/demo/negotiate/{thread_id}/supplier-respond — calls SupplierAgent via claude_openai_proxy; publishes result to Redis becn_on_select_results, resuming the parked LangGraph.
- **Notable behavior:** The only path through which the orchestrator's autonomous negotiation reaches negotiation_engine.

3.9.17 analytics (:8009)

- **Role:** Procurement reporting service querying PostgreSQL for aggregated KPIs, spend trends, and CPO benchmarks.
- **Run mode:** Docker container; host port 8009 → container port 8009.
- **Key responsibilities:**
 - GET /analytics?period=30d|90d|180d — full dashboard payload.
 - GET /benchmark?period=... — CPO benchmarking: contracted price vs best available market price.
- **Notable behavior:** Returns HTTP 503 (not mock data) when the DB connection pool is unavailable.

3.9.18 notification-dispatcher (:8010)

- **Role:** Kafka consumer fanning order status events out to Slack, Microsoft Teams, and email.
- **Run mode:** Docker container; no host port published. Consumer never starts if KAFKA_BOOTSTRAP is empty.
- **Key responsibilities:**
 - confirmed/delivered → all three channels; shipped/cancelled → Slack + Teams only.
 - asyncio.gather(return_exceptions=True) — one channel failure never blocks others.
- **Notable behavior:** KAFKA_BOOTSTRAP is empty by default — no notifications are sent in the default Docker stack.

3.9.19 onix-bap (:8081, Go)

- **Role:** BAP-side Go ONIX adapter validating Beckn schemas, routing outbound requests, and signing/verifying ED25519 signatures.

- **Run mode:** Docker container (`fidedocker/onix-adapter`, `linux/amd64`); host port 8081.
- **Key responsibilities:**
 - Signs outbound messages; applies split routing for `on_discover` (ADR-0001); routes all other `on_*` callbacks to `beckn-bap-client /bap/receiver/{action}`.
- **Notable behavior:** Target URLs in routing YAMLs must NOT include the action name — ONIX appends it. Schema validator pinned to commit `d43ec30d` — do not upgrade without an end-to-end signing test.

3.9.20 onix-bpp (:8082, Go)

- **Role:** BPP-side Go ONIX adapter mirroring `onix-bap`'s functionality for the inbound direction.
- **Run mode:** Docker container (`fidedocker/onix-adapter`, `linux/amd64`); host port 8082.
- **Notable behavior:** A `networkId` vs `domain` inconsistency in `config/generic-routing-BPPReceiver.yaml` is unresolved and may cause routing issues at runtime.

3.9.21 frontend (Next.js, :3000)

- **Role:** Buyer-facing Next.js 13.5 web application: procurement wizard, order tracking, approvals, analytics dashboard, agent reasoning panel, audit trail viewer.
- **Run mode:** Local dev (`npm run dev`). Not Dockerized.
- **Language / framework:** Next.js 13.5, Tailwind CSS, `shadcn/ui`, `recharts`, NextAuth 4, Keycloak OIDC.
- **Key responsibilities:**
 - All backend calls proxied through Next.js API routes: `/api/orchestrator/*`, `/api/analytics/*`, `/api/audit/*`, `/api/users/*`, `/api/demo/*`.
 - Authentication via NextAuth 4 + Keycloak only — no stub credentials mode, requires a live OIDC tenant.
- **Notable behavior:** No frontend tests, no App Router error boundaries. All `recharts` charts must be wrapped in `dynamic(..., { ssr: false })`.

3.9.22 shared/ (cross-service models)

- **Role:** Anti-corruption layer Python package providing canonical Pydantic v2 models shared across all Python services.
- **Run mode:** Python package; volume-mounted into Docker containers at `/app/shared`.
- **Notable behavior:**

Field	Canonical encoding	Common wrong form
delivery_timeline	72 (int hours)	"P3D" (ISO 8601)
location_coordinates	"12.9716,77.5946"	"Mumbai" (city name)
budget_constraints	BudgetConstraints(max=200.0,"max 200 INR" (raw string) min=0.0)	

3.9.23 Dependency Matrix

Checkmarks indicate that the row service makes HTTP, Redis, or Kafka calls to the column service.

Caller	IntentParser	data-normalizer	erp-adaptor	erp-mock	negotiation-engine	comparative-scoring	catalog-middleware	erp-simulator	frontend-analytic	onix-gateway	onix-backend	PostgreSQL	Redis	Kafka	Ollama
frontend		✓							✓	✓					
orc	✓	✓	✓	✓		✓			✓	✓					
IntentParser (local)								✓				✓			✓
backend-baj-client						✓				✓		✓			
data-normalizer												✓			✓
erp-adaptor				✓								✓	✓	✓	
erp-mock				✓											
negotiation-engine												✓	✓	✓	✓
comparative-scoring						✓									
Catalog-middleware												✓			
erp-simulator															
frontend-analytic															
onix-gateway															
onix-backend															
PostgreSQL															
Redis															
Kafka															
Ollama															✓

	beckn- IntentParser	data-erp- normalizer	erp- adapter	erp- rocknegotiation	C&S comparative pricing engine	log- mcp-sim- malware app	front-end analytics	onix- bap gateway	onix- bap gateway	PostgreSQL	Redis	OpenAI	Ollama
mcp- sidecar	✓										✓		
sin bpi		✓						✓				✓	
intention- parser (Docker)													✓
fro				✓				✓		✓			
analytics										✓			
not dis										✓		✓	
onix- bap								✓			✓		
oni bpi						✓					✓		
shared/													

Key: ✓ = service in that row calls the service in that column over the network. `discovery_engine` has no active callers or callees in the deployed stack (orphaned). `shared/` has no runtime dependencies (it is an imported library).

4 System Design

This document describes the structural principles, communication contracts, and key architecture decisions that govern the Procurement Agent. It is the primary reference for engineers who need to understand why the system is built the way it is, not just what it does. For the service-by-service operational reference, see [Architecture](#). For practical troubleshooting, see [Troubleshooting](#). For database schema details, see [Database](#).

4.1 1. Service Boundaries and Responsibilities

The system has sixteen Docker-deployed services and two local processes. Each service owns exactly one concern. The boundaries below are enforced by deployment topology, not just convention.

4.1.1 1.1 Service Responsibility Map

Service	Port	Single Responsibility
orchestrator	8004 (8000)	Pipeline state machine — sole entry point for the frontend; coordinates all other services
intention-parser	8001	NL → BecknIntent translation (Stages 1–3)
beckn-bap-client	8002	Beckn protocol client — all <code>discover</code> / <code>select</code> / <code>init</code> / <code>confirm</code> / <code>status</code> calls through <code>onix-bap</code>
catalog-normalizer	8005	Raw <code>on_discover</code> payload → typed <code>DiscoverOffering</code> list

Service	Port	Single Responsibility
comparative-scoring	8003	ML-scored ranking of DiscoverOffering objects
data-normalizer	8006	Sole write path to PostgreSQL — all other services use it for persistence
erp-adapter	8007	Budget gate + PO push to SAP/Oracle/mock; owns the erp_sync_records outbox
erp-mock	8008	Local ERP stub used when ERP_VENDORS=mock
analytics	8009	Read-only reporting queries against PostgreSQL
notification-dispatcher	8010	Kafka consumer → Slack/Teams/Email fan-out
negotiation-engine	18004	LangGraph automated price negotiation with multi-layer guardrails
demo-gateway	8015	Frontend BFF; bridges orchestrator to negotiation-engine and hosts the SupplierAgent
onix-bap	8081	Beckn BAP-side Go adapter — ED25519 signing, schema validation, DeDi routing
onix-bpp	8082	Beckn BPP-side Go adapter
sim-bpp	3002	Local BPP simulator replacing sandbox-2.0
mcp-sidecar	3000	MCP SSE bridge between IntentParser Stage 3 and the Beckn network

Two host-local processes not in Docker:

Process	Port	Role
IntentParser (local dev)	8001	Runs outside Docker when qwen3 : 8b complexity routing is needed

Process	Port	Role
claude_openai_proxy	8012	Wraps Claude Code CLI as an OpenAI-compatible endpoint; required by negotiation-engine and demo-gateway

4.1.2 1.2 Why the Orchestrator Is the Single Frontend Entry Point

The orchestrator exposes the `/run`, `/compare`, and `/commit` endpoints that the Next.js frontend calls. All other services are internal to the Docker network — they bind on `beckn_network` bridge addresses and are not directly reachable from a browser or external HTTP client.

This boundary exists for three reasons:

1. **State ownership.** The two-phase compare/commit flow requires session state (a `transaction_id` → session dict with 30-minute TTL) that must be consistent across both calls. Only one service can own that dict reliably on a single-host deployment.
2. **Approval workflow.** Spend-level routing (auto-approve / manager / CFO) requires a consistent view of the pipeline state and a blocking wait for the human decision. The orchestrator implements `POST /approvals/{id}/decide` and parks the pipeline until the decision arrives.
3. **Pipeline sequencing.** IntentParser, beckn-bap-client, comparative-scoring, and data-normalizer each do one thing. Sequencing their calls, handling partial failures, and enriching results with memory context is the orchestrator's job. Exposing those services directly to the frontend would push pipeline logic into the UI.

4.1.3 1.3 Why data-normalizer Is the Sole PostgreSQL Writer

Every service that needs to write to the database — orchestrator, beckn-bap-client, erp-adapter, sim-bpp, negotiation-engine — calls `data-normalizer :8006` instead of writing directly. This constraint is enforced by deployment: no other service has the PostgreSQL DSN injected as an environment variable.

Benefits of this design:

- **Single schema evolution point.** Adding a column or migration requires a change to one service's Pydantic models and one service's SQL, not six.
- **Centralised error handling.** The `db_error_middleware` in `data-normalizer` maps `asyncpg` exception types to structured HTTP error codes (409 for unique/FK violations, 422 for check/not-null

violations, 500 for all other `PostgresError`). Callers branch on HTTP status, never on database exception types.

- **Idempotency.** A 409 response from data-normalizer is an explicit signal that the row already exists; callers treat it as success. Without this, every caller would need to implement its own duplicate-detection logic.
- **Audit centralisation.** All `audit_trail_events` writes go through `POST /normalize/audit`. The 20+ call sites in the orchestrator alone would be scattered across the codebase if they wrote directly.

4.2 2. Communication Patterns

The system uses four distinct communication patterns. The choice of pattern for each interaction is determined by latency requirements, whether the caller can tolerate eventual consistency, and whether the transport needs to bridge separate processes.

Pattern	Services involved	When used	Why
Synchronous HTTP (blocking)	orchestrator → intention-parser, beckn-bap-client, comparative-scoring, erp-adapter	Critical-path steps where the pipeline result depends on the response	Pipeline sequencing requires ordered results; failures must halt the pipeline
Redis Pub/Sub	beckn-bap-client (PUBLISH) ↔ mcp-sidecar (SUBSCRIBE) on <code>beckn_results:{txn_id}</code> negotiation-engine (SUBSCRIBE) ↔ demo-gateway (PUBLISH) on <code>beckn_on_select_resu</code>	Beckn async discovery and negotiation engine resume	Beckn v2.0.0 returns only an ACK; the catalog arrives via webhook. Holding an HTTP connection open blocks the asyncio event loop — see ADR-0001
Kafka (deferred)	sim-bpp (PUBLISH), erp-adapter (PUBLISH) → notification-dispatcher (CONSUME) on <code>po.status.changed</code>	Order lifecycle state transitions and ERP audit events	Durable, replayable fan-out to multiple independent consumers (Slack, Teams, Email); decouples the BPP simulator from the notification layer

Pattern	Services involved	When used	Why
asyncio.create_task fire-and-forget	orchestrator → data-normalizer for memory write, audit writes	Non-critical side effects after a confirmed PO	Audit and memory writes must not add to the critical-path latency. A lost audit write is tolerable; a blocked user interface is not

4.2.1 2.1 Redis Pub/Sub Channels

Channel	Publisher	Subscriber	Purpose
beckn_results:{transaction_id}	beckn-api-client /on_discover handler	mcp-sidecar search_bpp_catalog tool	Delivers catalog payload from Beckn webhook to IntentParser Stage 3
beckn_on_select_result	demo-gateway (SupplierAgent response)	negotiation-engine OnSelectListener	Resumes a parked LangGraph graph with the supplier’s counter-offer
po.status_changed:{po_id}	mcp-adapter webhook handler	(Kafka fallback only)	Redis key written when Kafka is unavailable

4.2.2 2.2 Beckn Async Discovery — Why asyncio.create_task, Not await

The critical rule enforced by CLAUDE.md: the MCP sidecar must fire POST /discover as asyncio.create_task(...), never as await. Awaiting the POST blocks the event loop — the same event loop that must receive and process the on_discover callback. The result is a deadlock. The full five-step protocol is described in [ADR-0001](#) and shown in the sequence diagram in Architecture.

4.3 3. The Anti-Corruption Layer

shared/models.BecknIntent is the canonical model that crosses the boundary between natural language and the Beckn protocol. Every service that handles procurement intent either produces or consumes a BecknIntent. No service downstream of IntentParser knows about raw query strings, NLP intermediate representations, or locale-specific data formats.

4.3.1 3.1 Canonical Field Conventions

Three fields have invariants enforced by Pydantic v2 `field_validator` decorators. Violating them causes silent downstream errors that are hard to trace back to the parse step.

Field	Type	Invariant	Example	What breaks if violated
<code>delivery_timeline</code>	<code>int (hours)</code>	Never ISO 8601 duration strings	72 (not "P3D")	The Beckn fulfillment- <code>ment.start.time.duration</code> field expects a plain integer; downstream services that compute deadline timestamps fail if the value is a string
<code>location_coordinates</code>	<code>str "lat,lon" decimal</code>	Never city names or postal codes	"19.0760, 72.8777" (not "Mumbai")	pgvector ANN searches and Beckn location filtering require numeric coordinates; city names cause the BPP catalog match to silently return zero results
<code>budget_constraints</code>	<code>BudgetConstraints(max: float, min: float)</code>	Never raw strings	<code>{"max": 200.0, "min": 0.0}</code>	The ERP budget gate compares <code>amount_total</code> against <code>budget_constraints.max</code> ; a raw string causes a type comparison failure

4.3.2 3.2 Why These Invariants Exist

IntentParser Stages 1 and 2 use LLMs (qwen3:8b / qwen3:1.7b via Ollama) to extract structured data from natural language. LLMs without structured output enforcement will freely produce ISO 8601 durations, city names, and free-text budget ranges. The invariants exist because:

1. **LLM outputs are non-deterministic.** The same query may produce "3 days", "P3D", or 72 depending on the model's sampling state. Pydantic validation provides a deterministic contract.
2. **Downstream services do not know they received NL-derived data.** becn-bap-client, erp-adapter, and comparative-scoring treat BecknIntent as a structured business object. They would need NL-aware parsing logic if the invariants were not enforced at the boundary.
3. **Reuse across paths.** The same BecknIntent is consumed by IntentParser Stage 3 (for catalog ANN search), the orchestrator (for Beckn request construction), and data-normalizer (for persistence). A single enforcement point is more reliable than six separate validations.

4.3.3 3.3 Model Routing and the Docker Override

IntentParser uses a complexity heuristic to route between two LLMs:

- Queries longer than 120 characters, containing at least 2 numeric tokens, or containing procurement keywords (e.g. "urgent", "approval", "RFQ") → COMPLEX_MODEL (local default: qwen3:8b)
- All other queries → SIMPLE_MODEL (qwen3:1.7b)

Docker override discrepancy: `docker-compose.yml` sets `COMPLEX_MODEL=qwen3:1.7b` for the `intention-parser` container. This collapses the two-tier routing — the Dockerized service uses `qwen3:1.7b` for all queries. The `qwen3:8b` model and complexity routing are only active when IntentParser runs locally outside Docker. No warning is emitted. Engineers running the full Docker stack may observe lower extraction quality on complex multi-constraint queries without knowing why.

4.4 4. Error Handling Philosophy

Each layer of the system has a documented error contract. The contracts are chosen to prevent failures from propagating upward — each service absorbs its layer's failures and presents a uniform signal to its caller.

4.4.1 4.1 Never-Throw MCP Contract

The MCP sidecar (`search_bpp_catalog` tool) is the most strictly contained service in the system. It never raises a JSON-RPC error. Every outcome — timeout, Redis unavailability, BAP client unreachable, malformed response, blank argument, unhandled exception — returns HTTP 200 with a consistent JSON body:

```
{"found": false, "items": [], "probe_latency_ms": N}
```

A top-level `try/except Exception` in `server.py::search_bpp_catalog` enforces this. Intent-Parser Stage 3 branches on `result["found"]` without inspecting exception types. The contract means that if the entire Beckn discovery infrastructure is down, IntentParser degrades gracefully to its recovery flow rather than raising an unhandled exception.

The `not_found` recovery flow — query broadening (regex strip, then Claude Sonnet 4.6 via ANTHROPIC_API_KEY if set) → `retry` → `log_unmet_demand` / `notify_buyer_no_stock` / `trigger_open_rfq_flow` — is the designed degradation path. Note that the three terminal stubs in `IntentParser/recovery.py` are currently logger-only implementations; the real DB write, notification service call, and RFQ microservice integration are Phase 4 scope.

4.4.2 4.2 Fail-Closed ERP Budget Gate

The ERP budget gate is configured by two environment variables:

Variable	docker-compose.yml default	Production intent
ERP_BUDGET_CHECK_ENABLED	true	Always true
ERP_BUDGET_CHECK_REQUIRED	false (fail-open)	Must be true (fail-closed) in production
BUDGET_CHECK_TOTAL_TIMEOUT_MS	800	Hard ceiling enforced by <code>asyncio.wait_for</code>

Important: The docker-compose.yml default is fail-open (`ERP_BUDGET_CHECK_REQUIRED=false`) for development convenience. CLAUDE.md documents the intent as fail-closed. Engineers must explicitly set `ERP_BUDGET_CHECK_REQUIRED=true` before any deployment handling real procurement spend.

When `REQUIRED=true` and the ERP vendor does not respond within 800 ms (or its per-vendor py-breaker circuit breaker is OPEN), the commit is blocked and the orchestrator returns HTTP 402. No purchase order is created. This is the correct behaviour: an unknown budget status must block a spend commitment.

Per-vendor circuit breakers (one `pybreaker.CircuitBreaker` per vendor in `ERP_VENDORS`) trip to OPEN after `BREAKER_FAIL_MAX=5` consecutive failures and auto-probe after `BREAKER_RESET_TIMEOUT_SECS=60`. When a breaker is OPEN, budget checks fail immediately without consuming the 800 ms timeout. Business rejections (e.g. `INSUFFICIENT_FUNDS` returning a vendor-confirmed error) are not counted as circuit failures — only transport and infrastructure errors trip the breaker.

4.4.3 4.3 asyncio.gather Channel Isolation in Notifications

The notification-dispatcher fans out to Slack, Teams, and Email concurrently using:

```
await asyncio.gather(
    send_slack(...), send_teams(...), send_email(...),
    return_exceptions=True
)
```

`return_exceptions=True` ensures a `SlackApiError` does not cancel the Teams or Email coroutines. Each result is inspected individually; failures are logged at WARNING level. An empty string for any webhook URL (`SLACK_WEBHOOK_URL=""`, `TEAMS_WEBHOOK_URL=""`, `SMTP_HOST=""`) silently disables that channel — the service starts normally with any combination active.

Kafka consumer gap: No dead-letter queue exists for the Kafka consumer. Malformed or unrecognised messages are logged and skipped; `enable_auto_commit=True` advances the offset regardless. A message that cannot be JSON-parsed will never be reprocessed.

4.4.4 4.4 Always-200 Discovery Engine

The `discovery_engine` service (currently orphaned — it has no `docker-compose.yml` entry and no callers) returns HTTP 200 for all requests, including partial failures:

```
{"items": [...], "total": 3, "degraded": true, "failed_networks": ["network_b"]}
```

Callers must check `degraded: true` and `failed_networks`. The per-network circuit breaker (`DISCOVERY_CB_FAILURE_THRESHOLD=3`, `DISCOVERY_CB_RECOVERY_TIMEOUT_S=30`) excludes misbehaving networks from fan-out results rather than failing the entire response.

4.4.5 4.5 Negotiation Engine Three-Layer Guardrails

Counter-offers in the negotiation engine pass through three independent validation layers before being sent to a BPP:

1. **L1 Pydantic:** `discount_pct` field constraint `[0.0, 0.20]` — absolute maximum 20% discount enforced at schema level.
2. **L2 `validate_counter_offer()`:** Five guardrail rules (G1: absolute cap, G2: category-specific cap, G3: supplier cap, G5: lead time policy, G6: minimum order quantity). A G2 or G3 violation routes the LangGraph state machine to the `human_escalation` HITL interrupt node.

3. **L3 ONIX schema validation:** Beckn wire-format validation before the message reaches the protocol adapter.

For practical guidance on ONIX schema rejection causes (wrong `Contract.status.code`, misplaced billing/fulfillment fields, action names in routing URLs), see [Troubleshooting](#).

4.5 5. Data Consistency

The system has no distributed transactions. The consistency model is chosen deliberately: strong consistency only where financial commitments are being made; eventual consistency for observability and learning data.

4.5.1 5.1 Strong Consistency — ERP Outbox

The only path that uses a durability guarantee is the ERP PO push. When the orchestrator calls `POST /confirm`, the Beckn confirmation and the `erp_sync_records` outbox row are written in the **same PostgreSQL transaction** inside `data-normalizer`. If the PO row is committed, the outbox row is guaranteed to exist. The `erp-adapter` background worker picks it up, pushes to the ERP vendor, and updates the row status using `SELECT ... FOR UPDATE SKIP LOCKED` to prevent double-processing.

Retry schedule: 5, 30, 120, 600, 3600 seconds (exponential, configurable via `WORKER_BACKOFF_CSV`).

After exhausting retries the row moves to DLQ status; manual replay is available via `POST /api/v1/admin/outbox/{id}`.

See [Database](#) for the `erp_sync_records` schema and `19b_erp_sync_records_outbox.sql`.

4.5.2 5.2 Fire-and-Forget Writes — Eventual Consistency with Silent Failure Risk

Memory and audit writes are non-blocking side effects. The orchestrator wraps them in `asyncio.create_task(...)` and never awaits them:

```
asyncio.create_task(_persist_memory(...))      # agent_memory_vectors
asyncio.create_task(_persist_audit(...))       # audit_trail_events (20+ call sites)
```

A 5-second timeout is applied to pipeline data writes (`_persist`); memory and audit helpers catch all exceptions internally and log them. The pipeline continues regardless of persistence failures.

Consequences:

- If the orchestrator process crashes between task creation and task completion (e.g. during a hot-restart under Docker Compose), the audit event and memory record are permanently lost. There is no buffer, queue, or retry.
- The `audit_trail_events` table carries a `retention_until = NOW() + 7 years` marker (SOX 404 / GDPR / IT Act 2000), but the write itself is not durable. Phase 4 Kafka integration is intended to address this gap: all `_persist_audit` calls currently set `kafka_offset=0` with a `# TODO(kafka): real offset when topic is wired` comment.

The `time.sleep()` prohibition is directly related to this design. `time.sleep()` blocks the event loop thread — a fire-and-forget task created just before a `time.sleep()` call will never execute because the event loop cannot schedule it. Tests that assert on fire-and-forget side effects must use `await asyncio.sleep(0)` to yield control.

4.5.3 5.3 In-Memory Session State

The orchestrator’s compare/commit session dict (`_sessions`, TTL 1800 s) and approval workflow state (`_pending_approvals`) are in-process Python dicts. A service restart loses all in-flight sessions. There is no Redis or database backend for session state in Phases 1–3. See [Section 6](#) for the Phase 4 plan to externalise this.

4.6 6. Scalability Design and Current Limits

4.6.1 6.1 Current State (Phases 1–3)

The entire system runs on a single Docker Compose host. There is no horizontal scaling, rolling update, or pod-level health restart.

Constraint	Root cause	Phase 4 plan
orchestrator cannot run more than one replica	Session dict is in-process; two replicas would not share state	Externalise sessions to Redis; use a shared dict under <code>ORCHESTRATOR_REDIS_DSN</code>
negotiation-engine state is in-memory (<code>MemorySaver</code>) by default	<code>NEGOTIATION_POSTGRES_DSN</code> is optional; without it <code>AsyncPostgresSaver</code> is not used	Set <code>NEGOTIATION_POSTGRES_DSN</code> for durable LangGraph checkpoints

Constraint	Root cause	Phase 4 plan
<code>claude_openai_proxy</code> :8012 is a host-local process, not a Docker service	Requires the <code>claude</code> binary installed on the host; loopback-only bind	Will require special handling in Kubernetes (sidecars, or replacement with a managed Claude API endpoint)
Port collision workarounds	orchestrator maps both :8000 and :8004 for frontend compatibility; negotiation-engine uses :18004 to avoid conflict	Kubernetes service DNS eliminates host-port workarounds
<code>discovery_engine</code> multi-network fan-out is not deployed	No <code>docker-compose.yml</code> entry, no callers	Wire into orchestrator's discovery step in Phase 4 if multi-network support is needed

4.7 7. Architecture Decision Records

4.7.1 ADR-0001: Redis Pub/Sub for Async Beckn Discovery

Status: Accepted — May 2026. Formal ADR at `docs/architecture/decisions/0001-use-redis-pubsub-for-async-beckn-responses.md`.

Context: Beckn Protocol v2.0.0 makes discovery inherently asynchronous. `POST /discover` to `onix-bap` returns only an ACK; the actual catalog arrives later via an `on_discover` webhook callback. The MCP sidecar must fire a discover request and return a synchronous result to IntentParser Stage 3. The naive implementation — `await` the discover POST — blocks the `asyncio` event loop. The `on_discover` callback must be processed by the same event loop, creating a deadlock: the sidecar is suspended awaiting the callback while the callback can never be received because the event loop is blocked.

Decision: Use Redis Pub/Sub on per-transaction channels `beckn_results:{transaction_id}`. The five-step protocol:

The `/on_discover` handler also calls `CallbackCollector.handle_callback("on_discover", payload)` so the orchestrator's synchronous discovery path (which does not use the sidecar) is unblocked by the same webhook.

Rationale: - Breaks the deadlock by decoupling the two coroutines across an external message channel. - Redis was already a required dependency (`onix-bap` uses it for its internal cache); no new infrastructure is introduced. - The channel name embeds `transaction_id`, providing natural isolation between concurrent

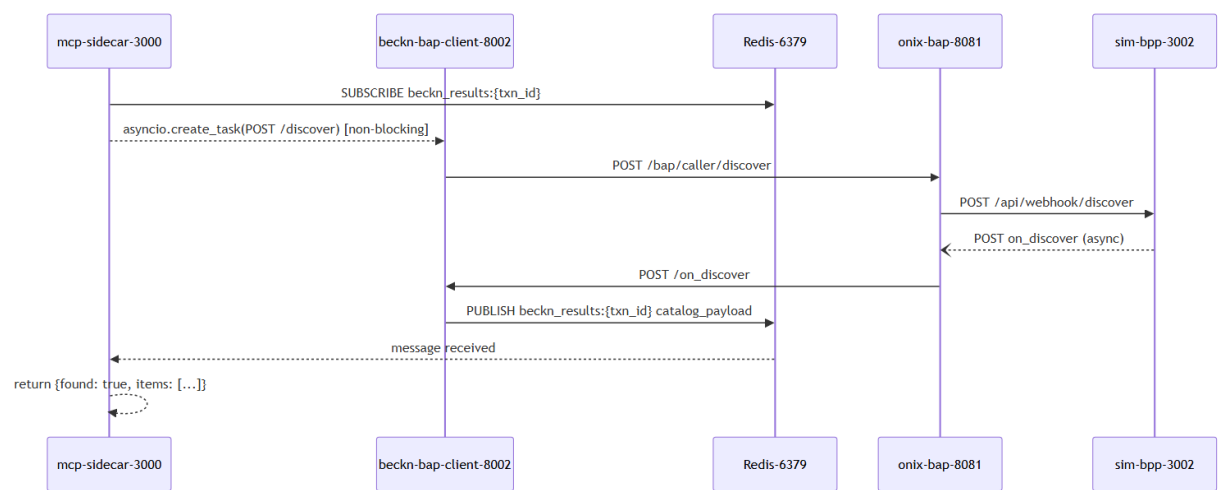


Figure 4.1: SYSTEM_DESIGN diagram 1

discovery probes. - Observable in real time via `redis-cli subscribe becn_results:<uuid>`. - Three rejected alternatives: HTTP polling (adds latency, requires server-side state), direct callback injection (circular dependency), in-process `asyncio.Queue` (breaks the microservices boundary — `mcp-sidecar` and `becn-bap-client` are separate processes).

Consequences: - Positive: Deadlock eliminated; producer/consumer fully decoupled; no new infrastructure. - Negative: Redis is a hard runtime dependency for IntentParser Stage 3. If Redis is unavailable, the sidecar degrades to `{"found": false, "items": []}` rather than raising an error. - Negative: Two code paths handle the same `on_discover` event — the Redis publish path (MCP sidecar) and Callback-Collector (orchestrator). Both must be maintained in sync. - Negative: `transaction_id` must be pre-generated and passed into the `discover` request so the channel name is known before the subscription is opened. - Timeout ceiling: `REDIS_RESULT_TIMEOUT` (default 15 s) is the effective latency ceiling for all discovery probes. `MCP_BAP_TIMEOUT` (default 8 s) governs only the HTTP fire-and-forget task and is not the meaningful end-to-end deadline.

4.7.2 pgvector Instead of Qdrant

Status: Implemented as-built deviation from the original spec.

Context: The original design specified Qdrant (self-hosted) or Pinecone (managed) as the vector store, with `text-embedding-3-large` (3072 dimensions) from the OpenAI embedding API. The system requires two vector stores: a BPP catalog semantic cache for IntentParser Stage 3 ANN validation

(bpp_catalog_semantic_cache), and an agent memory store for time-decayed supplier loyalty scoring (agent_memory_vectors).

Decision: Use pgvector exclusively. Two tables serve the vector workload, both at 384 dimensions:

Table	Migration	Index	Model
bpp_catalog_semantic_18_bpp_catalog_semantic_18	bpp_catalog_semantic_18_dim.sql	HNSW cosine	all-MiniLM-L6-v2 (sentence-transformers)
agent_memory_vectors	22_agent_memory_vect	HNSW cosine ef_search=100	BAAI/bge-small-en-v1.5 (fastembed ONNX)

Rationale: - The pilot corpus is below 100,000 records. At this scale, pgvector HNSW is computationally equivalent to Qdrant HNSW in both latency and recall. - Eliminating Qdrant removes one container, one StatefulSet, one backup strategy, and one infra dependency. - PostgreSQL was already the primary datastore; vector operations execute inside the same ACID-transactional connection pool, enabling atomic writes of relational data and embedding vectors. - Local embedding models (all-MiniLM-L6-v2 ~90 MB, BAAI/bge-small-en-v1.5 ~65 MB ONNX) replace the OpenAI embedding API, eliminating per-call cost, network dependency, and data egress.

Consequences: - Positive: One fewer external service; vector writes can be made atomic with FK-dependent relational writes. - Negative: At scale beyond ~500 K vectors, pgvector HNSW recall and throughput will degrade relative to a purpose-built ANN engine. Phase 4 should re-evaluate if the corpus grows. - Known schema bug: database/sql/00_extensions_and_types.sql defines embedding_model_type ENUM with only 'text-embedding-3-large' and 'e5-large-v2'. Migration 22_agent_memory_vector_dim.sql adds 'all-MiniLM-L6-v2' but never adds 'BAAI/bge-small-en-v1.5'. The embedding_model column DEFAULT is also never updated from 'text-embedding-3-large'. Inserts using the BAAI model name will fail with a PostgreSQL invalid enum value error until this is fixed. See Database for the migration sequence.

4.7.3 Local LLMs (qwen3:8b / qwen3:1.7b) Instead of GPT-4o

Status: Implemented as-built deviation from the original spec.

Context: The original design specified GPT-4o as the primary LLM (intent classification and BecknIntent extraction) and GPT-4o-mini for lightweight queries. Using cloud LLM APIs introduces per-call cost, network latency, procurement data egress to a third-party API, and external service availability risk.

Decision: Replace GPT-4o with qwen3:8b (complex queries) and GPT-4o-mini with qwen3:1.7b (simple queries), both served locally via Ollama at `OLLAMA_BASE_URL=http://localhost:11434/v1`. Claude Sonnet 4.6 is retained as a last-resort broadening fallback in IntentParser Stage 3 only — opt-in, requires `ANTHROPIC_API_KEY`, invoked solely when a `not_found` result needs semantic query broadening.

Complexity routing in `IntentParser/orchestrator.py`: queries longer than 120 characters, containing at least two numeric tokens, or containing procurement domain keywords are routed to `COMPLEX_MODEL`; all others to `SIMPLE_MODEL`.

Docker override discrepancy: `docker-compose.yml` sets `COMPLEX_MODEL=qwen3:1.7b`, collapsing complexity routing in the Docker container. The qwen3:8b model is only used when IntentParser runs locally.

Rationale: - Zero per-call LLM cost. Offline capability. Data sovereignty — procurement request content never leaves the developer machine. - Structured JSON extraction via `instructor Mode.JSON` with `max_retries=3` achieves required accuracy for the constrained `BecknIntent` schema using qwen3 models.

Consequences: - Positive: Zero LLM cost, offline operation, data sovereignty. - Negative: qwen3 is less capable than GPT-4o on ambiguous or multi-constraint queries. Stage 3 recovery (Claude broadening fallback) is the safety net but requires `ANTHROPIC_API_KEY`. - Negative: The Docker container silently degrades to qwen3:1.7b for all queries; no metric indicates that complex queries are not reaching the larger model.

4.7.4 Outbox Pattern for ERP PO Push Instead of Kafka

Status: Implemented as-built deviation from the original spec.

Context: The original design specified Apache Kafka as the central event bus for ERP integration: PO creation events published to a Kafka topic, consumed by an ERP sync worker pushing to SAP S/4HANA or Oracle ERP Cloud. No Kafka broker was deployed in Phases 1–3.

Decision: Replace the Kafka publish/consume pattern for PO push with a PostgreSQL transactional outbox. The `erp_sync_records` table (`database/sql/19b_erp_sync_records_outbox.sql`) acts as the outbox. The orchestrator writes a row to `erp_sync_records` within the same database transaction that creates the `purchase_orders` row. An `erp-adapter` background worker polls `WHERE status = 'pending'` with `FOR UPDATE SKIP LOCKED`, pushes to the vendor ERP, and updates the row status. A `kafka_offset` `BIGINT` column and `sqlunk_indexed` `BOOLEAN` are pre-wired as placeholders for Phase 4 Kafka migration.

Rationale: - Atomic guarantee: if the PO write succeeds, the outbox row is guaranteed to exist (same transaction). With Kafka, the PO could be committed to PostgreSQL while the Kafka publish fails. - FOR UPDATE SKIP LOCKED provides safe concurrent worker execution without a separate job queue service. - kafka_offset placeholder makes the Kafka migration non-breaking.

Consequences: - Positive: Atomic write + queue in a single transaction. No Kafka broker dependency in Phases 1–3. Retry with exponential backoff and DLQ replay. - Negative: Outbox worker polls on WORKER_POLL_INTERVAL_SECONDS (default 2 s) instead of being event-driven, introducing up to 2 s of additional PO-push latency. - Negative: PostgreSQL outbox does not support multi-consumer fan-out. Phase 4 Kafka migration is required when multiple independent consumers are needed.

4.7.5 DeDi Registry Bypass (targetType: url)

Status: Feature beyond spec, implemented in Phase 1.

Context: Beckn Protocol's standard topology uses a DeDi (Decentralised Distributed) registry to resolve BPP endpoints. A BAP performing discovery queries the DeDi registry to obtain the target BPP's ONIX caller URL. No DeDi registry is deployed in the local Docker Compose environment; routing Beckn traffic through a live registry would require external network access and registration of sim-bpp.

Decision: Set targetType: url in all four ONIX routing YAML files under config/ (generic-routing-BAPCaller.yaml, generic-routing-BAPReceiver.yaml, generic-routing-BPPCaller.yaml, generic-routing-BPPReceiver.yaml). This routes all Beckn traffic directly to configured target URLs, bypassing the DeDi registry lookup entirely.

Rationale: - Enables a complete Beckn protocol flow within the Docker bridge network without external dependencies. - Eliminates latency from two DeDi registry round-trips per transaction. - Switching to a real Beckn network requires only changing targetType to bap/bpp and setting registryUrl in the YAML files — no code changes.

Consequences: - Positive: Self-contained, fully offline Beckn lifecycle. - Negative: The DeDi trust model (BPP identity verification via registry) is bypassed in development. This is acceptable in a dev environment but must be reverted before any deployment handling real procurement data. - The config/README.md documents the exact 3-step production procedure for switching to DeDi and rotating key material.

Related constraint — ONIX schema validator pin at commit d43ec30d: Later ONIX commits introduced a \$ref resolution bug in SignatureHeader / AckSignatureHeader that causes 100% of Beckn transactions to fail schema validation. The fidedocker/onix-adapter image is pinned to the last known-

good commit. Do not upgrade without running a full end-to-end signing test covering discover → select → init → confirm → status with ED25519-signed payloads.

5 Data Flow Reference

This document describes every major data flow in the Procurement Agent system. Each section is self-contained: a developer new to the project can read any section without having read the others first. Numbered steps map directly to the accompanying Mermaid diagram.

Related documents

Document	Covers
ARCHITECTURE.md	Service topology, logical layers, port map
PROJECT_OVERVIEW.md	Goals, phases, and stakeholder context
GLOSSARY.md	Term definitions used throughout

Key invariants that apply to every flow

- `transaction_id` is a UUID generated once and propagated through every layer. Reuse is prohibited — Beckn pub/sub channels are keyed on it, and a reused ID delivers one transaction's catalog to a stale subscriber.
- All Beckn traffic passes through `onix-bap :8081`. No service posts directly to `sim-bpp :3002` or any external BPP.
- `data-normalizer :8006` is the sole write path into PostgreSQL. All `_persist_*` calls in the orchestrator use `asyncio.create_task` (fire-and-forget, 5 s timeout) and never raise on failure.
- `Contract.status.code` on `/confirm` must be `ACTIVE`. The valid enum is `DRAFT | ACTIVE | CANCELLED | COMPLETE`. ONIX rejects `CONFIRMED` and all other values.

5.1 1. Main Procurement Happy Path

The full end-to-end sequence from a buyer's natural-language query to a confirmed purchase order. This is the `/run` path in the orchestrator (see also the two-phase `/compare + /commit` variant in [Section 3](#)).

5.1.1 Steps

1. The frontend posts { query: "300 meters Cat6 UTP cable Mumbai 5 days" } to orchestrator :8004 POST /run.
2. The orchestrator calls data-normalizer :8006 POST /normalize/request to create a procurement_requests row and obtain a request_id. This is a fire-and-forget call; the pipeline continues immediately.
3. The orchestrator calls intention-parser :8001 POST /parse with the raw query.
 - **Stage 1 — Intent classification:** qwen3:8b (local) or qwen3:1.7b (Docker override) classifies intent as procurement, unknown, or emergency. Non-procurement queries return intent=unknown and the pipeline halts.
 - **Stage 2 — BecknIntent extraction:** qwen3:8b (complex queries) or qwen3:1.7b (simple queries) extracts a BecknIntent with fields item, quantity, location_coordinates ("lat,lon" decimal), delivery_timeline (int hours), and budget_constraints ({max, min}).
 - Stage 3 validation runs only on /parse/full; see [Section 2](#) for the sub-flow.
4. The orchestrator calls data-normalizer :8006 POST /normalize/intent to persist parsed_intents and beckn_intents.
5. The orchestrator calls beckn-bap-client :8002 POST /discover with the BecknIntent. Discovery is asynchronous — the Beckn protocol returns only an ACK; see [Section 2](#) for how the catalog arrives.
6. beckn-bap-client calls catalog-normalizer :8005 POST /normalize to translate the raw on_discover catalog payload into a list of DiscoverOffering objects.
7. The orchestrator calls data-normalizer :8006 POST /normalize/discovery to persist discovery_queries and seller_offerings.
8. The orchestrator optionally calls data-normalizer :8006 POST /normalize/memory/search to fetch up to three similar past transactions from agent_memory_vectors (cosine ANN, threshold ≥ 0.75). Results inject memory_context into reasoning_steps. See [Section 6](#) for the full memory sub-system.
9. The orchestrator calls comparative-scoring :8003 POST /score with the offering list. The service forwards to prediction-api :8004 (RankNet ML, Phase 2 MLOps stack); on failure or when SCORING_FALLBACK_ENABLED=true, falls back to cheapest-wins heuristic.
10. The orchestrator calls data-normalizer :8006 POST /normalize/scoring to persist scored_offers.
11. The orchestrator calls beckn-bap-client :8002 POST /select with the chosen offering. ONIX routes /select to sim-bpp :3002, which returns ACCEPTED.

- 12. The orchestrator returns the full result to the frontend including `transaction_id`, `recommended_item`, and `reasoning_steps`.
- 13. On buyer approval (or automatically in autonomous mode), the orchestrator executes the commit phase — see Section 3.

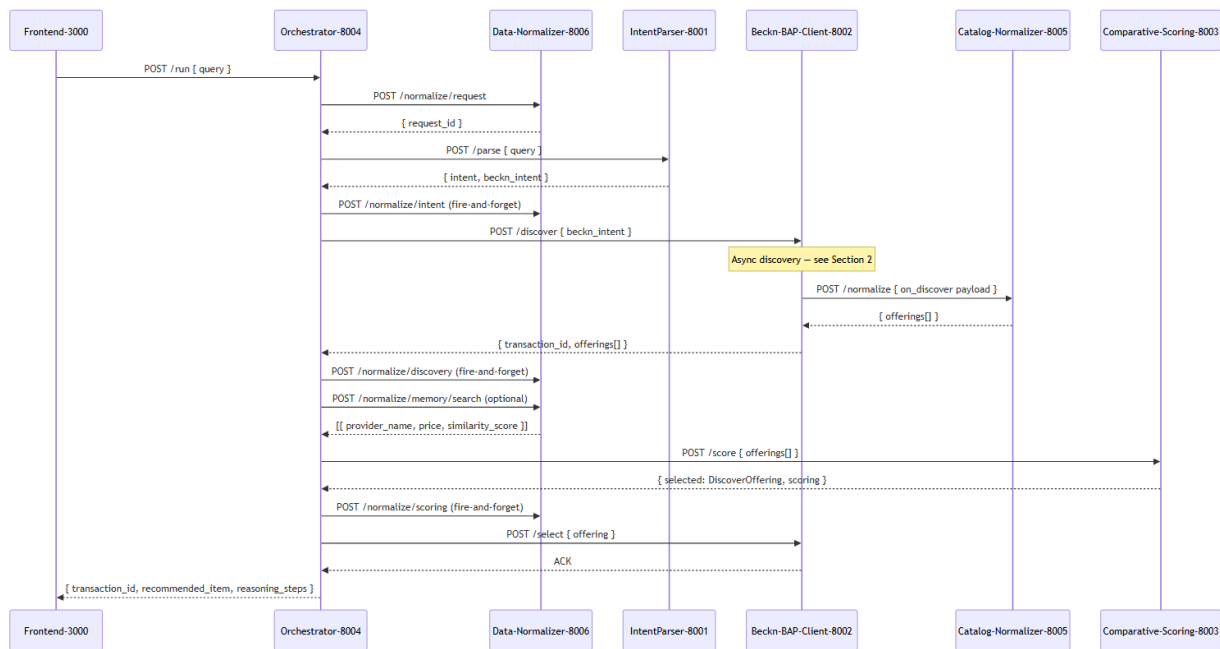


Figure 5.1: DATA_FLOW diagram 1

Docker LLM routing note: Inside the `intention-parser` container, `docker-compose.yml` sets both `COMPLEX_MODEL` and `SIMPLE_MODEL` to `qwen3:1.7b`, silently disabling complexity routing. The `qwen3:8b` model is only used when `IntentParser` runs locally outside Docker.

5.2 2. Beckn Async Discovery Flow (ADR-0001)

Beckn Protocol v2.0.0 makes discovery inherently asynchronous: `POST /discover` to ONIX returns only an `ACK`, and the catalog arrives later via an `on_discover` webhook callback. This section documents why a naïve implementation deadlocks and how ADR-0001 solves it.

5.2.1 The deadlock problem

If the MCP sidecar awaits the `POST /discover` HTTP call while also waiting for the `on_discover` webhook to arrive, the `asyncio` event loop is blocked. The webhook handler — which runs in the same loop

— can never be scheduled, so the catalog never arrives, and the `await` never resolves.

5.2.2 The solution — Redis Pub/Sub on per-transaction channels

The subscriber is opened **before** the discovery request is fired. The Redis channel name pattern is `beckn_results:{transaction_id}`. The five-step protocol:

1. Generate a UUID `transaction_id`.
2. **Subscribe** to `beckn_results:{transaction_id}` before sending anything.
3. Fire `asyncio.create_task(POST /discover { transaction_id, ... })` — **not** `await`. The task is non-blocking; the event loop remains free.
4. The `on_discover` handler in `beckn-bap-client` publishes the catalog payload to REDIS PUBLISH `beckn_results:{transaction_id}`.
5. The sidecar's Redis subscriber receives the message and returns the catalog to IntentParser Stage 3.

5.2.3 Full sequence (MCP sidecar path)

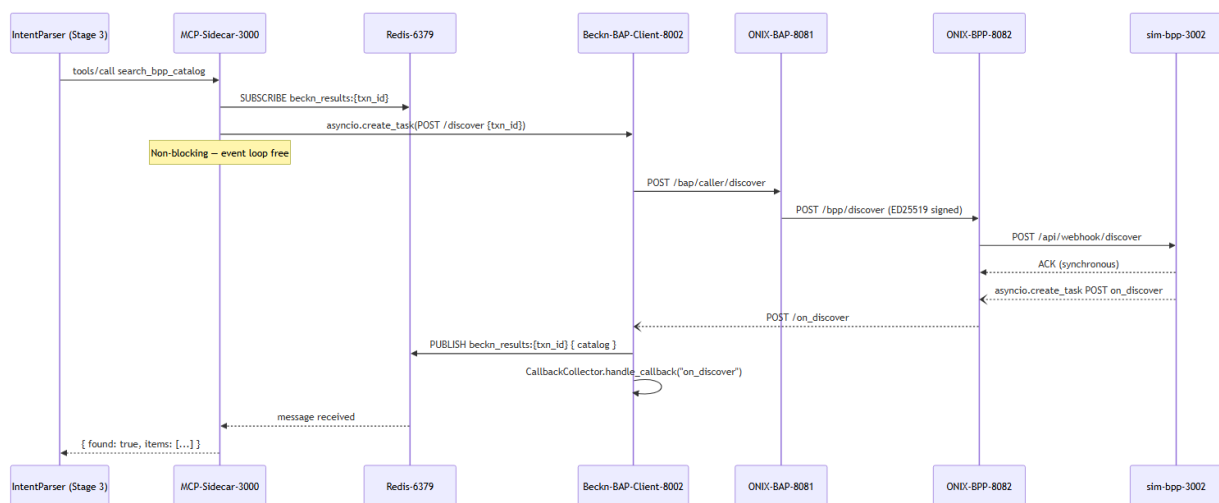


Figure 5.2: DATA_FLOW diagram 2

5.2.4 Dual unblock path

The `/on_discover` handler unblocks two independent waiters:

Waiter	Mechanism	Used by
MCP sidecar (Stage 3)	REDIS PUBLISH beckn_results:{txn_id}	IntentParser pgvector + probe path
Orchestrator (/run)	CallbackCollector.handle_callback	Main happy-path (Section 1)

5.2.5 Timeout behaviour

Env var	Default	Governs
REDIS_RESULT_TIMEOUT	15 s	Primary latency ceiling for the MCP sidecar SUBSCRIBE wait
MCP_BAP_TIMEOUT	8 s	Safety valve on the HTTP fire-and-forget task only

If no Redis message arrives within REDIS_RESULT_TIMEOUT, the sidecar returns { found: false, items: [], probe_latency_ms: 15000 }. If Redis is unavailable entirely, beckn-bap-client logs a warning and falls through to CallbackCollector only; the MCP sidecar still times out and returns found: false.

5.3 3. Compare / Commit Two-Phase Flow

The frontend separates discovery and scoring (Phase 1 — advisory) from the binding confirm action (Phase 2 — commit), enabling a human review step before spending is committed.

5.3.1 Phase 1 — Compare

1. The frontend posts { beckn_intent } (or a raw query) to orchestrator :8004 POST /compare.
2. The orchestrator executes Steps 2–10 from Section 1: persist request, parse intent, discover, score.
3. The orchestrator stores the session under transaction_id with a 30-minute TTL in an in-memory dict(_sessions, _session_times). This state is not durable — a container restart clears all pending sessions.

4. Returns{ transaction_id, offerings[], recommended_item_id, scoring, reasoning_steps } to the frontend.

5.3.2 Phase 2 — Commit

1. The buyer reviews the ranked offerings and may override recommended_item_id with chosen_item_id.
2. The frontend posts { transaction_id, chosen_item_id } to orchestrator :8004 POST /commit.
3. **ERP Budget Gate (synchronous, ≤ 800 ms, fail-closed):** The orchestrator calls erp-adapter :8007 POST /api/v1/budget/check. If ERP_BUDGET_CHECK_REQUIRED=true (default) and the gate returns allowed: false or times out, the commit is rejected with HTTP 402. In Docker dev stacks, docker-compose.yml sets ERP_BUDGET_CHECK_REQUIRED=false to keep the pipeline unblocked. See [Section 5](#) for the full ERP flow.
4. The orchestrator loads the session and resolves the chosen offering.
5. The orchestrator calls becn-bap-client :8002 POST /select → ONIX → sim-bpp (returns ACCEPTED).
6. The orchestrator calls becn-bap-client :8002 POST /init with buyer billing and fulfillment details → ONIX → sim-bpp (returns ACTIVE).
7. The orchestrator calls becn-bap-client :8002 POST /confirm with payment terms → ONIX → sim-bpp (returns ACTIVE, order_id).
8. The orchestrator calls data-normalizer :8006 POST /normalize/order to persist the full FK chain: negotiation_outcomes (strategy=skipped) → approval_decisions (auto_approved) → purchase_orders.
9. Fire-and-forget: asyncio.create_task(_persist_memory(...)) — writes the confirmed transaction to agent_memory_vectors. See [Section 6](#).
10. Fire-and-forget: multiple _persist_audit(...) calls write events to audit_trail_events. The confirm and order_confirmed event types are written here.
11. Returns{ transaction_id, order_id, order_state: "ACTIVE", status: "live" }.

5.3.3 Approval routing thresholds

Condition	Route
amount ≤ requester.approval_threshold	Auto-approved

Condition	Route
<code>amount > requester.threshold AND ≤ approver.threshold</code>	Manager approval
<code>amount > approver.threshold</code>	CFO approval
<code>is_emergency = TRUE</code>	CFO approval + 60-minute deadline

5.4 4. Automated Negotiation Flow

Autonomous price negotiation is a LangGraph state machine inside `negotiation_engine :18004`. It runs only on the `POST /run` path when `decision.requires_negotiation=True`; the `/compare + /commit` two-phase flow does not invoke it.

5.4.1 Routing chain

The orchestrator does **not** call `negotiation_engine` directly. The call chain is:

```
orchestrator :8004
  → demo-gateway :8015  POST /api/demo/negotiate
    → negotiation_engine :18004  POST /negotiate
      → Redis channel becn_on_select_results  (async resume)
```

The `claude_openai_proxy :8012` (a host-local process, not Dockerized) backs the `SupplierAgent` inside `demo-gateway`. Containers reach it via `host.docker.internal:8012`. If the proxy is not running, negotiation fails silently.

5.4.2 Steps

- Orchestrator calls `DEMO_GATEWAY_URL /api/demo/negotiate` (env var default `http://localhost:8015`) with `{ ranked_offers, policy: { max_discount_pct: 0.20 }, max_rounds: 3 }`.
- `demo-gateway` forwards to `negotiation_engine :18004 POST /negotiate`. The engine returns `202 { thread_id, paused_at: "wait_for_async_callback" }` immediately.

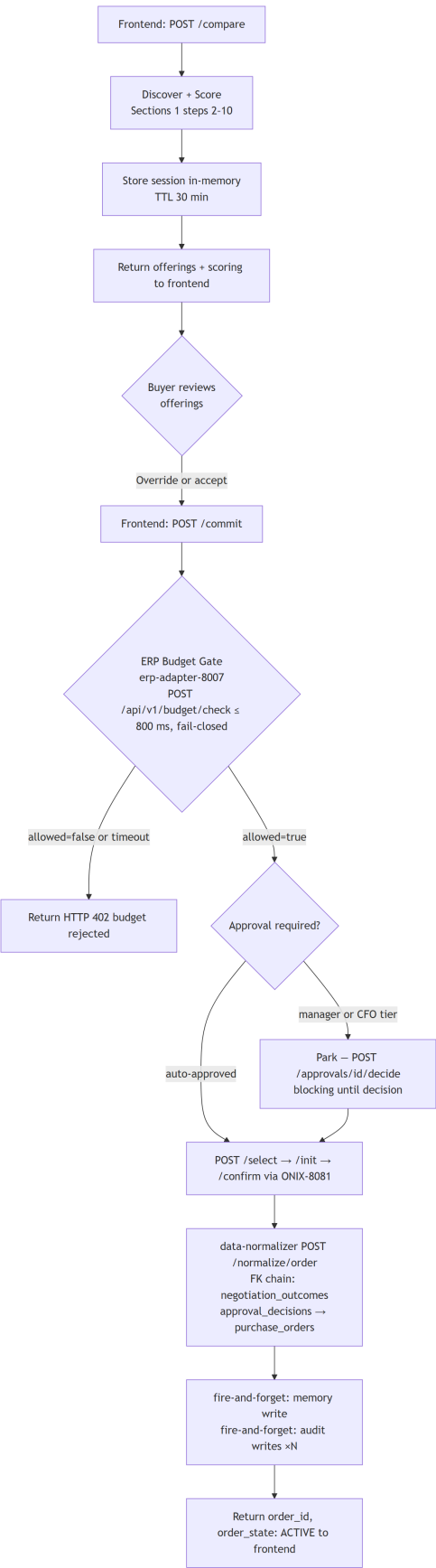


Figure 5.3: DATA_FLOW diagram 3

3. The LangGraph state machine runs inside `negotiation_engine`:

- **analyze_target**: determines category (`commodity` / `it_equipment` / `specialized` / `medical` / `unknown`) and base discount profile.
- **compute_counter_offer**: calculates a counter-offer price. Three independent guardrail layers apply:
 - L1 Pydantic field constraint: `discount_pct` $\in$ `[0.0, 0.20]`.
 - L2 `validate_counter_offer()`: G1 (20% absolute cap), G2 (category cap), G3 (supplier cap), G5 (lead time), G6 (quantity).
 - L3 ONIX schema validation.
- **policy_guardrail_check**: routes to `human_escalation` (HITL interrupt) on any violation; otherwise parks at `wait_for_async_callback`.

4. The orchestrator polls `GET {DEMO_GATEWAY_URL}/api/demo/negotiate/{thread_id}` every 1 s (up to 120 polls).

5. When `state.paused_at == "wait_for_async_callback"` and `state.awaiting_supplier == True`, the orchestrator calls `POST {DEMO_GATEWAY_URL}/api/demo/negotiate/{thread_id}/supplier` and responds.

6. `demo-gateway` invokes the `SupplierAgent` (Claude via `claude_openai_proxy :8012`) to generate a supplier counter-offer response.

7. `demo-gateway` publishes the supplier response to Redis channel `beckn_on_select_results`.

8. `OnSelectListener` in `negotiation_engine` receives the Redis message and calls `graph.ainvoke(Command(resume=payload))`, resuming the parked LangGraph graph.

9. **evaluate_response**: if `gap ≤ threshold` → `finalize` (accepted); if `gap > HITL threshold` or `max rounds reached` → `human_escalation`; otherwise → loop back to `analyze_target`.

10. On `finalize`, the orchestrator receives the negotiated price and proceeds to the commit step in [Section 3](#).

5.4.3 LangGraph node graph

5.4.4 Category discount profiles

Category	Base discount	Mode
<code>commodity</code>	10%	Negotiate
<code>specialized</code>	5%	Negotiate
<code>unknown</code>	5%	Negotiate

Category	Base discount	Mode
it_equipment	0%	Advisory only
medical	0%	Advisory only

Audit gap: No `_persist_audit` call exists inside `_run_autonomous_negotiation`. The `negotiate` audit event type is never written by the orchestrator. Negotiation outcomes are recorded only in the `negotiation_outcomes` table via the order persistence chain.

5.5 5. ERP Integration Flow

The ERP integration has two sub-flows that run at different points in the commit path: a synchronous budget gate that blocks the commit, and an asynchronous PO push via an outbox pattern that does not.

5.5.1 5a. Budget Gate (synchronous, ≤ 800 ms)

1. The orchestrator (commit path, Step 3 in [Section 3](#)) calls `erp-adapter :8007 POST /api/v1/budget/check` with `{ amount_total, cost_center, currency }`.
2. `erp-adapter` routes to the configured ERP vendor (`mock` / `SAP` / `Oracle`) via the `ERPAdapter Protocol` (selected by `ERP_VENDORS` env var).
3. In production SAP mode: `OAuth2client_credentials token fetch` → `OData API_PURCHASEORDER_PROCESSING` budget check endpoint.
4. Per-vendor `pybreaker` circuit breaker trips to `OPEN` after 5 consecutive failures and auto-probes after 60 s.
5. Hard wall: `BUDGET_CHECK_TOTAL_TIMEOUT_MS=800`. If ERP is unreachable and `ERP_BUDGET_CHECK_REQUIRED=true`, the gate returns `allowed: false` (fail-closed). Set `ERP_BUDGET_CHECK_REQUIRED=false` for a fail-open dev environment.
6. Returns `{ allowed: bool, available_balance, hold_id }`.

5.5.2 5b. PO Push (asynchronous outbox)

1. The orchestrator calls `erp-adapter :8007 POST /api/v1/po/sync` with `{ transaction_id, po_details }`. Returns 202 immediately.
2. `erp-adapter` inserts a row into `erp_sync_records` with `status=pending`.

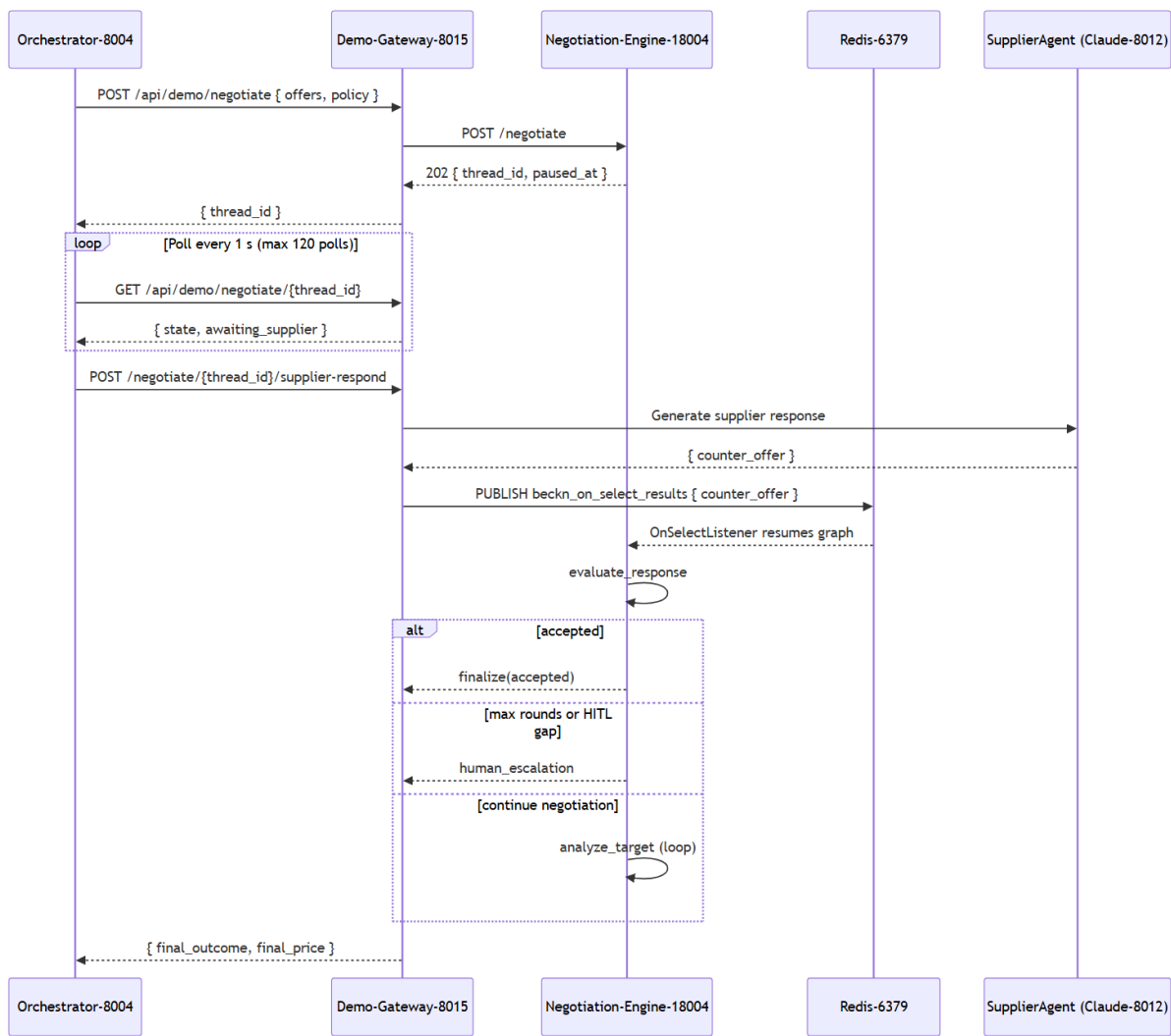


Figure 5.4: DATA_FLOW diagram 4

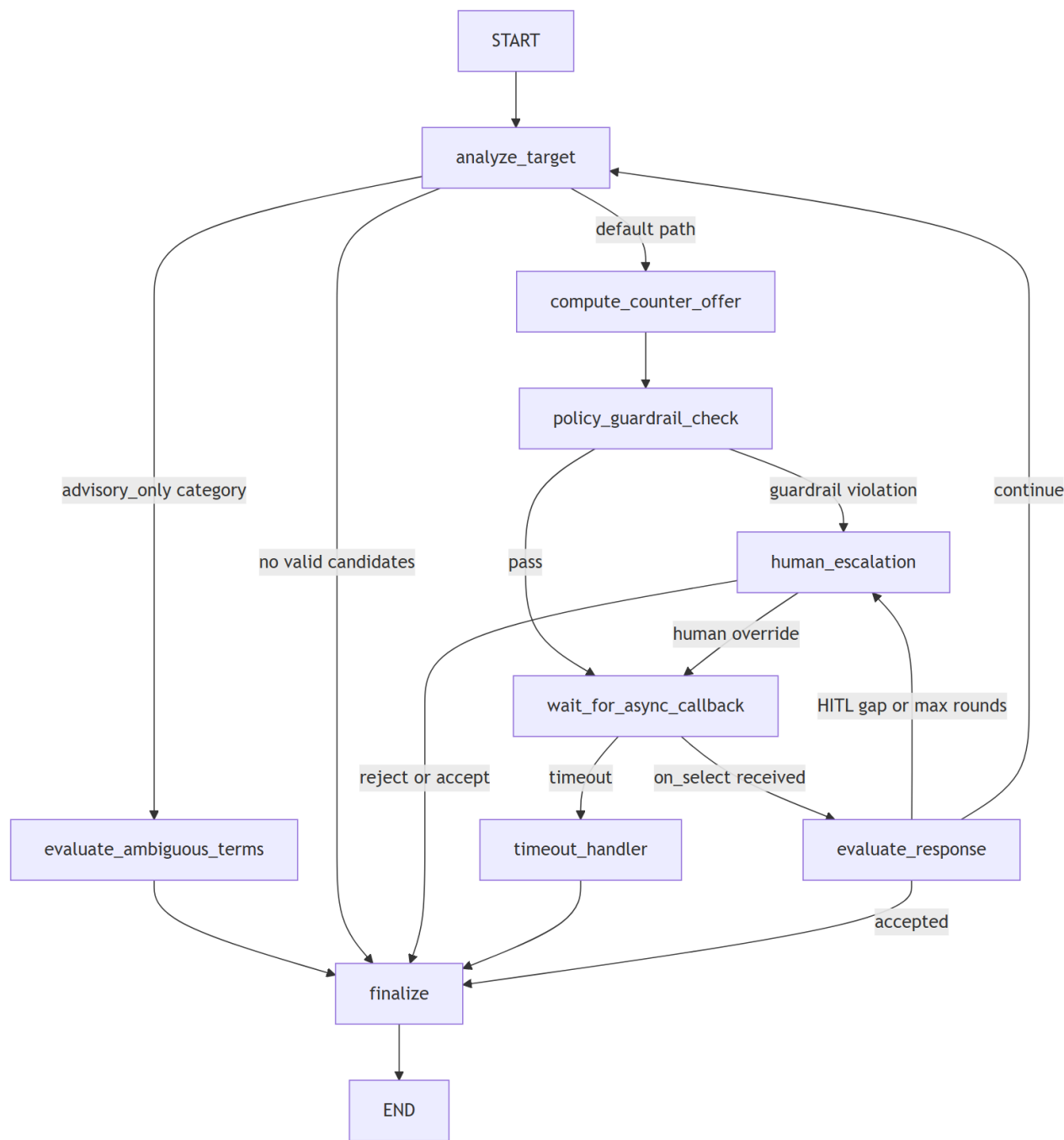


Figure 5.5: DATA_FLOW diagram 5

3. Background worker polls `erp_sync_records` using `FOR UPDATE SKIP LOCKED` (concurrency-safe), picks up pending rows.
4. Worker calls the ERP vendor REST/OData API to create the PO.
5. On success: updates `erp_sync_records.status=sent`, stores `erp_reference_id`.
6. On failure: increments `retry_count`, applies exponential backoff (`WORKER_BACKOFF_CSV=5,30,120,600` seconds). After max retries: `status → dlq` (dead-letter queue). If the database schema is absent, falls back to `InMemoryOutboxRepo` — POs are not durable in that mode.
7. Vendor webhook arrives at `erp-adapter POST /api/v1/webhooks/{vendor}/po-status`.
8. HMAC-SHA256 dual-secret verification: tries `{VENDOR}_WEBHOOK_HMAC_SECRET` first, then `{VENDOR}_WEBHOOK_HMAC_SECRET_NEXT` for zero-downtime rotation.
9. On a valid webhook: updates `purchase_orders.status`, publishes to Kafka topic `po.status.changed` (primary) or Redis key `po.status_changed:{txn_id}` (fallback when Kafka is unavailable).
10. `notification-dispatcher :8010` consumes the Kafka event — see [Section 7](#).

5.6 6. Agent Memory Learning Flywheel

The agent memory system enables time-decayed supplier loyalty adjustments in scoring. The pattern is **write on confirm, read before score**: past transactions are embedded and stored at purchase time, then retrieved by ANN search before comparative scoring to influence the recommendation.

5.6.1 Write path (triggered after /confirm)

1. After a successful `POST /confirm`, the orchestrator calls `asyncio.create_task(_persist_memory(...))` — fire-and-forget; never awaited; never raises.
2. `_persist_memory` posts to `data-normalizer :8006 POST /normalize/memory/write` with `{ item, provider_name, price, currency, delivery_hours, request_id }`.
3. `data-normalizer` embeds the text summary `"{item} ordered from {provider} at {price} {currency} delivery in {hours}h"` using `BAAI/bge-small-en-v1.5` (fastembed ONNX, 384 dimensions). The first call triggers a ~65 MB model download from HuggingFace (10–30 s cold start).
4. Inserts vector(384) into `agent_memory_vectors` with `entity_type='transaction'`, `indexed_at=NOW()`, `source_request_id`.

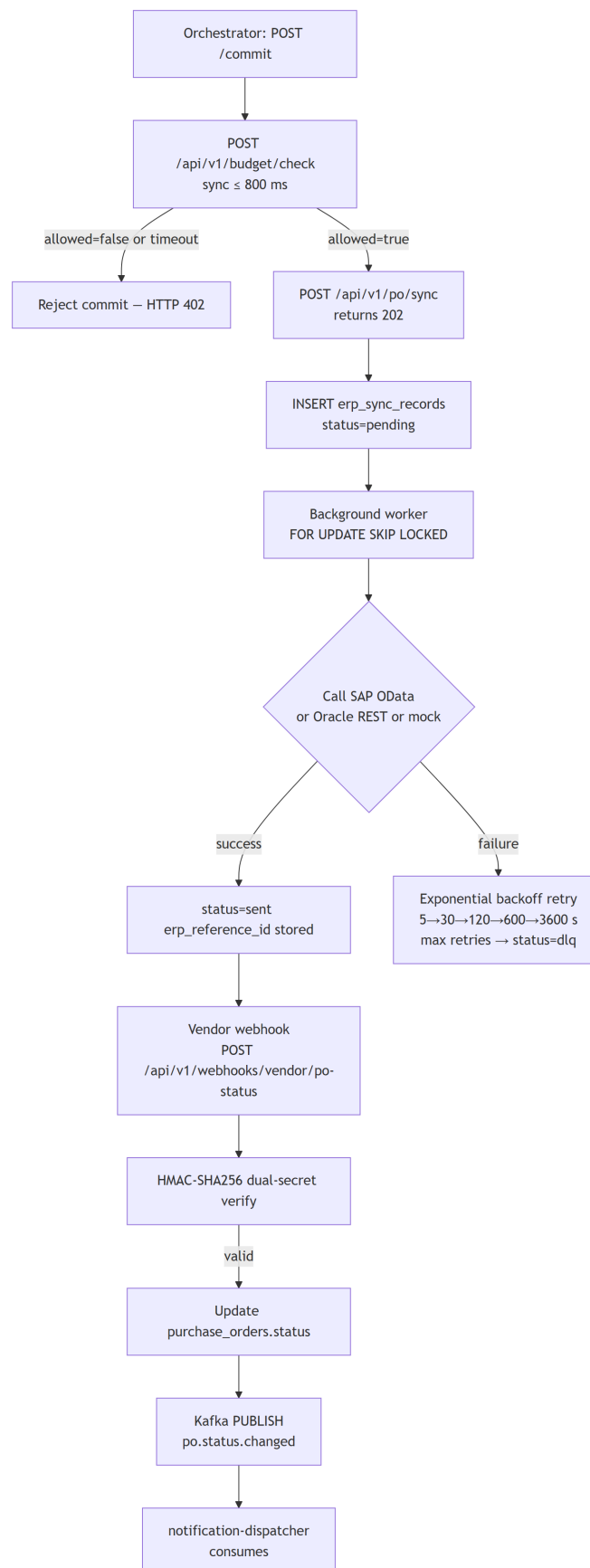


Figure 5.6: DATA_FLOW diagram 6

- Returns { stored: true }. Embedding failures are silently swallowed — the endpoint never errors due to embedding failure.

Database migration note: The `embedding_model_type` ENUM in `database/sql/00_extensions_and_ty` does not include `BAAI/bge-small-en-v1.5`. Migration `22_agent_memory_vector_dim.sql` adds it. Without this migration applied, the INSERT will fail. The column default remains `text-embedding-3-large` even after migration 22.

5.6.2 Read path (triggered before scoring)

- After discovery, before scoring (Step 8 in [Section 1](#)), the orchestrator calls `_fetch_memory_context(raw_query_limit=3)` with a 5 s timeout.
- Posts to `data-normalizer :8006 POST /normalize/memory/search` with { `item_text`, `limit: 3` }.
- `data-normalizer` performs a `pgvector` HNSW cosine ANN search against `agent_memory_vectors`. Items with similarity < 0.75 are filtered out.
- Returns up to three past transactions with `similarity_score`, `provider_name`, and `price`.
- The orchestrator appends `reasoning_steps += [{ node: "memory_context", role: "observe", content: [...hits] }]`.
- If the top recommendation changes after memory adjustments, an additional { `node: "memory_adjustment", role: "reason"` } step is appended.

5.6.3 Scoring delta formula

For each provider found in the memory results, a loyalty bonus is added to the composite score:

$$\text{memory_delta}(\text{provider}) = \sum \left[0.03 \times \exp(-\text{days_old} \times \ln(2) / 90) \right] \\ \text{over past POs for this provider (max 5 past orders)} \\ \text{capped at } \pm 0.10$$

New providers receive `delta = 0`. The half-life is 90 days. The volume cap of five past orders prevents runaway loyalty bias.

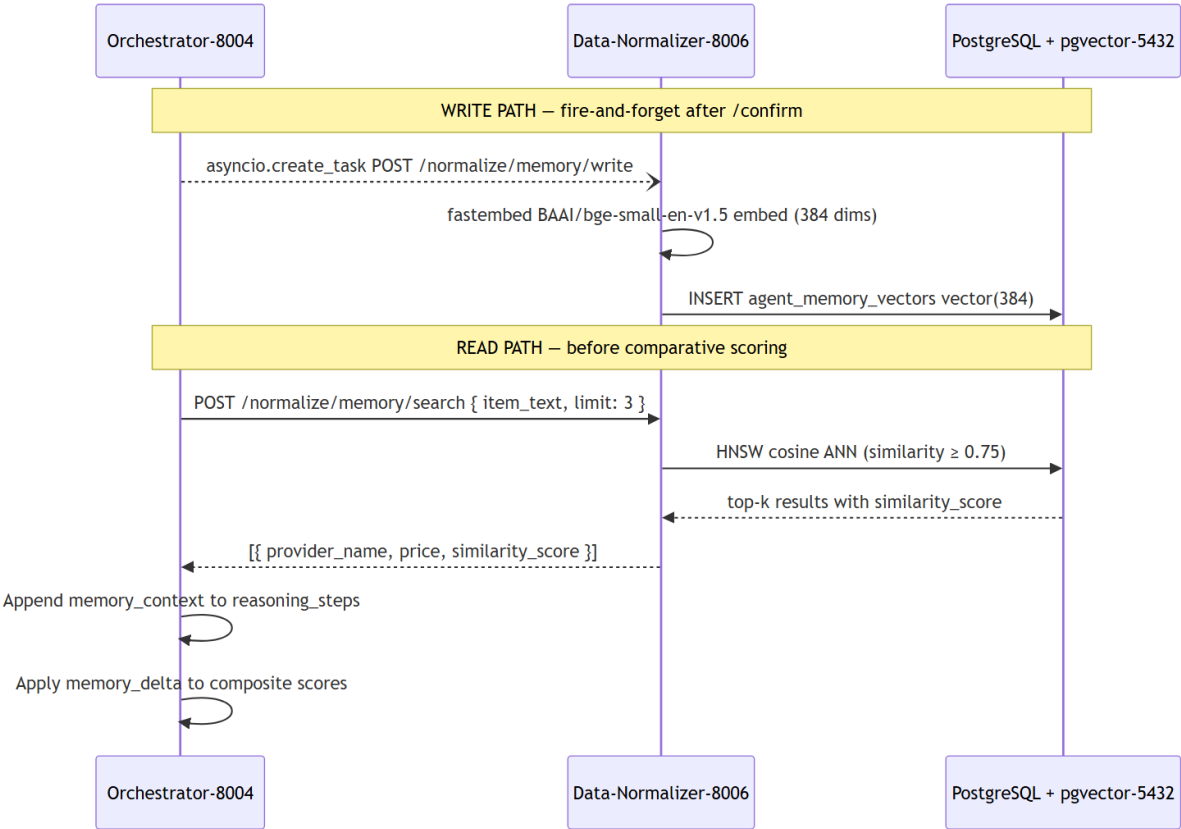


Figure 5.7: DATA_FLOW diagram 7

5.7 7. Notification Dispatch Flow

After a purchase order is confirmed, status changes flow through sim-bpp auto-advance simulation (or a real seller webhook), Kafka, and the notification dispatcher. Each channel operates independently; one channel's failure never blocks the others.

5.7.1 Order lifecycle (sim-bpp auto-advance)

When `SIM_BPP_AUTO_ADVANCE=true` (the default in `docker-compose.yml`), `sim-bpp :3002` automatically advances confirmed orders through a lifecycle every `SIM_BPP_ADVANCE_INTERVAL_SECS` seconds (default 5 s):

ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

For each transition, `sim-bpp`:

1. Patches `data-normalizer :8006 PATCH /normalize/po_status` with the new status.
2. Publishes to Kafka topic `po.status.changed`.

5.7.2 Notification routing rules

Order status	Slack	Teams	Email
confirmed	yes	yes	yes
shipped	yes	yes	no
delivered	yes	yes	yes
cancelled	yes	yes	no
anything else	no	no	no

5.7.3 Fan-out steps

1. `notification-dispatcher :8010 AIOKafkaConsumer` receives an event from `po.status.changed`. Malformed messages are logged as `WARNING` and skipped; there is no dead-letter queue.
2. Normalises the status field: tries `po_status` key first; `state` key as fallback.
3. Determines the channel set from the routing table above.

- 4. Calls `asyncio.gather([slack_task, teams_task, email_task], return_exceptions=True)` — channels are fully independent.
- 5. **Slack:** Block Kit message with header (emoji by status) + section (order_id, transaction_id, source, observed_at), posted to `SLACK_WEBHOOK_URL`.
- 6. **Teams:** Adaptive Card v1.4, posted to `TEAMS_WEBHOOK_URL`. Colour: good for confirmed/delivered, accent for shipped, attention for cancelled.
- 7. **Email:** Jinja2 HTML template (`email_confirmed.html`, `email_delivered.html`, or `email_generic.html`) sent via SMTP STARTTLS. Recipient is resolved from the database: `purchase_orders.beckn_confirm_ref = order_id` → FK chain → `users.email`. If the database is unavailable or no match is found, the email channel is silently skipped without affecting Slack or Teams.

5.8 Known Data Flow Anomalies

The following discrepancies between documentation and as-built behaviour are confirmed by code inspection and affect operational understanding of the flows above.

Anomaly	Affected flow	Impact
<code>COMPLEX_MODEL=qwen3:1.7b</code> in <code>docker-compose.yml</code> overrides the local default of <code>qwen3:8b</code>	Section 1 (IntentParser Stage 2)	Complexity routing is silently disabled in Docker; all queries use the smaller model regardless of classification
<code>negotiate</code> audit event type is never written by the orchestrator	Section 4 (Negotiation)	Negotiation decisions are absent from the audit trail; SOX reconstruction is incomplete for negotiated orders
<code>catalog-normalizer</code> LLM fallback uses Ollama at <code>OLLAMA_URL</code> , not OpenAI	Section 1 (catalog normalization)	<code>OPENAI_API_KEY</code> is irrelevant; operators who set it expecting LLM fallback via OpenAI will see no effect
<code>claude_openai_proxy :8012</code> is a host-local process absent from <code>docker-compose.yml</code>	Section 4 (Negotiation)	Demo and negotiation flows fail silently if the proxy is not started manually
<code>DEMO_GATEWAY_URL</code> defaults to <code>http://localhost:8015</code> in <code>workflow.py</code> ; the service README states port 8005	Section 4 (Negotiation)	Autonomous negotiation fails unless <code>DEMO_GATEWAY_URL</code> is explicitly set to the correct port

Anomaly	Affected flow	Impact
embedding_model_type ENUM missing all-MiniLM-L6-v2 and BAAI/bge-small-en-v1.5 until migration 22_agent_memory_vector_dim. is applied	Section 6 (Memory)	INSERT into agent_memory_vectors.embedding_model fails on a fresh database that has not run migration 22
discovery_engine :8006 is fully implemented but absent from docker-compose.yml and has no callers	Not active in any flow	Multi-network discovery fan-out is not operational; port 8006 conflicts with data-normalizer

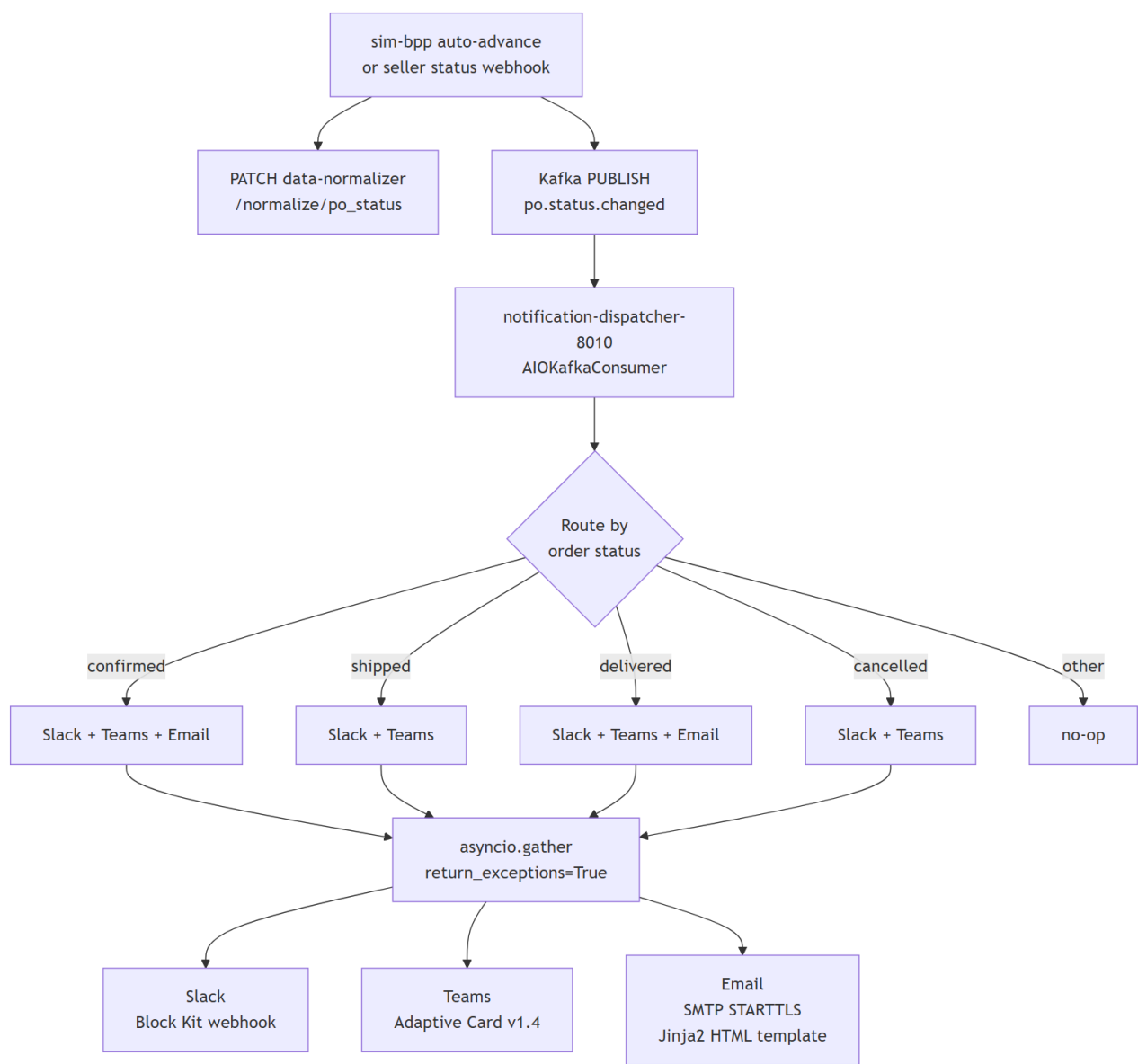


Figure 5.8: DATA_FLOW diagram 8

6 Installation Guide

This guide walks a developer through setting up the Procurement Agent from scratch on a local machine. It covers all prerequisites, the Python and LLM environment, the PostgreSQL database, the Docker stack, and the two local services that run outside Docker. Follow the sections in order — later steps depend on earlier ones.

For a map of how all services relate to each other, see [Architecture](#). For an explanation of every environment variable across all services, see [Environment Reference](#).

6.1 1. Prerequisites

Install and verify each tool before proceeding.

Requirement	Minimum version	Purpose	Notes
Docker Engine + Compose v2	Latest stable	Runs the 18-service becn_network stack	<code>docker compose</code> version must show v2.x — the legacy <code>docker-compose</code> binary is not supported
Conda (Miniconda or Anaconda)	Any current release	Manages the Python environment for the two local services	Miniconda installer
Ollama	Latest	Serves qwen3:8b and qwen3:1.7b locally at :11434	Must be running as a background daemon; <code>ollama list</code> should return without error
Git	2.x	Repository checkout	—

Requirement	Minimum version	Purpose	Notes
Python	3.11+	Installed through conda — do not use a system Python	Managed via the <code>infosys_project</code> conda environment created in §3
Node.js	18+	Frontend only (<code>npm run dev</code> in <code>frontend/</code>)	Optional if you are not running the Next.js frontend

6.2 2. Clone and Prepare the Repository

```
git clone <repository-url> Procurement-Agent
cd Procurement-Agent
```

The working directory after cloning has this top-level layout:

```
Procurement-Agent/
├── IntentParser/           # Stage 1+2+3 NL → BecknIntent pipeline (local, :8001)
├── services/               # Dockerised microservices + mcp-
sidecar (local, :3000)
├── database/              # 24 SQL migrations + setup_database.py
├── config/                # ONIX routing YAMLS for onix-bap / onix-bpp
├── frontend/              # Next.js 13 buyer-facing app
├── shared/                # Cross-service Pydantic models (BecknIntent, etc.)
├── .env.example           # Root environment template
└── docker-compose.yml     # 18-service stack
```

All commands in this guide run from the repository root unless a `cd` is shown.

6.3 3. Python Environment Setup

Both local services (IntentParser and mcp-sidecar) share the `infosys_project` conda environment.

Create and activate the environment:

```
conda create -n infosys_project python=3.11 -y
conda activate infosys_project
```

Install IntentParser dependencies:

```
pip install -r IntentParser/requirements.txt
```

Install mcp-sidecar dependencies:

```
pip install -r services/mcp-sidecar/requirements.txt
```

You must activate this environment in every terminal where you start IntentParser or mcp-sidecar. The Docker containers manage their own dependencies independently and do not use the conda environment.

6.4 4. Pull LLM Models with Ollama

IntentParser uses two models for the intent classification and extraction pipeline: `qwen3:8b` for complex queries and `qwen3:1.7b` for simpler ones. Both must be present before starting the service.

```
ollama pull qwen3:8b
ollama pull qwen3:1.7b
```

These are large downloads (`qwen3:8b` is approximately 5 GB). Run the pulls before continuing.

Verify that both models are available:

```
ollama list
```

Expected output includes both model names in the NAME column:

NAME	ID	SIZE	MODIFIED
qwen3:8b	...	5.2 GB	...
qwen3:1.7b	...	1.1 GB	...

Docker note: The `intention-parser` container reaches Ollama at `http://host.docker.internal:11434`. The full three-stage pipeline (Stage 3 `pgvector` + MCP sidecar validation) only runs when `IntentParser` is started locally, not in its Docker wrapper. Stage 3 requires `qwen3:8b` for complex-query routing — the Docker override collapses both tiers to `qwen3:1.7b`.

6.5 5. Environment Variables

Copy the root template:

```
cp .env.example .env
```

Edit `.env` and set these values for a working development setup:

Variable	Example value	Required for	Notes
DB_HOST	localhost	Database setup and local services	Use localhost when connecting from the host; Docker containers use <code>host.docker.internal</code> (set automatically by <code>docker-compose</code>)
DB_PORT	5432	Database	Default PostgreSQL port
DB_NAME	procurement_agent	Database	Main procurement database name
DB_USER	postgres	Database	PostgreSQL superuser
DB_PASSWORD	postgres123	Database	Change before any production deployment
BAP_API_KEY	dev-key	mcp-sidecar	Any non-empty string works in development; the sidecar refuses to start without it

Variable	Example value	Required for	Notes
REDIS_URL	redis://localhost:6379	mcp-sidecar	Must also be exported as a real shell variable (see §8)
REDIS_RESULT_TIMEOUT	15	mcp-sidecar	Seconds to wait for a Beckn discovery callback; must also be a real shell variable

The ANTHROPIC_API_KEY is optional. When set, it enables Claude Sonnet 4.6 as a last-resort Stage 3 query-broadening fallback in IntentParser. Without it, Stage 3 uses regex-only broadening.

For the complete list of environment variables across all 18 services — including production-required secrets, ERP integration variables, and Keycloak OIDC configuration — see [Environment Reference](#).

6.6 6. Database Initialization

The main procurement_agent database runs outside the Docker stack (on the host, or in a standalone named container). Docker containers reach it via host.docker.internal:5432.

6.6.1 6.1 Start a PostgreSQL container with pgvector

```
docker run -d \
  --name procurement-postgres \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres123 \
  -e POSTGRES_DB=procurement_agent \
  -p 5432:5432 \
  pgvector/pgvector:pg16
```

This container must be running before setup_database.py is executed and must remain running whenever any service queries the database.

6.6.2 6.2 Apply all migrations

```
cd database
export $(grep -v '^#' .env | xargs)
python setup_database.py --create-db
```

The `--create-db` flag creates `procurement_agent` if it does not exist (requires the `CREATEDB` privilege held by the `postgres` superuser). Without the flag, the script assumes the database already exists.

`setup_database.py` reads every `.sql` file in `database/sql/` in lexicographic order and executes them against the database. The 24 files are numbered with a `NN_` prefix that matches the foreign-key dependency chain:

Phase	Files	What is created
Extensions and types	<code>00_extensions_and_types.sql</code>	<code>uuid-ossdp</code> , <code>pgcrypto</code> , vector extensions; 20 ENUM types
Core tables	<code>01_</code> through <code>16_</code>	All 16 primary tables: <code>users</code> , <code>requests</code> , <code>intents</code> , <code>discovery</code> , <code>scoring</code> , <code>negotiation</code> , <code>approval</code> , <code>orders</code> , <code>ERP</code> , <code>audit</code> , and vector stores
Indexes	<code>17_indexes.sql</code>	22 named B-tree and partial indexes
Cache and outbox	<code>18_</code> , <code>19_</code> , <code>19b_</code>	BPP semantic cache table (HNSW index); ERP outbox columns
Negotiation schema	<code>20_negotiation_schema.sql</code>	Negotiation audit tables (also used by the <code>postgres</code> Docker service)
Fidelity and corrections	<code>21_</code> , <code>22_</code> , <code>22b_</code>	Order detail columns; agent memory vector dimension correction (3072 → 384); approval-resumption columns

All migrations are idempotent (`IF NOT EXISTS` / `IF EXISTS` guards). Re-running `setup_database.py` without `--create-db` is safe and applies any new migrations:

```
python setup_database.py --continue-on-exists
```

6.6.3 6.3 Verify the schema

```
pytest test_database.py -v --tb=short -q
```

The test suite runs 67 assertions across 8 test classes covering connection, extensions, ENUM types, tables, indexes, workflow rows, end-to-end queries, and constraints. All 67 should pass on a clean setup.

6.7 7. Start the Docker Stack

Return to the repository root and bring up all 18 services:

```
docker compose up -d
```

Check that all containers reach a healthy or running state:

```
docker compose ps
```

Expected healthy state — every service should show running or healthy, not restarting or exited:

Service	Host port	Expected status
redis	6379	running
onix-bap	8081	healthy (waits for redis)
onix-bpp	8082	healthy (waits for redis)
sim-bpp	3002	running (waits for kafka)
kafka	9092	healthy
beckn-bap-client	8002	running
catalog-normalizer	8005	running
comparative-scoring	8003	running
data-normalizer	8006	running
analytics	8009	running (waits for postgres healthy)

Service	Host port	Expected status
orchestrator	8000,8004	running
erp-adapter	8007	running
erp-mock	8008	running
negotiation-engine	18004	healthy
demo-gateway	8015	running
postgres	55432	healthy
intention-parser	8001	running
notification-dispatcher	(none)	running

The `postgres` service on host port 55432 holds only the `negotiation` database used by the negotiation engine. It is distinct from the `procurement-postgres` container started in §6 which holds `procurement_agent`.

If only Beckn discovery infrastructure is needed (for `IntentParser` development without the full pipeline), a lighter subset suffices:

```
docker compose up -d redis onix-bap onix-bpp sim-bpp
```

Tail logs for a specific service:

```
docker compose logs -f becn-bap-client
```

6.8 8. Start Local Services

Two services run outside Docker and must be started in separate terminals with the `conda` environment active. They implement parts of the pipeline that require direct access to the host's Ollama daemon and local Redis.

6.8.1 Terminal 1 — `IntentParser`

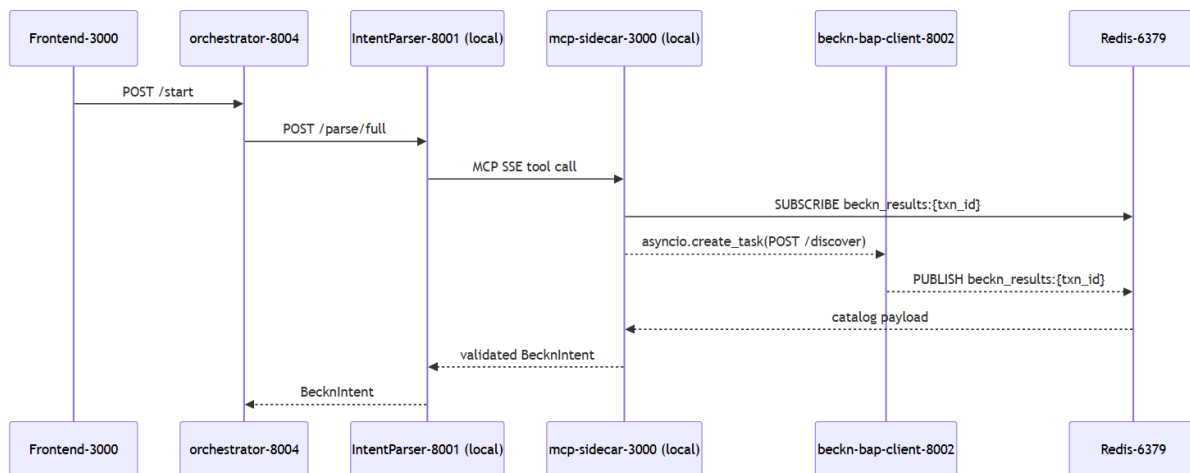


Figure 6.1: INSTALLATION diagram 1

```
conda activate infosys_project
cd IntentParser
uvicorn api:app --port 8001 --reload
```

IntentParser runs the full three-stage pipeline: LLM intent classification (Stage 1), BecknIntent extraction (Stage 2), and pgvector ANN + MCP sidecar validation (Stage 3). The `--re load` flag restarts the server automatically when source files change. Ollama must be running before this starts.

6.8.2 Terminal 2 — mcp-sidecar

`REDIS_URL` and `REDIS_RESULT_TIMEOUT` must be exported as real shell variables, not only present in `.env`. The sidecar reads them with `os.getenv()` in `bap_client.py`, bypassing the Pydantic Settings loader.

```
conda activate infosys_project
cd services/mcp-sidecar
export BAP_API_KEY="dev-key"
export REDIS_URL="redis://localhost:6379"
export REDIS_RESULT_TIMEOUT="15"
uvicorn server:app --port 3000
```

The sidecar will refuse to start if `BAP_API_KEY` is empty or unset. Any non-empty string is valid in development.

6.9 9. Smoke Test

With all services running, send a natural-language procurement query to IntentParser:

```
curl -X POST http://localhost:8001/parse/full \
-H "Content-Type: application/json" \
-d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}'
```

Healthy response shape:

```
{
  "status": "success",
  "intent": {
    "item": "Cat6 UTP cable",
    "quantity": 300,
    "unit": "meters",
    "location_coordinates": "19.0760,72.8777",
    "delivery_timeline": 120,
    "budget_constraints": {
      "max": null,
      "min": null
    },
    "currency": "INR",
    "compliance_requirements": []
  },
  "validation_result": {
    "status": "VALIDATED",
    "confidence": 0.91
  }
}
```

Key fields to check:

Field	Expected	If wrong
status	"success"	A "error" with a stack trace points to an Ollama connectivity or model-not-found error

Field	Expected	If wrong
<code>intent.delivery_timeline</code>	An integer (hours, not a string like "5 days")	IntentParser is using an old BecknIntent model; check <code>shared/models.py</code>
<code>intent.location_coordinates</code>	"lat,lon" decimal pair	Stage 2 extraction defaulted; Ollama may be under load
<code>validation_result.status</code>	"VALIDATED", "AMBIGUOUS", or "CACHE_MISS"	"CACHE_MISS" on the first run is normal — the semantic cache is empty

A 503 or connection-refused response from :8001 means IntentParser is not running or crashed on startup.

6.10 10. Troubleshooting Installation

Symptom	Likely cause	Fix
<code>docker compose ps</code> shows a service in restarting	Missing env var or upstream dependency not yet healthy	Run <code>docker compose logs <service></code> to see the startup error; check that all <code>depends_on</code> targets are healthy
<code>onix-bap</code> or <code>onix-bpp</code> crashes immediately	Redis not ready at startup	Run <code>docker compose up -d redis</code> first, wait for it to show running, then <code>docker compose up -d onix-bap onix-bpp</code>
<code>setup_database.py</code> fails with <code>FATAL: database "procurement_agent" does not exist</code>	<code>--create-db</code> flag omitted on first run	Re-run with <code>python setup_database.py --create-db</code>
<code>setup_database.py</code> fails with <code>could not connect to server</code>	<code>procurement-postgres</code> container not running	Run the <code>docker run</code> command from §6.1, then retry
IntentParser returns <code>{"error": "model not found"}</code>	Ollama model not pulled	Run <code>ollama pull qwen3:8b</code> && <code>ollama pull qwen3:1.7b</code>

Symptom	Likely cause	Fix
mcp-sidecar exits immediately with BAP_API_KEY is required	Environment variable not set	<code>export BAP_API_KEY="dev-key"</code> in the same terminal before <code>uvicorn</code>
Smoke test returns <code>{"found": false}</code> for validation	REDIS_URL or REDIS_RESULT_TIMEOUT not exported as shell variables	Export both with <code>export</code> (not just in <code>.env</code>) before starting the sidecar
<code>pytest test_database.py</code> fails on extension tests	pgvector extension not available in the PostgreSQL image	Confirm you used <code>pgvector/pgvector:pg16</code> (not <code>postgres:16</code>) in the <code>docker run</code> command
analytics container shows unhealthy	postgres service (negotiation DB) still initialising	Wait 30–60 seconds for the postgres health check to pass; analytics depends on it
Docker containers cannot reach Ollama	Ollama bound to loopback only	On Linux, Ollama defaults to <code>127.0.0.1</code> ; set <code>OLLAMA_HOST=0.0.0.0</code> in Ollama's service config so containers can reach it via <code>host.docker.internal</code>

For issues beyond initial installation — Beckn protocol errors, ONIX routing failures, database schema conflicts — see [Troubleshooting](#).

7 Configuration Guide

This guide covers every configuration surface in the Procurement Agent: environment variables, `.env` files, ONIX routing YAMLS, LLM model selection, ERP vendor wiring, and Beckn network parameters. For the architecture that these variables serve see [Architecture](#); for the runtime topology that consumes them see [System Design](#).

7.1 1. Configuration Philosophy

Configuration in the Procurement Agent follows four rules:

1. **Env-var driven throughout.** Every tuneable value is read from the environment. No service embeds connection strings or credentials in application code.
 2. **Module-level `config.py` per service.** Each Python service owns a single `config.py` that imports all `os.getenv()` calls and exposes a typed Pydantic `Settings` object. Application code imports from `config` — never from `os` directly. Two documented exceptions exist in `beckn-bap-client` and `mcp-sidecar` (see §2).
 3. **Separate `.env` files per environment.** `docker-compose.yml` injects variables into containers via `environment:` blocks; local services read from shell exports or a root `.env` file. Never commit secrets.
 4. **Safe dev defaults, unsafe-by-design production gaps.** Variables that cannot have a safe universal default (secrets, BAP identifiers, public URLs) are set to obvious placeholder values such as `dev-internal-token-CHANGE_ME`. These are documented in §2 and the production checklist is in §6 of [System Design](#).
-

7.2 2. Configuration Files

7.2.1 How It Works

```
repo root
├── .env                ← loaded by docker-compose.yml; sets defaults for all containers
├── IntentParser/
│   ├── config.py      ← Pydantic BaseSettings; reads env vars at import time
├── services/
│   ├── orchestrator/src/config.py
│   ├── becn-bap-client/src/config.py
│   ├── mcp-sidecar/config.py
│   └── ...            ← every service has its own config.py
└── frontend/
    └── .env.local      ← Next.js convention; never committed (git-ignored)
```

Docker deployments: `docker-compose.yml` reads the root `.env` file and passes variables into each container's environment: `block`. All containers share the same `.env` but each service only reads the variables it declares in its own `config.py`.

Local services (IntentParser, mcp-sidecar): These run outside Docker and must have variables exported into the shell before starting:

```
# Inside conda activate infosys_project
export $(grep -v '^#' .env | xargs)
cd IntentParser && uvicorn api:app --port 8001 --reload
cd services/mcp-sidecar && BAP_API_KEY="any-dev-string" uvicorn server:app
↪ --port 3000
```

7.2.2 File Map

File	Purpose	Read by
.env (repo root)	Primary local dev secrets — DB credentials, CLAUDE_PROXY_BINARY_PATH	database/setup_database.py; shell export \$(...); docker-compose.yml via env_file:

File	Purpose	Read by
<code>.env.example</code> (repo root)	Checked-in template with safe placeholder values	Developers — copy to <code>.env</code> and edit
<code>docker-compose.yml</code>	Canonical runtime config for all 18 Docker services; overrides code defaults	Docker Compose; CI/CD
<code>IntentParser/config.py</code>	IntentParser Pydantic Settings: Ollama URLs, model names, DB pool, MCP params	IntentParser API and pipeline stages
<code>services/*/src/config.py</code>	Per-service Pydantic Settings (orchestrator, erp-adapter, becn-bap-client, etc.)	Each service at startup
<code>services/mcp-sidecar/config.py</code>	MCP sidecar Pydantic Settings	mcp-sidecar SSE server
<code>config/*.yaml</code>	ONIX routing YAMLS for onix-bap and onix-bpp	ONIX Go adapter at container startup
<code>frontend/.env.local</code>	Next.js secrets: Keycloak OIDC client, NextAuth session secret	Next.js build and runtime
<code>frontend/.env.example</code>	Checked-in frontend template	Developers

7.2.3 Root .env Template

```

# Copy and adjust for your machine
cp .env.example .env

# PostgreSQL reached by Docker containers via host.docker.internal
DB_HOST=host.docker.internal
DB_PORT=5432
DB_NAME=procurement_agent
DB_USER=postgres
DB_PASSWORD=postgres123          # change in production

# Claude OpenAI Proxy (only needed for negotiation demo)
CLAUDE_PROXY_BINARY_PATH=/home/<user>/.local/bin/claude  # update per machine
CLAUDE_PROXY_KEY=          # any non-empty string enables auth

```

7.2.4 The Two `os.getenv()` Exceptions

`REDIS_URL` and `REDIS_RESULT_TIMEOUT` in `services/mcp-sidecar/bap_client.py`, and `REDIS_URL` in `services/beckn-bap-client/src/handler.py`, are read via bare `os.getenv()` rather than through the Pydantic Settings model. These two variables **must** be present in the process environment — setting them only in a `.env` file that is not sourced into the shell will silently give them their code defaults.

7.3 3. Complete Environment Variable Reference

7.3.1 orchestrator (: 8004, host ports 8000 and 8004)

The orchestrator is dual-mapped: host port 8000 exists because `NEXT_PUBLIC_API_BASE_URL=http://localhost` is the frontend default; 8004 is exposed for direct testing.

Variable	Default	Required	Purpose
INTENTION_PARSER_URL	<code>http://localhost:8001</code>	Yes	Step 1: NL intent parsing service
BECKN_BAP_URL	<code>http://localhost:8002</code>	Yes	Steps 2 and 4: Beckn BAP client
COMPARATIVE_SCORING_URL	<code>http://localhost:8003</code>	Yes	Step 3: comparative scoring adapter
DATA_NORMALIZER_URL	<code>http://localhost:8004</code>	Yes	Persistence layer for all DB writes
DEMO_GATEWAY_URL	<code>http://localhost:8015</code>	Yes (negotiation)	LangGraph negotiation demo gateway — must be port 8015, not 8005 as some README versions state
ANALYTICS_URL	<code>http://localhost:8006</code>	No	Analytics dashboard data
ERP_ADAPTER_URL	<code>http://localhost:8007</code>	No	ERP budget gate and PO push
ERP_INTERNAL_TOKEN	<code>dev-internal-token-CHANGE_ME</code>	Yes (production)	Bearer token sent to erp-adapter; rotate before any external deployment

Variable	Default	Required	Purpose
ERP_BUDGET_CHECK_ENABLED	true	No	Enables the budget check call to erp-adapter
ERP_BUDGET_CHECK_REQ	false	No	true = fail-closed; false = fail-open. docker-compose default is false — set true in production
ERP_BUDGET_CHECK_TIMEOUT_MS	5000	No	Hard timeout for the synchronous budget gate call
ERP_SYNC_ENABLED	true	No	Enables async PO outbox sync to ERP
ERP_DEFAULT_COST_CENTRE	IND-PROC-01	No	Default cost-centre code in PO payloads
SELLER_WEBHOOK_HMAC_CHANGE_ME	dev-seller-hmac-	Yes (production)	HMAC-SHA256 secret for inbound seller push webhooks
BUYER_NAME	Procurement Agent	No	Buyer display name in Beckn init/confirm contracts
BUYER_EMAIL	procurement@example.	No	Buyer contact email in Beckn contracts
BUYER_PHONE	+91-0000000000	No	Buyer phone for Beckn contracts
BUYER_ADDRESS_CITY	Bangalore	No	Buyer city
BUYER_ADDRESS_STATE	Karnataka	No	Buyer state
BUYER_ADDRESS_COUNTR	IND	No	Buyer country (ISO 3166-1 alpha-3)
KAFKA_BOOTSTRAP	"" (disabled)	No	Kafka broker; empty disables the consumer. Deferred to Phase 4.
KAFKA_TOPIC	po.status.changed	No	Kafka topic for order status events

7.3.2 IntentParser (: 8001, runs locally or as intention-parser container)

IntentParser has two execution modes. When run locally (`uvicorn api:app --port 8001 --reload`) it uses the code defaults below and runs all three pipeline stages. The `intention-parser` Docker container applies the overrides in the third column and only runs Stages 1 and 2.

Variable	Code Default	Docker Override	Required	Purpose
OLLAMA_URL	<code>http://localhost:11434/v1</code>	<code>http://host.docker.internal:11434/v1</code>	Yes	OLLama API base URL
COMPLEX_MODEL	<code>qwen3:8b</code>	<code>qwen3:1.7b</code>	No	LLM for Stage 1 classification and complex Stage 2 extraction. Docker override collapses two-tier routing — both model paths use <code>qwen3:1.7b</code> in the container.
SIMPLE_MODEL	<code>qwen3:1.7b</code>	<code>qwen3:1.7b</code>	No	LLM for simple/short Stage 2 queries
EMBEDDING_MODEL	<code>all-MiniLM-L6-v2</code>	(not set)	No	sentence-transformers model for Stage 3 semantic cache
ANTHROPIC_API_KEY	<code>"</code> (disabled)	(not set)	No	Enables Claude Sonnet 4.6 as last-resort Stage 3 broadening fallback. Empty = Claude fallback disabled. See §4.
CLAUDE_MODEL	<code>claude-sonnet-4-6</code>	(not set)	No	Claude model name when ANTHROPIC_API_KEY is set
MCP_SSE_URL	<code>http://localhost:3000/sse</code>	(not set)	No	MCP sidecar SSE endpoint for Stage 3 BPP validation
MCP_PROBE_TIMEOUT	<code>8.0</code>	(not set)	No	Timeout in seconds for MCP tool calls

Variable	Code Default	Docker Override	Required	Purpose
BECKN_DOMAIN	procurement	(not set)	No	Beckn domain identifier passed to search_bpp_catalog
BECKN_VERSION	1.1.0	(not set)	No	Beckn protocol version passed to the sidecar
DB_HOST	localhost	host.docker.internal	Yes	PostgreSQL host for Stage 3 pgvector semantic cache
DB_PORT	5432	(not set)	No	PostgreSQL port
DB_NAME	procurement_agent	(not set)	No	Database name
DB_USER	postgres	(not set)	No	Database user
DB_PASSWORD	""	(not set)	Yes (production)	Database password
DB_MIN_POOL	5	(not set)	No	asyncpg minimum connection pool size
DB_MAX_POOL	20	(not set)	No	asyncpg maximum connection pool size
DB_CMD_TIMEOUT	5.0	(not set)	No	asyncpg command timeout in seconds
HNSW_EF_SEARCH	100	(not set)	No	pgvector HNSW ef_search parameter for ANN queries

VALIDATED_THRESHOLD (0.85) and AMBIGUOUS_THRESHOLD (0.45) are named constants in Intent-Parser/config.py but are **not env-configurable** — overriding them requires a code change.

7.3.3 beckn-bap-client (: 8002)

Variable	Default	Required	Purpose
BAP_ID	bap.example.com	Yes (production)	Beckn Application Platform identifier registered on the network

Variable	Default	Required	Purpose
BAP_URI	http://localhost:8000	Yes (production)	Public callback URI for ONIX to route on_* callbacks. Do not include a trailing action name — ONIX appends /{action}.
ONIX_URL	http://localhost:8081	Yes	Base URL of the onix-bap Go adapter
DOMAIN	nic2004:52110	No	Beckn domain code; docker-compose sets beckn.one/testnet
COUNTRY	IND	No	Buyer country for Beckn context
CITY	std:080	No	Buyer city code for Beckn context
CORE_VERSION	2.0.0	No	Beckn core spec version sent in all request contexts
REQUEST_TIMEOUT	30	No	HTTP request timeout in seconds for outbound Beckn calls
CALLBACK_TIMEOUT	10.0	No	Seconds the CallbackCollector waits for on_select / on_init / on_confirm / on_status
CATALOG_NORMALIZER_URL	http://localhost:8000	Yes	URL of catalog-normalizer for on_discover payload normalisation
REDIS_URL	redis://localhost:6379	No	Redis URL for Pub/Sub. Must be a real env var (not only in .env) — read via os.getenv() in handler.py.

7.3.4 mcp-sidecar (:3000, runs locally only)

Variable	Default	Required	Purpose
BAP_API_KEY	(none)	Yes — service refuses to start	Bearer token for the BAP client API. Any non-empty string works in dev; never commit.
BAP_CLIENT_URL	http://localhost:800	Yes	URL of becn-bap-client's /discover endpoint
REDIS_URL	redis://localhost:637	Yes	Redis Pub/Sub URL. Must be a real env var — read via <code>os.getenv()</code> in <code>bap_client.py</code> .
REDIS_RESULT_TIMEOUT	15	Yes	Seconds to wait on the Redis channel before returning <code>found: false</code> . Must be a real env var. This is the primary latency ceiling for discovery probes.
MCP_BAP_TIMEOUT	3.0	No	HTTP safety valve for the fire-and-forget POST to becn-bap-client
RANKING_MIN_SIMILARI	0.30	No	Cosine similarity floor; items below this are filtered from results

Start command: `cd services/mcp-sidecar && BAP_API_KEY="any-string" uvicorn server:app --port 3000` (inside conda activate `infosys_project`).

7.3.5 erp-adapter (:8007)

See §5 for ERP vendor selection and HMAC rotation details.

Variable	Default	Required	Purpose
ERP_INTERNAL_TOKEN	dev-internal-token-CHANGE_ME	Yes (production)	Bearer token validated on all <code>/api/v1/*</code> routes from the orchestrator

Variable	Default	Required	Purpose
ERP_VENDORS	mock	Yes	Comma-separated vendor list: mock, sap, oracle, or sap,oracle
DB_HOST	procurement-postgres	Yes	PostgreSQL host for the outbox table
DB_PASSWORD	postgres123	Yes (production)	Database password
SAP_CLIENT_ID	mock-sap-client	Yes (SAP)	SAP OAuth2 client ID
SAP_CLIENT_SECRET	mock-sap-secret	Yes (SAP production)	SAP OAuth2 client secret
SAP_WEBHOOK_HMAC_SECRET	dev-sap-hmac-CHANGE_ME	Yes (production)	Primary HMAC-SHA256 key for X-Sap-Signature
SAP_WEBHOOK_HMAC_SEC "" (disabled)		No	Rotation key — both primary and next are accepted simultaneously
ORACLE_CLIENT_ID	mock-oracle-client	Yes (Oracle)	Oracle OAuth2 client ID
ORACLE_CLIENT_SECRET	mock-oracle-secret	Yes (Oracle production)	Oracle OAuth2 client secret
ORACLE_WEBHOOK_HMAC_SECRET	dev-oracle-hmac-CHANGE_ME	Yes (production)	Primary HMAC-SHA256 key for Oracle webhook verification
ORACLE_WEBHOOK_HMAC_ "" (disabled)		No	Oracle HMAC rotation key
BUDGET_CHECK_TOTAL_TIMEOUT_MS	3000	No	Hard timeout for the synchronous budget gate
BREAKER_FAIL_MAX	5	No	Per-vendor circuit breaker consecutive failure threshold
BREAKER_RESET_TIMEOUT_SECS	60	No	Seconds in OPEN state before auto-probe (half-open)
LOG_PAYLOADS	false	No	When true, full request/response bodies are logged. Never set true in production — full PO and budget payloads contain PII.

7.3.6 data-normalizer (:8006)

Variable	Default	Required	Purpose
DB_HOST	localhost	Yes	PostgreSQL host; host.docker.internal in Docker
DB_PASSWORD	postgres123	Yes (production)	Database password
SYSTEM_USER_ID	00000000-0000-0000- 0000-000000000001	No	UUID for the fallback system user upserted into users when no requester_id is provided. Not in docker-compose.yml — hardcoded default is used in all deployments unless explicitly overridden.

The all-MiniLM-L6-v2 model (~90 MB) is downloaded from Hugging Face on first startup of the memory write endpoint. Cold start for this download is 10–30 seconds.

7.3.7 sim-bpp (:3002)

Variable	Default	Required	Purpose
BPP_ID	bpp.example.com	No	BPP identifier embedded in all response contexts
BPP_URI	http://onix- bpp:8082/bpp/receive	No	BPP receiver URI sent in catalog responses
ONIX_BPP_CALLER	http://onix- bpp:8082/bpp/caller	Yes	Base URL of the onix-bpp caller module
CATALOG_PATH	/app/catalog.json	No	Path to the catalog JSON file; bind-mounted at . /services/sim- bpp/catalog.json. Re-read on every request — no rebuild needed to update the catalog.

Variable	Default	Required	Purpose
SIM_BPP_AUTO_ADVANCE	false (code) / true (docker-compose)	No	When enabled, automatically transitions a confirmed order through ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED
SIM_BPP_ADVANCE_INTE	5	No	Seconds between each auto-advance lifecycle step
DATA_NORMALIZER_URL	http://data-normalizer:8006	Yes (auto-advance)	URL for PATCHing purchase_orders.status during lifecycle transitions

Important: SIM_BPP_AUTO_ADVANCE defaults to false in code but is set to true in docker-compose.yml. A developer running sim-bpp outside Docker will not see automatic lifecycle transitions unless the variable is explicitly exported.

7.3.8 frontend (:3000)

Variable	Default	Required	Purpose
NEXTAUTH_URL	http://localhost:3000	Yes	Public URL of this Next.js app; must match the Keycloak redirect URI exactly
NEXTAUTH_SECRET	(none)	Yes	Signs and verifies session JWTs. Generate with openssl rand -base64 32.
KEYCLOAK_ISSUER	https://euc1.auth.ac	Yes	OIDC issuer URL. Phase Two cloud: https://app.phasetwo.io/auth/ Self-hosted: https://<host>/realms/<realm>
KEYCLOAK_CLIENT_ID	procurement-frontend	Yes	Client ID in the Keycloak realm

Variable	Default	Required	Purpose
KEYCLOAK_CLIENT_SECRET	(none)	Yes	Client secret (confidential client) — copy from the Keycloak dashboard
NEXT_PUBLIC_API_BASE	http://localhost:8000	Yes	Orchestrator base URL; matches the orchestrator's 8000 host port in docker-compose

The frontend has no stub credentials provider. Even local development requires a live Keycloak-compatible instance. The development tenant used by this project is Phase Two at `euc1.auth.ac`, realm `procurement-agent`.

7.3.9 Secrets That Must Be Rotated Before Production

Variable	Service	Risk if left at default
NEXTAUTH_SECRET	frontend	Session JWTs are unsigned; any JWT is accepted
KEYCLOAK_CLIENT_SECRET	frontend	OIDC flow breaks
ERP_INTERNAL_TOKEN	orchestrator, erp-adapter	Default is public knowledge; any caller can hit <code>/api/v1/*</code> on erp-adapter
SELLER_WEBHOOK_HMAC_SECRET	orchestrator	Forged seller webhooks accepted
SAP_WEBHOOK_HMAC_SECRET	erp-adapter, erp-mock	Forged SAP webhooks accepted
ORACLE_WEBHOOK_HMAC_SECRET	erp-adapter, erp-mock	Forged Oracle webhooks accepted
DB_PASSWORD	data-normalizer, analytics, erp-adapter	Default postgres123 grants full DB access
BAP_ID	beckn-bap-client	Placeholder; must be the registered BAP identifier on the target network
BAP_URI	beckn-bap-client	Must be a publicly reachable URL for ONIX to route callbacks

7.4 4. ONIX Routing Configuration

The six YAML files in `config/` control how the `onix-bap` and `onix-bpp` Go adapters route, sign, and validate Beckn traffic. Both containers bind-mount the `config/` directory at `/app/config`.

7.4.1 File Map

File	Adapter	Pipeline role	Purpose
<code>generic-routing-BAPCaller.yaml</code>	<code>onix-bap</code>	Outbound from BAP	Signs outbound BAP requests with ED25519 and routes them to the BPP receiver
<code>generic-routing-BAPReceiver.yaml</code>	<code>onix-bap</code>	Inbound to BAP	Validates inbound <code>on_*</code> callback signatures and routes to <code>beckn-bap-client</code>
<code>generic-routing-BPPCaller.yaml</code>	<code>onix-bpp</code>	Outbound from BPP	Signs and routes <code>on_*</code> responses from <code>sim-bpp</code> back to the BAP receiver
<code>generic-routing-BPPReceiver.yaml</code>	<code>onix-bpp</code>	Inbound to BPP	Validates inbound Beckn action requests and routes to <code>sim-bpp</code>
<code>specific-routing-BAPCaller.yaml</code>	<code>onix-bap</code>	Override entries	Action-level overrides for the caller pipeline
<code>specific-routing-BAPReceiver.yaml</code>	<code>onix-bap</code>	Override entries	Action-level overrides for the receiver pipeline

7.4.2 Pipeline Stages

The **caller** pipeline (outbound from BAP): `addRoute` → `sign` → `validateSchema`

The **receiver** pipeline (inbound to BAP): `validateSign` → `addRoute` → `validateSchema`

7.4.3 targetType: url — DeDi Registry Bypass

All routing YAMLs use `targetType: url` rather than `targetType: bpp` / `targetType: bap`. This routes directly to the configured URLs without consulting the Beckn DeDi registry, enabling a full Beckn protocol flow inside the Docker `beckn_network` bridge without a public registry subscription.

```
# Example entry from generic-routing-BAPCaller.yaml
routes:
  - action: discover
    target:
      type: url
      url: http://onix-bpp:8082/bpp/receiver # Docker DNS name; ONIX appends
      ↪ /discover
```

Do not include the action name in target URLs. ONIX appends `{action}` automatically. Including it duplicates the path and causes 404 errors.

When connecting to a live Beckn network, change `targetType: url` to `targetType: bpp / targetType: bap` and register real BAP/BPP identifiers in the target network's registry.

7.4.4 Schema Validator Pin

The `schemav2validator.so` binary inside the `fidedocker/onix-adapter` image is **pinned to commit d43ec30d**. Later commits of this binary introduced a `$ref` resolution bug in `Signature-Header` and `AckSignatureHeader` that causes all signed requests to fail schema validation.

Do not upgrade the ONIX schema validator without running a full end-to-end signing test (`discover` → `select` → `init` → `confirm` → `status`) first.

7.4.5 ED25519 Key Rotation

The current `config/*.yaml` files contain **testnet sandbox ED25519 key pairs** from the Beckn Developer Program. To rotate keys before production:

1. Generate a new ED25519 key pair using the Beckn key generation utility.
2. Register the new public key in the target Beckn network's registry.
3. Update the `privateKey` and `keyId` fields in `config/generic-routing-BAPCaller.yaml`.
4. Restart the `onix-bap` container. The new key takes effect immediately; no rolling restart is needed.

7.5 5. LLM Configuration

IntentParser uses a two-tier routing scheme based on query complexity. The same Ollama endpoint serves both tiers.

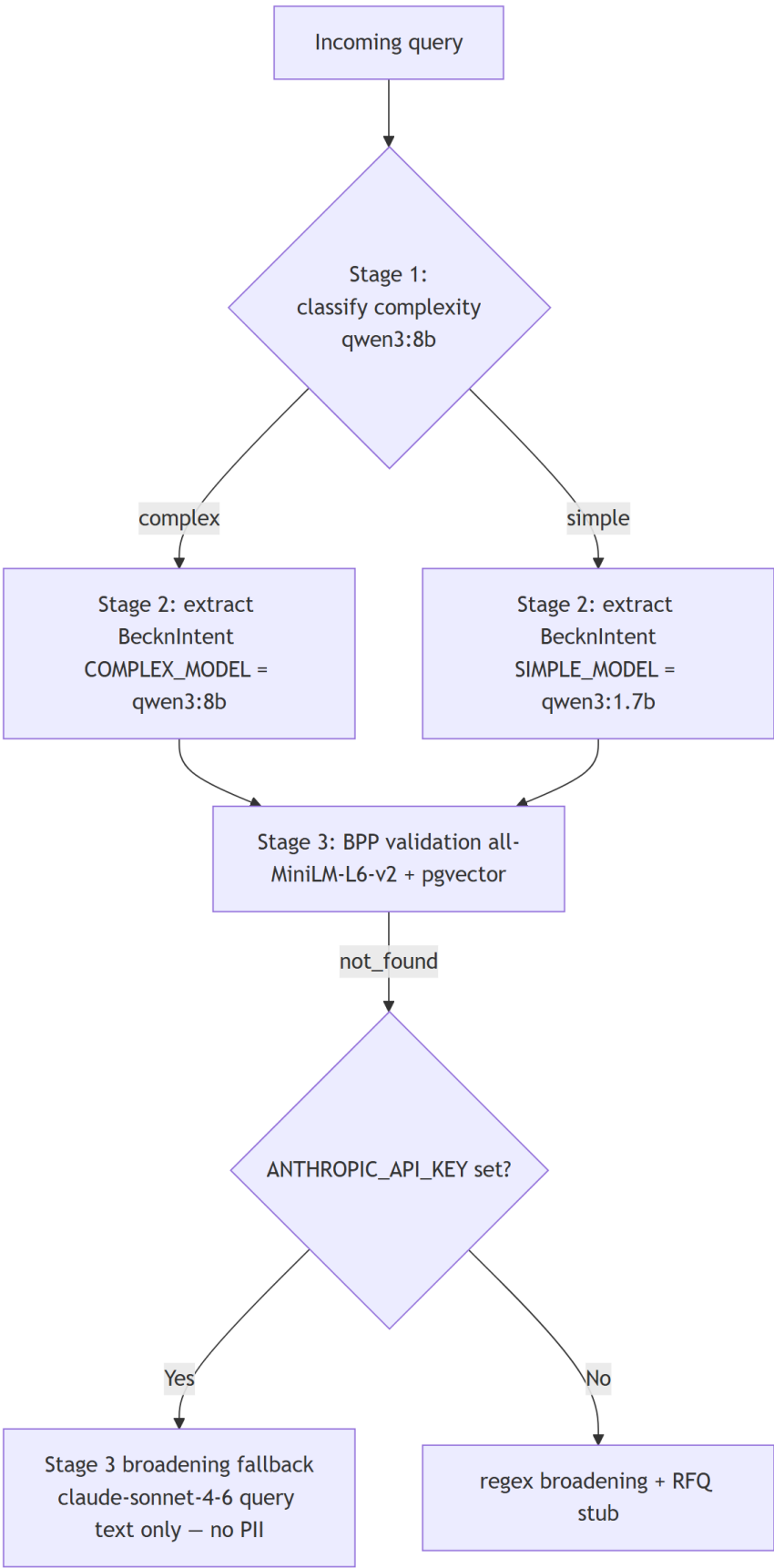


Figure 7.1: CONFIGURATION diagram 1

7.5.1 Switching Models

To change the primary LLM, set these variables in the shell before starting IntentParser locally:

```
export COMPLEX_MODEL="qwen3:8b"      # default; handles complex procurement
↪ requests
export SIMPLE_MODEL="qwen3:1.7b"    # default; handles short/simple queries
ollama pull qwen3:8b
ollama pull qwen3:1.7b
```

The `docker-compose.yml` overrides `COMPLEX_MODEL=qwen3:1.7b`, collapsing both routing branches to the smaller model inside the `intention-parser` container. Restore `qwen3:8b` for both models on a GPU host to enable full two-tier routing.

7.5.2 ANTHROPIC_API_KEY — Stage 3 Broadening Fallback

`ANTHROPIC_API_KEY` is **unset by default**. When unset, the Claude fallback is skipped entirely and the recovery flow falls through to stub functions (`log_unmet_demand`, `notify_buyer_no_stock`, `trigger_open_rfq_flow`).

When set:

- Only the broadening prompt is sent to the Anthropic API.
- The prompt contains the original procurement query text but **no buyer identity, pricing, or supplier data**.
- The model is `claude-sonnet-4-6` (configurable via `CLAUDE_MODEL`).

This is an opt-in feature. All LLMs are local by default — no data leaves the host unless `ANTHROPIC_API_KEY` is exported.

7.5.3 negotiation_engine LLM (:8004)

The negotiation engine routes through the local `claude_openai_proxy` service rather than Ollama directly:

Variable	Default	Purpose
NEGOTIATION_OPENAI_BASE_URL	http://host.docker.internal:8004	Point to <code>claude_openai_proxy</code> on the host

Variable	Default	Purpose
NEGOTIATION_OPENAI_MODEL	claude-3-5-sonnet	OpenAI alias mapped to sonnet by the proxy
NEGOTIATION_OPENAI_TIMEOUT_S	30.0	LLM call timeout
NEGOTIATION_ADVISORY_MAX_TC	512	Max token budget for advisory LLM node responses

The proxy must be running (`uvicorn services.claude_openai_proxy.main:app --host 0.0.0.0 --port 8012`) and `CLAUDE_PROXY_BINARY_PATH` must point to the installed `claude` CLI binary on the host.

7.6 6. ERP Configuration

7.6.1 Vendor Selection

The `ERP_VENDORS` variable in `erp-adapter` selects which ERP backend is active:

ERP_VENDORS value	Effect
mock (default)	Routes all ERP calls to <code>erp-mock:8008</code> . No external connectivity required.
sap	Activates the SAP S/4HANA adapter. Requires <code>SAP_CLIENT_ID</code> , <code>SAP_CLIENT_SECRET</code> , <code>SAP_BASE_URL</code> , and <code>SAP_OAUTH_TOKEN_URL</code> .
oracle	Activates the Oracle ERP Cloud adapter. Requires <code>ORACLE_CLIENT_ID</code> , <code>ORACLE_CLIENT_SECRET</code> , <code>ORACLE_BASE_URL</code> , and <code>ORACLE_OAUTH_TOKEN_URL</code> .
sap,oracle	Both vendors active simultaneously (multi-vendor mode).

7.6.2 Switching from Mock to SAP

1. Set `ERP_VENDORS=sap` in the orchestrator and `erp-adapter` environment.

2. Replace SAP_BASE_URL with the real SAP S/4HANA OData base URL (remove the erp-mock:8008 default).
3. Replace SAP_OAUTH_TOKEN_URL with the real SAP OAuth2 token endpoint.
4. Set SAP_CLIENT_ID and SAP_CLIENT_SECRET with the production OAuth2 credentials.
5. Set SAP_WEBHOOK_HMAC_SECRET to the key that SAP will use to sign inbound webhooks.
6. Restart erp-adapter.

The same pattern applies for Oracle, substituting the ORACLE_* equivalents.

7.6.3 Budget Gate Flags

Variable	Service	Dev default	Production recommendation
ERP_BUDGET_CHECK_ENABLED	orchestrator	true	true
ERP_BUDGET_CHECK_REQUIRED	orchestrator	false (fail-open)	true (fail-closed — prevents budget overruns even if erp-adapter is unreachable)
ERP_BUDGET_CHECK_TIMEOUT_MS	orchestrator	800	800 (matches BUDGET_CHECK_TOTAL_TIMEOUT_MS in erp-adapter)

Setting ERP_BUDGET_CHECK_REQUIRED=false means the orchestrator's /commit path proceeds even when erp-adapter returns an error. This is the docker-compose default for development convenience.

7.6.4 HMAC Dual-Secret Rotation

The erp-adapter accepts inbound vendor webhooks signed with HMAC-SHA256. The dual-secret pattern enables zero-downtime key rotation.

Step-by-step rotation procedure:

1. Generate a new secret: `openssl rand -hex 32`
2. Set SAP_WEBHOOK_HMAC_SECRET_NEXT=<new-secret> in erp-adapter and restart the container.
3. Update the SAP system to sign outbound webhooks with the new secret.
4. Confirm that new signatures are arriving and being accepted (check erp-adapter logs).

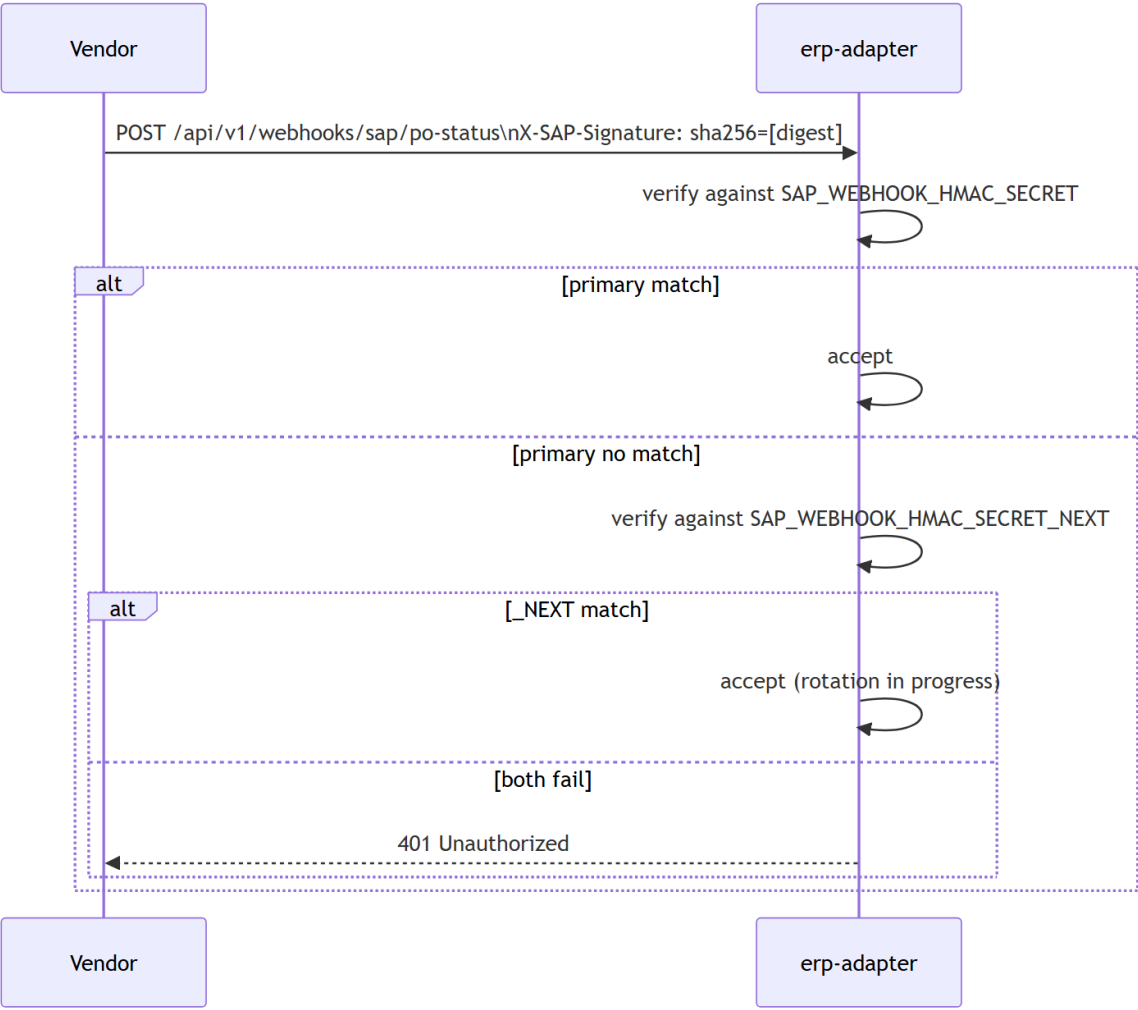


Figure 7.2: CONFIGURATION diagram 2

- 5. Move the new secret to the primary: `SAP_WEBHOOK_HMAC_SECRET=<new-secret>`.
- 6. Clear `SAP_WEBHOOK_HMAC_SECRET_NEXT=""` and restart `erp-adapter`.

The same procedure applies for `ORACLE_WEBHOOK_HMAC_SECRET` / `ORACLE_WEBHOOK_HMAC_SECRET_NEXT` and for `SELLER_WEBHOOK_HMAC_SECRET` in the orchestrator (single-secret; no `_NEXT` variant).

7.7 7. Beckn Network Configuration

7.7.1 Pointing the Stack at a Different Beckn Network

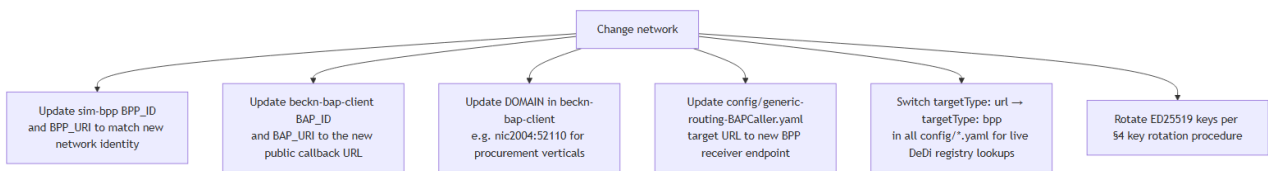


Figure 7.3: CONFIGURATION diagram 3

The key variables to change when switching networks:

Variable	Service	What to set
BAP_ID	beckn-bap-client	Registered BAP identifier on the target network
BAP_URI	beckn-bap-client	Publicly reachable callback URL (ONIX appends <code>/ {action}</code>)
BPP_ID	sim-bpp	BPP identifier for this simulator on the target network
DOMAIN	beckn-bap-client	Beckn domain code for the target vertical (e.g. <code>nic2004:52110</code>)

7.7.2 Testnet vs Production Keys

The `config/*.yaml` files currently contain testnet ED25519 key pairs from the Beckn Developer Program. These keys are checked into the repository and are known to the sandbox environment — they provide no security guarantee. Follow the key rotation procedure in §4 before connecting to any non-sandbox network.

7.8 8. sim-bpp Catalog Configuration

The BPP catalog is served from `services/sim-bpp/catalog.json`. Docker Compose bind-mounts this file into the container at `/app/catalog.json`:

```
# docker-compose.yml (sim-bpp volumes section)
volumes:
  - ./services/sim-bpp/catalog.json:/app/catalog.json
```

`sim-bpp` re-reads `catalog.json` on every inbound request. **You can edit the catalog and see the changes immediately without rebuilding or restarting the container.**

7.8.1 Catalog Hot-Reload

```
# Edit catalog.json directly
vim services/sim-bpp/catalog.json

# Verify the update
curl -s http://localhost:3002/api/catalog | python -m json.tool | head -40
```

The catalog JSON structure follows the Beckn v2.0.0 `on_discover` payload shape: a top-level `catalogs[]` array with `providers[]`, `items[]`, `fulfillments[]`, and `payments[]` as flat resources.

7.8.2 Automated Lifecycle Progression

`SIM_BPP_AUTO_ADVANCE=true` (the docker-compose default) causes `sim-bpp` to advance a confirmed order through the following states automatically at `SIM_BPP_ADVANCE_INTERVAL_SECS` (default 5 seconds):

ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

Set `SIM_BPP_AUTO_ADVANCE=false` to freeze the order lifecycle and inspect intermediate states during testing. When running `sim-bpp` locally (outside Docker), `SIM_BPP_AUTO_ADVANCE` defaults to `false` in code — set it explicitly if you want auto-advance:

```
SIM_BPP_AUTO_ADVANCE=true SIM_BPP_ADVANCE_INTERVAL_SECS=10 \
uvicorn services.sim-bpp.src.main:app --port 3002
```

7.9 9. Adding a New Configuration Variable

Follow these steps to add a configurable parameter to any service. The procedure is the same whether you are adding to IntentParser, a dockerised service, or the mcp-sidecar.

7.9.1 Step 1 — Add to the module-level config.py

Open the service's config.py (e.g. services/orchestrator/src/config.py) and add the field to the Pydantic Settings class:

```
# services/orchestrator/src/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # ... existing fields ...
    MY_NEW_TIMEOUT_MS: int = 500 # safe default for development

settings = Settings()
```

Use Pydantic v2 types with `@field_validator` decorators for any value that needs normalisation. Never use bare `os.getenv()` in application code — that would bypass the Settings model.

7.9.2 Step 2 — Add to .env.example with a safe placeholder

```
# .env.example
# Description of what this variable controls and its units
MY_NEW_TIMEOUT_MS=500
```

If the variable has no safe default (a secret or a host-specific path), leave the value empty and add a comment marking it as required:

```
# Required before production deployment
MY_WEBHOOK_SECRET=CHANGE_ME
```

7.9.3 Step 3 — Add to docker-compose.yml if a different value is needed in Docker

```
# docker-compose.yml (inside the relevant service's environment block)
services:
  orchestrator:
    environment:
      MY_NEW_TIMEOUT_MS: "800" # override for containerised environment
```

7.9.4 Step 4 — Document in this file

Add the new variable to the relevant table in §3, including: variable name, default value, whether it is required in production, and a one-line purpose description.

7.9.5 Step 5 — Document in ENVIRONMENT.md

If the project maintains an ENVIRONMENT.md file, add a row to the relevant service's section there as well. Keeping both documents consistent ensures the configuration reference and the deployment guide stay in sync.

Database Reference — Procurement Agent

8 Database Reference

PostgreSQL 16 + pgvector persistence layer for the Procurement Agent. All schema objects are defined in numbered migration scripts under `database/sql/`. Redis 7 is used alongside PostgreSQL for ephemeral async coordination.

For architectural context on why Redis Pub/Sub is used for async Beckn discovery, see [ADR-0001](#). For the full service topology, see [ARCHITECTURE.md](#).

8.1 1. Overview

Aspect	Detail
Engine	PostgreSQL 16
Vector extension	pgvector 0.7.0 (vector type, HNSW indexes)
UUID extensions	uuid-oss (<code>uuid_generate_v4()</code>) and pgcrypto (<code>gen_random_uuid()</code>)
Primary database	<code>procurement_agent</code> (container <code>procurement-postgres</code> , host port 5432)
Secondary database	<code>negotiation</code> (same container, host port 55432) — negotiation engine LangGraph checkpoints
Migration files	24 numbered SQL scripts in <code>database/sql/</code> (NN_ prefix; lexicographic order = FK dependency order)
Migration strategy	Idempotent — all DDL uses <code>IF NOT EXISTS / IF EXISTS / DO \$\$... \$\$</code> guards
ENUM types	20 custom types (defined in <code>00_extensions_and_types.sql</code> ; 2 receive additive values via migrations 19 and 22a)

Aspect	Detail
Named indexes	22 B-tree/partial indexes in 17_indexes.sql + 8 additional indexes defined inline in later migrations
Schema setup	python database/setup_database.py [--create-db] [--drop-all] [--continue-on-exists]

The procurement_agent database holds all 16 primary application tables. The negotiation database (port 55432) holds the three negotiation audit tables from 20_negotiation_schema.sql plus LangGraph checkpoint tables created at engine startup by AsyncPostgresSaver.setup().

8.2 2. Schema by Domain

8.2.1 2.1 Core Procurement

Figure 1 — Main procurement pipeline (procurement_agent database)

Figure 2 — Supporting and observability tables

8.2.1.1 users (migration 01_users.sql)

Column	Type	Notes
user_id	UUID PK	uuid_generate_v4() default
email	VARCHAR(255) UNIQUE NOT NULL	—
name	VARCHAR(255) NOT NULL	—
role	user_role ENUM	requester \ approver \ admin
department	VARCHAR(100)	—
approval_threshold	DECIMAL(15,2)	Maximum PO value this user may self-approve
keycloak_id	VARCHAR(255) UNIQUE	JWT sub claim; Keycloak is source of truth for identity

Column	Type	Notes
idp_provider	idp_provider_type ENUM	keycloak \ okta \ azure_ad

8.2.1.2 procurement_requests (migration 03_procurement_requests.sql, extended by 22b)

Column	Type	Notes
request_id	UUID PK	—
requester_id	UUID FK → users	ON DELETE RESTRICT
raw_input_text	TEXT NOT NULL	Original natural-language query
channel	channel_type ENUM	web \ slack \ teams
urgency_flag	BOOLEAN DEFAULT FALSE	—
category	VARCHAR(100)	Top-level commodity class
status	procurement_status ENUM	State machine: draft → parsing → discovering → scoring → negotiating → pending_approval → confirmed \ cancelled
pending_chosen_item_id	VARCHAR(255)	Added by migration 22b — survives session TTL expiry for approval resumption
pending_transaction_id	VARCHAR(255)	Added by migration 22b — Beckn transaction ID preserved across session boundary

Root anchor of every purchase request. All downstream tables reference this row.

8.2.1.3 parsed_intents (migration 04_parsed_intents.sql)

1:1 with procurement_requests (UNIQUE FK). Stores Stage 1 IntentParser output.

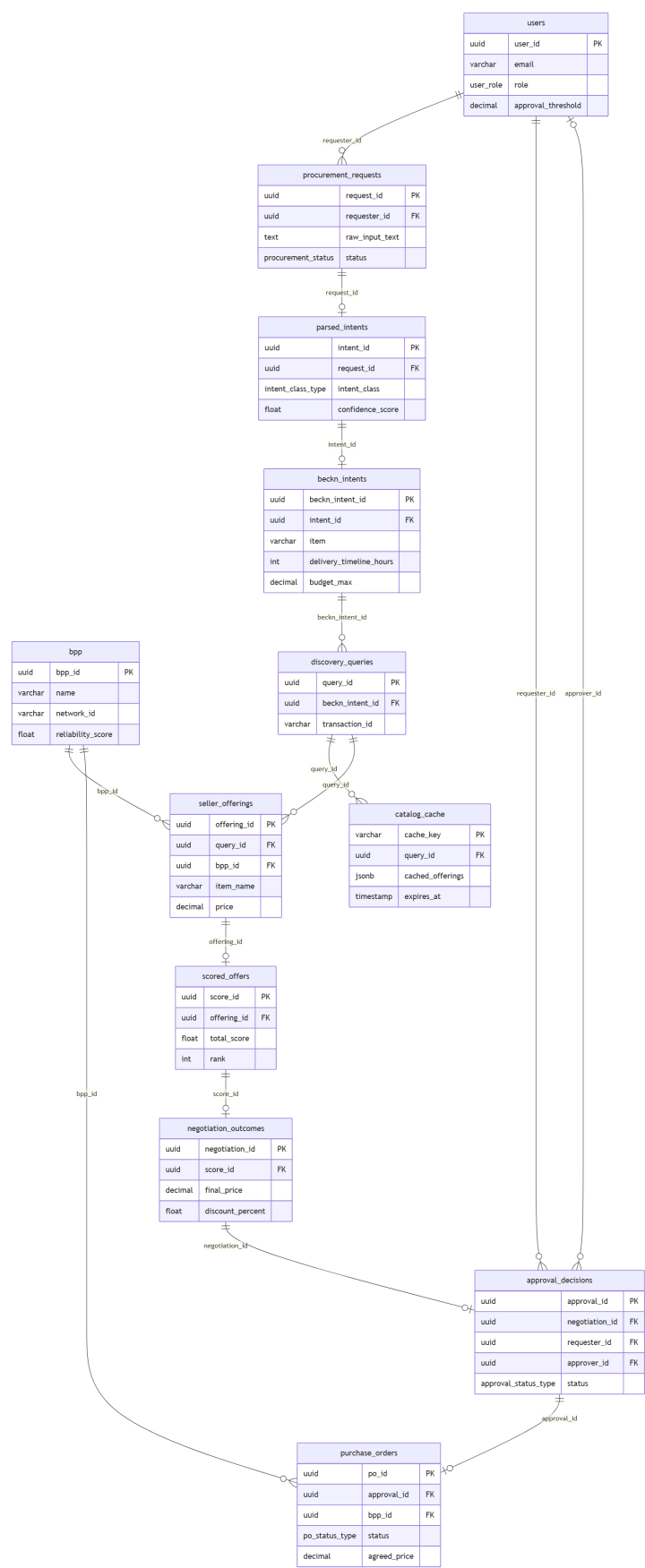


Figure 8.1: DATABASE diagram 1

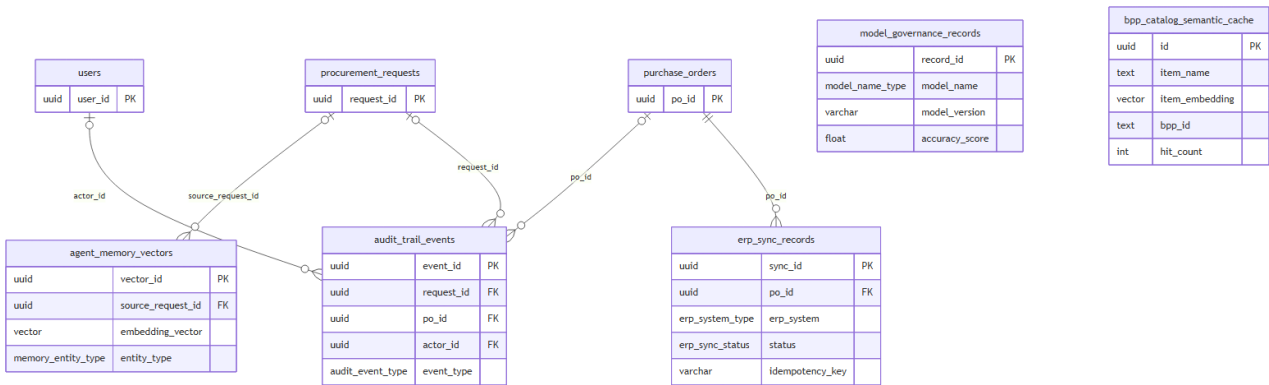


Figure 8.2: DATABASE diagram 2

Column	Type	Notes
intent_id	UUID PK	—
request_id	UUID FK → procurement_requests	UNIQUE, ON DELETE CASCADE
intent_class	intent_class_type ENUM	procurement \ query \ support \ out_of_scope
confidence_score	FLOAT	[0, 1]
model_version	VARCHAR(50)	e.g. qwen3:8b-v1
parsed_at	TIMESTAMP	—

8.2.1.4 becn_intents (migration 05_becn_intents.sql)

1:1 with parsed_intents. Anti-corruption layer — canonical structured intent used by all downstream services.

Column	Type	Notes
becn_intent_id	UUID PK	—
intent_id	UUID FK → parsed_intents	UNIQUE, ON DELETE CASCADE
item	VARCHAR(255) NOT NULL	Commodity name
descriptions	JSONB[]	Spec attributes array
quantity	INT	—
unit	VARCHAR(50)	e.g. meters, units

Column	Type	Notes
location_coordinates	VARCHAR(50)	Format: "lat, lon" decimal — never city names
delivery_timeline_hours	INT	Always integer hours — never ISO 8601
budget_min / budget_max	DECIMAL(15,2)	Typed bounds — never raw strings
currency	CHAR(3)	ISO 4217
compliance_requirements	JSONB[]	Regulatory / certification constraints

8.2.1.5 bpp (migration 02_bpp.sql)

Defined early (before seller_offerings and purchase_orders) because both reference it.

Column	Type	Notes
bpp_id	UUID PK	—
name	VARCHAR(255) NOT NULL	—
network_id	VARCHAR(100) NOT NULL	Beckn network identifier
endpoint_url	VARCHAR(500) NOT NULL	Must route through onix-bap:8081 — never call BPP directly
reliability_score	FLOAT	[0, 1]
on_time_delivery_rate	FLOAT	[0, 1]
last_seen_at	TIMESTAMP	Updated on each on_discover callback

8.2.1.6 discovery_queries (migration 06_discovery_queries.sql)

Many per becn_intents row — one row per POST /discover call.

Column	Type	Notes
query_id	UUID PK	—
beckn_intent_id	UUID FK → becn_intents	ON DELETE RESTRICT
network_id	VARCHAR(100)	Beckn network targeted

Column	Type	Notes
cache_hit	BOOLEAN	TRUE = response served from Redis (15-min TTL)
results_count	INT	Number of offerings returned
queried_at	TIMESTAMP	—

8.2.1.7 catalog_cache (migration 07_catalog_cache.sql)

PostgreSQL-layer cache of raw on_discover catalog payloads. A cache entry is keyed by a hash of the query parameters so that identical intents within the TTL window skip a live Beckn network round-trip. Distinct from the Redis 15-minute short-lived cache — this table stores the normalised catalog payload for audit and replay.

Column	Type	Notes
cache_key	VARCHAR(500) PK	SHA-256 hash of the BecknIntent parameters (item, location, budget, timeline)
query_id	UUID FK → discovery_queries	ON DELETE CASCADE
cached_offerings	JSONB NOT NULL	Full on_discover catalog payload at time of caching
created_at	TIMESTAMP	—
expires_at	TIMESTAMP	Cache row considered stale after this timestamp; eviction job removes expired rows

8.2.1.8 seller_offerings (migrations 08_seller_offerings.sql + 21_order_detail_fidelity.sql)

Column	Type	Notes
offering_id	UUID PK	—
query_id	UUID FK → discovery_queries	ON DELETE RESTRICT
bpp_id	UUID FK → bpp	ON DELETE RESTRICT
item_id	VARCHAR(255) NOT NULL	Beckn catalog item ID

Column	Type	Notes
item_name	VARCHAR(255)	Added by migration 21
price	DECIMAL(15,2) CHECK > 0	—
currency	CHAR(3)	—
delivery_eta_hours	INT CHECK > 0	—
quality_rating	FLOAT	[0, 5]
certifications	JSONB	—
inventory_count	INT	—
format_variant	INT	[1–5] — original catalog format before normalization (1=BECKN_V2_FLAT_RESOURCES ... 5=UNKNOWN/LLM)
is_normalized	BOOLEAN	—

8.2.2 2.2 Scoring and Approval

8.2.2.1 scored_offers (migration 09_scored_offers.sql)

1:1 with seller_offerings. Comparative Scoring Engine output.

Column	Type	Notes
score_id	UUID PK	—
offering_id	UUID FK → seller_offerings	UNIQUE, ON DELETE CASCADE
rank	INT CHECK > 0	Ordinal rank among competing offers
total_score	FLOAT	[0, 100] — composite_score × 100
price_score/delivery_score / quality_score / compliance_score	FLOAT	Each [0, 100]
tco_value	DECIMAL(15,2)	Total Cost of Ownership estimate
explanation_text	TEXT	Human-readable scoring rationale
user_overridden	BOOLEAN	TRUE feeds the override-rate calibration metric in model_governance_records

Column	Type	Notes
model_version	VARCHAR(50)	Scoring model version

8.2.2.2 negotiation_outcomes (migration 10_negotiation_outcomes.sql)

1:1 with scored_offers.

Column	Type	Notes
negotiation_id	UUID PK	—
score_id	UUID FK → scored_offers	UNIQUE, ON DELETE CASCADE
strategy_applied	negotiation_strategy_type ENUM	aggressive \\ accept_margin \\ advisory \\ escalate \\ skipped
initial_price	DECIMAL(15,2)	—
counter_offer_price	DECIMAL(15,2)	NULL when strategy is skipped or advisory
final_price	DECIMAL(15,2)	—
discount_percent	FLOAT	[0, 20] — CHECK constraint enforces hard cap at 20.0; not bypassable
acceptance_status	acceptance_status_type ENUM	accepted \\ rejected \\ advisory \\ escalated \\ skipped

A strategy=skipped row is auto-created when negotiation is bypassed to maintain referential integrity.

8.2.2.3 approval_decisions (migration 11_approval_decisions.sql)

1:1 with negotiation_outcomes. Approval state machine.

Column	Type	Notes
approval_id	UUID PK	—

Column	Type	Notes
negotiation_id	UUID FK → negotiation_outcomes	UNIQUE, ON DELETE RESTRICT
requester_id	UUID FK → users	ON DELETE RESTRICT
approver_id	UUID FK → users	SET NULL on delete; NULL for auto approvals
approval_level	approval_level_type ENUM	Routing: amount_total ≤ requester.threshold → auto; ≤ approver.threshold → manager; else → cfo
amount_total	DECIMAL(15,2) CHECK > 0	—
status	approval_status_type ENUM	pending approved rejected escalated auto_approved
is_emergency	BOOLEAN	TRUE sets deadline_at = NOW() + 60min
deadline_at	TIMESTAMP	—
notification_channel	notification_channel_type ENUM	slack teams email
decided_at	TIMESTAMP	—

8.2.2.4 purchase_orders (migrations 12_purchase_orders.sql + 21_order_detail_fidelity.sql)

Created only after approval_decisions.status IN ('approved', 'auto_approved') and ERP budget check passes.

Column	Type	Notes
po_id	UUID PK	—
approval_id	UUID FK → approval_decisions	UNIQUE, ON DELETE RESTRICT
bpp_id	UUID FK → bpp	ON DELETE RESTRICT
item_id	VARCHAR(255)	Beckn catalog item ID
quantity	INT CHECK > 0	—
unit	VARCHAR(50)	—

Column	Type	Notes
agreed_price	DECIMAL(15,2)	—
currency	CHAR(3)	—
beckn_confirm_ref	VARCHAR(255) UNIQUE	Beckn /confirm protocol reference
erp_po_ref	VARCHAR(255) UNIQUE	NULL until ERP sync completes
status	po_status_type ENUM	pending \ confirmed \ shipped \ delivered \ cancelled
fulfillment_eta	TIMESTAMPTZ	Added by migration 21

8.2.3 2.3 ERP Synchronisation

8.2.3.1 erp_sync_records (migrations 13_erp_sync_records.sql + 19_erp_enum_extensions.sql + 19b_erp_sync_records_outbox.sql)

ERP sync operations against SAP S/4HANA or Oracle ERP Cloud. Migration 19b promotes the table into a retryable outbox with pessimistic locking via FOR UPDATE SKIP LOCKED.

Core columns:

Column	Type	Notes
sync_id	UUID PK	—
po_id	UUID FK → purchase_orders	RESTRICT; nullable post-19b (outbox row may be enqueued before the PO row exists)
erp_system	erp_system_type ENUM	sap_s4hana \ oracle_erp_cloud \ mock
sync_type	erp_sync_type ENUM	budget_check \ po_creation \ goods_receipt \ invoice_matching
status	erp_sync_status ENUM	success \ failed \ pending \ in_progress
erp_reference_id	VARCHAR(255)	Vendor-assigned reference for the synced object

Column	Type	Notes
budget_available	BOOLEAN	NOT NULL when sync_type='budget_check' (CHECK constraint) — blocking prerequisite before /confirm
synced_at	TIMESTAMP	—

Outbox columns added by migration 19b:

Column	Type	Purpose
idempotency_key	VARCHAR(128) UNIQUE	sha256(transaction_id vendor sync_type) — forwarded as Idempotency-Key header to SAP/Oracle
attempts	INT DEFAULT 0	Total push attempts; row moves to DLQ when status='failed' AND attempts >= MAX_ATTEMPTS
next_attempt_at	TIMESTAMPTZ DEFAULT NOW()	Earliest wall-clock for retry; bumped by exponential backoff (5 → 30 → 120 → 600 → 3600 s)
lease_until	TIMESTAMPTZ	Set by worker on claim; another worker may reclaim if expired
worker_id	VARCHAR(64)	hostname:pid or Kubernetes pod name; diagnostic only
last_error	TEXT	Last exception message from a failed push attempt
payload	JSONB	Normalized PO JSON snapshot at enqueue time; enables stateless retry by any worker replica
updated_at	TIMESTAMPTZ DEFAULT NOW()	Outbox row mutation timestamp

8.2.4 2.4 Audit and Observability

8.2.4.1 audit_trail_events (migration 14_audit_trail_events.sql)

Complete agent decision log. SOX 404 / GDPR / IT Act 2000 compliant.

Column	Type	Notes
event_id	UUID PK	—
request_id	UUID FK → procurement_requests	SET NULL on delete; nullable
po_id	UUID FK → purchase_orders	SET NULL on delete; nullable
actor_id	UUID FK → users	SET NULL; NULL for all autonomous agent actions — this is by design
event_type	audit_event_type ENUM NOT NULL	discover \ normalize \ score \ negotiate \ approve \ confirm \ override \ erp_sync \ notification
agent_action	TEXT NOT NULL	Human-readable description of the action
reasoning_payload	JSONB DEFAULT '{}'	LLM chain-of-thought; will carry LangSmith trace IDs when wired in Phase 4
kafka_offset	BIGINT NOT NULL	Placeholder — hardcoded to 0 (Kafka not deployed in Phases 1-3)
splunk_indexed	BOOLEAN DEFAULT FALSE	Placeholder — never set to TRUE; SIEM exporter not deployed
event_timestamp	TIMESTAMP	—
retention_until	TIMESTAMP DEFAULT NOW() + 7 years	Populated correctly; archival deletion job deferred to Phase 4

8.2.4.2 model_governance_records (migration 16_model_governance_records.sql)

Weekly AI model evaluation registry. Not yet populated (governance pipeline deferred to Phase 4).

Column	Type	Notes
record_id	UUID PK	—
model_name	model_name_type ENUM NOT NULL	intent_parsing \\ comparison_scoring \\ negotiation_strategy \\ memory_retrieval
model_version	VARCHAR(50)	—
provider	ai_provider_type ENUM NOT NULL	openai \\ anthropic (note: ollama missing from ENUM — see §3)
accuracy_score	FLOAT [0, 1]	Review thresholds: intent_parsing < 0.95, comparison_scoring < 0.85, negotiation_strategy < 0.80
override_rate	FLOAT [0, 1]	Override thresholds: comparison_scoring > 30%, negotiation_strategy > 25%
evaluation_date	DATE	UNIQUE with (model_name, model_version, evaluation_date)
status	governance_status_type ENUM	active \\ review_triggered \\ deprecated

8.2.5 2.5 AI Memory

See §6 — [pgvector Similarity Search](#) for query patterns and thresholds.

8.2.5.1 agent_memory_vectors (migrations 15_agent_memory_vectors.sql + 22_agent_memory_vector_dim.sql)

pgvector similarity store for agent learning and supplier loyalty scoring.

Column	Type	Notes
vector_id	UUID PK	—
source_request_id	UUID FK → procurement_requests	SET NULL; nullable

Column	Type	Notes
entity_type	memory_entity_type ENUM NOT NULL	transaction \\\ negotiation \\\ seasonal \\\ supplier \\\ override
embedding_vector	vector (384) NOT NULL	Corrected from 3072 by migration 22a
metadata	JSONB DEFAULT '{}'	Structured context for retrieval
embedding_model	embedding_model_type ENUM	DEFAULT 'text-embedding-3-large' (stale – actual inserts use all-MiniLM-L6-v2; see §3)
indexed_at	TIMESTAMP	—

8.2.5.2 bpp_catalog_semantic_cache (migration 18_bpp_catalog_semantic_cache.sql)

Stage 3 BPP item existence validation cache, populated by IntentParser.

Column	Type	Notes
id	UUID PK	—
item_name	TEXT NOT NULL	—
item_embedding	vector (1536) NOT NULL	spec dimension; actual model (all-MiniLM-L6-v2) produces 384 dims
descriptions	TEXT[]	Spec attributes
bpp_id	TEXT	—
bpp_uri	TEXT	BPP endpoint
provider_id	TEXT	—
category_tag	TEXT	—
source	TEXT CHECK IN ('bpp_publish', 'mcp_feedback')	Write path identifier
embedding_strategy	TEXT CHECK IN ('item_name_only', 'item_name_and_specs')	—
hit_count	INT DEFAULT 0	Updated on cache hit for eviction heuristics
last_seen_at / created_at	TIMESTAMPTZ	—

Column	Type	Notes
UNIQUE	(item_name, bpp_id)	One entry per item per BPP

8.2.6 2.6 Negotiation Engine (negotiation database, port 55432)

These three tables live in the separate negotiation database, not in procurement_agent. They are provisioned by 20_negotiation_schema.sql, which is bind-mounted as a Docker init script. Lang-Graph checkpoint tables (checkpoints, checkpoint_writes, checkpoint_blobs) are created at engine startup by AsyncPostgresSaver.setup().

Table	PK	Purpose
negotiation_session	transaction_id TEXT	One row per negotiation lifecycle. status CHECK IN ('active', 'completed', 'escalated', 'failed') Root of the FK chain.
negotiation_round	round_id UUID	One row per round per session. UNIQUE (transaction_id, round_no). decision CHECK IN ('counter', 'accept', 'reject', 'escalate')
negotiation_policy_decision	decision_id UUID	Guardrail audit log. rule_violated TEXT (G1-G12), severity CHECK IN ('LOW', 'MEDIUM', 'HIGH'), action_taken CHECK IN ('CLAMP', 'ESCALATE', 'LOG', 'BLOCK'). Mirrors the Kafka procurement.negotiation.policy_violations.v topic when Kafka is available.

8.3 3. ENUMs

All ENUM types are defined in database/sql/00_extensions_and_types.sql. Two ENUMs receive additive values via later migrations.

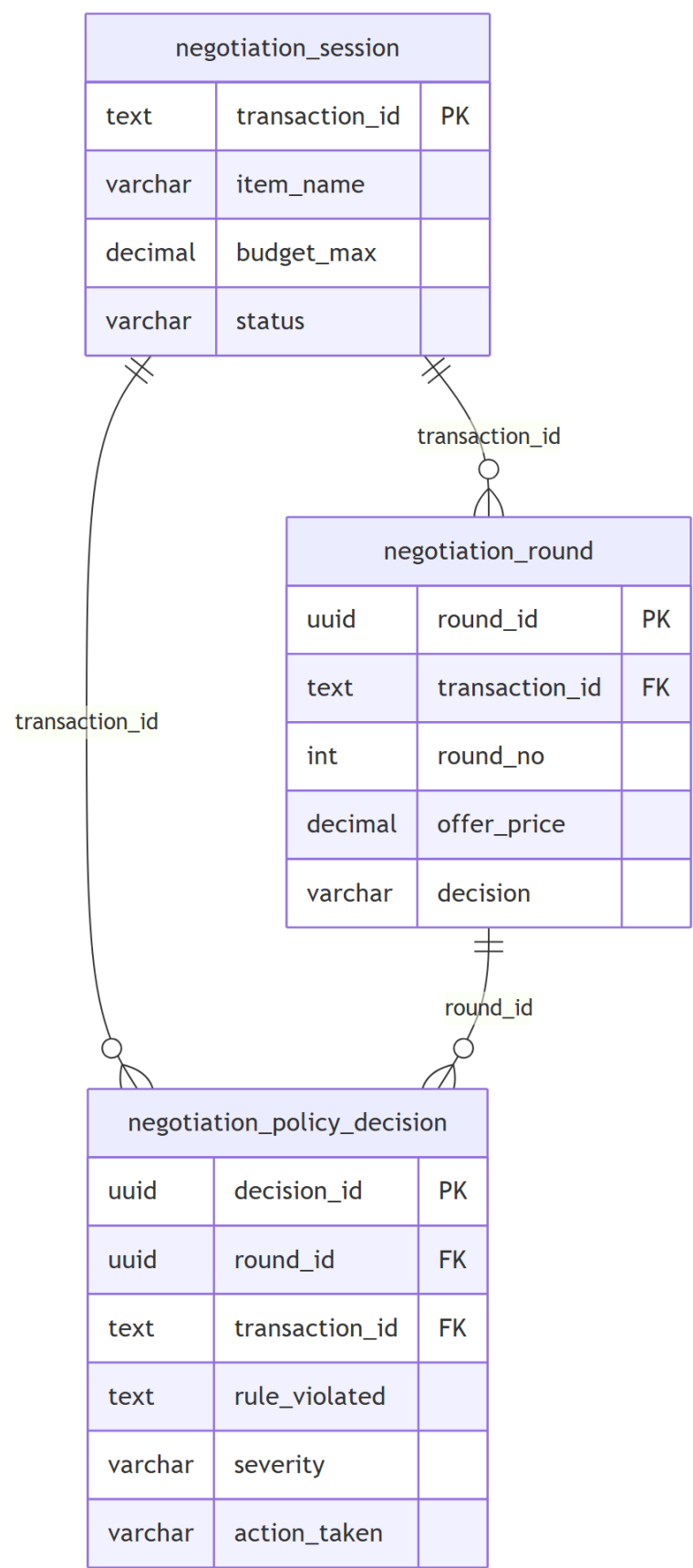


Figure 8.3: DATABASE diagram 3

ENUM Type	Values	Used By	Migration
user_role	requester, approver, admin	users.role	00
idp_provider_type	keycloak, okta, azure_ad	users.idp_provider	00
channel_type	web, slack, teams	procurement_requests.channel	00
procurement_status	draft, parsing, discovering, scoring, negotiating, pending_approval, confirmed, cancelled	procurement_requests	00
intent_class_type	procurement, query, support, out_of_scope	parsed_intents.intent	00class
negotiation_strategy	aggressive, accept_margin, advisory, escalate, skipped	negotiation_outcomes	00
acceptance_status_type	accepted, rejected, advisory, escalated, skipped	negotiation_outcomes	00acceptance_status
approval_level_type	auto, manager, cfo	approval_decisions.a	00
approval_status_type	pending, approved, rejected, escalated, auto_approved	approval_decisions.s	00status
notification_channel	slack, teams, email	approval_decisions.n	00
po_status_type	pending, confirmed, shipped, delivered, cancelled	purchase_orders.statu	00
erp_system_type	sap_s4hana, oracle_erp_cloud (base); mock added by migration 19	erp_sync_records.erp	00 + 19
erp_sync_type	budget_check, po_creation, goods_receipt, invoice_matching	erp_sync_records.sync	00type

ENUM Type	Values	Used By	Migration
erp_sync_status	success, failed, pending (base); in_progress added by migration 19	erp_sync_records.status	00 + 19
audit_event_type	discover, normalize, score, negotiate, approve, confirm, override, erp_sync, notification	audit_trail_events.event_type	00
memory_entity_type	transaction, negotiation, seasonal, supplier, override	agent_memory_vectors.entity_type	00
embedding_model_type	text-embedding-3-large, e5-large-v2 (base); all-MiniLM-L6-v2 added by migration 22a	agent_memory_vectors.embedding_model	00 + 22a
model_name_type	intent_parsing, comparison_scoring, negotiation_strategy, memory_retrieval	model_governance_records.model_name	00
ai_provider_type	openai, anthropic	model_governance_records.provider	00
governance_status_type	active, review_triggered, deprecated	model_governance_records.governance_status	00

Known schema discrepancies:

- `embedding_model_type` does not include `BAAI/bge-small-en-v1.5`, which is the actual model used by data-normalizer for agent memory writes. Any INSERT that stores the actual model name will fail at the DB constraint unless the column type is changed to TEXT.
- `ai_provider_type` does not include `ollama`, which is the primary LLM provider in development.
- The DEFAULT on `agent_memory_vectors.embedding_model` remains 'text-embedding-3-large' even after migration 22a — it was not updated to 'all-MiniLM-L6-v2'.

8.4 4. Indexes

8.4.1 4.1 Named B-tree / Partial Indexes (17_indexes.sql)

Index Name	Table	Column(s)	Type	Purpose
idx_procurement_procurement_requests	procurement_requests	(requester_id, created_at DESC)	B-tree	Fast lookup of all requests by user, newest first
idx_procurement_procurement_requ	procurement_requ	(status) — partial: status NOT IN ('con-firmed','cancell	B-tree partial	Filter active-only requests
idx_audit_events_audit_trail_event	audit_trail_event	(request_id, event_timestamp) — partial: request_id IS NOT NULL	B-tree partial	Primary traceability query: full audit chain for a request
idx_audit_events_audit_trail_even	audit_trail_even	(event_type, event_timestamp)	B-tree	SOX reporting queries by event type and time window
idx_audit_events_audit_trail_event	audit_trail_event	(po_id, event_timestamp) — partial: po_id IS NOT NULL	B-tree partial	Audit events linked to a specific PO
idx_audit_events_audit_trail_even	audit_trail_even	(splunk_indexed, event_timestamp) — partial: splunk_indexed = FALSE	B-tree partial	Batch SIEM push job — un-indexed events
idx_seller_offerings_query	seller_offerings	(query_id)	B-tree	All offerings returned by a discovery query
idx_seller_offer_seller_offerings	seller_offerings	(bpp_id)	B-tree	All offerings from a given BPP

Index Name	Table	Column(s)	Type	Purpose
idx_scored_offers	scored_offers	(user_overridden, scored_at) — partial: user_overridden = TRUE	B-tree partial	30-day override-rate calibration query
idx_purchase_order	purchase_orders	(bpp_id, status) — partial: status NOT IN ('delivered', 'cancelled')	B-tree partial	Active orders per BPP for operations dashboard
idx_purchase_order_status	purchase_orders	(status, created_at DESC)	B-tree	Status-filtered order listing
idx_model_governance	model_governance	(model_name, evaluation_date DESC)	B-tree	Latest evaluation per model name
idx_model_governance_status	model_governance	(status) — partial: status <> 'deprecated'	B-tree partial	Active / under-review models only
idx_bpp_network	bpp	(network_id)	B-tree	Lookup BPPs by Beckn network ID
idx_erp_sync_po	erp_sync_records	(po_id, sync_type)	B-tree	ERP sync history for a PO grouped by operation
idx_erp_sync_pending	erp_sync_records	(status, synced_at) — partial: status = 'pending'	B-tree partial	ERP outbox retry queue
idx_discovery_queries_intent	discovery_queries	(beckn_intent_id, queried_at DESC)	B-tree	All queries triggered by a beckn_intent, newest first
idx_approval_decisions	approval_decisions	(status) — partial: status IN ('pending', 'escalated')	B-tree partial	Approval dashboard — actionable decisions only

Index Name	Table	Column(s)	Type	Purpose
idx_approval_decisions_approver	approval_decisions	(approver_id, status) – partial: approver_id IS NOT NULL	B-tree partial	Approver inbox – open decisions per user
idx_agent_memory_agent_entity	agent_memory_vec	(entity_type)	B-tree	RAG retrieval routing by entity type
idx_agent_memory_agent_request	agent_memory_vec	(source_request_id) – partial: source_request_id IS NOT NULL	B-tree partial	Vectors from a specific request

8.4.2 4.2 Migration-Added Indexes

Index Name	Migration	Table	Column(s)	Type	Purpose
hnsf_bpp_catalog_embeddings	18_bpp_catalog_embeddings	catalog_embeddings	(instance_id, vec – tor_cosine_ops)	HNSW m=16, ef_construction=64	Cosine ANN search for Stage 3 item validation
uq_erp_sync_idempotency_key	19b_erp_sync_erp_sync_reco	erp_sync_reco	(idempotency_key) – partial: WHERE idempotency_key IS NOT NULL	Unique partial:	At-most-once enqueue per (txn, vendor, operation)
idx_erp_sync_reco_status	19b_erp_sync_erp_sync_reco	erp_sync_reco	(status) – partial: WHERE status IN ('pending', 'in_progress')	B-tree partial:	Outbox worker hot-path claim query
idx_neg_round	20_negotiation_negotiation_r	negotiation_r	(transaction_id, round_no)	B-tree	Replay access for negotiation rounds
idx_neg_policy	20_negotiation_negotiation_p	negotiation_p	(transaction_id, decided_at)	B-tree	Compliance queries by transaction
idx_neg_policy	20_negotiation_negotiation_p	negotiation_p	(rule_violated, severity)	B-tree	Compliance queries by guardrail rule

Index Name	Migration	Table	Column(s)	Type	Purpose
idx_agent_memory_22_agent_memory	22_agent_memory	agent_memory_	embedding_vector_cosine_ops	HNSW m=16, ef_construction=64	Sub-linear cosine ANN search for memory retrieval (<100ms target)
idx_agent_mem	22_agent_memo	agent_memory_	(entity_type)	B-tree	Entity-type pre-filter before ANN search

8.5 5. Running Migrations

8.5.1 Applying migrations

```
# First-time setup (creates the procurement_agent database)
cd database
export $(grep -v '^#' .env | xargs)
python setup_database.py --create-db

# Idempotent re-run (safe to run on an existing database)
python setup_database.py

# Drop everything and recreate from scratch
python setup_database.py --drop-all --create-db

# Continue past already-existing objects without error
python setup_database.py --continue-on-exists
```

The setup script executes all .sql files in database/sql/ in lexicographic order (the NN_ numeric prefix is the dependency ordering). No migration framework (e.g. Alembic) is used; idempotency is achieved at the SQL level.

8.5.2 Adding a new migration

1. Find the next free NN_ prefix. Never skip or reorder existing numbers — the FK dependency chain is implicit in lexicographic order.

- 2. Name the file `NN_descriptive_name.sql` (underscores, lowercase).
- 3. Write all DDL as `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, or inside a `DO $$ ... $$` guard.
- 4. Run `python setup_database.py` to apply. The script is safe to re-run against an existing database.

Never renumber or reorder existing files. The FK chain depends on execution order. Breaking it causes `relation does not exist` errors during a fresh setup.

8.5.3 Verification

```
# Run database integration tests
cd database
pytest test_database.py -v --tb=short -q
```

8.6 6. pgvector Similarity Search

8.6.1 6.1 Agent Memory (agent_memory_vectors)

The orchestrator uses agent memory to apply a time-decayed loyalty delta to composite scores. The write path is fire-and-forget; the read path has a 5-second timeout.

Parameter	Value
Vector dimension	384
Embedding model	BAAI/bge-small-en-v1.5 (fastembed ONNX, ~65 MB, no PyTorch dependency)
Index type	HNSW cosine, m=16, ef_construction=64
ef_search at query time	100 (set per session)
Similarity threshold	>= 0.75 (results below this score are excluded)
Top-k	3 per query
Text stored per row	"{item} ordered from {provider} at {price} {currency} delivery in {hours}h"

Write path:

```
orchestrator._persist_memory() (fire-and-forget asyncio.create_task)
→ POST /normalize/memory/write (data-normalizer :8006)
→ fastembed embed text
→ INSERT INTO agent_memory_vectors
```

Read path:

```
orchestrator._fetch_memory_context(raw_query, limit=3, timeout=5s)
→ POST /normalize/memory/search (data-normalizer :8006)
→ pgvector HNSW ANN cosine query
→ return top-3 results above threshold
```

Example similarity query:

```
-- Set ANN search beam width for this session
SET hnsw.ef_search = 100;

-- Retrieve the top-3 most similar past transactions for a query embedding
SELECT
    vector_id,
    entity_type,
    metadata,
    1 - (embedding_vector <=> $1::vector) AS cosine_similarity
FROM agent_memory_vectors
WHERE entity_type = 'transaction'
    AND 1 - (embedding_vector <=> $1::vector) >= 0.75
ORDER BY embedding_vector <=> $1::vector
LIMIT 3;
-- $1 = float array of length 384 produced by BAAI/bge-small-en-v1.5
```

Memory delta formula applied to composite scores after retrieval:

```
memory_delta(provider) = sum[ 0.03 × exp(-days_old × ln(2) / 90) ]
                        for each past PO from this provider
                        (max 5 orders considered, capped at ±0.10)
```

New providers receive delta = 0. Half-life is 90 days. This keeps the scoring bias bounded and time-decayed.

8.6.2 6.2 BPP Catalog Semantic Cache (bpp_catalog_semantic_cache)

Used by Stage 3 of the IntentParser pipeline to validate that a queried item actually exists in a known BPP catalog before committing to a discovery call.

Parameter	Value
Vector dimension	1536 (spec); actual model all-MiniLM-L6-v2 produces 384 dims
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Index type	HNSW cosine, m=16, ef_construction=64
ef_search at query time	100 (set per session)

Three-zone decision boundary:

Cosine similarity	Zone	Action
>= 0.92	VALIDATED	Item confirmed to exist in BPP catalog
0.75 – 0.91	AMBIGUOUS	Accept with lower confidence; may trigger MCP sidecar probe
< 0.75	CACHE_MISS	Falls through to mcp-sidecar probe at :3000

Note: IntentParser runtime code uses VALIDATED >= 0.85 and AMBIGUOUS >= 0.45. The runtime values take precedence over the SQL comment values.

Two write paths:

Path A – CatalogCacheWriter (on_discover callback):
embed(item_name) → INSERT source='bpp_publish', strategy='item_name_only'

Path B – MCPResultAdapter (successful MCP probe):
embed(item_name + " | " + join(descriptions)) → INSERT source='mcp_feedback', strat

Path B entries carry richer embedding context. hit_count and last_seen_at are updated on cache hit for eviction heuristics.

8.7 7. Redis Schema

Redis 7 is used for two distinct purposes: async Beckn discovery coordination (see [ADR-0001](#)) and ONIX adapter response caching. All entries are ephemeral — no persistent data model.

8.7.1 7.1 Beckn Discovery Pub/Sub

Channel pattern: `beckn_results:{transaction_id}`

Property	Value
Type	Pub/Sub channel (not a key — no TTL, no persistence)
Publisher	beckn-bap-client /on_discover handler — publishes the raw on_discover ONIX callback payload
Subscribers	mcp-sidecar (bap_client.py subscribes before firing POST /discover); orchestrator CallbackCollector
Message format	Raw JSON of the Beckn on_discover callback payload (message.catalogs[] in Beckn v2 flat-resource wire format)
Lifetime	Until the subscriber reads the message; no explicit TTL
Timeout	REDIS_RESULT_TIMEOUT env var (default 15 s); sidecar returns {"found": false, ...} on timeout
Key reuse	Never — transaction_id must be unique per request. Channels are derived from it; reuse delivers one transaction's catalog to a stale subscriber.

Five-step sequence (ADR-0001):

The SUBSCRIBE must be created before the POST /discover is fired. Reversing the order introduces a race between PUBLISH and SUBSCRIBE where the catalog payload is lost.

8.7.2 7.2 Negotiation Engine Channel

Channel pattern: `beckn_on_select_results`

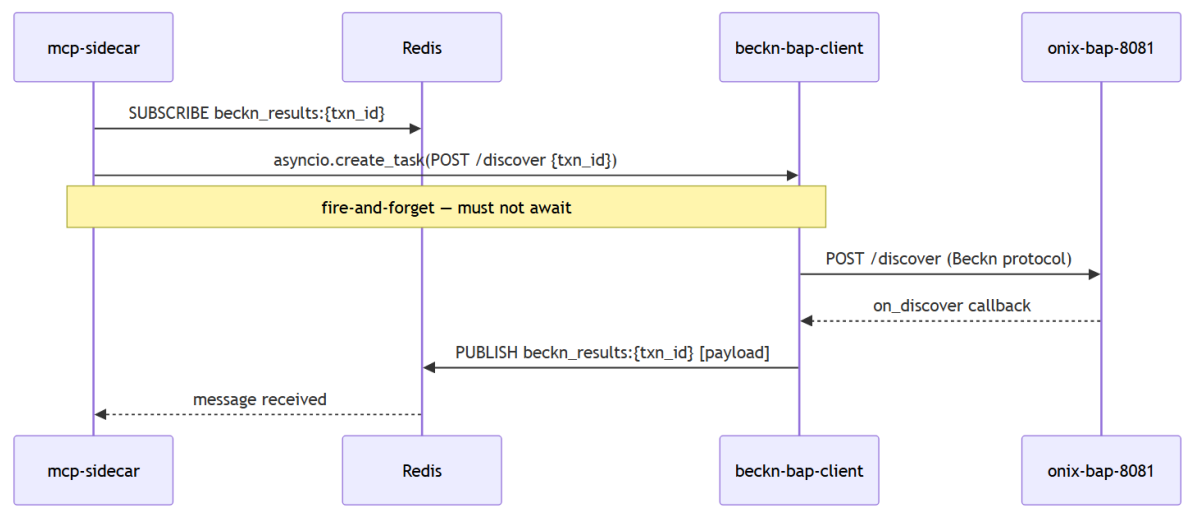


Figure 8.4: DATABASE diagram 4

Property	Value
Type	Pub/Sub channel
Publisher	demo-gateway (:8015) — publishes after SupplierAgent (Claude proxy :8012) returns a supplier response
Subscriber	negotiation_engine (:8004) — OnSelectListener resumes the parked LangGraph buyer graph via graph.ainvoke(Command(resume=payload))
Purpose	Async resume of a LangGraph negotiation state machine parked at wait_for_async_callback
Env var	BECKN_ON_SELECT_CHANNEL (negotiation_engine) and REDIS_ON_SELECT_CHANNEL (demo-gateway) — both default to becn_on_select_results

8.7.3 7.3 ONIX Adapter Response Cache

Property	Value
Purpose	Avoids redundant Beckn network calls; caches on_discover responses
TTL	15 minutes

Property	Value
Publisher / Manager	onix-bap Go process (internal to the fidedocker/onix-adapter image)
Key format	Internal to the ONIX adapter image — not directly accessible

8.7.4 7.4 Redis Connection Reference

Service	Env var	Docker Compose value
beckn-bap-client	REDIS_URL	redis://redis:6379
mcp-sidecar (local, not dockerised)	REDIS_URL	redis://localhost:6379 — must be set as a real shell env var, not only in .env
onix-bap / onix-bpp	REDIS_ADDR	redis:6379
erp-adapter	REDIS_URL	redis://redis:6379
negotiation_engine	REDIS_URL	redis://redis:6379/0
demo-gateway	REDIS_URL	redis://redis:6379/0
orchestrator	REDIS_URL	"" (disabled by default; enable for WebSocket push tracking)

9 Security

This document covers the complete security posture of the Procurement Agent as implemented across Phases 1-3. It distinguishes what is enforced today from what is planned for Phase 4 hardening. For environment variable references, see [Configuration](#). For service topology context, see [Architecture](#).

9.1 1. Authentication Model

9.1.1 1.1 Frontend Authentication

The Next.js frontend (port 3000) enforces authentication via **NextAuth v4** using an exclusive **Keycloak OIDC provider** backed by the Phase Two (phasetwo.io) hosted Keycloak service. There is no stub credentials provider, no local fallback login, and no dev-mode bypass.

```
// frontend/src/lib/auth.ts -- only provider configured
KeycloakProvider({
  clientId:    process.env.KEYCLOAK_CLIENT_ID!,
  clientSecret: process.env.KEYCLOAK_CLIENT_SECRET!,
  issuer:      process.env.KEYCLOAK_ISSUER!,
})
```

All three environment variables use the non-null assertion operator (!). The application throws at startup if any is absent.

Session parameters:

Parameter	Value
Strategy	JWT
Max age	28,800 s (8 hours)

Parameter	Value
Sign-in page	/login
Realm roles claim	realm_access.roles in ID token

Role model:

Role	Capability
admin	Full access; configure thresholds and approval policies
approver	Approve or reject requests above the requester's threshold
requester	Default role; cannot call /commit directly above their own threshold

Role values are read from `realm_access.roles` in the ID token. If no recognized role is present, the session falls back to `requester`. The code also falls back to a manual `base64url` decode of `access_token` when the ID token claim is absent.

Required env vars (see [Configuration](#) for full reference):

Variable	Purpose
NEXTAUTH_URL	Public URL of the Next.js app; must match the Keycloak redirect URI exactly
NEXTAUTH_SECRET	Signs and verifies session JWTs; generate with <code>openssl rand -base64 32</code>
KEYCLOAK_ISSUER	OIDC issuer — Phase Two cloud: <code>https://euc1.auth.ac/auth/realms/procurement-agent</code>
KEYCLOAK_CLIENT_ID	Client ID in the Keycloak realm (procurement-frontend)
KEYCLOAK_CLIENT_SECRET	Client secret from the Keycloak dashboard; must never be committed

Minimum Keycloak client configuration:

Setting	Required value
Client authentication	ON (confidential client)
Standard flow	Enabled
Valid Redirect URIs	<code>\${NEXTAUTH_URL}/api/auth/callback/keycloak</code>
Web origins	<code>\${NEXTAUTH_URL}</code>
Realm roles mapper	Token claim name <code>realm_access.roles</code> , Add to ID token: ON

Warning: No Keycloak realm export or setup guide exists in the repository. Developers cloning the repo cannot authenticate to the frontend without access to the Phase Two tenant (`euc1.auth.ac`, `realm procurement-agent`).

9.1.2 1.2 Internal Service Authentication

Only the `erp-adapter` service enforces inbound authentication. All other inter-service communication relies on Docker bridge network isolation (`beckn_network`).

Boundary	Mechanism
<code>orchestrator → erp-adapter</code>	Bearer token: <code>Authorization: Bearer \${ERP_INTERNAL_TOKEN}</code>
<code>erp-adapter ← vendor webhooks</code>	HMAC-SHA256 dual-secret rotation (see §1.3)
All other service-to-service calls	None — Docker <code>beckn_network</code> bridge only

The default `ERP_INTERNAL_TOKEN` in `docker-compose.yml` is `dev-internal-token-CHANGE_ME`. This must be replaced via environment variable before any deployment exposed to untrusted networks. Phase 4 plan: Kubernetes RBAC service accounts with per-service mTLS.

9.1.3 1.3 ERP Webhook Authentication (HMAC-SHA256)

Inbound vendor webhooks to `erp-adapter` carry a per-vendor HMAC-SHA256 signature:

`X-{Vendor}-Signature: sha256=<hex_digest>`

Example: X-SAP-Signature: sha256=abcdef01...

The adapter verifies the signature using `hmac.compare_digest` (constant-time comparison, preventing timing oracle attacks).

Dual-secret rotation pattern:

Two secrets are accepted simultaneously during key rotation, enabling zero-downtime rotation without a maintenance window.

Variable	Purpose
<code>SAP_WEBHOOK_HMAC_SECRET</code>	Primary active secret for SAP webhooks
<code>SAP_WEBHOOK_HMAC_SECRET_NEXT</code>	New secret being rolled in; empty when no rotation is in progress
<code>ORACLE_WEBHOOK_HMAC_SECRET</code>	Primary active secret for Oracle webhooks
<code>ORACLE_WEBHOOK_HMAC_SECRET_NEXT</code>	New Oracle secret during rotation
<code>SELLER_WEBHOOK_HMAC_SECRET</code>	Primary secret for inbound seller push webhooks (orchestrator)

Zero-downtime rotation procedure:

1. Set `{VENDOR}_WEBHOOK_HMAC_SECRET_NEXT` to the new secret value and deploy.
2. Configure the vendor's outbound webhook delivery to sign with the new secret.
3. Verify that new webhook deliveries arrive and are accepted.
4. Promote `_NEXT` to the primary variable; clear `_NEXT`.

9.1.4 1.4 Claude OpenAI Proxy Authentication

The `claude_openai_proxy` service (port 8012) implements `BearerAuthMiddleware` on all `/v1/*` paths. The auth key is controlled by `CLAUDE_PROXY_KEY`.

Path	Auth required
<code>GET /healthz</code>	No
<code>GET /v1/models</code>	Yes — Authorization: Bearer <code>\${CLAUDE_PROXY_KEY}</code>
<code>POST /v1/chat/completions</code>	Yes — Authorization: Bearer <code>\${CLAUDE_PROXY_KEY}</code>

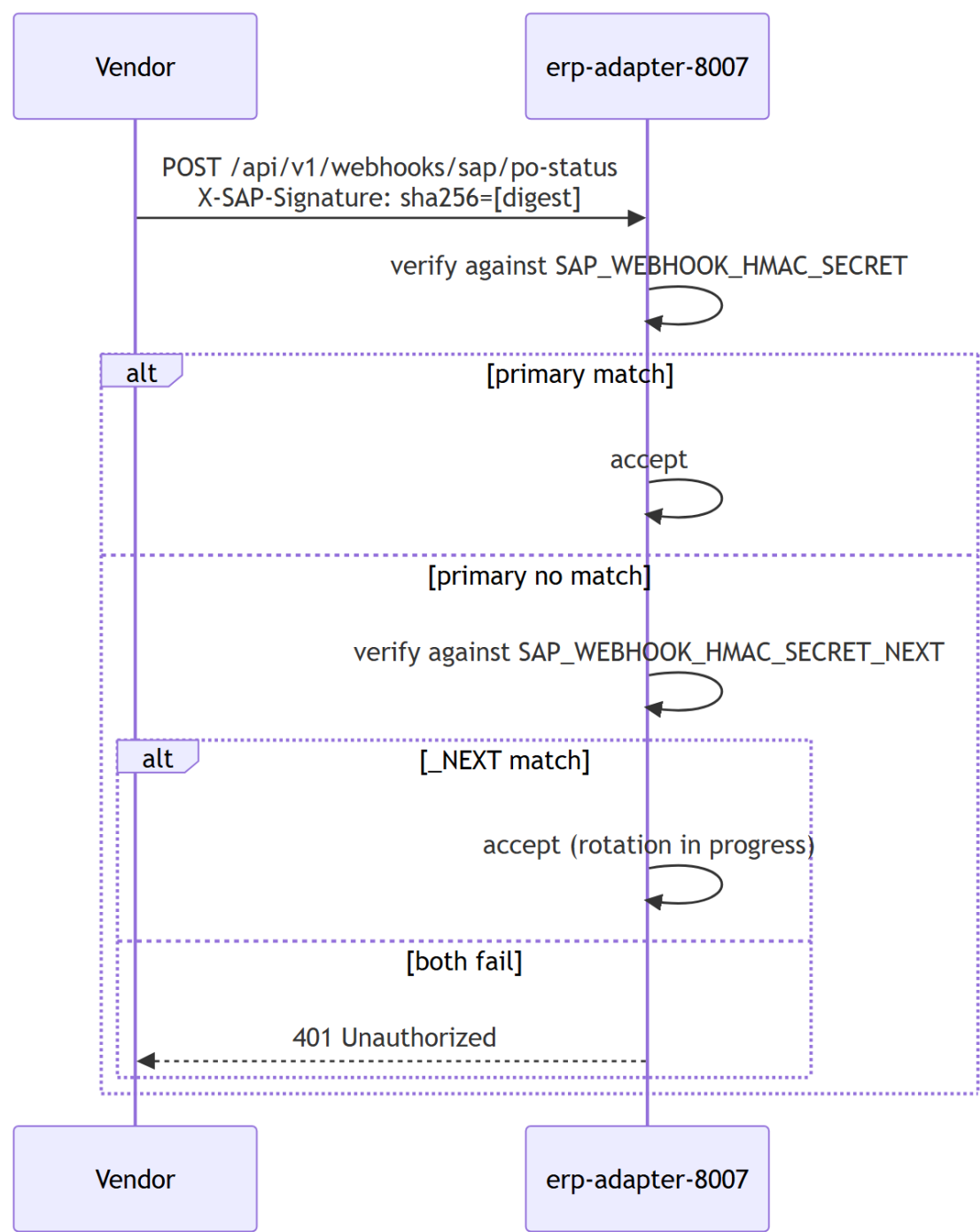


Figure 9.1: SECURITY diagram 1

If `CLAUDE_PROXY_KEY` is empty, authentication is **disabled** with a startup warning. In development, the proxy binds `127.0.0.1`. The production systemd unit (`claude-proxy.service`) binds `0.0.0.0` and relies on a UFW rule restricting port 8012 to loopback and the Docker bridge subnet `172.16.0.0/12`.

9.2 2. Beckn Protocol Security

9.2.1 2.1 ED25519 Request Signing

All outbound Beckn traffic from the BAP passes through the **onix-bap** Go adapter (port 8081), which signs every request with **ED25519**. The `beckn-bap-client` service never calls a BPP URL directly; all traffic is mediated by ONIX.

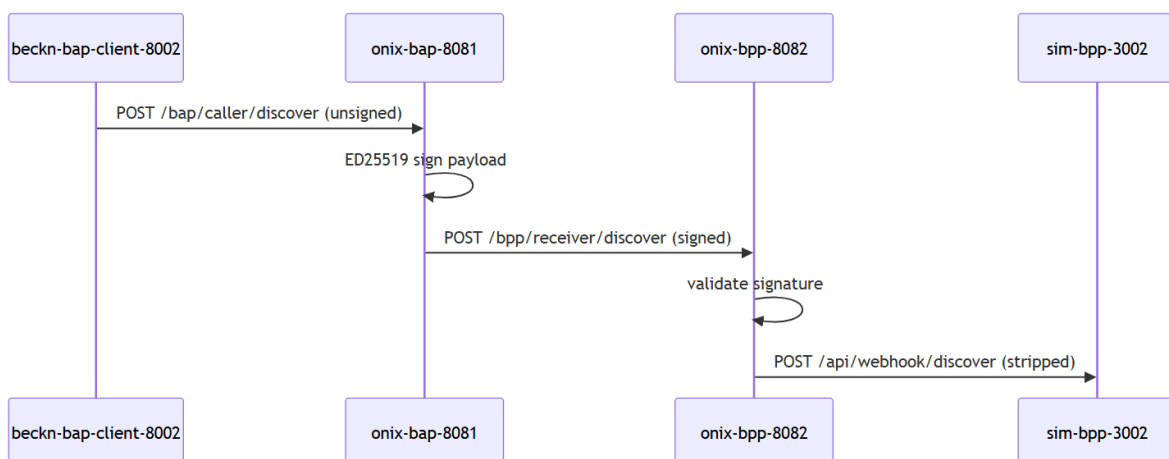


Figure 9.2: SECURITY diagram 2

The two-adapter model (BAPCaller+ BAPReceiver, BPPCaller+ BPPReceiver) is defined in `config/generic-routing-*.yaml`. The signing pipeline for the caller is: `addRoute` → `sign` → `validateSchema`. The receiver pipeline is: `validateSign` → `addRoute` → `validateSchema`.

Rule: Never POST directly to a BPP. All Beckn traffic must go through `onix-bap:8081`. The test suite enforces that `select_url` always contains `caller`.

9.2.2 2.2 Schema Validator Pin at Commit d43ec30d

The ONIX schema validator (`schemav2validator.so`) is **pinned to commit d43ec30d** of the `fidedocker/onix-adapter` image. Later commits introduced a `$ref` resolution bug in `Signa-`

tureHeader and AckSignatureHeader that causes all signed Beckn messages to fail validation even when the signature and payload are correct.

Do not upgrade the ONIX schema validator without first running a full end-to-end signing test covering discover → select → init → confirm → status with ED25519-signed payloads.

The validator binary is compiled into the .so plugin and cannot be patched at runtime.

The validator enforces several contract-level constraints that all services must respect:

Constraint	Detail
Contract.status.code valid values	DRAFT, ACTIVE, CANCELLED, COMPLETE — CONFIRMED is rejected with a 400 NACK
Contract additional properties	additionalProperties: false; only nine keys allowed
Buyer info placement	participants[role=buyer] — not inline in Contract
Fulfillment placement	performance[] — not inline in Contract
Payment placement	settlements[] — not inline in Contract

9.2.3 2.3 Testnet Key Material

The current configuration uses **testnet sandbox ED25519 key pairs** from the Beckn Developer Program. These key pairs are checked into the repository in `config/generic-routing-BAPCaller.yaml` and `config/generic-routing-BAPReceiver.yaml`.

Production key rotation procedure (from `config/README.md`):

1. Generate a new ED25519 key pair using the Beckn key generation utility.
2. Register the public key in the target Beckn network's registry.
3. Update `config/generic-routing-BAPCaller.yaml` with the new private key and key ID.
4. Restart `onix-bap`. The new key takes effect immediately; no rolling restart is needed.

9.2.4 2.4 DeDi Registry Bypass

All four ONIX routing YAML files use `targetType: url` instead of `targetType: bpp / targetType: bap`. This bypasses DeDi registry lookup and routes directly to configured URLs, enabling a complete Beckn lifecycle flow inside the Docker network without a live registry.

This bypass must be reverted before connecting to any live Beckn network. Revert by setting `targetType`: `bpp/bap` and providing real registry entries in the YAML files. No application code changes are required.

9.3 3. Data Sovereignty

In the default deployment, all data, model inference, and vector search remain on the local host. No procurement data leaves the machine without explicit opt-in.

Component	Provider	Data leaves host?
Intent classification (Stage 1)	qwen3:8b via local Ollama	No
BecknIntent extraction (Stage 2)	qwen3:8b / qwen3:1.7b via local Ollama	No
BPP catalog semantic cache	all-MiniLM-L6-v2 (sentence-transformers, local ONNX)	No
Agent memory embeddings	BAAI/bge-small-en-v1.5 (fastembed ONNX, local)	No
Vector search	pgvector (PostgreSQL on host)	No
Stage 3 broadening fallback	claude-sonnet-4-6 via Anthropic API (opt-in)	Yes — requires <code>ANTHROPIC_API_KEY</code>
SupplierAgent in demo flows	claude-sonnet-4-6 via <code>claude_openai_proxy:8012</code>	Yes — bills the host's Claude account

9.3.1 3.1 Claude Stage-3 Broadening Fallback

`ANTHROPIC_API_KEY` is **unset by default**. When unset, `IntentParser` skips the Claude broadening step and the recovery flow falls through to stub functions (`log_unmet_demand`, `notify_buyer_no_stock`, `trigger_open_rfq_flow`).

When `ANTHROPIC_API_KEY` is set, only the broadening prompt is sent to the Anthropic API. The prompt contains the original procurement query text but no buyer identity, pricing, or supplier data. The model used is controlled by `CLAUDE_MODEL` (default: `claude-sonnet-4-6` in `IntentParser/config.py`).

9.3.2 3.2 Claude OpenAI Proxy Loopback Binding

The `claude_openai_proxy` service invokes the host-installed `claude` CLI (`CLAUDE_PROXY_BINARY_PATH`) for each inference request. Every request bills the host's Claude account (`~/.claude/.credentials.json`). The service binds `127.0.0.1` by default; the production systemd unit (`claude-proxy.service`) binds `0.0.0.0` and relies on UFW to restrict access to loopback and the Docker bridge subnet `172.16.0.0/12`.

9.4 4. Input Validation

9.4.1 4.1 BecknIntent Field Validation

All NL-derived intent data passes through `shared.models.BecknIntent` (Pydantic v2) before entering the pipeline. Validators use `@field_validator` decorators (Pydantic v2 — `@validator` is deprecated and not used).

Canonical encoding rules enforced at this layer:

Field	Enforcement
quantity	Must be <code>> 0</code> ; raises <code>ValidationError</code> otherwise
delivery_timeline	Must be <code>> 0</code> if provided; value is int hours (not ISO 8601 duration)
location_coordinates	String <code>"lat,lon"</code> decimal — not city names
budget_constraints	Typed <code>BudgetConstraints(max, min)</code> object — not a raw string

9.4.2 4.2 ONIX Schema Validation

Every Beckn message (outbound and inbound) is validated against the Beckn v2.0.0 JSON schema by the ONIX adapter before forwarding. Invalid messages are rejected with a 400 NACK before reaching any application service. See §2.2 for the pin and the constraints it enforces.

9.4.3 4.3 MCP Sidecar Never-Throw Contract

The MCP sidecar’s `search_bpp_catalog` tool never raises a JSON-RPC error. All failure paths — network timeouts, Redis unavailability, BPP returning no results — return a structured response:

```
{"found": false, "items": [], "probe_latency_ms": N}
```

This prevents unhandled exceptions from propagating to the LLM-side caller in IntentParser Stage 3 and enables safe graceful degradation.

9.4.4 4.4 Database Error Middleware

`data-normalizer` wraps all PostgreSQL errors in `db_error_middleware`, mapping them to structured HTTP responses that do not expose raw database messages to callers:

PostgreSQL error	HTTP response
UniqueViolationError	409 {"error": "duplicate"}
ForeignKeyViolationError	409 {"error": "fk_violation"}
CheckViolationError	422
NotNullViolationError	422
Other PostgresError	500

9.4.5 4.5 ERP Budget Gate Fail-Closed

The `ERPbudgetcheck` (`POST /api/v1/budget/check`) is configured with `ERP_BUDGET_CHECK_REQUIRED` (default `false` in `docker-compose.yml`, should be `true` in production). When `true`, if the `erp-adapter` is unreachable or exceeds the 800 ms timeout (`ERP_BUDGET_CHECK_TIMEOUT_MS`), the orchestrator’s `/commit` path **rejects** the purchase order rather than allowing it to proceed. This prevents orders from bypassing the budget gate due to infrastructure failures.

Production requirement: Set `ERP_BUDGET_CHECK_REQUIRED=true`. The Docker Compose default of `false` is a development convenience only.

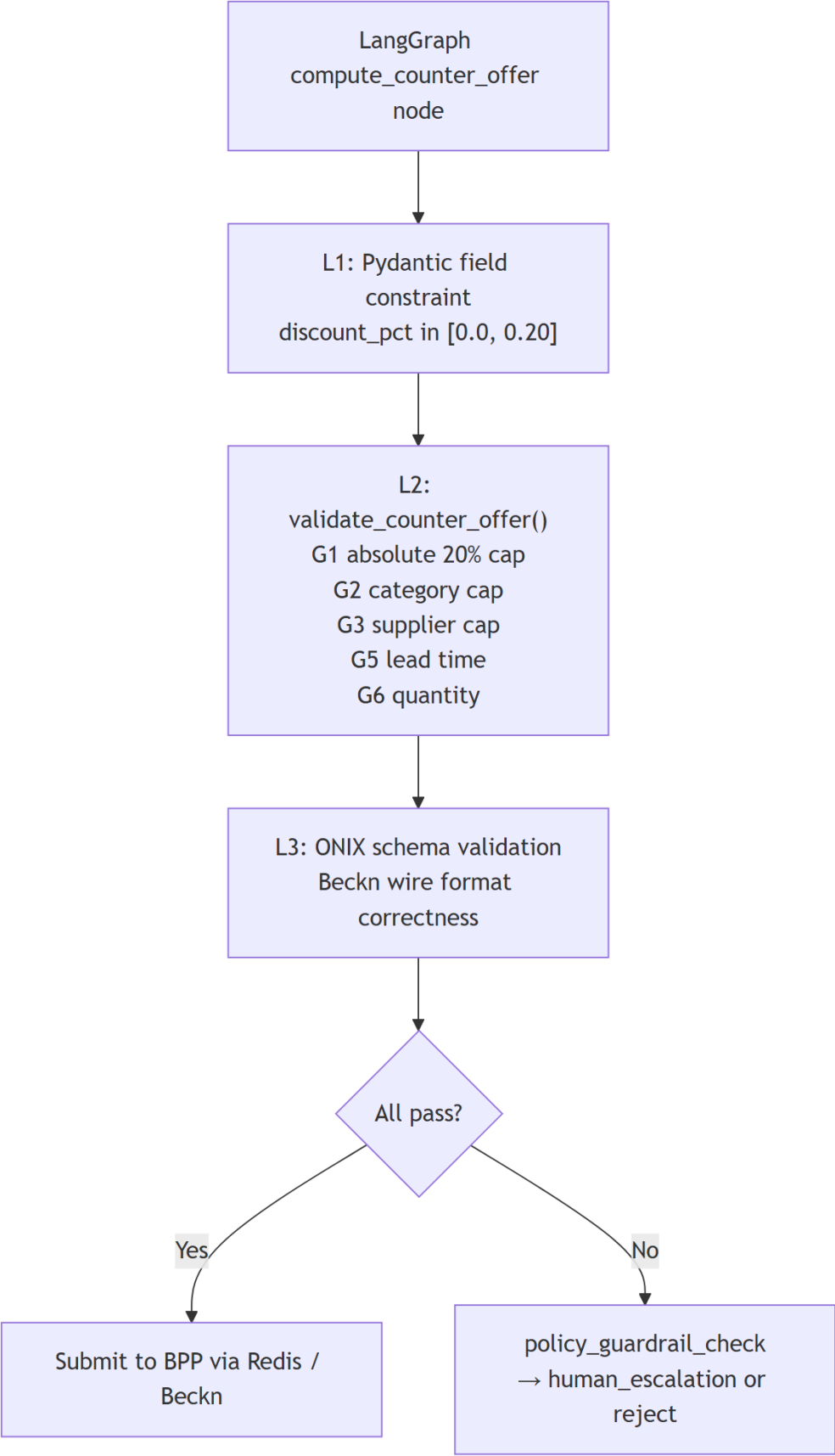


Figure 9.3: SECURITY diagram 3

9.4.6 4.6 Negotiation Engine Guardrails

The negotiation engine enforces a three-layer guardrail stack to prevent the LLM from agreeing to out-of-policy terms:

The hard cap is **20% maximum discount** regardless of category profile or LLM output. This constraint is enforced at both the Pydantic model level (L1) and the imperative validation function (L2).

9.5 5. Secrets Management

9.5.1 5.1 Env-Var Pattern

All service secrets are loaded from environment variables via module-level `config.py` files (`IntentParser/config.py`, `Bap-1/src/config.py`, `services/*/config.py`). Inline `os.getenv(...)` calls in application logic are forbidden. Exceptions: `REDIS_URL` and `REDIS_RESULT_TIMEOUT` in `services/mcp-sidecar/bap_client.py` and `services/beckn-bap-client/src/handler.py` use `os.getenv()` directly because they predate the Pydantic Settings migration — they must be set as real process environment variables, not just in `.env` files.

9.5.2 5.2 .env Files

`.env` files are gitignored. Each service's `.env.example` documents all variables with safe placeholder values and serves as the onboarding template. The root `.env` is the canonical source for the Docker stack (loaded by `docker-compose.yml`).

9.5.3 5.3 Secrets That Must Be Rotated Before Any Production Deployment

The variables below have no safe default and will leave the system insecure or non-functional if not replaced:

Variable	Service	Risk if not replaced
NEXTAUTH_SECRET	frontend	Session JWTs are unsigned; any JWT is accepted
KEYCLOAK_CLIENT_SECRET	frontend	OIDC flow breaks; no user can authenticate

Variable	Service	Risk if not replaced
KEYCLOAK_ISSUER	frontend	Auth goes to the Phase Two dev tenant instead of your realm
ERP_INTERNAL_TOKEN (dev-internal-token-CHANGE_ME)	orchestrator, erp-adapter	Any caller can hit /api/v1/* on erp-adapter
SELLER_WEBHOOK_HMAC_SECRET (dev-seller-hmac-CHANGE_ME)	orchestrator	Forged seller webhooks are accepted
SAP_WEBHOOK_HMAC_SECRET (dev-sap-hmac-CHANGE_ME)	erp-adapter, erp-mock	Forged SAP webhooks are accepted
ORACLE_WEBHOOK_HMAC_SECRET (dev-oracle-hmac-CHANGE_ME)	erp-adapter, erp-mock	Forged Oracle webhooks are accepted
DB_PASSWORD (postgres123)	data-normalizer, analytics, erp-adapter	Trivial password grants full DB access
CLAUDE_PROXY_KEY	claude_openai_proxy	Any client can trigger Claude CLI invocations that bill the host account
BAP_ID (bap.example.com)	beckn-bap-client	Placeholder — must be the registered BAP identifier on the target Beckn network
BAP_URI (http://localhost:8000/beckn)	beckn-bap-client	Localhost URI; ONIX cannot route callbacks back correctly from a remote network

9.6 6. Known Security Gaps

The following gaps are confirmed in the Phases 1-3 implementation. All are either deferred to Phase 4 or carry a documented workaround.

Gap	Severity	Current Mitigation	Phase 4 Plan
No authentication on internal HTTP APIs (intention-parser :8001, becn-bap-client :8002, data-normalizer :8006, catalog-normalizer :8005, comparative-scoring :8003, analytics :8009)	Medium	Docker becn_network bridge; services are not published to the public internet	Kubernetes NetworkPolicy + service mesh mTLS; API Gateway (Kong) RBAC on every request
Testnet ED25519 key pairs committed to config/generic-routing-*.yaml	High	Development only; testnet keys have no value outside the sandbox	Follow the 3-step key rotation procedure in config/README.md before any real deployment
dev-*.CHANGE_ME HMAC secrets in docker-compose.yaml (SAP, Oracle, seller)	High	Docker bridge isolation	Replace with real secrets via environment variables before external deployment
ERP_INTERNAL_TOKEN=d internal-token-CHANGE_ME	High	Docker bridge isolation	Rotate via env var before deployment
Live Keycloak client secret potentially in frontend/.env.local (gitignored but in working tree)	High	File is gitignored; must not be committed	Rotate KEY-CLOAK_CLIENT_SECRET in the Phase Two admin console if the file was ever shared
No Keycloak realm export or setup guide in the repository	Medium	Developers must have Phase Two tenant access	Document realm configuration and export realm JSON
mTLS to SAP / Oracle not activated in erp-adapter	Low	SSL context hook is wired but not called; vendor traffic is over plain HTTP in development	Activate per-vendor mTLS in Phase 4 production ERP connectivity
No container image scanning (Trivy)	Low	No containers pushed to a public registry in Phases 1-3	Trivy in CI pipeline; block builds on HIGH/CRITICAL CVEs
Secrets passed as plain env vars in docker-compose.yaml	Low	Local Docker Compose; no shared orchestration platform	Kubernetes Secrets + KMS in Phase 4

Gap	Severity	Current Mitigation	Phase 4 Plan
ERP_BUDGET_CHECK_REQ in <code>docker-compose.yml</code> (fail-open)	Medium	Development default; budget overruns are possible if erp-adapter fails	Set <code>true</code> in all non-development environments
<code>LOG_PAYLOADS=false</code> default in erp-adapter; must not be set <code>true</code>	Low	Default is safe; <code>true</code> would log full PO and budget payloads containing PII	No change needed — flag must remain <code>false</code> in production
Bap-1 hardcoded test users with cleartext passwords	Low	Bap-1 is a Phase 1 integration test harness, not in the production service path	Bap-1 test frontend is not user-exposed; resolved when Bap-1 is retired

9.7 7. Security Checklist for New Features

Before merging any feature that introduces a new service, API endpoint, secret, or external dependency, verify all of the following:

- ☐ **No inline secrets.** All secrets loaded from environment variables via the service's `config.py`. No `os.getenv(...)` calls outside `config.py` (except the documented exceptions in `mcp-sidecar/bap_client.py` and `beckn-bap-client/handler.py`).
- ☐ **No new `.env` values committed.** Updated `.env.example` only; actual secret values in `.env` (git-ignored).
- ☐ **No direct BPP calls.** All Beckn traffic routes through `onix-bap:8081`. `select_url` must contain caller.
- ☐ **No action name in ONIX target URLs.** ONIX appends the action automatically; including it in the target URL causes a 404 double-path.
- ☐ **No `Contract.status.code` = `"CONFIRMED"`.** The valid enum is `DRAFT` | `ACTIVE` | `CANCELLED` | `COMPLETE`.
- ☐ **Pydantic v2 validators on new `BecknIntent` fields.** Use `@field_validator`; never `@validator`.
- ☐ **MCP tools return structured failure, not exceptions.** Failure paths must return `{"found": false, ...}`.
- ☐ **No `time.sleep()` in async code.** Use `await asyncio.sleep(0)` to yield to the event loop.

- ❑ **No `await` on the `/discover` POST in the MCP sidecar.** It must be `asyncio.create_task(...); await` reintroduces the deadlock described in ADR-0001.
- ❑ **New SQL migrations use the next free `NN_` prefix.** Do not reorder or renumber existing files. Migrations must be idempotent (`IF NOT EXISTS`).
- ❑ **No reuse of `transaction_id`.** Beckn channels (`beckn_results:{transaction_id}`) are derived from it; reuse delivers a stale catalog to a waiting subscriber.
- ❑ **`ERP_BUDGET_CHECK_REQUIRED=true` in any non-development deployment.** The Docker Compose default of `false` is unsafe outside a local dev environment.
- ❑ **`CLAUDE_PROXY_KEY` set before exposing `claude_openai_proxy` beyond loopback.** Empty key disables authentication.
- ❑ **New webhook endpoints use `hmac.compare_digest` for signature verification.** Never use `==` for HMAC comparison.
- ❑ **PII not included in LLM prompts.** The Stage-3 Claude broadening prompt must contain only the procurement query text, not buyer identity, pricing, or supplier data.

For contribution guidelines and the PR process, see [Architecture](#).

10 Deployment Guide

This guide covers the complete deployment topology for all phases: the current Docker Compose stack used in Phases 1–3, the host-side services that run outside Docker, the MLOps sub-stack, database migration procedure, observability setup, and the Phase 4 Kubernetes plan.

For configuration variable reference (full env var tables per service), see [Configuration Reference](#). For the Beckn async discovery architecture, see [Architecture](#) and [ADR-0001](#).

10.1 1. Current Deployment: Docker Compose (Phases 1–3)

10.1.1 1.1 Stack Overview

18 containers run on the `beckn_network` Docker bridge network. Three services run on the host outside Docker (see [Section 2](#)). All inter-container communication uses Docker DNS: containers reference each other by service name (e.g. `http://data-normalizer:8006`). Host-side services are reached from Docker containers via `host.docker.internal`.

Two separate PostgreSQL instances:

Instance	Where it runs	Host port	Database name	Used by
Main procurement DB	Host (Docker container procurement-postgres, or native install)	5432	procurement_agent	data-normalizer, analytics, erp-adapter, notification-dispatcher, IntentParser
Negotiation DB	postgres service inside beckn_network	55432	negotiation	negotiation-engine only

Docker containers reach the main procurement DB at `host.docker.internal:5432`. They reach the negotiation DB at `postgres:5432` via Docker DNS.

10.1.2 1.2 Container Reference

Note on port assignments: - orchestrator maps host ports **8000 and 8004** to container port 8004. Port 8000 exists because `frontend/.env.local` defaults `NEXT_PUBLIC_API_BASE_URL=http://localhost:8000`. - `negotiation-engine` maps to host port **18004** (not 8004) to avoid collision with orchestrator. Docker-internal traffic still uses `negotiation-engine:8004`. - `notification-dispatcher` has **no published host port**; it is reachable only within `beckn_network`.

Container	Image / Build context	Host port	Container port	Key env vars (docker-compose values)	Health check	Depends on
intention-parser	./services/intention-parser	8004	8001	OLLAMA_URL=http://host.docker.internal:11434; COM- PLEX_MODEL=qwen3:1.7b; SIM- PLE_MODEL=qwen3:1.7b	HTTP /health	ollama (soft)
beckn-bap-client	./services/bap-client (repo root context)	8002	8002	BAP_ID, BAP_URI, ONIX_URL=http://host.docker.internal:8081, RE- DIS_URL=redis; CATALOG_NORMALIZER=log-normalizer	HTTP /health	redis, catalog-normalizer
catalog-normalizer	./services/catalog-normalizer (repo root context)	8005	8005	OLLAMA_URL=http://host.docker.internal:11434	HTTP /health	
comparative-scoring	./services/comparative-scoring	8003	8003	PREDICTION_API:8004, SCOR- ING_FALLBACK	HTTP /health	on-

Container	Image / Build context	Host port	Container port	Key env vars (docker-compose values)	Health check	Depends on
data-normalizer	./services/data-normalizer (repo root context)	8006	8006	DB_HOST=host DB_PASSWORD	HTTP /health	Host, PostgreSQL (soft)
analytics	./services/	8009	8009	DB_HOST=hos DB_PASSWORD	HTTP /health	postgres (healthy)
orchestrator	./services/	8004	8004	ERP_BUDGET_CHECK_REQUIRE ERP_BUDGET_CHECK_ENABLED ERP_INTERNAL_TOKEN	HTTP /health	Interface parser, Detku, app-client, comparative-scoring, data-normalizer, analytics, erp-adapter, demo-gateway, kafka
erp-adapter	./services/adapter (repo root context)	8007	8007	ERP_VENDORS ERP_INTERNAL_RE- DIS_URL=red KAFKA_BOOTSTRAP	HTTP /health	redis, erp-mock, kafka
erp-mock	./services/erp-mock (repo root context)	8008	8008	MOCK_SCENARIO WEBHOOK_TARGET_URL	HTTP /health	— http://erp-adapter:8007
redis	redis:alpine	6379	6379	—	redis-cli ping	—
kafka	apache/kafka	9092	9092	KAFKA_AUTO_CREATE_TOPICS_ENABLE=true; KRaft mode (no Zookeeper); cluster ID 5L6g3nShT- eMCtK-- X86sw	broker-api-versions.sh (interval 10 s, start_period 30 s)	—

Container	Image / Build context	Host port	Container port	Key env vars (docker-compose values)	Health check	Depends on
notification-dispatcher	./services/notification-dispatcher (repo root context)	(none)	—	KAFKA_BOOTSTRAP_SERVERS= KAFKA_TOPIC=	—	kafka (healthy)
onix-bap	fidedocker/onix-bap-adapter (linux/amd64)	8081	8081	ONIX_ROUTING_YAMLS bind-mounted from ./config	HTTP health	redis (healthy)
onix-bpp	fidedocker/onix-bpp-adapter (linux/amd64)	8082	8082	ONIX_ROUTING_YAMLS bind-mounted from ./config	HTTP health	redis (healthy)
sim-bpp	./services/sim-bpp	3002	3002	SIM_BPP_AUTO_HTTP_ADVANCE=true; /health ONIX_BPP_CALLER=http://onix-bpp:8082/bpp/caller; KAFKA_BOOTSTRAP_SERVERS=kafka:9092	HTTP health	kafka (healthy)
postgres	pgvector/pgvector	5432	5432	POSTGRES_DB_SCHEMA auto-init from database/sql	pg_isready	—
negotiation-engine	./services/negotiation-engine	18004	18004	NEGOTIATION_ENGINE_OPENAI_BASE_URL=http://host.docker.internal:8081/health REDIS_URL=redis://redis:6379/0	HTTP health	postgres (healthy), redis (healthy)
demo-gateway	./services/demo-gateway (repo root context)	8015	8015	OLLAMA_BASE_URL=http://host.docker.internal:11434/health	HTTP health	negotiation-engine (healthy), redis (healthy)

⚠ Notable docker-compose overrides that differ from code defaults:

Service	Variable	docker-compose value	Code default	Effect
intention- parser	COMPLEX_MODEL	qwen3:1.7b	qwen3:8b	Two-tier complexity routing is disabled in Docker; all queries use qwen3:1.7b. Full routing only works when IntentParser runs locally.
sim-bpp	SIM_BPP_AUTO_ADV	true	false	Confirmed orders auto-advance through ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED every 5 s. Running sim-bpp locally without Docker requires setting this explicitly.
orchestrator	ERP_BUDGET_CHECK_REQUIRED	false	true	Budget gate is fail-open in the Docker stack: ERP connectivity errors do not block /commit. Set true for production to make it fail-closed.

10.1.3 1.3 Key Bind Mounts

Container	Host path	Container path	Purpose
intention-parser	./IntentParser	/app/IntentParser	Live-mount IntentParser package
intention-parser	./shared	/app/shared	Shared Pydantic models
catalog-normalizer	./CatalogNormalizer	/app/CatalogNormalizer	Live-mount CatalogNormalizer package

Container	Host path	Container path	Purpose
catalog-normalizer	./shared	/app/shared	Shared Pydantic models
data-normalizer	./DataNormalizer	/app/DataNormalizer	Live-mount DataNormalizer library
data-normalizer	./shared	/app/shared	Shared Pydantic models
sim-bpp	./services/sim-bpp/catalog.json	/app/catalog.json	Hot-reload BPP catalog without rebuilding the container
onix-bap	./config	/app/config	ONIX routing YAML files
onix-bpp	./config	/app/config	ONIX routing YAML files
postgres	./database/sql/20_ne	/docker-entrypoint-initdb.d/01_negotiat	Auto-init negotiation schema on first start
postgres	postgres_data (named volume)	/var/lib/postgresql/data	Negotiation DB persistence across restarts

10.1.4 1.4 Common Operations

Start the full stack:

```
docker compose up -d
```

Start only Beckn discovery infrastructure (fastest for IntentParser and BAP development):

```
docker compose up -d redis onix-bap onix-bpp sim-bpp
```

Tail logs for a single service:

```
docker compose logs -f becn-bap-client
```

Restart and rebuild a single container:

```
docker compose up -d --build catalog-normalizer
```

Teardown — stop all containers and remove them, keep named volumes (preserves negotiation DB data):

```
docker compose down
```

Full teardown including all volumes (destructive — deletes negotiation DB data):

```
docker compose down -v
```

10.1.5 1.5 Deployment Topology Diagram

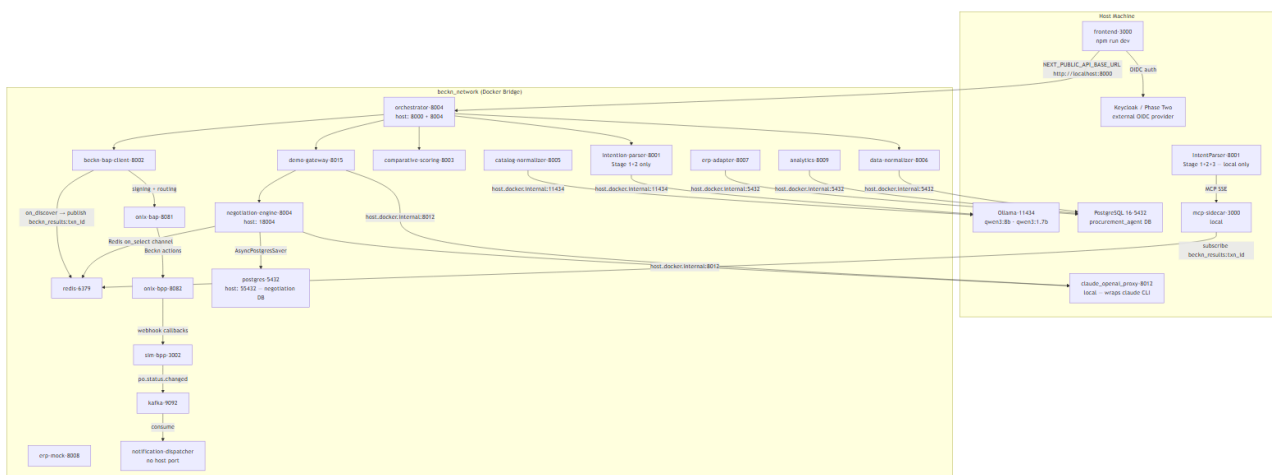


Figure 10.1: DEPLOYMENT diagram 1

10.2 2. Services That Run Outside Docker

Three services run on the host and are not in `docker-compose.yml`. Docker containers reach them via `host.docker.internal`. All three require the `infosys_project` conda environment.

```
conda activate infosys_project
```

10.2.1 2.1 IntentParser API (: 8001)

The full three-stage pipeline — Stage 1 (LLM intent classification) → Stage 2 (BecknIntent extraction) → Stage 3 (pgvector ANN + mcp-sidecar BPP validation) — only operates when IntentParser runs locally.

The `intention-parser` Docker container runs only Stages 1+2, and collapses both model tiers to `qwen3:1.7b` via the `COMPLEX_MODEL` override.

```
# Terminal 1
conda activate infosys_project
cd IntentParser
uvicorn api:app --port 8001 --reload
```

Required env vars:

Variable	Purpose
OLLAMA_URL	Ollama base URL; default <code>http://localhost:11434/v1</code>
COMPLEX_MODEL	Large model for Stage 1 + complex Stage 2; default <code>qwen3:8b</code>
SIMPLE_MODEL	Small model for short queries; default <code>qwen3:1.7b</code>
DB_HOST	PostgreSQL host for Stage 3 pgvector cache; default <code>localhost</code>
DB_PASSWORD	PostgreSQL password; empty is unsafe in production
ANTHROPIC_API_KEY	Optional. Enables Claude Sonnet 4.6 as Stage 3 broadening fallback. Without it Stage 3 uses regex-only broadening.

Pull required Ollama models before first run:

```
ollama pull qwen3:8b
ollama pull qwen3:1.7b
```

10.2.2 2.2 MCP Sidecar (:3000)

The `mcp-sidecar` implements the Redis Pub/Sub subscriber side of [ADR-0001](#). It subscribes to `beckn_results:{txn_` **before** firing the `/discover` POST so it never misses the callback. All failures return `{"found": false, "items": [], "probe_latency_ms": elapsed}` — the service never throws (see [Configuration Reference](#) for full contract).

```
# Terminal 2
conda activate infosys_project
cd services/mcp-sidecar
export BAP_API_KEY="any-value-in-dev"
uvicorn server:app --port 3000
```

Important: BAP_API_KEY, REDIS_URL, and REDIS_RESULT_TIMEOUT are read via `os.getenv()` in `bap_client.py`, not through the Pydantic Settings model. They must be present as real process environment variables, not only in a `.env` file.

Variable	Default	Notes
BAP_API_KEY	(none)	Mandatory. Service refuses to start without it. Any non-empty string works in dev.
REDIS_URL	redis://localhost:6379	Must be a real env var, not only <code>.env</code> .
REDIS_RESULT_TIMEOUT	15	Seconds to wait on the Redis channel before returning <code>found: false</code> . This is the primary latency ceiling for discovery probes. Must be a real env var.
BAP_CLIENT_URL	http://localhost:8002	URL of becn-bap-client's <code>/discover</code> endpoint.
RANKING_MIN_SIMILARITY	0.30	Cosine similarity floor; items below this score are filtered from results.

10.2.3 2.3 Claude OpenAI Proxy (: 8012)

A FastAPI service that wraps the host-installed `claude` CLI as an OpenAI-compatible `/v1/chat/completions` endpoint. Required by `negotiation-engine` and `demo-gateway` (both configured with `NEGOTIATION_OPENAI_BASE_URL=http://host.docker.internal:8012/v1`). It must bind to `0.0.0.0` (not `127.0.0.1`) so Docker bridge containers can reach it via `host.docker.internal:8012`.

Development (loopback only — does not serve Docker containers):

```
# Terminal 3
export CLAUDE_PROXY_KEY="any-dev-string"
uvicorn services.claude_openai_proxy.main:app --host 127.0.0.1 --port 8012
```

Persistent with Docker access (systemd user service — recommended):

```
cp services/claude_openai_proxy/claude-proxy.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now claude-proxy

# Allow the service to persist after logout (one-time, requires sudo):
sudo loginctl enable-linger $USER

# Tail logs:
journalctl --user -u claude-proxy -f
```

The systemd unit binds to 0.0.0.0:8012. A UFW rule documented in claude-proxy.service restricts access to loopback plus the Docker subnet 172.16.0.0/12.

Variable	Default	Notes
CLAUDE_PROXY_BINARY_PATH	/home/carbaje/.local/bin/claude-proxy	Must be updated per machine.
CLAUDE_PROXY_KEY	"" (auth disabled)	Empty disables auth with a startup warning.
CLAUDE_PROXY_HOST	127.0.0.1	Set 0.0.0.0 for Docker bridge access (the systemd unit sets this).
CLAUDE_PROXY_MAX_CONCURRENC	2	Max concurrent claude -p subprocesses; excess requests queue.
CLAUDE_PROXY_TIMEOUT_S	120.0	Hard timeout per request; CLI subprocess is killed if exceeded.
CLAUDE_PROXY_DISABLE_TOOLS	true	Disallows interactive tools in -p mode to prevent permission hangs.

OpenAI → Claude model mapping:

OpenAI name received	Claude alias used
gpt-4o	sonnet
gpt-4o-mini	haiku
gpt-4-turbo, gpt-4	sonnet, opus
gpt-3.5-turbo	haiku
claude-3-5-sonnet	sonnet

OpenAI name received	Claude alias used
Any c <code>laude</code> -* prefix	passed through unchanged

Known limitations: temperature, top_p, and max_tokens are accepted in requests but silently ignored — the Claude CLI exposes no sampling knobs. Each request bills the host’s Claude account and incurs a ~1–3 s CLI cold start.

10.3 3. MLOps Sub-Stack (ComparativeAndScoreing)

The MLOps stack lives in a separate Compose file and runs independently of the main stack.

File: services/ComparativeAndScoreing/docker-compose.mlops.yaml

Service	Host port	Purpose
mlflow-db	— (internal)	PostgreSQL backend for MLflow metadata
mlflow-server	5000	MLflow tracking server and model registry
prediction-api	8004	RankNet inference endpoint; falls back to static weights when no Production model is registered
training-pipeline	—	Batch CLI: builds training pairs, trains RankNet, auto-promotes to Staging if NDCG@5 ≥ 0.85
validation-service	—	Batch CLI: weekly drift check; exits 1 if degradation > NDCG_DRIFT_THRESHOLD (default 0.05)

Start the inference API and MLflow tracking server:

```
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
  up -d mlflow-db mlflow-server prediction-api
```

Run a training cycle (training profile):

```
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
  --profile training up training-pipeline
```

Auto-promotes to Staging when $\text{NDCG@5} \geq 0.85$ ($\text{AUTO_PROMOTE_NDCG}=0.85$).

Run the weekly drift check (validation profile):

```
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
  --profile validation up validation-service
```

Activating the ML scoring path:

On first start there is no Production model in MLflow. `prediction-api` falls back to static weights $[0.4, 0.3, 0.3]$ (price / speed / risk). To enable ML scoring:

1. After training completes, promote the best version to Production:

```
mlflow models transition-stage \
  --name ProcurementRanker --version 1 \
  --to-stage Production --archive-existing-versions
```

2. Hot-reload without restarting the container:

```
curl -X POST http://localhost:8004/reload
```

Set `ALLOW_FALLBACK_WEIGHTS=false` to disable the heuristic fallback for canary deployments where ML failures must surface rather than silently degrade.

10.4 4. Applying Database Migrations

The main `procurement_agent` database runs on the host (not inside the main Docker Compose stack). Docker containers reach it via `host.docker.internal:5432`.

10.4.1 4.1 First-Time Setup

Provision the PostgreSQL container (Windows / Docker Desktop recommended):

```
docker run -d \
  --name procurement-postgres \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres123 \
  -e POSTGRES_DB=procurement_agent \
  -p 5432:5432 \
  pgvector/pgvector:pg16
```

Apply all 24 SQL migrations:

```
cd database
export $(grep -v '^#' .env | xargs)
python setup_database.py --create-db
```

10.4.2 4.2 Adding New Migrations

Migrations in database/sql/ run in lexicographic order. This order encodes FK dependency: always use the next available numeric prefix. Never reorder or renumber existing files — the FK chain depends on lexicographic execution order.

```
# After adding a new NN_*.sql file:
cd database
python setup_database.py --continue-on-exists
```

--continue-on-exists treats PostgreSQL “already exists” errors as SKIP rather than FAIL, making re-runs safe.

10.4.3 4.3 setup_database.py Flags

Flag	Effect
(none)	Run all 24 SQL scripts against the existing database

Flag	Effect
<code>--create-db</code>	Create DB_NAME if it does not exist (requires CREATEDB privilege)
<code>--drop-all</code>	DROP all tables and ENUM types CASCADE, then recreate (destructive)
<code>--continue-on-exists</code>	Treat “already exists” errors as SKIP — use for idempotent re-runs

10.4.4 4.4 Verify Schema

```
pytest test_database.py -v --tb=short -q
# 67 tests: Connection, Extensions, EnumTypes, Tables, Indexes,
# WorkflowRows, EndToEndQuery, Constraints
```

10.4.5 4.5 Key Migrations

File	Effect
<code>00_extensions_and_types.sql</code>	uuid-oss, pgcrypto, vector extensions; 20 ENUM types
<code>15_agent_memory_vectors.sql</code>	Creates agent_memory_vectors with vector(3072)
<code>18_bpp_catalog_semantic_cache.sql</code>	Standalone vector store for Stage 3 BPP semantic cache (vector(384))
<code>19b_erp_sync_records_outbox.sql</code>	Adds outbox columns to erp_sync_records for async PO push
<code>20_negotiation_schema.sql</code>	Negotiation-engine tables (also auto-init'd in the postgres Docker service)
<code>22_agent_memory_vector_dim.sql</code>	Alters agent_memory_vectors from vector(3072) to vector(384); adds all-MiniLM-L6-v2 to embedding_model_type ENUM

Known ENUM gap: The `embedding_model_type` ENUM defined in `00_extensions_and_types.sql` includes only `text-embedding-3-large` and `e5-large-v2`. Migration `22_agent_memory_vector_dim.` adds `all-MiniLM-L6-v2`. The value `BAAI/bge-small-en-v1.5` — used by the data-normalizer memory service — is absent from all migrations. Any INSERT that stores this value in a column typed `embedding_model_type` will fail with a PostgreSQL type error.

10.4.6 4.6 Root .env Template

```
cp .env.example .env
```

```
# DB_HOST options:
#   localhost          — when PostgreSQL is in a named Docker container visible
#   ↪ on host
#   host.docker.internal — when PostgreSQL on host must be reached by Docker
#   ↪ containers
DB_HOST=host.docker.internal
DB_PORT=5432
DB_NAME=procurement_agent
DB_USER=postgres
DB_PASSWORD=postgres123

# Required for claude_openai_proxy (negotiation demo):
CLAUDE_PROXY_BINARY_PATH=/home/<your-user>/.local/bin/claude
CLAUDE_PROXY_KEY=          # any non-empty string enables auth
```

10.5 5. Monitoring and Observability

10.5.1 5.1 Service Health Endpoints

All FastAPI services expose a `GET /health` endpoint that returns HTTP 200 when the service is up. The `analytics` service returns HTTP 503 (not mock data) when the DB connection pool is unavailable.


```
# Spot-check all running services:
curl -s http://localhost:8001/health # intention-parser
curl -s http://localhost:8002/health # becn-bap-client
curl -s http://localhost:8003/health # comparative-scoring
curl -s http://localhost:8004/health # orchestrator
curl -s http://localhost:8005/health # catalog-normalizer
curl -s http://localhost:8006/health # data-normalizer
curl -s http://localhost:8007/health # erp-adapter
curl -s http://localhost:8009/health # analytics
curl -s http://localhost:3002/health # sim-bpp
```

10.5.2 5.2 erp-adapter Prometheus Metrics

erp-adapter exposes a Prometheus-format metrics endpoint with 9 series:

```
curl http://localhost:8007/metrics
```

Tracked series include: budget check latency histogram, outbox queue depth, outbox retry counts, circuit breaker state per vendor, and webhook HMAC validation pass/fail counts. This endpoint is the primary integration point for the Phase 4 Prometheus scrape config.

10.5.3 5.3 Log Streaming

```
# Single service (follow mode):
docker compose logs -f becn-bap-client

# Multiple services:
docker compose logs -f orchestrator becn-bap-client data-normalizer

# Last 100 lines without following:
docker compose logs --tail=100 orchestrator

# For the local IntentParser process, logs go to the terminal where uvicorn is
↪ running.
```

Never set LOG_PAYLOADS=true on erp-adapter in production. This logs full PO and budget payloads containing PII and financial data.

10.5.4 5.4 Phase 4: Prometheus + Grafana (Planned)

Phase 4 will add a Prometheus scrape job targeting the `erp-adapter /metrics` endpoint plus OpenTelemetry collectors for all FastAPI services. A Grafana deployment will provide dashboards for procurement pipeline latency (P95 < 5 s acceptance criterion), BPP discovery success rate, and ERP outbox backlog depth. LangSmith tracing for LangGraph negotiation flows will be enabled via `LANGCHAIN_TRACING_V2=true` on `negotiation-engine`.

The `audit_trail_events` table already has a `splunk_indexed` boolean column and a `retention_until` timestamp placeholder for the Phase 4 SIEM and retention-enforcement CronJob.

10.6 Appendix: Orphaned Service — `discovery_engine`

`services/discovery_engine/` contains a standalone multi-network Beckn discovery fan-out implementation. It is **absent from `docker-compose.yml`** and **not called by any other service**. Its documented port (8006) conflicts with `data-normalizer`. Its architectural status is undocumented — it may be a planned replacement for `beckn-bap-client`'s single-gateway discovery, a dead branch, or a future integration point.

Features it provides: concurrent fan-out to N Beckn network gateways via `asyncio.gather`, per-network circuit breakers (3 failures, 30 s recovery), result deduplication by `provider_id:item_id:currency` with geo-proximity merge at 0.5 km, always HTTP 200 even on partial failures.

To evaluate it standalone (use port 8011 to avoid conflict with `data-normalizer`):

```
export DISCOV-
↪ ERY_NETWORKS_JSON='[{"name":"main","base_url":"http://localhost:8002","timeout_s":5.0}]'
cd services/discovery_engine
uvicorn src.main:app --host 0.0.0.0 --port 8011
```

11 Testing Guide

This document is the single reference for running, writing, and understanding tests across the Procurement Agent monorepo. It covers every test suite in the repository, their infrastructure requirements, and known coverage gaps.

Related: [Architecture](#) | [Installation](#) | [Development Guide](#)

11.1 1. Test Framework

The project uses **pytest** with **pytest-asyncio** throughout. All async test configuration lives in `pytest.ini` (or `pyproject.toml` / `setup.cfg`) at the root of each tested package.

11.1.1 `asyncio_mode = auto` — Critical Rule

Every `pytest.ini` that governs async tests sets:

```
[pytest]
asyncio_mode = auto
```

This means:

- `async def test_*` functions are collected and run automatically — **no decorator needed**.
- `async def fixtures` are recognised automatically.
- **Do NOT add `@pytest.mark.asyncio` to any `async def test_*` function.** Under `pytest-asyncio` 0.21+, adding the decorator when `asyncio_mode = auto` is already active produces a deprecation warning or causes test collection failures.

This setting is active in `IntentParser/`, `Bap-1/`, and `services/data-normalizer/`.

11.1.2 HTTP Mocking

All Bap-1 tests mock outbound HTTP with aioresponses. Fixtures in `Bap-1/tests/conftest.py` set `onix_url="http://mock-onix.test"` — a non-routable sentinel hostname that only resolves inside an `aioresponses` context manager. Do not run Bap-1 tests against a real ONIX stack; use `python run.py` for that.

11.1.3 asyncpg Mocking (DataNormalizer unit layer)

The `DataNormalizer/tests/unit` layer mocks `asyncpg` entirely with `AsyncMock`. No database connection is made. Only `services/data-normalizer/tests/` (integration tests) hits a real PostgreSQL instance.

11.2 2. Test Organization

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
<code>IntentParser/tests/test_async_pipeline.py</code>	<code>IntentParser</code>	Full Stage 1+2+3 pipeline in mock mode and live mode. 4 cases: VALIDATED, AMBIGUOUS, CACHE_MISS+MCP_VALIDATED, CACHE_MISS+not_found+recovery	No (mock) / Yes (live)	No (mock) / Yes (live)
<code>IntentParser/tests/test_integration.py</code>	<code>IntentParser</code>	Section A: 18 integration tests (Ollama); Section B: 6 orchestrator unit tests; Section C: 5 Stage 3 validation unit tests	Partial	Partial

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
Bap-1/tests/test_agent.py	Bap-1 BAP	LangGraph ReAct state transitions: parse_intent, discover, rank_and_select, send_select, present_results (14 tests)	No	No
Bap-1/tests/test_callback_collecto	Bap-1 BAP	CallbackCollecto payload parsing, ACK response, ignores unregistered transactions, collect timeout, concurrent callbacks (10 tests)	No	No
Bap-1/tests/test_discover.py	Bap-1 BAP	BecknIntent validation, discover URL must contain local-host:8081/bap/caller/discover, numeric price_value (17 tests)	No	No
Bap-1/tests/test_select	Bap-1 BAP	/select wire format, select URL must contain /bap/caller/select HTTP 500 raises exception (9 tests)	No	No

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
Bap-1/tests/test_intent_parser.py	Bap-1 BAP	NL facade unit tests (7) + integration tests (3 — require Ollama qwen3:8b). Integration asserts: “3 days” → delivery_timeline=72, “Bangalore” → "12.9716,77.5946", descriptions as list[str]	Partial	No
Bap-1/tests/ (7 other files)	Bap-1 BAP	Adapter URL construction, session store TTL, graph build helpers, mock ONIX responses (69 tests)	No	No
database/test_data/PostgreSQL/schema	PostgreSQL	TestConnection, TestExtensions, TestEnumTypes (20 types), TestTables (16 tables), TestIndexes (22 indexes), TestWorkflowRows, TestEndToEndQuery, TestConstraints (67 tests)	No	Yes (PostgreSQL only)
DataNormalizer/t	DataNormalizer	INSERT, channel coercion to 'web', SYSTEM_USER_ID fallback, UPDATE status (4 tests)	No	No

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
DataNormalizer/tests/test_normalize_intent.py	DataNormalizer	NSBT, confidence clamping [0,1], invalid intent_class → 'out_of_scope', BecknIntent defaults (6 tests)	No	No
DataNormalizer/tests/test_normalize_bpp.py	DataNormalizer	BPP upsert, negative fulfillment_hours clamped to 1, non-numeric price → 0.0 (5 tests)	No	No
DataNormalizer/tests/test_normalize_composite.py	DataNormalizer	composite_score × 100 scaling, clamping > 1 to 100, entries without offering_id skipped (4 tests)	No	No
DataNormalizer/tests/test_normalize_fk.py	DataNormalizer	Full FK chain negotiation_outcomes → approval_decisions → purchase_orders, quantity 0 → 1 (6 tests)	No	No
DataNormalizer/tests/test_normalize_audit.py	DataNormalizer	NSBT with JSON reasoning_payload, invalid event_type → ValueError, empty agent_action → ValueError (5 tests)	No	No
DataNormalizer/tests/test_normalize_delegation.py	DataNormalizer	Delegation for all 5 normalize methods (5 tests)	No	No

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
services/data-normalizer/tests/test_endpoints.py	data-normalizer	Happy path for all 8 HTTP endpoints, validation 400 responses, DB errors → 409, defaults verified in DB (23 tests)	No	Yes (PostgreSQL)
services/data-normalizer/tests	data-normalizer	9-step E2E pipeline from request creation → confirmed order → delivered PO with 3 audit events (1 test)	No	Yes (PostgreSQL)
services/data-normalizer/tests/test_memory_performance.py	data-normalizer	P99 search latency < 100ms (20 specs, 20 iterations); ground-truth similarity for 4 known provider queries (2 tests, @pytest.mark.integration)	No	Yes (PostgreSQL + pgvector)
services/sim-bpp/tests/test_a	sim-bpp	Lifecycle ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED, cancel mid-advance, idempotency, SIM_BPP_AUTO_ADV	No	Yes (data-normalizer running)
services/negotiating-engine/tests/test_policy_direct.py	negotiating-engine	Layer 2 policy direct: G1 (absolute 20% cap), G2 (category cap), G3 (supplier cap), G5 (lead time), G6 (quantity) (16 tests)	No	No

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
services/negotia	negotiation_engine	LangGraph interrupt/resume mechanics, HITL accept/reject, gap 25% → hu-man_escalation, transaction_id extraction from 6 Redis channel patterns (7 tests)	No	No
services/erp-adapter/tests/smoke_m31.py	erp-adapter	Budget gate smoke	No	Yes (erp-mock running)
services/erp-adapter/tests/sr	erp-adapter	Outbox + DLQ	No	Yes (PostgreSQL + erp-mock)
services/erp-adapter/tests/smoke_m33.py	erp-adapter	Bidirectional webhooks (HMAC secrets)	No	Yes (erp-mock)
services/erp-adapter/tests/sr	erp-adapter	SAP/Oracle real + circuit breaker	No	Yes (erp-mock)
services/erp-adapter/tests/smoke_m35.py	erp-adapter	readyz + DLQ replay	No	Yes (PostgreSQL)
services/erp-adapter/tests/te	erp-adapter	NormalizedPO Pydantic model + byte-equal JSON against pinned SAP/Oracle snapshots (standalone script, not pytest-discoverable)	No	No
services/orchestration/tests/	orchestration	Real-time tracking WebSocket flow only	No	Yes (Redis + Kafka)
services/notification-dispatcher/tests	notification-dispatcher	Kafka consumer routing, channel fan-out	No	Yes (Kafka mock)
services/frontend-gateway/tes	frontend-gateway	Scoring pipeline, negotiation mock	No	No

Test file path	Service / Package	What it covers	Requires Ollama	Requires DB / Redis
services/discover	discovery_engine	Multi-network fan-out, circuit breaker, deduplication	No	No (aiohttp mock)

Services with zero tests: services/catalog-normalizer/, services/analytics/, services/beckn-bap-client/, services/mcp-sidecar/, services/intention-parser/ (Docker wrapper only – the underlying IntentParser/ package has tests), services/erp-mock/, frontend/.

11.3 3. Test Markers

Marker	Meaning	Infrastructure required	How to skip
@pytest.mark.integration	Requires live PostgreSQL + pgvector (IntentParser, data-normalizer) or live Ollama qwen3:8b (Bap-1)	Yes	-m "not integration"
(no marker)	Pure unit / in-process	None	Always runs

Use the integration marker for any test that opens a network connection to PostgreSQL, Redis, Ollama, or any external service. Do not mark tests that use AsyncMock or aioresponses as integration – they require no live infrastructure.

11.4 4. Running Tests

All commands assume the conda environment is activated (conda activate infosys_project) and you are in the repo root (Procurement-Agent/).

11.4.1 IntentParser — unit only (no infrastructure)

```
pytest IntentParser/ -m "not integration" -v
```

11.4.2 IntentParser — full suite (requires Ollama at localhost:11434)

```
pytest IntentParser/ -v
```

11.4.3 IntentParser Stage 3 async pipeline — live mode (requires PostgreSQL + pgvector + Ollama + MCP sidecar)

```
INTENT_PARSER_TEST_MODE=live pytest IntentParser/tests/test_async_pipeline.py -v
↩ -S
```

11.4.4 Bap-1 — unit only (no infrastructure; all HTTP mocked with aioresponses)

```
pytest Bap-1/tests/ -v -k "not integration"
```

11.4.5 Bap-1 — Ollama integration tests only (requires qwen3:8b pulled)

```
# Prerequisite: ollama serve is running and qwen3:8b has been pulled
ollama pull qwen3:8b
pytest Bap-1/tests/test_intent_parser.py -v -m integration
```

11.4.6 Database schema (requires PostgreSQL 16 + pgvector with procurement_agent DB applied)

```
cd database && export $(grep -v '^#' .env | xargs)
pytest test_database.py -v --tb=short -q
```

11.4.7 DataNormalizer unit layer (no infrastructure; asyncpg mocked)

```
python -m pytest DataNormalizer/tests/ -v
```

11.4.8 data-normalizer integration tests (requires procurement_agent_test DB or Docker)

```
# Auto-detects testcontainers (Docker) or falls back to local Postgres
python -m pytest services/data-normalizer/tests/ -v

# Force local Postgres instead of testcontainers
USE_LOCAL_PG=1 TEST_DB_HOST=localhost python -m pytest
↪ services/data-normalizer/tests/ -v
```

11.4.9 Phase 3 memory performance tests only

```
pytest services/data-normalizer/tests/test_memory_performance.py -v -m
↪ integration
```

11.4.10 Phase 3 audit trail tests only

```
PYTHONPATH=$(pwd) .venv-test/bin/python -m pytest \
services/data-normalizer/tests/test_endpoints.py -k "audit" -v
```

11.4.11 negotiation_engine (no infrastructure; uses MemorySaver)

```
pytest services/negotiation_engine/tests/ -v
```

11.4.12 erp-adapter contract test (standalone script — not pytest-discoverable)

```
python services/erp-adapter/tests/test_contracts.py
# Exit code 0 = all snapshots match
# Exit code 1 = wire shape drift detected (diff printed to stdout)
```

11.4.13 erp-adapter smoke tests (requires Docker stack with erp-mock and PostgreSQL)

```
pytest services/erp-adapter/tests/smoke_m31.py \
services/erp-adapter/tests/smoke_m32.py \
services/erp-adapter/tests/smoke_m33.py \
services/erp-adapter/tests/smoke_m34.py \
services/erp-adapter/tests/smoke_m35.py -v
```

11.4.14 MLOps stack (requires docker-compose.mlops.yaml)

```
# Start the MLOps stack
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
up -d mlflow-db mlflow-server prediction-api

# Train a model (auto-promotes to Staging if NDCG@5 >= 0.85)
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
--profile training up training-pipeline

# Weekly drift validation (exit code 1 = drift; writes
↳ /tmp/model_drift_detected.flag)
docker compose -f services/ComparativeAndScoreing/docker-compose.mlops.yaml \
--profile validation up validation-service
```

11.4.15 Bap-1 smoke test (end-to-end with ONIX Docker stack)

```
# Prerequisites: docker compose up -d onix-bap onix-bpp sim-bpp redis && ollama
↳ serve
cd Bap-1
python run.py "500 reams A4 paper 80gsm Bangalore 3 days max 200 INR"
```

Expected console output shows five labelled steps in sequence: [parse_intent] → [discover] → [rank_and_select] → [send_select] → [present_results].

11.5 5. Writing New Tests

11.5.1 Async test functions — no decorator

```
# CORRECT
async def test_parse_intent_returns_beckn_intent(mock_pool):
    result = await parse_intent("300 meters Cat6 cable Mumbai")
    assert result.item_description is not None

# WRONG — do not do this
@pytest.mark.asyncio          # <-- causes failures under asyncio_mode=auto
async def test_parse_intent_returns_beckn_intent(mock_pool):
    ...
```

11.5.2 Async fixtures

```
import pytest_asyncio

@pytest_asyncio.fixture
async def db_pool():
    pool = await asyncpg.create_pool(dsn=settings.DATABASE_URL)
    yield pool
    await pool.close()
```

11.5.3 Integration marker for infra-dependent tests

```
import pytest

@pytest.mark.integration
async def test_vector_search_returns_similar_results(db_pool):
```

```
# This test requires live PostgreSQL + pgvector
results = await search_memory("A4 paper reams", limit=3)
assert len(results) > 0
assert all(r["similarity"] >= 0.75 for r in results)
```

Run only fast tests (no marker): `pytest -m "not integration" -v`

11.5.4 Config via environment variables

Read all configuration from the module-level `config.py` in your service or package. Never call `os.getenv()` inline in test files. Override values in tests by setting environment variables before importing:

```
import os
os.environ["INTENT_PARSER_TEST_MODE"] = "mock"

from IntentParser.orchestrator import run_pipeline
```

11.5.5 asyncpg mock pattern (DataNormalizer unit tests)

```
from unittest.mock import AsyncMock, MagicMock
import pytest

@pytest.fixture
def mock_pool():
    conn = AsyncMock()
    conn.fetchrow = AsyncMock(return_value={"request_id": "uuid-test-1234"})
    pool = MagicMock()
    pool.acquire = AsyncMock(return_value=conn)
    pool.acquire.return_value.__aenter__ = AsyncMock(return_value=conn)
    pool.acquire.return_value.__aexit__ = AsyncMock(return_value=False)
    return pool

async def test_insert_request_returns_id(mock_pool):
    repo = RequestRepo(pool=mock_pool)
    result = await repo.insert(item_text="100 units USB-C cable", channel="api")
    assert result["request_id"] == "uuid-test-1234"
```

11.5.6 aioresponses pattern (Bap-1 tests)

```
from aioresponses import aioresponses

async def test_discover_posts_to_onix(bap_client):
    with aioresponses() as m:
        m.post(
            "http://mock-onix.test/bap/caller/discover",
            payload={"context": {}, "message": {"ack": {"status": "ACK"}}}
        )
        result = await bap_client.discover_async(intent)
        assert result["message"]["ack"]["status"] == "ACK"
```

All Bap-1 fixtures configure BecknConfig with onix_url="http://mock-onix.test".

11.5.7 LangGraph in-memory checkpointing (negotiation_engine tests)

```
from langgraph.checkpoint.memory import MemorySaver
import pytest

@pytest.fixture
def graph():
    return build_negotiation_graph(checkpointer=MemorySaver())
```

MemorySaver replaces AsyncPostgresSaver in tests. Leave NEGOTIATION_POSTGRES_DSN unset — the service falls back to MemorySaver automatically when the DSN is empty.

11.6 6. Phase 3 Acceptance Test Suites

Two acceptance test suites in KnowledgeBase/tests/phase3/ validate Phase 3 features. They combine automated pytest assertions with manual observation steps.

11.6.1 6.1 Agent Memory and Learning (phase3_agent_memory_learning.md)

Test 1 — Write path

Complete a full order via the frontend (/compare → /commit), then verify:

```
SELECT COUNT(*) FROM agent_memory_vectors;
-- Row count must increase by exactly 1
```

The new row must have: - text_summary matching "{item} ordered from {provider} at {price} {currency} delivery in {hours}h" - entity_type = 'transaction' - embedding_model consistent with the deployed all-MiniLM-L6-v2 model

Test 2 — Read path (API assertion)

Submit a second request for a similar item, then verify the similarity search endpoint:

```
curl -X POST http://localhost:8006/normalize/memory/search \
-H "Content-Type: application/json" \
-d '{"item_text": "A4 paper reams", "limit": 3}'
# Expected: {"results": [...], "count": N} with similarity scores >= 0.75
```

The orchestrator's Agent Reasoning panel must also display a memory_context node.

Test 3 — Isolation (unrelated item must not surface)

```
curl -X POST http://localhost:8006/normalize/memory/search \
-H "Content-Type: application/json" \
-d '{"item_text": "Dell laptops 16GB RAM", "limit": 3}'
# Expected: {"results": [], "count": 0}
```

The 0.75 cosine similarity threshold must prevent unrelated items from appearing.

Test 4 — DB integrity

All rows in agent_memory_vectors must have entity_type = 'transaction'.

Phase 3 memory acceptance checklist:

- agent_memory_vectors has vector (384) column with HNSW cosine index (set by migration 22_agent_memory_vector_dim.sql)
- POST /normalize/memory/write returns {"stored": true} in under 2 seconds (fastembed ONNX cold-start is 10–30 s on first call)

- POST /normalize/memory/search returns results with similarity scores
- After confirming order: orchestrator logs [memory] stored transaction for <item>
- Next similar request: reasoning panel shows memory_context node
- Memory enrichment latency under 5 seconds total

11.6.2 6.2 Audit Trail System (phase3_audit_trail_system.md)

Run the 10 automated pytest tests with:

```
PYTHONPATH=$(pwd) .venv-test/bin/python -m pytest \
  services/data-normalizer/tests/test_endpoints.py -k "audit" -v
```

Test name	What it asserts
test_post_audit_appends_event	POST /normalize/audit returns {"event_id": "..."} with status 201
test_validation_errors_return_400 (x2)	Invalid event_type → 400; missing agent_action → 400
test_audit_get_by_request_id_returns_events	GET /normalize/audit?request_id=X returns events in chronological order
test_audit_get_by_request_id_empty_when_unknown	Unknown request_id returns empty list, not 404
test_audit_get_by_po_id_returns_events	GET /normalize/audit?po_id=X returns events scoped to that PO
test_audit_get_missing_param_returns_400	GET /normalize/audit with no params → 400
test_audit_get_event_by_id	GET /normalize/audit/{event_id} returns full event including reasoning_payload
test_audit_get_event_by_id_not_found_404	Non-existent event_id → 404
test_audit_get_limit_respected	Response respects the maximum 500-event limit

4 manual test cases (run after a full procurement flow):

1. At least 4 events exist for the request: request_created, intent_persisted, discovery_executed, order_confirmed (minimum set)
2. retention_until = event_timestamp + interval '7 years' is set on all events
3. The frontend /request/{id}/audit page renders a vertical timeline with per-type icons and collapsible reasoning_payload

4. Decision chain is fully reconstructable from events alone — no external business context required (SOX 404 compliance check)

11.7 7. Coverage Assessment

11.7.1 Well-tested areas

Area	Approximate coverage	Notes
IntentParser pipeline (Stages 1–3)	Good	2 test files; mock + live modes; all three stages exercised
Bap-1 BAP layer	Good	12 files, 129 tests; full HTTP mocking with aioresponses; all LangGraph state transitions
PostgreSQL schema	Good	67 tests across 8 classes; all 16 tables, 22 indexes, 20 ENUM types, FK constraints
DataNormalizer unit layer	Good	35 tests; 7 files; all repository classes; asyncpg fully mocked
data-normalizer integration	Good	26 tests; real PostgreSQL; 9-step E2E pipeline with FK chain
negotiation_engine	Good	Guardrail policy (16 tests) + LangGraph interrupt/resume mechanics (7 tests); no infrastructure required
sim-bpp lifecycle	Good	Auto-advance, cancel, idempotency, SIM_BPP_AUTO_ADVANCE=false toggle
erp-adapter contracts	Partial	Wire-shape snapshots are pinned but the runner is not pytest-discoverable; smoke tests require a live Docker stack

11.7.2 Known coverage gaps

Area	Gap	Severity
frontend/	Zero tests of any kind — no Jest, no Vitest, no Playwright	High
services/beckn-bap-client/	No tests; the Beckn protocol HTTP client that publishes to Redis on <code>on_discover</code> has no automated verification	High
services/catalog-normalizer/	No tests; the <code>CatalogNormalizer</code> library has no unit coverage despite format detection and LLM fallback logic	High
Orchestrator pipeline (<code>/run</code> , <code>/compare</code> , <code>/commit</code>)	Only real-time tracking WebSocket is tested; the main pipeline state machine has no automated test	High
E2E automated test	No test drives the full NL → Beckn → PostgreSQL → audit trail path automatically; all E2E verification is manual curl	High
CI/CD	No CI pipeline runs tests automatically; all test execution is manual	High
services/analytics/	No tests; SQL queries returning dashboard KPIs are untested	Medium
services/mcp-sidecar/	No tests; the never-throw contract (<code>{"found": false, ...}</code> on all failures) is enforced by convention only	Medium
services/erp-mock/	No tests; mock server responses are not contract-pinned except through erp-adapter smoke tests	Medium
Memory performance	<code>test_memory_performance.py</code> is <code>@pytest.mark.integration</code> ; skipped in most contexts without a live pgvector instance	Low

11.7.3 Infrastructure requirements matrix

Test suite	PostgreSQL	Ollama	Redis	Docker stack	MLflow
IntentParser (unit, -m "not integration")	No	No	No	No	No
IntentParser (live, IN-TENT_PARSER_T	Yes	Yes	Yes	No	No
Bap-1 (unit, -k "not integration")	No	No	No	No	No
Bap-1 (integration, qwen3:8b)	No	Yes	No	No	No
Database tests	Yes	No	No	No	No
DataNormalizer unit	No	No	No	No	No
data-normalizer integration	Yes	No	No	No	No
negotiation_engine	No	No	No	No	No
erp-adapter smoke tests	Yes	No	No	Yes	No
ComparativeAndSc MLOps	No	No	No	Yes (mlops compose)	Yes

11.8 8. End-to-End Smoke Test

This is a manual verification procedure to confirm the full IntentParser → Beckn pipeline is healthy.

11.8.1 Prerequisites

```
# 1. Start core Docker services
docker compose up -d redis onix-bap onix-bpp sim-bpp
```

```
# 2. Start IntentParser locally
conda activate infosys_project
cd IntentParser
uvicorn api:app --port 8001 --reload &

# 3. Start MCP sidecar locally (BAP_API_KEY can be any non-empty string in dev)
cd services/mcp-sidecar
BAP_API_KEY="dev-key" uvicorn server:app --port 3000 &

# 4. Confirm Ollama is running with the required model
ollama serve &
ollama pull qwen3:8b
```

11.8.2 Smoke request

```
curl -s -X POST http://localhost:8001/parse/full \
-H "Content-Type: application/json" \
-d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}' \
| python -m json.tool
```

11.8.3 Expected healthy response shape

A healthy response includes all three stages and has status: "success":

```
{
  "status": "success",
  "intent": {
    "item_description": ["Cat6 UTP cable"],
    "quantity": 300,
    "unit": "meters",
    "delivery_timeline": 120,
    "location_coordinates": "19.0760,72.8777",
    "budget_constraints": { "max": null, "min": null }
  },
  "validation": {
    "stage": "VALIDATED",
    "source": "pgvector_cache"
  }
}
```

```
  },
  "transaction_id": "txn-..."
}
```

Key fields to check:

Field	What to look for	Common failure
status	Must be "success" — not "error" or absent	IntentParser not started or Ollama not running
intent.delivery_timeline	Integer hours (120 for “5 days”) — never an ISO 8601 string	BecknIntent field validator failed
intent.location_coordinates	Decimal "lat, lon" string — never a city name	Stage 2 extraction regression
validation.stage	VALIDATED, AMBIGUOUS, or not_found	Stage 3 unavailable if mcp-sidecar is not running
transaction_id	Non-null UUID	DB write or Redis connection failure

11.8.4 Distinguishing partial failure

Symptom	Likely cause
status: "error", no intent field	Ollama is not running or qwen3:8b is not pulled
intent present but validation absent	MCP sidecar (:3000) is not running; Stage 3 was skipped
validation.stage: "not_found" with recovery_triggered: true	No catalog match found; recovery flow attempted query broadening
transaction_id is null	PostgreSQL or data-normalizer (:8006) unreachable
Response takes > 30 s	Ollama cold-starting the qwen3:8b model; subsequent calls will be faster

12 Development Guide

This guide covers every coding convention, workflow pattern, and pitfall that a developer needs to know when contributing to the Procurement Agent. It is written for someone who has already read [Architecture](#) and [Installation](#) but is about to write their first line of production code.

12.1 Table of Contents

1. [Code Conventions](#)
 2. [Things NOT to Do](#)
 3. [Repository Layout Quick-Reference](#)
 4. [Local Development Workflow](#)
 5. [Adding a New Service](#)
 6. [Adding a New Database Table](#)
 7. [Working with the IntentParser Pipeline](#)
 8. [Working with LangGraph \(Negotiation Engine\)](#)
 9. [Error Handling Patterns](#)
 10. [Known Problems and Workarounds](#)
-

12.2 1. Code Conventions

12.2.1 1.1 Async-First

Every new feature in `IntentParser/`, `services/`, and `Bap-1/` ships as a coroutine.


```
# correct
async def fetch_offerings(intent: BecknIntent) -> list[DiscoverOffering]:
    ...

# wrong – do not write sync functions for new service code
def fetch_offerings(intent: BecknIntent) -> list[DiscoverOffering]:
    ...
```

The only exception is `IntentParser.core.parse_request`, a sync wrapper that exists solely for legacy callers. It explicitly disables Stage 3 and must not be extended.

12.2.2 1.2 Module-Level Config

Every service and package has exactly one `config.py` at module level. That file is the single source of truth for environment variables. Never call `os.getenv()` inline in business logic.

```
IntentParser/config.py          # LLM model names, Ollama URL, DB DSN, MCP sidecar U
services/orchestrator/src/config.py  # orchestrator env vars (Pydantic Settings v2)
services/erp-adapter/src/config.py    # ERP vendor selection, budget-
check flags
services/mcp-sidecar/config.py      # BAP client URL, port, timeouts, ranking thresh
```

When you add a new env var:

1. Add it to the relevant `config.py` with a type annotation and a safe default.
2. Add it to the service's `.env.example` (or the root `.env.example` if it is a cross-service concern).
3. Document it in [Environment](#).

Known exception: `services/mcp-sidecar/bap_client.py` reads `REDIS_URL` and `REDIS_RESULT_TIMEOUT` via `os.getenv()` because they must be real shell environment variables — `dotenv` loading does not work for these at MCP sidecar startup.

12.2.3 1.3 Pydantic v2

Use `field_validator` decorators. The deprecated `@validator` is not acceptable.

```
# correct (Pydantic v2)
from pydantic import BaseModel, field_validator

class BecknIntent(BaseModel):
    delivery_timeline: int # hours, not ISO 8601

    @field_validator("delivery_timeline")
    @classmethod
    def must_be_positive(cls, v: int) -> int:
        if v <= 0:
            raise ValueError("delivery_timeline must be positive hours")
        return v
```

Follow `shared/models.BecknIntent` as the canonical reference. The anti-corruption layer fields have precise semantics:

Field	Type	Unit / format
delivery_timeline	int	Hours (not ISO 8601)
location_coordinates	str	"lat,lon" decimal string
budget_constraints	BudgetConstraints	Typed {max, min} (not raw strings)

12.2.4 1.4 Numbered SQL Migrations

All schema changes live in `database/sql/` with a two-digit numeric prefix.

`NN_description.sql`

Rules:

- **Never reorder or renumber** existing files. The setup script executes them in lexicographic order; FK dependencies rely on that order.
- New migrations go at the **next free prefix only**. If the last file is `23_foo.sql`, the next is `24_bar.sql`.
- All migrations must be **idempotent** using `IF NOT EXISTS` or `DO $$ BEGIN ... EXCEPTION WHEN duplicate_* THEN NULL; END $$`.

- Known naming issue: two files share the 22_ prefix on disk (22_agent_memory_vector_dim.sql and 22_pending_approval_columns.sql). They are independent and safe in either order, but the ambiguity should be resolved by renaming one to 22b_ or moving it to 23_ in a future clean-up migration.

See [Database](#) for the full schema overview.

12.2.5 1.5 Bind-Mounted Libraries

Large Python packages are not copied into Docker images at build time; they are bind-mounted at run-time.

Root package	Consumer container	Mount path
IntentParser/	intention-parser	/app/IntentParser
shared/	intention-parser, beckn-bap-client, catalog-normalizer, data-normalizer	/app/shared
CatalogNormalizer/	catalog-normalizer	/app/CatalogNormalizer
DataNormalizer/	data-normalizer	/app/DataNormalizer

Consequence for development: changes to these packages take effect in running containers immediately without a rebuild. Changes to a service's own src/ code require `docker compose up -d --build <service>`.

12.2.6 1.6 Library vs Service Boundary

When adding logic to the persistence layer, write it in DataNormalizer/ (the library package), not in services/data-normalizer/src/handler.py (the HTTP adapter). The same split applies to CatalogNormalizer/ versus services/catalog-normalizer/. This keeps business logic unit-testable independently of the HTTP layer.

12.2.7 1.7 Test Conventions

- Use `pytest-asyncio` with `asyncio_mode = auto`. Never decorate `async def test_*` with `@pytest.mark.asyncio`.

- Tests that require live PostgreSQL, Ollama, or the Docker stack must be tagged `@pytest.mark.integration` so they can be excluded in CI with `-m "not integration"`.
- Unit tests in `IntentParser/` and `DataNormalizer/` must not require any running infrastructure.

Test locations by scope:

Scope	Directory	Infrastructure required
IntentParser unit	IntentParser/tests/	None
DataNormalizer unit	DataNormalizer/tests/	None
Negotiation guardrails	services/negotiation_engine/tests/test_guardrails.py	None
Bap-1 unit	Bap-1/tests/	None (HTTP mocked with aioresponses)
data-normalizer integration	services/data-normalizer/tests/	PostgreSQL (procurement_agent_test DB)
Database schema	database/test_database.py	PostgreSQL 16 + pgvector
IntentParser live pipeline	IntentParser/tests/test_async_pipeline.py	Ollama + PostgreSQL + MCP sidecar

12.3 2. Things NOT to Do

These rules appear in `CLAUDE.md` and are reproduced here because violating them silently breaks the system — the error often appears in a completely different service, making the root cause hard to trace.

12.3.1 2.1 Never POST Directly to a BPP

All Beckn traffic must flow through `onix-bap:8081`. The ONIX Go adapter handles ED25519 signing and Beckn schema validation. Direct posts bypass both and produce unsigned, unvalidated messages that real BPPs will reject.

```
# correct flow
orchestrator -> becn-bap-client:8002 -> onix-bap:8081 -> sim-bpp:3002

# wrong – never do this
orchestrator -> sim-bpp:3002 (direct)
```

12.3.2 2.2 Never Include the Action Name in ONIX Routing URLs

ONIX appends the Beckn action name automatically to the target URL. Including it duplicates the path and produces a 404.

```
# correct – ONIX will append /on_discover
target: http://beckn-bap-client:8002

# wrong – resolves to http://beckn-bap-client:8002/on_discover/on_discover
target: http://beckn-bap-client:8002/on_discover
```

12.3.3 2.3 Never Use Contract.status.code = “CONFIRMED”

The ONIX schema validator rejects this value. Valid `Contract.status.code` values are: DRAFT, ACTIVE, CANCELLED, COMPLETE.

12.3.4 2.4 Never Await the /discover POST in the MCP Sidecar

The discovery flow in `services/mcp-sidecar/bap_client.py` must fire-and-forget the POST to the BAP client using `asyncio.create_task`, not `await`. Awaiting reintroduces the deadlock that [ADR-0001](#) was written to fix: the sidecar would block waiting for a response that can only arrive after the event loop processes the `/on_discover` callback — which cannot run while the event loop is blocked.

```
# correct
asyncio.create_task(_post_discover(url, payload))

# wrong – deadlock
await _post_discover(url, payload)
```

12.3.5 2.5 Never Use time.sleep() to Flush asyncio.create_task Side Effects

`time.sleep()` blocks the thread, preventing the event loop from scheduling the task that was just created. Use `await asyncio.sleep(0)` to yield control.

```
# correct – yields to the event loop, allowing the task to run
await asyncio.sleep(0)
assert await db.get_audit_record(txn_id) is not None
```

```
# wrong – the task never executes; the assertion will always fail
time.sleep(0.1)
assert await db.get_audit_record(txn_id) is not None
```

12.3.6 2.6 Never Reuse a transaction_id

Beckn Redis Pub/Sub channels are named `beckn_results:{transaction_id}`. Reusing a `transaction_id` delivers one transaction’s catalog payload to a stale subscriber waiting on a different transaction.

12.3.7 2.7 Never Put Billing or Fulfillment Inline in Contract

`ContractisadditionalProperties: false`. The ONIX schema validator will reject any extra fields. Correct placement:

Data	Location in wire shape
Buyer billing address	participants[role=buyer]
Fulfillment details	performance[]
Payment	settlements[]

See `Bap-1/CLAUDE.md` wire-shape gotchas for the full list of Contract pitfalls.

12.4 3. Repository Layout Quick-Reference

For a full service map see [Components](#). Key directories a developer touches frequently:

IntentParser/	NL-to-BecknIntent pipeline – Stage 1 + 2 + 3
api.py	FastAPI app; /parse, /parse/batch, /parse/full
orchestrator.py	Three-stage driver; complexity routing
config.py	All env vars for the pipeline
core/	Stage 3 ANN validator, recovery flow
recovery.py	log_unmet_demand, notify_buyer_no_stock, trigger_open_rfq_flow

(currently log-only stubs – see §10)

shared/models.py	Canonical BecknIntent, BudgetConstraints, DiscoverOffering
services/	
orchestrator/src/workflow.py	Pipeline state machine –
primary integration point	
beckn-bap-client/src/bap_client.py	discover/select/init/confirm/status + Callbacks
data-normalizer/src/handler.py	HTTP adapter (logic lives in DataNormalizer/)
erp-adapter/src/routes/budget.py	Synchronous budget gate
negotiation_engine/src/graph.py	LangGraph StateGraph definition
mcp-sidecar/server.py	FastMCP; never-throw search_bpp_catalog tool
sim-bpp/catalog.json	Bind-mounted catalog; edit without container restart
DataNormalizer/normalizer.py	All PostgreSQL write logic
CatalogNormalizer/normalizer.py	on_discover catalog format detection + mapping
database/sql/	24 numbered SQL migration files
config/	ONIX routing YAMLS (4 files)
docs/architecture/decisions/	ADR-0001 (Redis Pub/Sub)
frontend/src/app/	Next.js 13 App Router pages

12.4.1 Port Map

Range	Services
:8001	IntentParser (local) / intention-parser (Docker)
:8002	beckn-bap-client
:8003	comparative-scoring
:8004 / :18004	orchestrator (8004) / negotiation-engine (host 18004)
:8005	catalog-normalizer
:8006	data-normalizer
:8007	erp-adapter
:8008	erp-mock
:8009	analytics
:8015	frontend_demo_gateway (demo-gateway)

Range	Services
:8081 / :8082	onix-bap / onix-bpp (Go adapters)
:3000	mcp-sidecar (local only — not in Docker)
:3002	sim-bpp (Node.js)
:6379	Redis
:8012	claude_openai_proxy (local only — not in Docker)
:9092	Kafka
:5432	Main procurement_agent PostgreSQL (native host)
:55432	Negotiation-only PostgreSQL (Docker)

12.5 4. Local Development Workflow

12.5.1 4.1 Prerequisites

```
conda activate infosys_project
docker compose up -d # full stack (or subset — see below)
```

12.5.2 4.2 The Three-Terminal Pattern

Three processes must run on the host outside Docker:

Terminal 1 — IntentParser

```
conda activate infosys_project
cd IntentParser
uvicorn api:app --port 8001 --reload
```

Hot-reload is active. Any change to IntentParser/ or shared/ takes effect immediately on the next request.

Terminal 2 — MCP Sidecar


```
conda activate infosys_project
cd services/mcp-sidecar
BAP_API_KEY="dev-key" uvicorn server:app --port 3000
```

BAP_API_KEY must be non-empty; any value works in development. The sidecar is required for Intent-Parser Stage 3 (pgvector ANN + live catalog probe). Without it, Stage 3 degrades to `found=false` for every query.

Terminal 3 — `claude_openai_proxy` (only needed for negotiation and demo flows)

```
export CLAUDE_PROXY_KEY=your-local-key
uvicorn services.claude_openai_proxy.main:app --host 0.0.0.0 --port 8012
```

The proxy wraps the locally installed `claude` CLI. It is required by `negotiation-engine` (env var `NEGOTIATION_OPENAI_BASE_URL`) and `demo-gateway` (env var `OLLAMA_BASE_URL`). For persistent operation across reboots use the provided systemd unit: `services/claude_openai_proxy/claude-proxy.service`.

12.5.3 4.3 Starting a Subset of the Docker Stack

```
# Discovery infrastructure only
docker compose up -d redis onix-bap onix-bpp sim-bpp

# Full pipeline without the ML scoring stack
docker compose up -d redis onix-bap onix-bpp sim-bpp \
    becn-bap-client catalog-normalizer data-normalizer \
    orchestrator comparative-scoring erp-adapter erp-mock

# MLOps stack (RankNet prediction-api + training)
docker compose -f docker-compose.mlops.yaml up -d
```

12.5.4 4.4 Tailing Logs

```
docker compose logs -f becn-bap-client          # single service
docker compose logs -f orchestrator data-normalizer # multiple services
```

12.5.5 4.5 Hot-Reloading sim-bpp Catalog

Edit `services/sim-bpp/catalog.json` directly. The file is bind-mounted into the container; the next HTTP request reads the updated catalog with no restart required.

12.5.6 4.6 Running Tests

```
# IntentParser – unit only (no infrastructure)
pytest IntentParser/ -m "not integration" -v

# IntentParser – full suite (requires Ollama + PostgreSQL + MCP sidecar)
INTENT_PARSER_TEST_MODE=live pytest IntentParser/ -v

# IntentParser – live async pipeline specifically
INTENT_PARSER_TEST_MODE=live pytest IntentParser/tests/test_async_pipeline.py -v
↪ -S

# Bap-1 – unit (HTTP mocked; no Docker required)
pytest Bap-1/tests/ -v -k "not integration"

# Database schema
cd database && export $(grep -v '^#' .env | xargs)
pytest test_database.py -v --tb=short

# Negotiation guardrails (no infrastructure)
pytest services/negotiation_engine/tests/test_guardrails.py -v

# data-normalizer integration (requires PostgreSQL)
pytest services/data-normalizer/tests/ -v
```

12.5.7 4.7 End-to-End Smoke Test

```
curl -X POST http://localhost:8001/parse/full \
  -H "Content-Type: application/json" \
  -d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}'
```

A successful response includes `validation_status`, `mcp_validated`, and a populated `BecknIntent`.

12.6 5. Adding a New Service

Follow these steps to add a new Dockerised Python microservice.

12.6.1 Step 1: Create the service directory

```
services/new-service/
  src/
    main.py          # FastAPI app, lifespan, routes
    config.py        # Pydantic Settings v2 – ALL env vars here
    requirements.txt
    Dockerfile
    README.md
```

12.6.2 Step 2: Write the Dockerfile

Follow the pattern used by existing Python services (e.g. `services/erp-adapter/Dockerfile`):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8XXX"]
```

12.6.3 Step 3: Add to docker-compose.yml

```
new-service:
  build: ./services/new-service
  container_name: new-service
  ports:
    - "8XXX:8XXX"
  environment:
    -
    DATABASE_URL=postgresl+asyncpg://procurement_user:${DB_PASSWORD}@host.docker.internal
```

```

networks:
  - becn_network
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8XXX/health"]
  interval: 30s
  timeout: 5s
  retries: 3

```

Use the next available port in the :8001–:8015 range. Check the port map in [Components](#) before claiming a port.

12.6.4 Step 4: Add /health endpoint

Every service must expose a /health endpoint that returns {"status": "ok"} when the service is ready. This is used by Docker healthchecks and the readiness probe pattern.

12.6.5 Step 5: Update documentation

- Add the service to [Components](#) with its port, role, and key files.
- Add any new env vars to [Environment](#).
- Add any new endpoints to [API Reference](#).

12.7 6. Adding a New Database Table

12.7.1 Step 1: Create the migration file

Create database/sql/NN_description.sql where NN is the next free two-digit prefix.

```

-- NN_new_table.sql
-- Idempotent – safe to re-run

CREATE TABLE IF NOT EXISTS new_table (
  id          UUID          DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at  TIMESTAMPTZ   DEFAULT NOW() NOT NULL,
  -- your columns here

```

```
);
```

```
CREATE INDEX IF NOT EXISTS idx_new_table_created_at
ON new_table (created_at DESC);
```

Rules: - Use IF NOT EXISTS everywhere — the setup script may be re-run on an existing schema. - Use gen_random_uuid() (not uuid_generate_v4()) for UUID primary keys. - Follow existing naming conventions: snake_case for tables and columns, idx_{table}_{column} for indexes.

12.7.2 Step 2: Apply the migration

```
cd database
export $(grep -v '^#' .env | xargs)
python setup_database.py # idempotent re-run
```

12.7.3 Step 3: Add repository class

If the table is owned by the data-normalizer persistence layer, add a DataNormalizer/repositories/new_table that follows the asyncpg repository pattern used by request_repo.py, audit_repo.py, and memory_repo.py.

12.7.4 Step 4: Add integration test

Add a test class to database/test_database.py that confirms the table exists, has the expected columns, and that FK constraints hold. Run pytest database/test_database.py to verify.

12.7.5 ENUM types — important gotcha

If your table needs a new ENUM type, define it in database/sql/00_extensions_and_types.sql AND create a migration that adds it via ALTER TYPE ... ADD VALUE. Do not just add the value to the migration file; the base definition must also be updated so a fresh install from scratch works correctly.

Existing ENUM types embedding_model_type and ai_provider_type contain spec-era values that do not match the as-built system. If you write code that stores an embedding model name in a column typed as embedding_model_type, verify first that your model name is present in the ENUM. See [Known Problems §10.1](#) for details.

12.8 7. Working with the IntentParser Pipeline

12.8.1 7.1 Pipeline Stages

Stage 3 is only active when IntentParser is run **locally** (not via the Docker `intention-parser` container). The container's `docker-compose.yml` `env` block sets `COMPLEX_MODEL=qwen3:1.7b` and `SIMPLE_MODEL=qwen3:1.7b`, collapsing the two-tier model routing.

12.8.2 7.2 Complexity Routing

`IntentParser/orchestrator.py::_is_complex()` routes to `COMPLEX_MODEL` (default `qwen3:8b`) when any of the following are true:

- Query length exceeds 120 characters
- Query contains two or more numeric tokens
- Query contains procurement-domain keywords

All other queries use `SIMPLE_MODEL` (default `qwen3:1.7b`). This routing is active only when running locally; the Docker container overrides both to `qwen3:1.7b`.

12.8.3 7.3 Adding a New Intent Type

1. Add the new intent type to the intent classification prompt in `IntentParser/core/classifier.py`.
2. Add a corresponding branch in `IntentParser/orchestrator.py` if the new intent requires different Stage 2 extraction behaviour.
3. Update `shared/models.BecknIntent` if the new intent needs new fields.
4. Update all downstream consumers of `BecknIntent` in `services/orchestrator/src/workflow.py`, `services/beckn-bap-client/src/bap_client.py`, and any other service that reads `BecknIntent` fields.
5. Add a unit test in `IntentParser/tests/`.

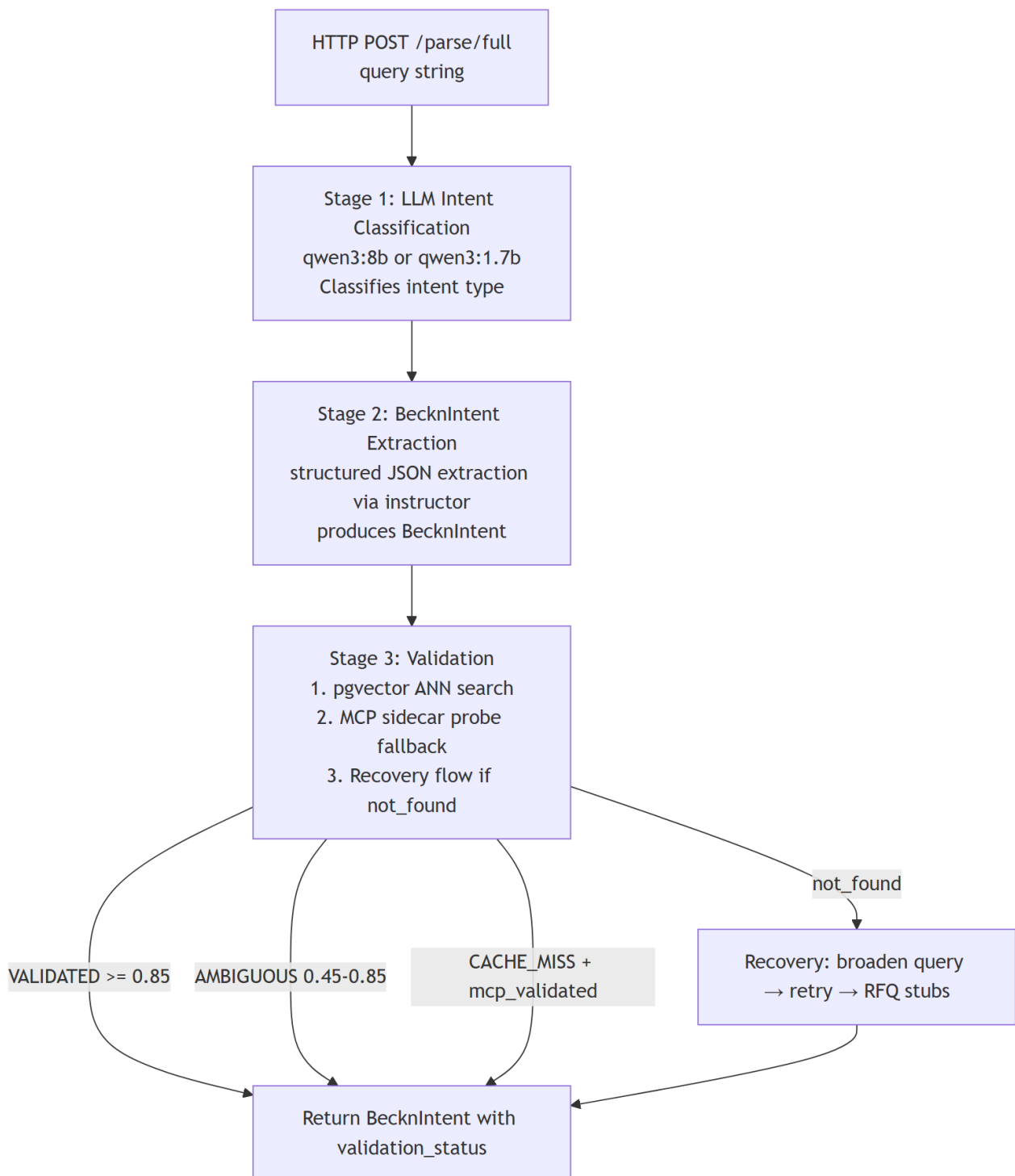


Figure 12.1: DEVELOPMENT_GUIDE diagram 1

12.8.4 7.4 Modifying BecknIntent Fields

shared/models.BecknIntent is the anti-corruption layer boundary. When you change it:

- 1. Update shared/models.py (Pydantic v2 — use field_validator, not @validator).
- 2. Search for all usages across services:

```
grep -r "BecknIntent" services/ IntentParser/ Bap-1/ --include="*.py" -l
```

- 3. Update each consumer. The primary consumers are services/orchestrator/src/workflow.py and services/beckn-bap-client/src/bap_client.py.
- 4. Update API Reference if the change affects the /parse/full response shape.

12.8.5 7.5 Stage 3 Validation Thresholds

The similarity thresholds for Stage 3 ANN search are currently **hardcoded constants** in IntentParser/core/validator.py, not environment variables, despite appearing in the IntentParser README configuration table. Setting env vars for these thresholds has no effect. To change them, edit the constants directly:

Constant	Value	Meaning
VALIDATED_THRESHOLD	0.85	ANN cosine similarity above which an item is accepted
AMBIGUOUS_THRESHOLD	0.45	Below VALIDATED_THRESHOLD but above this → ambiguous

Items below AMBIGUOUS_THRESHOLD trigger the MCP sidecar probe. If the probe also returns found=false, the recovery flow runs.

12.8.6 7.6 Recovery Flow

The recovery stubs in IntentParser/recovery.py (log_unmet_demand, notify_buyer_no_stock, trigger_open_rfq_flow) are **logger-only** — they write to the application log but do not write to the database, call any external service, or send notifications. The module docstring marks them for replacement when infrastructure is provisioned. If you are implementing these integrations, replace the stub body rather than calling the stub from a wrapper.

12.9 8. Working with LangGraph (Negotiation Engine)

12.9.1 8.1 Graph Overview

The negotiation engine (`services/negotiation_engine/src/graph.py`) is a `StateGraph` where each node is an `async` function that reads and writes a shared `NegotiationState` dict.

12.9.2 8.2 Adding a New Node

1. Write an `async def my_node(state: NegotiationState) -> dict` function in `services/negotiation_engine/src/graph.py`. Return only the keys of `NegotiationState` that the node changes.
2. Add the node to the `StateGraph` with `graph.add_node("my_node", my_node)`.
3. Add edges with `graph.add_edge(...)` or `graph.add_conditional_edges(...)`.
4. Add a unit test in `services/negotiation_engine/tests/test_guardrails.py` (no infrastructure) or `test_async_flow.py` (uses `MemorySaver`).

12.9.3 8.3 Guardrail Layers

Every counter-offer passes three independent validation layers before being sent to a BPP:

Layer	Mechanism	On failure
L1	Pydantic <code>discount_pct</code> field constraint (0.0–0.20)	<code>ValidationError</code> returned to the calling node
L2	<code>validate_counter_offer()</code> in <code>src/guardrails.py</code> (G1–G6 rules)	Routes to <code>human_escalation</code> HITL interrupt
L3	ONIX schema validation on the outbound Beckn message	Routes to error terminal state

G2 (category cap) and G3 (supplier cap) violations escalate to human review. The operator can either approve the counter-offer (resuming the graph) or reject the transaction (routing to finalize with `status=REJECTED`).

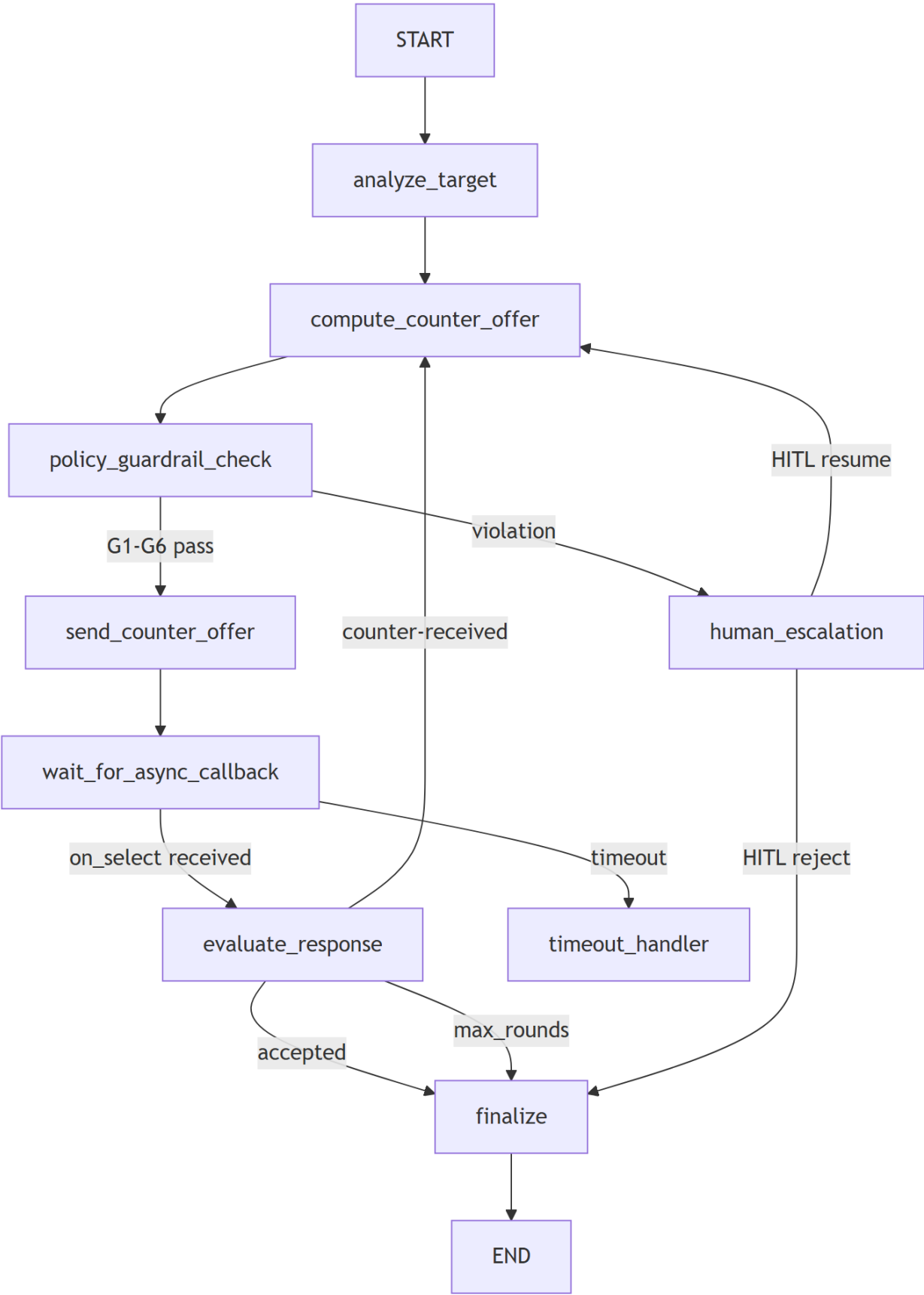


Figure 12.2: DEVELOPMENT_GUIDE diagram 2

12.9.4 8.4 HITL Interrupt and Resume

The graph is interrupted at `human_escalation` using LangGraph's `Command(interrupt=...)` pattern. The orchestrator polls for escalated negotiations and surfaces them in the approval queue. An operator response resumes the graph with `Command(resume=operator_decision)`.

The graph checkpoint is stored in `AsyncPostgresSaver` (configured via `NEGOTIATION_POSTGRES_DSN`). A restarted service can resume an interrupted graph from the last checkpoint. Without `NEGOTIATION_POSTGRES_DSN`, the engine falls back to `MemorySaver` (in-memory, state lost on restart).

12.9.5 8.5 Orchestrator Routes Negotiation via demo-gateway

The orchestrator does not call `negotiation-engine` directly. `workflow.py::_run_autonomous_negotiation` calls `DEMO_GATEWAY_URL` (default: `http://localhost:8015`), which is the `frontend_demo_gateway` service. The demo-gateway then calls `negotiation-engine:8004`. This extra hop exists for demo-flow reasons and is the production routing path in the current codebase. Keep `DEMO_GATEWAY_URL` in the orchestrator environment block when modifying negotiation flows.

12.10 9. Error Handling Patterns

See [System Design](#) for architecture rationale. This section is the implementation reference.

12.10.1 9.1 Never-Throw Contract — MCP Tool Handlers

MCP tool handlers in `services/mcp-sidecar/server.py` must never raise a JSON-RPC error. All failure paths return a valid JSON object with `found: false`.

```
# correct — top-level try/except in search_bpp_catalog
async def search_bpp_catalog(item_name: str, ...) -> dict:
    try:
        # ...
        return {"found": True, "items": [...], "probe_latency_ms": elapsed}
    except Exception:
        return {"found": False, "items": [], "probe_latency_ms": elapsed}
```

This applies to any new MCP tool you add to the sidecar.

12.10.2 9.2 Fire-and-Forget for Non-Critical Writes

Audit trail, agent memory, and ERP sync writes must not block the user-facing pipeline response.

```
# correct – orchestrator pattern for non-critical writes
asyncio.create_task(_persist_audit(txn_id, event_type, payload))
asyncio.create_task(_persist_memory(intent, result))

# wrong – blocks the pipeline for DB write latency
await _persist_audit(txn_id, event_type, payload)
```

Fire-and-forget helpers must catch all exceptions internally and log them; they must never propagate an exception to the caller.

Note: audit writes are not retried. If data-normalizer is unavailable when a task runs, the audit event is permanently lost. The `kafka_offset` placeholder in every `_persist_audit` call (`kafka_offset=0`) is reserved for Phase 4 Kafka integration, which will provide durable delivery.

12.10.3 9.3 asyncio.gather with return_exceptions=True for Fan-Out

When dispatching to multiple independent targets (notification channels, network probes), use `asyncio.gather` with `return_exceptions=True` so a failure in one target does not cancel the others.

```
results = await asyncio.gather(
    send_slack(message),
    send_teams(message),
    send_email(message),
    return_exceptions=True,
)
for r in results:
    if isinstance(r, Exception):
        logger.warning("notification channel failed: %s", r)
```

12.10.4 9.4 ERP Budget Gate — Fail-Closed vs Fail-Open

The budget gate is configurable per environment:

Variable	docker-compose.yml default	Behaviour
ERP_BUDGET_CHECK_ENABLED	true	Whether the gate is called at all
ERP_BUDGET_CHECK_REQUIRED	false	true = fail-closed (deny on error), false = fail-open (allow on error)

The `docker-compose.yml` default is **fail-open** (`false`) for developer convenience. Production deployments that handle real purchase orders must explicitly set `ERP_BUDGET_CHECK_REQUIRED=true`.

12.10.5 9.5 Database Error Middleware

The `data-normalizer` maps `asyncpg` exceptions to structured HTTP responses via `db_error_middleware`. Callers must handle 409 as an idempotency signal (duplicate write is not fatal):

asyncpg exception	HTTP status	Body
UniqueViolationError	409	{"error": "duplicate"}
ForeignKeyViolationError	409	{"error": "fk_violation"}
CheckViolationError	422	{"error": "check_violation"}
NotNullViolationError	422	{"error": "not_null_violation"}
Any other PostgresError	500	{"error": "db_error"}

12.11 10. Known Problems and Workarounds

These are confirmed bugs or gaps. Each entry describes the symptom, the root cause, and the correct workaround until a fix is applied.

12.11.1 10.1 Database ENUM Types Contain Spec-Era Values

Symptom: Inserting an agent memory record fails with `invalid input value for enum embedding_model_type`.

Root cause: `database/sql/00_extensions_and_types.sql` defines `embedding_model_type` with only `text-embedding-3-large` and `e5-large-v2`. The actual deployed models (`all-MiniLM-L6-v2`, `BAAI/bge-small-en-v1.5`) are absent. `Migration22_agent_memory_vector_dim.sql` adds `all-MiniLM-L6-v2` but never adds `BAAI/bge-small-en-v1.5`. Similarly, `ai_provider_type` is missing `ollama`.

Workaround: After running `python setup_database.py`, run:

```
ALTER TYPE embedding_model_type ADD VALUE IF NOT EXISTS 'BAAI/bge-small-en-v1.5';
ALTER TYPE ai_provider_type ADD VALUE IF NOT EXISTS 'ollama';
```

Fix required: Add the missing values to `00_extensions_and_types.sql` (base definition) and update the `DEFAULT` in `15_agent_memory_vectors.sql` from `text-embedding-3-large` to `all-MiniLM-L6-v2`.

12.11.2 10.2 Docker Container Collapses Complexity Routing to qwen3:1.7b

Symptom: Complex procurement queries (multi-constraint, long descriptions) return lower-quality Beckn-Intent extractions when running via Docker.

Root cause: `docker-compose.yml` sets `COMPLEX_MODEL=qwen3:1.7b` and `SIMPLE_MODEL=qwen3:1.7b` for the `intention-parser` container. The complexity routing in `IntentParser/orchestrator.py` always resolves to `qwen3:1.7b` regardless of query complexity. The code default of `qwen3:8b` for `COMPLEX_MODEL` is only active when running `IntentParser` locally.

Workaround: Run `IntentParser` locally (not via Docker) for Stage 3 development and for testing complex query extraction quality.

12.11.3 10.3 catalog-normalizer LLM Fallback Uses Ollama, Not OpenAI

Symptom: Setting `OPENAI_API_KEY` has no effect on the catalog normalizer's `UNKNOWN-format` fallback.

Root cause: `CatalogNormalizer/llm_fallback.py` uses `instructor.from_openai(OpenAI(base_url=api_key="ollama"))` — the OpenAI SDK is used as an Ollama shim. The correct env vars are `OLLAMA_URL` (default `http://localhost:11434/v1`) and `NORMALIZER_MODEL` (default `qwen3:1.7b`).

12.11.4 10.4 demo-gateway Port Mismatch

Symptom: Running `frontend_demo_gateway` per its README on port 8005 causes connection refused errors from the orchestrator.

Root cause: The service README documents `--port 8005`, but `docker-compose.yml` runs it on port 8015, and `services/orchestrator/src/workflow.py` defaults `DEMO_GATEWAY_URL` to `http://localhost:8015`.

Workaround: Always start `demo-gateway` on port 8015: `uvicorn src.main:app --port 8015`.

12.11.5 10.5 Frontend Requires Live Keycloak — No Stub Credentials

Symptom: The Next.js frontend throws at runtime: `KEYCLOAK_CLIENT_ID must not be null`.

Root cause: `frontend/src/lib/auth.ts` configures only `KeycloakProvider`. There is no `CredentialsProvider` and no stub login. The credentials in `frontend/.env.local` connect to the Phase Two (`phasetwo.io`) hosted Keycloak tenant at `euc1.auth.ac/auth/realms/procurement-agent`.

Workaround: Obtain the Phase Two tenant credentials from the project lead. Add them to `frontend/.env.local` (git-ignored). Do not commit this file.

12.11.6 10.6 negotiate Audit Events Are Never Written

Symptom: The audit trail for a confirmed negotiated order contains no `negotiate` event type entries.

Root cause: `services/orchestrator/src/workflow.py::_run_autonomous_negotiation` contains zero calls to `_persist_audit`. Negotiation outcomes are written only to `result['messages']` in memory.

Fix required: Add `_persist_audit` calls at negotiation start, at each round's counter-offer generation, and at negotiation finalization. This is needed for SOX 404 compliance.

12.11.7 10.7 discovery_engine Is Orphaned

The `services/discovery_engine/` directory contains a complete `MultiNetworkCoordinator` implementation with per-network circuit breakers and deduplication. It has no entry in `docker-compose.yml` and is not called by any service. If you need multi-network Beckn discovery, this service must be wired into the stack — it is not currently reachable. Do not start extending the single-gateway path

in `beckn-bap-client` for multi-network use without first evaluating whether `discovery_engine` can be plugged in directly.

12.11.8 10.8 recovery.py Stubs Never Notify or Write to DB

The functions `log_unmet_demand`, `notify_buyer_no_stock`, and `trigger_open_rfq_flow` in `IntentParser/recovery.py` are logger-only. When a procurement item cannot be found in the catalog after query broadening, the buyer receives no notification and no RFQ is created. The recovery flow completes silently with a `not_found` result.

12.12 See Also

- [Architecture](#) — system context, Beckn lifecycle, async discovery model
- [System Design](#) — ADR-0001 Redis Pub/Sub, design decision rationale
- [Components](#) — full service map with ports and responsibilities
- [API Reference](#) — endpoint contracts for all services
- [Database](#) — schema overview, migration history
- [Environment](#) — all environment variables with defaults
- [Security](#) — HMAC rotation, ERP budget gate, ONIX signing
- [Configuration](#) — ONIX routing YAMLs, Docker Compose overrides
- [Bap-1/CLAUDE.md](#) — Beckn v2 wire-shape gotchas (read before modifying the Beckn adapter)
- [docs/architecture/decisions/0001-use-redis-pubsub-for-async-beckn-responses.md](#) — ADR-0001

13 Contributing Guide

This guide covers Git conventions, the pre-submission checklist, documentation expectations, code style, and PR review criteria. For environment setup and first-time configuration see [Installation](#).

Cross-references: [Architecture](#) · [API Reference](#) · [Configuration](#) · [Installation](#) · [Database](#)

13.1 1. Git Workflow

13.1.1 1.1 Branch Naming

Prefix	When to use	Example
feature/	New capability or service	feature/erp-cancel-po-sap
fix/	Bug correction	fix/audit-trail-missing-negotiate-event
docs/	Documentation only	docs/update-contributing-setup
refactor/	Internal restructuring, no behaviour change	refactor/extract-bap-client-retry-logic

Branch from `main`. Submit PRs back to `main`. Long-running feature work should rebase onto `main` before opening the PR.

13.1.2 1.2 Commit Message Format

`<type>(<scope>): <imperative summary, ≤72 characters>`

Optional body: explain **why**, not **what**. The diff shows what changed;

the commit message explains the motivation and context.

Optional footer: Refs #123, Co-authored-by: Name <email>

- **Imperative, present tense:** “add audit event for negotiation round” not “added” or “adds”.
- **Scope** is the service or module: orchestrator, intent-parser, data-normalizer, frontend, database, mcp-sidecar, etc.
- **Types:** feat, fix, refactor, test, docs, chore.

13.1.3 1.3 PR Process

1. Open the PR against `main` with a description that answers:
 - **What** changed (brief summary).
 - **Why** it was needed (motivation, linked issue if any).
 - **How to test** it manually (curl command, unit test path, or acceptance step).
2. Keep PRs focused. A PR that touches both a new feature and an unrelated refactor should be split.
3. Ensure the pre-submission checklist (Section 2) is complete before requesting review.
4. At least one approval is required before merging.

13.2 2. Pre-Submission Checklist

Work through this list before marking a PR ready for review.

13.2.1 Async and Python conventions

- ☐ All new functions in `IntentParser`, `services`, and `Bap-1` are `async def` coroutines. The sync wrapper in `IntentParser.core.parse_request` exists only for legacy callers and disables Stage 3 — do not extend it.
- ☐ No `@pytest.mark.asyncio` decorators anywhere. `asyncio_mode = auto` in `pytest.ini` handles coroutine collection globally; the decorator breaks `pytest-asyncio 0.21+`.
- ☐ No `time.sleep()` inside coroutines. Use `await asyncio.sleep(0)` or `await asyncio.sleep(n)` to yield the event loop so `asyncio.create_task` side effects (e.g. cache writes) can complete.

- ❑ No inline `os.getenv(...)` calls outside `config.py`. The exception is `services/mcp-sidecar/bap_client.py` which reads `REDIS_URL` and `REDIS_RESULT_TIMEOUT` via `os.getenv()` because these must be real shell variables — do not extend this pattern elsewhere.

13.2.2 Configuration

- ❑ New environment variables are added in three places: the service's `config.py`, the root `.env.example`, and `CONFIGURATION.md`.
- ❑ Config values are read through Pydantic Settings v2 in the service's `config.py` — not accessed inline.

13.2.3 Database

- ❑ New tables have an idempotent migration script at `database/sql/NN_description.sql`, using the next available two-digit prefix.
- ❑ Migration scripts use `IF NOT EXISTS / DO $$ BEGIN ... EXCEPTION WHEN duplicate_* THEN NULL; END $$` so `setup_database.py` can re-run without errors.
- ❑ Existing `database/sql/` files have not been reordered or renumbered. The FK chain depends on lexicographic sort order.
- ❑ New ENUM values added to an existing type go in the base definition (`00_extensions_and_types.sql`) **and** in an `ALTER TYPE ... ADD VALUE` migration at the next free prefix.

13.2.4 Tests

- ❑ Unit tests added for all new business logic.
- ❑ Tests that require live infrastructure (PostgreSQL, Ollama, Redis, Docker stack) are marked `@pytest.mark.integration`.
- ❑ MCP sidecar changes manually verify the never-throw contract: test that a BAP Client timeout, an unreachable BAP Client, zero ONIX matches, a malformed response, a blank required argument, and an unhandled internal exception all return `{"found": false, "items": [], "probe_latency_ms": <elapsed>}` — not a JSON-RPC error.

13.2.5 Beckn routing

- ❑ All Beckn protocol calls (`discover/select/init/confirm/status`) go through `onix-bap:8081`. No service POSTs directly to a BPP URL.
- ❑ ONIX routing target URLs in `config/generic-routing-*.yaml` do **not** include the action name. ONIX appends it: `http://host:8000/bpp+actiondiscover` → `http://host:8000/bpp/discover`.

13.2.6 Documentation

- ☐ If a service API changed, `API_REFERENCE.md` is updated.
- ☐ If a service was added or removed, the Component Reference section of `ARCHITECTURE.md` and `docker-compose.yml` comments are updated.
- ☐ `CLAUDE.md` “What NOT to do” section was reviewed; no new code violates its constraints.

13.2.7 Wire-shape

- ☐ `Contract.status.code` uses only `DRAFT` | `ACTIVE` | `CANCELLED` | `COMPLETE`. ONIX rejects other values.
- ☐ No billing or fulfillment data is placed inline in `Contract` (`additionalProperties: false`). Buyer info belongs in `participants[role=buyer]`, fulfillment in `performance[]`, payment in `settlements[]`.
- ☐ Read Bap-1/`CLAUDE.md` wire-shape gotchas before modifying any Beckn adapter, ONIX URL construction, or `Contract` shape.

13.3 3. Documentation Standards

13.3.1 3.1 New Service

Every new service directory must contain a `README.md` with these sections:

Section	Contents
Role	One paragraph: what problem this service solves, which other services call it, which it calls.
Architecture diagram	Mermaid flowchart TD or sequenceDiagram showing inputs, outputs, and the main data path.
API endpoints	Table: method, path, request schema summary, response schema summary.
Configuration	Table: env var name, description, default, required/optional.
Run command	Exact command for local dev and for Docker.
Testing	How to run the service’s tests; infra required.

13.3.2 3.2 Shared Library Changes

`shared/`, `DataNormalizer/`, and `CatalogNormalizer/` are volume-mounted into multiple containers. Changes take effect immediately in running containers without a rebuild, but they affect all consumers simultaneously.

When modifying a shared model:

- Check all services that import from the changed module (use `grep -r "from shared" / from DataNormalizer across services/`).
- Update the Pydantic model and all consumers in the same PR.
- Run the unit tests for every consumer that has a test suite before opening the PR.

13.3.3 3.3 Significant Features

For substantial new features (a new pipeline stage, a new async flow, a new integration):

- Add a record to `docs/architecture/decisions/` following the ADR format in `0001-use-redis-pubsub-for-async-beckn-responses.md`.
- Update the relevant `KnowledgeBase/project_scaffold/` notes if they cover the changed area.
- Consider whether `ARCHITECTURE.md` (including §9 Component Reference) needs a structural update.

13.4 4. Code Style

13.4.1 4.1 Python

- PEP 8. Line length 100 characters.
- **Type hints on all function signatures.** Return types included.
- **Pydanticv2 field_validator decorators**, not the deprecated `@validator`. Use `model_validator(mod` for cross-field validation.
- `BecknIntent` is the canonical NL-derived intent model in `shared/models.py`. Its fields have strict types: `delivery_timeline` is `int` (hours, not ISO 8601 strings), `location_coordinates` is `"lat, lon"` decimal string (not a city name), `budget_constraints` is typed `{max: float, min: float}`.

- **Error handling in MCP tools:** return `{"found": False, ...}` on all failure paths. Never raise. See `services/mcp-sidecar/server.py::search_bpp_catalog`.
- Discovery POSTs from the MCP sidecar use `asyncio.create_task(...)`, not `await`. Awaiting the POST reintroduces the deadlock that [ADR-0001](#) was written to prevent.

13.4.2 4.2 TypeScript (Frontend)

- Strict mode (`"strict": true` in `tsconfig.json`).
- Component library: Radix UI primitives with Tailwind utility classes. Do not import a second component library.
- All backend calls go through the Axios wrappers in `frontend/src/lib/api.ts`. Do not add raw fetch calls in page components.
- The frontend uses NextAuth with Phase Two hosted Keycloak (KeycloakProvider only). There are no stub credentials for local dev — a live Phase Two tenant is required. See `frontend/.env.example` for the required variables (`KEYCLOAK_CLIENT_ID`, `KEYCLOAK_CLIENT_SECRET`, `KEYCLOAK_ISSUER`).

13.4.3 4.3 SQL Migrations

`NN_description.sql` ← two-digit numeric prefix, lowercase, underscores

- NN is the next free number in `database/sql/`. Check the directory listing before picking a prefix.
- Keep migrations idempotent. A re-run of `setup_database.py` must be safe.
- One logical change per file.

13.5 5. Reviewing Others' PRs

When reviewing a PR, check for the following categories of issue in addition to correctness and logic.

13.5.1 Async patterns

```
# BAD – blocks the event loop; create_task side effects never run
time.sleep(1)

# GOOD – yields control so tasks can run
await asyncio.sleep(0)
```

- Are new `asyncio.create_task` calls for fire-and-forget operations (not fire-and-await)?
- Is any new coroutine accidentally blocking on I/O with a sync call?
- Are there any new `@pytest.mark.asyncio` decorators? Flag them for removal.

13.5.2 Error handling

- MCP tool handlers must return `{"found": False, ...}` on all failure paths, never raise.
- New service endpoints should return structured error bodies, not bare exception strings.

13.5.3 Configuration discipline

- Every new `os.getenv()` outside `config.py` is a convention violation (except the documented carve-out in `mcp-sidecar/bap_client.py`).
- New env vars not in `config.py` will not be visible to services running in Docker (env injection goes through the config object).

13.5.4 Database migrations

- Is the new migration idempotent? Run `setup_database.py` twice and confirm it does not error.
- Does the prefix follow the correct lexicographic order relative to existing files?

13.5.5 Beckn traffic routing

Look for any line that constructs a URL ending in `/bpp`, `/seller`, or a port that is not `8081`. All Beckn protocol traffic must route through `onix-bap:8081`. The presence of `select_url` in tests is a good proxy — it must always contain `caller` (i.e. the ONIX caller path).

13.5.6 Shared library consumers

If `shared/`, `DataNormalizer/`, or `CatalogNormalizer/` changed, confirm that all services importing from those packages are updated and tested in the same PR. A partial update breaks all containers on the next `docker compose up` without a rebuild.

14 Troubleshooting Guide

This guide is for developers who need to diagnose and fix problems in the Procurement Agent on Beckn Protocol system. For architecture context see [ARCHITECTURE.md](#). For deployment setup see [DEPLOYMENT.md](#).

14.1 1. Service Health Check

14.1.1 1.1 Health Endpoints

The table below covers every service in the stack. Three services run on the host outside Docker (marked **local**); the rest are addressed via their published host ports.

Service	Health URL	Expected HTTP status	Key fields in response	Notes
orchestrator	http://localhost:28000/health	—	—	Dual-mapped to host ports 8000 and 8004
beckn-bap-client	http://localhost:200	—	—	—
comparative-scoring	http://localhost:28003/health	—	—	ML path active only when prediction-api is up
catalog-normalizer	http://localhost:200	—	—	—
data-normalizer	http://localhost:28006/health	—	—	—
erp-adapter	http://localhost:200	—	circuit_breakers db, redis	Only confirmed /readyz endpoint in the stack; reports per-vendor circuit state
erp-mock	http://localhost:28008/health	—	—	Default ERP_VENDORS=mock

Service	Health URL	Expected HTTP status	Key fields in response	Notes
analytics	http://localhost:2000	200	—	Returns 503 when PostgreSQL is unavailable
negotiation-engine	http://localhost:18004/readyz	200	—	Host port 18004; container port 8004
demo-gateway	http://localhost:2000	200	—	Required for autonomous negotiation
sim-bpp	http://localhost:20002/health	200	—	—
onix-bap	http://localhost:2000	200	—	ONIX Go adapter; schema validator pinned to d43ec30d
onix-bpp	http://localhost:20082/health	200	—	—
mcp-sidecar	http://localhost:2000	200	status	local — must be started manually
IntentParser	http://localhost:20001/health	200	—	local for Stage 1+2+3; Docker wrapper is Stage 1+2 only
claude_openai_proxy	http://localhost:2000	200	—	local — required for negotiation demo flows
redis	redis-cli -p 6379 PING	PONG	—	Via Docker: docker exec \$(docker compose ps -q redis) redis-cli PING
kafka	internal healthcheck only	—	—	kafka-broker-api-versions.sh run inside container

14.1.2 1.2 Check All Docker Services at Once

```
# Show container names, status, and health from the repo root
docker compose ps
```

```
# Spot-check every HTTP service in one pass (host-accessible ports only)
for entry in \
    "orchestrator:8000" \
    "beckn-bap-client:8002" \
    "comparative-scoring:8003" \
    "catalog-normalizer:8005" \
    "data-normalizer:8006" \
    "erp-adapter:8007/readyz" \
    "erp-mock:8008" \
    "analytics:8009" \
    "sim-bpp:3002" \
    "demo-gateway:8015" \
    "negotiation-engine:18004/readyz" \
    "mcp-sidecar:3000" \
    "intention-parser:8001"; do
name="${entry%%:*}"
rest="${entry#*:}"
port="${rest%%/*}"
path="${rest#*/}"
[ "$path" = "$rest" ] && path="health"
code=$(curl -s -o /dev/null -w "%{http_code}"
↪ "http://localhost:${port}/${path}" 2>/dev/null || echo "UNREACHABLE")
printf "%-30s HTTP %s\n" "$name" "$code"
done
```

14.2 2. Common Issues

14.2.1 2.1 Natural Language Parsing

14.2.1.1 Stage 3 returns "found": false for every query

Symptom. Every call to `/parse/full` returns `validation_status: "not_found"` or `mcp_validated: false` regardless of the search term.

Cause. The `mcp-sidecar` process is not running, or its Redis connection is broken.

Diagnosis.

```
# 1. Confirm the process is listening
curl http://localhost:3000/health

# 2. If down, start it
conda activate infosys_project
cd services/mcp-sidecar
export BAP_API_KEY="dev-key"
export REDIS_URL="redis://localhost:6379" # must be a real env var, not just in
↳ .env
uvicorn server:app --port 3000

# 3. Verify Redis is reachable from the sidecar's perspective
redis-cli -u redis://localhost:6379 PING
```

Note. REDIS_URL and REDIS_RESULT_TIMEOUT must be exported shell variables, not merely set in a .env file, because `bap_client.py` reads them via `os.getenv()` at import time.

14.2.1.2 qwen3 model error / LLM call timeout

Symptom. Stage 1 or Stage 2 returns an error such as `OllamaRequestError` or a timeout; `BecknIntent` fields are populated with `null`.

Cause. Ollama is not running, the model has not been pulled, or the `OLLAMA_BASE_URL` env var points to the wrong host.

Diagnosis and fix.

```
# Check Ollama is responding
curl http://localhost:11434/api/tags

# Pull required models (first-time or after image prune)
ollama pull qwen3:8b
ollama pull qwen3:1.7b

# If running IntentParser inside Docker, the container calls host Ollama via
# host.docker.internal:11434. Verify the bridge is reachable from inside the
↳ container:
docker exec $(docker compose ps -q intention-parser) \
  curl -s http://host.docker.internal:11434/api/tags
```

Note (Docker vs local). The `intention-parser` Docker container overrides `COMPLEX_MODEL=qwen3:1.7b` (see `docker-compose.yml`), collapsing both routing branches to the 1.7b model. The full two-tier routing (`qwen3:8b` for complex queries) only operates when `IntentParser` is run locally outside Docker. If quality on complex queries is under investigation, use the local process.

14.2.1.3 BecknIntent fields are null after Stage 2

Symptom. `/parse/full` succeeds with HTTP 200 but critical fields such as `quantity`, `delivery_timeline`, or `location_coordinates` are null.

Cause. Stage 2 LLM extraction failed silently; instructor exhausted `max_retries=3` and returned a partially populated model.

Diagnosis.

```
# Enable verbose logging
INTENT_PARSER_TEST_MODE=live uvicorn api:app --port 8001 --reload

# Re-run the failing query with full logging
curl -X POST http://localhost:8001/parse/full \
  -H "Content-Type: application/json" \
  -d '{"query": "<your query here>"}'
```

Watch the log output for instructor retry attempts. Common causes: the query is ambiguous (LLM cannot extract a numeric quantity), or `delivery_timeline` unit confusion (hours vs days). The anti-corruption layer normalises `delivery_timeline` to **integer hours** and `location_coordinates` to `"lat,lon"` decimal strings — check that Stage 2 output conforms before passing to Stage 3.

14.2.1.4 Stage 3 recovery flow does not notify the buyer or trigger an RFQ

Expected behaviour. When Stage 3 returns `not_found` after query broadening, the recovery chain calls `log_unmet_demand`, `notify_buyer_no_stock`, and `trigger_open_rfq_flow`.

Current status. All three functions in `IntentParser/recovery.py` are logger-only stubs. They print to stdout (`UNMET_DEMAND`, `BUYER_NOTIFY`, `OPEN_RFQ` prefixes) and do not call any external service. This is a known limitation with no assigned owner. See [Known Limitations](#).

14.2.2 2.2 Beckn Discovery

14.2.2.1 on_discover never arrives — timeout after REDIS_RESULT_TIMEOUT

Symptom. The mcp-sidecar waits the full 15 seconds and returns {"found": false, "items": [], "probe_latency_ms": 15000}. The orchestrator path returns an empty offerings list.

Cause A — Redis Pub/Sub not working.

```
# Open a subscriber in one terminal to watch the channel
redis-cli SUBSCRIBE becn_results:test

# Fire a discovery in another terminal
curl -X POST http://localhost:8002/discover \
  -H "Content-Type: application/json" \
  -d '{"transaction_id":"test","item":"Cat6 cable"}'

# If nothing appears on the subscriber within 10 s, the on_discover callback
# is not reaching Redis. Check the becn-bap-client log:
docker compose logs -f becn-bap-client
```

Cause B — on_discover routing misconfigured. Check config/generic-routing-BAPReceiver.yaml. The target URL must **not** include the action name — ONIX appends it automatically. Correct: http://becn-bap-client:8002. Wrong: http://becn-bap-client:8002/on_discover (causes a 404 double-path error).

Cause C — Redis unavailable at the moment on_discover fires. If Redis was unreachable when the /on_discover webhook arrived, the handler logs a warning and falls back to the CallbackCollector queue only. The mcp-sidecar has no retry; it times out after REDIS_RESULT_TIMEOUT seconds.

14.2.2.2 ONIX routing returns 404 on Beckn actions

Symptom. docker compose logs onix-bap shows 404 responses when routing discover, select, init, or confirm.

Cause. The action name is included in the target URL in a routing YAML file. ONIX appends the action name automatically; including it in the YAML doubles the path.

Fix. Open the relevant file in `config/` and remove the trailing action from the `target` field.

```
# WRONG – results in http://host:8002/on_discover/on_discover
target: http://beckn-bap-client:8002/on_discover

# CORRECT – ONIX appends /on_discover itself
target: http://beckn-bap-client:8002
```

14.2.2.3 Schema validation failure at ONIX — Beckn message rejected

Symptom. `docker compose logs onix-bap` shows a schema validation error for a `Contract` field or a `status.code` value.

Common causes and fixes.

Mistake	Fix
<code>Contract.status.code = "CONFIRMED"</code>	Use DRAFT, ACTIVE, CANCELLED, or COMPLETE
Billing address placed directly inside Contract	Move to <code>participants[role=buyer]</code>
Fulfillment placed directly inside Contract	Move to <code>performance[]</code>
Payment placed directly inside Contract	Move to <code>settlements[]</code>
Action name appended to routing target URL	Remove it (see previous item)

The ONIX schema validator is pinned to commit `d43ec30d`. Do not upgrade the `fidedocker/onix-adapter` image before running an end-to-end signing test — later commits introduced a `$ref` resolution bug in `SignatureHeader` / `AckSignatureHeader`.

14.2.2.4 sim-bpp returns zero results for multi-noun queries

Symptom. Discovery returns an empty `on_discover` catalog for queries containing two product categories (e.g. "office supplies and networking cable").

Cause. `sim-bpp` tokenises the query and applies AND-logic: every non-filler token must match at least one item. No single item matches tokens from two unrelated categories simultaneously.

Workaround. Issue one discovery per product category, or edit `services/sim-bpp/catalog.json` (bind-mounted at runtime — no container rebuild needed) to add an item that matches the combined tokens.

14.2.3 2.3 Database

14.2.3.1 Migration fails with FK violation

Symptom. `python setup_database.py` exits with a PostgreSQL FK constraint error midway through applying migrations.

Cause. Migration files were reordered. The 24 SQL scripts in `database/sql/` must execute in lexicographic order because FK dependencies follow the numeric prefix chain.

Fix. Restore the original filenames and ordering. Do not renumber existing files — add new migrations at the next free prefix only.

```
ls database/sql/ | sort # verify lexicographic order looks correct
python setup_database.py --continue-on-exists
```

14.2.3.2 pgvector extension missing

Symptom. `CREATE EXTENSION IF NOT EXISTS vector` error during schema setup, or `column "embedding" is of type vector but expression is of type text` errors at runtime.

Fix. The extension must be installed in the `procurement_agent` database.


```
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent \
  -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Verify the pgvector/pgvector:pg16 image is used — the standard postgres:16 image does not include the vector extension.

14.2.3.3 agent_memory_vectors inserts fail with “invalid input value for enum”

Symptom. The data-normalizer returns {"stored": false} for all memory writes, and PostgreSQL logs show invalid input value for enum embedding_model_type.

Root cause. The base ENUM in database/sql/00_extensions_and_types.sql defines only text-embedding-3-large and e5-large-v2. Migration 22_agent_memory_vector_dim.sql adds all-MiniLM-L6-v2, but BAAI/bge-small-en-v1.5 (used by the data-normalizer memory service) is absent from all migration files.

Fix. Apply migration 22 if not already applied, then manually add the missing value:

```
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent \
  -c "ALTER TYPE embedding_model_type ADD VALUE IF NOT EXISTS
  ↪ 'BAAI/bge-small-en-v1.5';"
```

Similarly, ai_provider_type lacks ollama. Add it if model governance inserts are failing:

```
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent \
  -c "ALTER TYPE ai_provider_type ADD VALUE IF NOT EXISTS 'ollama';"
```

14.2.3.4 agent_memory_vectors table empty after confirmed orders

Symptom. Supplier memory appears absent in subsequent procurements; similarity scores are not improving.

Cause. The orchestrator fires memory writes as `asyncio.create_task(_persist_memory(...))` and never awaits them. If the data-normalizer is temporarily unavailable, or if the fastembed ONNX model call fails inside `POST /normalize/memory/write`, the failure is logged at `DEBUG` level and silently dropped. The `{"stored": false}` response from data-normalizer is not retried.

Diagnosis.

```
# Check orchestrator for _persist_memory errors
docker compose logs orchestrator | grep
↪ "_persist_memory\|memory.*error\|stored.*false"

# Verify data-normalizer is healthy and can reach the DB
curl http://localhost:8006/health
```

14.2.4 2.4 ERP Integration

14.2.4.1 Budget check always denies

Symptom. Every `/confirm` request is blocked with a budget-check failure, even for low-value orders.

Diagnosis.

```
# Check which ERP scenario is active
docker compose exec erp-mock env | grep ERP MOCK_SCENARIO

# Switch to the happy path
curl -X POST http://localhost:8008/mock/scenario \
  -H "Content-Type: application/json" \
  -d '{"scenario": "happy"}'
```

Also verify the fail-open/fail-closed setting. The Docker stack defaults to `ERP_BUDGET_CHECK_REQUIRED=false` (fail-open), meaning a budget check error should allow the confirm to proceed. If `ERP_BUDGET_CHECK_REQUIRED=true` is set and the erp-adapter cannot reach the ERP endpoint within 800 ms, the confirm is blocked regardless.

```
docker compose exec orchestrator env | grep ERP_BUDGET_CHECK_REQUIRED
```

14.2.4.2 Circuit breaker is OPEN

Symptom. GET /readyz on erp-adapter returns a circuit breaker state of 1 (OPEN) for a vendor. All budget checks and PO syncs for that vendor fail immediately.

Diagnosis.

```
# Check per-vendor Prometheus gauge
curl -s http://localhost:8007/metrics | grep erp_circuit_state
# 0 = CLOSED, 1 = OPEN, 2 = HALF_OPEN
```

The breaker enters OPEN after BREAKER_FAIL_MAX (default 5) consecutive failures, and auto-probes to HALF_OPEN after BREAKER_RESET_TIMEOUT_SECS (default 60 seconds). VendorPermanentError (e.g. INSUFFICIENT_FUNDS) does not increment the failure counter.

Fix. Restart the erp-adapter container to reset all breakers to CLOSED immediately, or wait 60 seconds for the auto-probe.

```
docker compose restart erp-adapter
```

14.2.4.3 PO stuck in outbox / DLQ

Symptom. A purchase order was confirmed but has not appeared in the ERP system. Retries have stopped.

Diagnosis.

```
# Inspect outbox row state and last error
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent \
  -c "SELECT id, status, last_error, next_attempt_at FROM erp_sync_records
      WHERE status IN ('DLQ','failed') ORDER BY updated_at DESC LIMIT 10;"
```

Fix. Trigger a manual replay for a specific sync record:

```
curl -X POST http://localhost:8007/api/v1/admin/outbox/<sync_id>/replay
```

Note. If the `erp_sync_records` table does not exist (schema not applied), `erp-adapter` falls back to `In-MemoryOutboxRepo` at startup. POs are not durable in this mode — a service restart loses all pending sync records. Apply the database migrations and restart the service.

14.2.4.4 ERP cancellation silently drops for SAP / Oracle

Expected behaviour. `PATCH /cancel` on the orchestrator should cancel the PO in the ERP system.

Current status. `SAPS4HanaAdapter.cancel_po` and `OracleERPCloudAdapter.cancel_po` raise `NotImplementedError`. Because the orchestrator calls `_cancel_erp_record()` as a fire-and-forget task, this exception is swallowed. The PO status is patched to `cancelled` in local PostgreSQL only; SAP and Oracle are never notified. This is a known Phase 4 gap — `ERP_VENDORS=mock` is the only fully functional configuration for cancellation.

14.2.5 2.5 Frontend

14.2.5.1 Analytics page shows no data / blank dashboard

Symptom. The analytics page is blank or displays a “database unavailable” error banner.

Cause. The `analytics` service returns `HTTP 503` for all three endpoints (`/analytics`, `/business-impact`, `/benchmark`) when it cannot acquire a PostgreSQL connection pool. There is no stale-data cache and no fallback mock.

Fix. Verify that `procurement-postgres` is running and accepting connections:

```
docker ps --filter "name=procurement-postgres"
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent -c "SELECT 1;"
```

Then restart the analytics container to force connection pool re-initialisation:

```
docker compose restart analytics
```

14.2.5.2 WebSocket order tracking not updating

Symptom. The order tracking page shows “Polling every 30s” instead of live updates, or status does not advance after a confirmed order.

Cause. The orchestrator’s session state is an in-process Python `dict`. Restarting the orchestrator container drops all active sessions. The frontend’s `StatusPoller.tsx` falls back to 30-second HTTP polling when the Kafka consumer loop is unavailable.

Fix. Reload the page to start a new session. If Kafka is down, notifications from `sim-bpp` auto-advance events (`SIM_BPP_AUTO_ADVANCE=true`) are silently dropped because `sim-bpp` cannot publish to the `po.status.changed` topic.

```
# Verify Kafka is running and the topic exists
docker compose logs kafka | tail -20
docker exec $(docker compose ps -q kafka) \
  kafka-topics.sh --bootstrap-server localhost:9092 --list
```

14.2.5.3 Keycloak login fails / frontend refuses to start

Symptom. The Next.js frontend crashes at startup with a missing env var error, or the login page redirects loop.

Cause. The frontend uses only `KeycloakProvider` — there are no stub credentials. All three required env vars (`KEYCLOAK_CLIENT_ID`, `KEYCLOAK_CLIENT_SECRET`, `KEYCLOAK_ISSUER`) use the non-null assertion operator and throw at runtime if absent.

Minimum frontend/.env.local required:

```
KEYCLOAK_CLIENT_ID=procurement-frontend
KEYCLOAK_CLIENT_SECRET=<client secret from Phase Two / Keycloak admin>
KEYCLOAK_ISSUER=https://<realm-host>/auth/realms/procurement-agent
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=<any random string>
```

The working tenant used during development is Phase Two (`phasetwo.io`) at `eucl.auth.ac/auth/realms/pro` agent. No Keycloak container is defined in `docker-compose.yml` — a live external IdP is required in all environments including local dev. See [DEPLOYMENT.md](#) for the required realm and client configuration.

14.2.6 2.6 Negotiation Engine

14.2.6.1 Negotiation connection refused — `http://localhost:8012`

Symptom. `POST /negotiate` or `POST /api/demo/negotiate/{thread_id}/supplier-` respond fails with `Connection refused at host.docker.internal:8012`.

Cause. The `claude_openai_proxy` service is not running. This service wraps the local `claude` CLI as an OpenAI-compatible endpoint and is a hard dependency for both `negotiation-engine` and `demo-gateway`. It is not in `docker-compose.yml` and must be started manually on the host.

Fix.

```
# Development (loopback only)
export CLAUDE_PROXY_KEY="dev-key"
uvicorn services.claude_openai_proxy.main:app --host 0.0.0.0 --port 8012

# Persistent (systemd — required for Docker bridge access)
cp services/claude_openai_proxy/claude-proxy.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now claude-proxy
journalctl --user -u claude-proxy -f
```

Prerequisite: the `claude` CLI must be installed and authenticated on the host. Default binary path: `CLAUDE_PROXY_BINARY_PATH=/home/<user>/.local/bin/claude`.

The `negotiation-engine /readyz` probe does not check connectivity to the proxy, so the container may report healthy even when this dependency is missing.

14.2.6.2 Negotiation stuck waiting for on_select

Symptom. The LangGraph negotiation graph stalls in the `waiting_on_select` state and never advances.

Cause. The `beckn-bap-client`'s `on_select` handler is not publishing to Redis after receiving the BPP callback. The negotiation engine subscribes to a Redis channel and blocks until that message arrives.

Diagnosis.

```
# Watch the Redis channel the engine is waiting on
# (substitute the actual transaction_id from the negotiation result)
redis-cli SUBSCRIBE beckn_results:<transaction_id>

# Check that on_select events are arriving at beckn-bap-client
docker compose logs -f beckn-bap-client | grep "on_select"
```

14.2.6.3 LangGraph checkpoint error on negotiation-engine startup

Symptom. `negotiation-engine` logs `AsyncPostgresSaver` errors or raises on first `/negotiate` request.

Cause. The LangGraph `AsyncPostgresSaver` requires the negotiation schema tables defined in `database/sql/20_negotiation_schema.sql`. These are auto-applied to the dedicated postgres Docker service at startup, but if the postgres service was not healthy when the negotiation engine first connected, the tables may not exist.

Fix. Restart the postgres Docker service (not the host PostgreSQL), then restart `negotiation-engine`:

```
docker compose restart postgres
docker compose restart negotiation-engine
```

With `MemorySaver` (in-memory fallback), all negotiation state is lost on restart and cannot be resumed. `AsyncPostgresSaver` requires the `NEGOTIATION_POSTGRES_DSN` environment variable to be set.

14.3 3. Known Limitations (Expected Behavior)

The behaviours below are by design or are deferred to Phase 4. Do not attempt to work around them without understanding the rationale.

Behaviour	Why it happens	Workaround
<code>analytics</code> returns HTTP 503 when PostgreSQL is down	The service has no stale-data cache or fallback mock by design; all three <code>analytics</code> endpoints require a live DB connection.	Restore PostgreSQL, then <code>docker compose restart analytics</code> . No partial degradation mode exists.
All orchestrator sessions lost on container restart	Compare/commit state, pending approvals, ERP state cache, and order enrichments are stored in five module-level Python dict objects (<code>_sessions</code> , <code>_pending_approvals</code> , <code>_erp_state_cache</code> , <code>_order_enrichments</code> , <code>_run_id_index</code>). No Redis-backed or DB-backed session store is wired. TTL is 1800 s.	Reload the frontend page to start a new session after any orchestrator restart. Phase 4 will move sessions to Redis.
Kafka-dependent notifications not delivered	<code>notification-dispatcher</code> will not start its consumer loop if <code>KAFKA_BOOTSTRAP</code> is empty or unreachable. <code>sim-bpp</code> auto-advance state transitions are silently dropped. <code>erp-adapter</code> webhook fan-out events are discarded.	Verify Kafka is healthy: <code>docker compose ps kafka</code> . The Kafka KRaft broker in <code>docker-compose.yml</code> requires a 30 s <code>start_period</code> before its healthcheck clears.
<code>negotiate</code> event type never written to <code>audit_trail_events</code>	<code>_run_autonomous_negotiation</code> in <code>workflow.py</code> contains zero <code>_persist_audit</code> calls. Negotiation outcomes are appended to <code>result["messages"]</code> in memory only.	For compliance audit reconstruction, extract negotiation outcomes from orchestrator logs manually. This is a tracked SOX 404 gap (see analysis <code>22_problems_detected.md</code> §1.9).
IntentParser Docker container always uses <code>qwen3:1.7b</code>	<code>docker-compose.yml</code> overrides <code>COMPLEX_MODEL=qwen3:1.7b</code> , collapsing the two-tier complexity routing. The larger <code>qwen3:8b</code> model is only invoked when IntentParser runs locally outside Docker.	Run IntentParser locally via <code>conda activate infosys_project && cd IntentParser && uvicorn api:app --port 8001 --reload</code> to activate the full routing.

Behaviour	Why it happens	Workaround
discovery_engine multi-network fan-out never runs	services/discovery_engine/ is a complete implementation but has no entry in docker-compose.yml and is not called by any service. The only operational discovery path is through becn-bap-client.	If multi-network search is needed, run discovery_engine standalone on port 8011 (use 8011 to avoid conflict with data-normalizer:8006).
catalog-normalizer UNKNOWN format returns empty offerings when Ollama is down	The LLM fallback in CatalogNormalizer/llm_fallback.py wraps the Ollama call in a top-level try/except. Any Ollama failure returns [] with HTTP 200 and no error surface.	Ensure Ollama is running and NORMALIZER_MODEL=qwen3:1.7b is pulled.
ERP PO cancellation silently dropped for SAP / Oracle	cancel_po raises NotImplementedError in both production adapters; fire-and-forget call swallows the error.	Use ERP_VENDORS=mock for cancellation testing. Production SAP/Oracle cancellation is a Phase 4 item.
Bap-1/ is reference code, not the production BAP	Bap-1/src/server.py runs a standalone FastAPI server on port 8000 with its own CLAUDE.md and 129 tests. It is not wired into docker-compose.yml and is not called by any production service.	The production BAP functions live in: becn-bap-client (protocol), orchestrator (pipeline), data-normalizer (persistence). Do not extend Bap-1 for production features.

14.4 4. Useful Debug Commands

14.4.1 4.1 Docker Logs and Containers

```
# Tail a specific service log
docker compose logs -f orchestrator
docker compose logs -f becn-bap-client
docker compose logs -f negotiation-engine

# Show all container statuses and health
docker compose ps
```

```
# Restart a single service
docker compose restart catalog-normalizer

# Rebuild and restart a single container (picks up code changes)
docker compose up -d --build catalog-normalizer

# Open a shell in a running container
docker exec -it $(docker compose ps -q orchestrator) /bin/bash
```

14.4.2 4.2 Redis

```
# Verify Redis is responding
docker exec $(docker compose ps -q redis) redis-cli PING

# Watch a Beckn discovery callback channel in real time
# Replace <txn_id> with the transaction_id from your /discover call
redis-cli SUBSCRIBE bekn_results:<txn_id>

# List all active bekn_results channels
redis-cli --scan --pattern 'bekn_results:*'

# Flush all Pub/Sub channels (dev only – drops all in-flight callbacks)
docker exec $(docker compose ps -q redis) redis-cli FLUSHDB
```

14.4.3 4.3 PostgreSQL

```
# Connect to the main procurement_agent database
docker exec -it procurement-postgres \
  psql -U postgres -d procurement_agent

# List all tables
\dt
```

```
# Check for ENUM types (useful when debugging embedding_model_type inserts)
SELECT enumlabel FROM pg_enum
  JOIN pg_type ON pg_enum.enumtypid = pg_type.oid
 WHERE pg_type.typname = 'embedding_model_type';

# Inspect audit trail for a specific transaction
SELECT event_type, event_timestamp, actor_id, outcome
FROM audit_trail_events
WHERE transaction_id = '<your_txn_id>'
ORDER BY event_timestamp;

# Find the last 10 failed ERP sync records
SELECT id, status, last_error, next_attempt_at
FROM erp_sync_records
WHERE status IN ('DLQ', 'failed')
ORDER BY updated_at DESC
LIMIT 10;

# Replay a dead-letter outbox record
curl -X POST http://localhost:8007/api/v1/admin/outbox/<sync_id>/replay

# Verify pgvector extension and vector index on bpp_catalog_semantic_cache
SELECT relname, amname FROM pg_class
  JOIN pg_am ON pg_class.relam = pg_am.oid
 WHERE relname LIKE '%bpp_catalog%';

# Check agent memory vector count
SELECT COUNT(*), embedding_model FROM agent_memory_vectors GROUP BY
  ↪ embedding_model;
```

14.4.4 4.4 ERP / Mock Control

```
# Switch erp-mock to the happy path (budget checks pass)
curl -X POST http://localhost:8008/mock/scenario \
  -H "Content-Type: application/json" \
  -d '{"scenario": "happy"}'
```

```
# Switch to the sad path (budget checks deny)
curl -X POST http://localhost:8008/mock/scenario \
  -H "Content-Type: application/json" \
  -d '{"scenario": "insufficient_funds"}'

# Read per-vendor circuit breaker state from Prometheus
curl -s http://localhost:8007/metrics | grep erp_circuit_state

# erp-adapter readiness probe (reports db, redis, circuit states)
curl http://localhost:8007/readyz
```

14.4.5 4.5 IntentParser End-to-End Smoke Test

```
# Full three-stage parse (requires local IntentParser and mcp-sidecar)
curl -X POST http://localhost:8001/parse/full \
  -H "Content-Type: application/json" \
  -d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}'

# Stage 1 only (intent classification)
curl -X POST http://localhost:8001/parse/classify \
  -H "Content-Type: application/json" \
  -d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}'

# Stage 2 only (BecknIntent extraction – no validation)
curl -X POST http://localhost:8001/parse/intent \
  -H "Content-Type: application/json" \
  -d '{"query": "300 meters Cat6 UTP cable Mumbai 5 days"}'
```

14.4.6 4.6 Beckn Protocol / ONIX

```
# Check onix-bap routing config is loaded
docker compose logs onix-bap | grep "loaded\|config\|error"
```

```
# Fire a raw discover directly to onix-bap (bypasses becn-bap-client signing)
# Use only for schema debugging – never POST directly to a BPP
curl -X POST http://localhost:8081/discover \
  -H "Content-Type: application/json" \
  -d @<your_becn_discover_payload.json>

# Watch ONIX routing activity
docker compose logs -f onix-bap | grep -E "route|404|error|action"
```

14.4.7 4.7 MLOps / Comparative Scoring

```
# Start the MLOps sub-stack (separate compose file)
docker compose -f services/ComparativeAndScoring/docker-compose.mlops.yaml \
  up -d mlflow-db mlflow-server prediction-api

# Check prediction-api is using ML model (not static fallback weights)
curl http://localhost:8004/health

# Hot-reload after promoting a new model to Production in MLflow
curl -X POST http://localhost:8004/reload

# Run a scoring request directly
curl -X POST http://localhost:8003/score \
  -H "Content-Type: application/json" \
  -d '{"offerings": [{"provider_id": "p1", "price": 100, "delivery_hours": 48}]}'
```

15 API Reference

This document covers every HTTP API surface in the Procurement Agent stack. All services communicate over the `beckn_network` Docker bridge; inside that network the container name is the hostname. Outside Docker (host-to-container) use `localhost:{host-port}`.

Base URL pattern: `http://{service-name}:{port}` (inside Docker) or `http://localhost:{host-port}` (from the host).

Related reading: [Architecture](#) · [Data Flow](#) · [Configuration](#) · [Components](#)

15.1 Service Port Map

Service	Container port	Host port	Protocol
orchestrator	8004	8004, 8000 (legacy)	HTTP/JSON + WebSocket
IntentParser	8001	8001	HTTP/JSON
beckn-bap-client	8002	8002	HTTP/JSON
data-normalizer	8006	8006	HTTP/JSON
catalog-normalizer	8005	8005	HTTP/JSON
comparative-scoring	8003	8003	HTTP/JSON
erp-adapter	8007	8007	HTTP/JSON
erp-mock	8008	8008	HTTP/JSON
negotiation-engine	8004	18004	HTTP/JSON
frontend-demo-gateway	8015	8015	HTTP/JSON
discovery-engine	8006	—	HTTP/JSON (orphaned — not in docker-compose)
mcp-sidecar	3000	3000	HTTP/SSE + JSON-RPC 2.0

Service	Container port	Host port	Protocol
sim-bpp	3002	3002	HTTP/JSON
analytics	8009	8009	HTTP/JSON
notification-dispatcher	8010	—	Kafka consumer only (no HTTP API)

Port conflict note: both orchestrator and negotiation-engine bind container port 8004. `docker-compose.yml` resolves this by mapping negotiation-engine to host port **18004**. Use `negotiation-engine:8004` inside Docker and `localhost:18004` on the host.

15.2 Shared Data Models

These types appear across multiple endpoints. Always refer to the canonical definitions in `shared/models.py`.

15.2.1 BecknIntent

Field	Type	Required	Notes
item	str	Yes	Canonical item name, e.g. "A4 paper"
descriptions	list[str]	No	Atomic technical specifications, e.g. ["80gsm", "A4"]
quantity	int	Yes	Must be > 0
unit	str	No	Default "units". E.g. "reams", "meters"
location_coordinates	str null	No	"lat, lon" decimal string — never a city name
delivery_timeline	int null	No	Positive integer in hours . 1 day = 24, 1 week = 168 — never ISO 8601

Field	Type	Required	Notes
budget_constraints	BudgetConstraints \\ null	No	{"max": 200.0, "min": 0.0} — never a raw string

15.2.2 DiscoverOffering

Returned by /discover and stored in all session state.

Field	Type	Notes
bpp_id	str	Beckn Provider Platform identifier
bpp_uri	str	ONIX BPP caller URL (always contains caller in path)
provider_id	str	
provider_name	str	
item_id	str	
item_name	str	
price_value	str	Numeric string, e.g. "168.00"
price_currency	str	Default "INR"
available_quantity	int \\ null	
rating	str \\ null	
specifications	list[str]	
fulfillment_hours	int \\ null	Delivery lead time in hours
category	str \\ null	

15.3 orchestrator (:8004)

The **sole entry point** for the Next.js frontend. No other service in the stack should be called directly by the frontend. The orchestrator proxies or orchestrates all downstream calls.

Host port note: `docker-compose.yml` publishes two host ports for this container. Port 8004 is the primary (microservices mode, `BAP_URL=localhost:8004`). Port 8000 is the legacy Bap-1 compatible mode.

15.3.1 Pipeline Entry Points

Method	Path	Description
POST	<code>/run</code>	Unified single-call orchestration: NL query → discover → score → (negotiate) → optional auto-commit
POST	<code>/parse</code>	Proxy to intention-parser <code>/parse</code> (Stages 1+2 only, synchronous)
POST	<code>/discover</code>	Steps 2–4 with a pre-parsed BecknIntent body
POST	<code>/compare</code>	Steps 2–3 only. Stores session. Returns offerings + scoring
POST	<code>/commit</code>	Loads a compare session, then runs select → init → confirm
PATCH	<code>/cancel</code>	Mark a procurement request as cancelled

15.3.2 Session and Order Tracking

Method	Path	Description
GET	<code>/run/{run_id}</code>	Current stage + state snapshot for a <code>/run</code> session. 404 for unknown, 410 Gone if expired
POST	<code>/run/{run_id}/decide</code>	Advance a run after a human decision (advisory / HITL)
GET	<code>/status/{txn_id}/{order_id}</code>	Poll Beckn order lifecycle via becn-bap-client. Always returns 200
GET	<code>/order/{request_id}</code>	Full order detail from data-normalizer, overlaid with in-memory enrichment

15.3.3 WebSocket

Method	Path	Description
GET (WS)	/ws/status/{txn_id}	Real-time order status push. Read-only; sends initial snapshot then Kafka-driven events

15.3.4 Admin, Approvals, Analytics

Method	Path	Description
GET	/health	Liveness. Returns upstream service URL map
GET	/analytics	Proxy to analytics service. Query param: period=30d\ 90d\ 180d
GET	/admin/users	Proxy to data-normalizer /admin/users
PATCH	/admin/users/{user_id}	Proxy to data-normalizer. Updates approval_threshold / department
GET	/approvals	List pending approval requests (in-memory store)
POST	/approvals/{request_id}/dec	Approve or reject a pending order

15.3.5 Inbound Webhook

Method	Path	Auth	Description
POST	/webhooks/seller/status	Signature: base64(HMAC-SHA256(body, SELLER_WEBHOOK_HMAC_SECRET))	BPP pushes state change. Persists, audits, publishes to Kafka status . changed

15.3.6 POST /run

Unified orchestration. Accepts BecknIntent fields plus mode extras. Internally runs Steps 1–4 and, for the autonomous mode, drives automated negotiation via the demo-gateway.

Request body

```
{
  "item": "A4 paper",
  "descriptions": ["80gsm"],
  "quantity": 500,
  "unit": "reams",
  "location_coordinates": "12.9716,77.5946",
  "delivery_timeline": 72,
  "budget_constraints": { "max": 200.0 },
  "raw_query": "500 reams A4 paper Bangalore 3 days",
  "execution_mode": "advisory",
  "actor": { "user_id": "...", "role": "requester" }
}
```

raw_query, actor, and execution_mode are optional extras not part of BecknIntent.

execution_mode	Behaviour
advisory (default)	Returns ranked offerings. Human selects via POST /run/{run_id}/decide
hitl	Agent recommends one item. Human approves or rejects
autonomous	Agent commits automatically when within the requester’s RBAC threshold

Response 200 — advisory, stage awaiting_selection

```
{
  "run_id": "abc123",
  "transaction_id": "uuid",
  "request_id": "uuid",
  "stage": "awaiting_selection",
  "execution_mode": "advisory",
  "offerings": [],
  "recommended_item_id": "item_id_string",
}
```

```

"scoring": {
  "recommended_item_id": "...",
  "criteria": [{ "key": "price", "label": "Price", "weight": 1.0, "direction":
    ↪ "min", "scores": [] }],
  "ranking": [{ "item_id": "...", "composite_score": 0.9, "rank": 1 }]
},
"reasoning_steps": [],
"messages": ["Step 1 ...", "Step 2 ..."],
"decision": {},
"status": "live"
}

```

status is "mock" when the ONIX Docker stack is offline.

Response 202 — RBAC approval required, stage awaiting_rbac_approval

```

{
  "run_id": "abc123",
  "stage": "awaiting_rbac_approval",
  "request_id": "uuid",
  "amount_total": 95000.0,
  "explanation": "Amount exceeds requester threshold ...",
  "decision": {}
}

```

Response 200 — autonomous path, stage confirmed

```

{
  "run_id": "abc123",
  "stage": "confirmed",
  "order_id": "order_uuid",
  "order_state": "ACTIVE",
  "transaction_id": "uuid",
  "contract_id": "uuid",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "reasoning_steps": [],
  "messages": [],
  "status": "live",
  "offerings": [],
  "negotiation_settled_price": null
}

```

15.3.7 POST /compare

Runs Steps 2–3 (discover + score). Stores session keyed by `transaction_id` with a 30-minute TTL. Does **not** run Step 4 (select).

Request body — BecknIntent fields plus optional `raw_query`: str.

Response 200

```
{
  "transaction_id": "uuid",
  "request_id": "uuid",
  "offerings": [],
  "recommended_item_id": "item_id_string",
  "scoring": {
    "recommended_item_id": "...",
    "criteria": [],
    "ranking": []
  },
  "reasoning_steps": [{ "node": "memory_context", "role": "observe", "content":
    ↪ "..."}],
  "messages": ["Step 2: Discovered 5 offerings", "Step 3: Scored offerings"],
  "status": "live"
}
```

15.3.8 POST /commit

Loads the compare session by `transaction_id`, overrides the selected item with `chosen_item_id`, then drives `/select` → `/init` → `/confirm` through `beckn-bap-client`.

If `order_total > user.approval_threshold`, the commit is queued to `_pending_approvals` and returns 202 Accepted instead of 200.

Request body

```
{ "transaction_id": "uuid", "chosen_item_id": "item_id_string" }
```

Response 200

```
{
  "transaction_id": "uuid",
  "request_id": "uuid",
  "order_id": "order_uuid",
  "order_state": "ACTIVE",
  "payment_terms": {
    "type": "ON_FULFILLMENT",
    "collected_by": "BPP",
    "currency": "INR",
    "status": "COMMITTED"
  },
  "fulfillment_eta": null,
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "contract_id": "uuid",
  "reasoning_steps": [],
  "messages": [],
  "status": "live"
}
```

Response 202 — approval gate

```
{
  "status": "awaiting_approval",
  "message": "Order requires manager approval ...",
  "request_id": "uuid",
  "transaction_id": "uuid",
  "amount_total": 120000.0
}
```

15.3.9 POST /run/{run_id}/decide

Advances a run after a human decision.

Request body

```
{
  "decision": "proceed",
  "chosen_item_id": "item_id_string",
  "approver_id": "user_uuid"
}
```

Stage	decision values	Notes
awaiting_selection	proceed	chosen_item_id required
awaiting_approval (HITL)	proceed	Uses decision.final_item_id from session; chosen_item_id is an optional override
Any	reject	Cancels run. For HITL, transitions to awaiting_selection instead

Response 200 — same shape as the /run confirmed response. **Response 202** — if the approved commit triggers a further RBAC escalation.

15.3.10 POST /approvals/{request_id}/decide

decision must be "approved" or "rejected". Returns 404 for unknown request_id.

- rejected — persists cancelled status, removes from pending queue.
- approved — resumes Beckn select → init → confirm from the saved session; falls back to a mock response if the session has expired.

Request body

```
{ "decision": "approved" }
```

15.3.11 GET /status/{txn_id}/{order_id}

Polls order lifecycle via becn-bap-client /status. Always returns 200. Session data provides bpp_id/bpp_uri; query params ?bpp_id=&bpp_uri= are the fallback when the session has expired.

Response 200

```
{
  "state": "SHIPPED",
  "observed_at": "2026-07-07T10:00:00.000Z",
  "fulfillment_eta": null,
  "tracking_url": null,
  "erp_state": {
    "state": "SHIPPED",
    "vendor": "sap",
    "erp_reference_id": "4500123456",
    "event_ts": "2026-07-07T09:45:00Z",
    "vendor_event_id": "sap-evt-001"
  },
  "status": "live"
}
```

erp_state is null when no ERP inbound event has been cached for the transaction.

15.3.12 orchestrator Environment Variables

Variable	Default	Notes
INTENTION_PARSER_URL	http://localhost:8001	
BECKN_BAP_URL	http://localhost:8002	
COMPARATIVE_SCORING_URL	http://localhost:8003	
DATA_NORMALIZER_URL	http://localhost:8006	
ANALYTICS_URL	http://localhost:8009	
ERP_ADAPTER_URL	http://localhost:8007	
DEMO_GATEWAY_URL	http://localhost:8015	Required for autonomous negotiation; correct port is 8015

Variable	Default	Notes
ERP_BUDGET_CHECK_ENABLED	"true"	
ERP_BUDGET_CHECK_REQUIRED	"false"	"true" = fail-closed; blocks commit if ERP is unreachable
ERP_BUDGET_CHECK_TIMEOUT_MS	800	Hard ceiling for the synchronous budget gate
ERP_SYNC_ENABLED	"true"	
KAFKA_BOOTSTRAP	""	Empty disables the Kafka producer/consumer
KAFKA_TOPIC	po.status.changed	
SELLER_WEBHOOK_HMAC_SECRET	dev-seller-hmac-CHANGE_ME	Change before production
BUYER_NAME	Procurement Agent	Used in billing and fulfillment blocks
BUYER_EMAIL	procurement@example.com	

15.4 IntentParser (:8001)

FastAPI service. Runs **locally by default** (not Dockerised). The `intention-parser` Docker container wraps only Stages 1+2; Stage 3 (pgvector + MCP sidecar validation) runs only in the local process.

All three parsing stages depend on a local [Ollama](#) instance serving `qwen3:8b` and `qwen3:1.7b`.

Complexity routing rule (Stage 2 model selection): query longer than 120 chars, or at least 2 numeric tokens, or contains a procurement keyword (delivery, budget, timeline, etc.) → `COMPLEX_MODEL` (default `qwen3:8b` locally, `qwen3:1.7b` in Docker). Otherwise → `SIMPLE_MODEL` (`qwen3:1.7b`).

15.4.1 Endpoints

Method	Path	Stages	Ollama required	Notes
POST	/parse	1+2	Yes	Synchronous; Stage 3 disabled

Method	Path	Stages	Ollama required	Notes
POST	/parse/batch	1+2	Yes	Parallel batch via ThreadPoolExecutor
POST	/parse/full	1+2+3+recovery	Yes + pgvector + MCP sidecar	Async; full pipeline

There is no GET /health endpoint in api.py. Probe liveness with a minimal POST /parse request.

15.4.2 POST /parse

Runs Stage 1 (intent classification) then Stage 2 (BecknIntent extraction). Synchronous; blocks on Ollama inference.

Request body

```
{ "query": "500 reams A4 paper Bangalore 3 days" }
```

Response — ParseResult

```
{
  "intent": "SearchProduct",
  "confidence": 0.95,
  "beckn_intent": {
    "item": "A4 paper",
    "descriptions": ["80gsm", "A4"],
    "quantity": 500,
    "unit": "reams",
    "location_coordinates": "12.9716,77.5946",
    "delivery_timeline": 72,
    "budget_constraints": null
  },
  "routed_to": "qwen3:1.7b"
}
```

For a non-procurement query:

```
{ "intent": "GeneralInquiry", "confidence": 0.98, "beckn_intent": null,
  ↪ "routed_to": "qwen3:1.7b" }
```

15.4.3 POST /parse/batch

Runs Stage 1+2 on multiple queries in parallel using `ThreadPoolExecutor` (default `max_workers=4`).

Request body

```
{ "queries": ["query one", "query two"], "max_workers": 4 }
```

Response — `list[ParseResult]` in the same order as input.

15.4.4 POST /parse/full

Full async pipeline: Stage 1 → Stage 2 → Stage 3 (pgvector ANN cache + MCP sidecar ONIX probe) → recovery flow if not found.

Requires: local Ollama, PostgreSQL 16 with pgvector, and mcp-sidecar on port 3000.

Request body — same as `/parse`.

Response — `ParseResponse`

```
{
  "intent": "SearchProduct",
  "confidence": 0.91,
  "beckn_intent": {
    "item": "Cat6 UTP cable",
    "descriptions": ["Cat6", "UTP", "ethernet"],
    "quantity": 300,
    "unit": "meters",
    "location_coordinates": "19.0760,72.8777",
    "delivery_timeline": 120,
```

```
    "budget_constraints": null
  },
  "validation": {
    "zone": "VALIDATED",
    "top_match": {
      "item_name": "Cat6 Cable UTP",
      "bpp_id": "bpp.example.com",
      "similarity": 0.91
    },
    "mcp_validated": false
  },
  "recovery_log": [],
  "routed_to": "qwen3:1.7b"
}
```

validation.zone values

Zone	Condition	Action
VALIDATED	ANN cosine similarity >= 0.85	Return immediately
AMBIGUOUS	Similarity 0.45–0.85	MCP sidecar probe attempted
CACHE_MISS	Similarity < 0.45	MCP probe; if not found, recovery flow triggered

Recovery flow (triggered when zone = CACHE_MISS and MCP returns found: false):

1. broaden_procurement_query() — strips specifics; optionally calls Claude Sonnet 4.6 if ANTHROPIC_API_KEY is set.
2. Stage 3 retry with the broadened query.
3. log_unmet_demand() — stub (logs only; no DB write in current implementation).
4. notify_buyer_no_stock() — stub.
5. trigger_open_rfq_flow() — stub.

recovery_log contains messages from each step.

15.4.5 IntentParser Environment Variables

Variable	Default	Notes
OLLAMA_BASE_URL	http://localhost:11434/v1	
COMPLEX_MODEL	qwen3:8b (local) / qwen3:1.7b (Docker)	Docker collapses both tiers to qwen3:1.7b
SIMPLE_MODEL	qwen3:1.7b	
ANTHROPIC_API_KEY	""	Optional; required only for Stage 3 broadening fallback
DB_HOST	localhost	Required for Stage 3 pgvector cache
DB_PORT	5432	
DB_NAME	procurement_agent	
DB_USER	postgres	
DB_PASSWORD	""	

15.5 becn-bap-client (:8002)

aihttp service. This service is the **exclusive gateway for all Beckn protocol traffic**. All Beckn actions route through onix-bap:8081. Never POST directly to a BPP — all `select_url` values must contain `caller` in the path.

The service exposes two route groups: orchestrator-facing routes (trigger Beckn actions) and ONIX callback receivers.

15.5.1 Orchestrator-Facing Routes

Method	Path	Description
GET	/health	Returns {"status":"ok","service":"bekn-bap-client","bap_id":"..."}
POST	/discover	Runs discovery via ONIX, awaits on_discover callback
POST	/select	Sends /select through ONIX

Method	Path	Description
POST	/init	Sends /init through ONIX, awaits on_init callback
POST	/confirm	Sends /confirm through ONIX, awaits on_confirm callback
POST	/status	Sends /status through ONIX, awaits on_status callback. Never returns 5xx

15.5.2 ONIX Callback Receivers

Method	Path	Description
POST	/on_discover	Dedicated async on_discover webhook; publishes to Redis and unblocks CallbackCollector
POST	/bap/receiver/{action}	Generic callback receiver: on_select, on_init, on_confirm, on_status
POST	{action}	Wildcard for direct real Beckn network callbacks

15.5.3 POST /discover

Accepts a BecknIntent JSON body plus an optional transaction_id. Fires a Beckn /discover to onix-bap via asyncio.create_task (non-blocking), then awaits the on_discover callback via CallbackCollector. CALLBACK_TIMEOUT (default 10 s) is the wait ceiling.

Request body

```
{
  "item": "Cat6 UTP cable",
  "descriptions": ["Cat6", "UTP"],
  "quantity": 300,
  "unit": "meters",
```

```
"location_coordinates": "19.0760,72.8777",
"delivery_timeline": 120,
"transaction_id": "pre-generated-uuid"
}
```

transaction_id is injected by the MCP sidecar so the Redis channel name (beckn_results:{transaction_id} matches the channel it has already subscribed to. The orchestrator omits it and lets the service generate one.

Response 200

```
{
  "transaction_id": "uuid",
  "offerings": []
}
```

15.5.4 POST /select

Sends a Beckn /select action through onix-bap. Required fields: transaction_id, bpp_id, bpp_uri.

Request body

```
{
  "transaction_id": "uuid",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "provider_id": "provider_001",
  "item_id": "item_001",
  "item_name": "Cat6 UTP cable",
  "quantity": 300,
  "price_value": "45.00",
  "price_currency": "INR"
}
```

Response 200

```
{ "ack": "ACK" }
```

15.5.5 POST /init

Sends a Beckn /init action through onix-bap and awaits the on_init callback. Required fields: transaction_id, contract_id, bpp_id, bpp_uri.

Request body

```
{
  "transaction_id": "uuid",
  "contract_id": "uuid",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "items": [
    { "id": "item_001", "quantity": 300, "name": "Cat6 UTP cable",
      "price_value": "45.00", "price_currency": "INR" }
  ],
  "billing": {
    "name": "Infosys Limited",
    "email": "procurement@infosys.com",
    "phone": "+91-80-28520261",
    "address": { "city": "Bangalore", "area_code": "560100", "country": "IND" }
  },
  "fulfillment": {
    "end_location": "12.9716,77.5946",
    "end_address": { "city": "Bangalore", "area_code": "560100", "country": "IND"
      ↪ },
    "contact_name": "Infosys Limited",
    "contact_phone": "+91-80-28520261",
    "delivery_timeline": 72
  }
}
```

billing and fulfillment are optional; the orchestrator always supplies them via _build_billing_info() / _build_fulfillment_info() from BUYER_* env vars.

Response 200


```
{
  "payment_terms": {
    "type": "ON_FULFILLMENT",
    "collected_by": "BPP",
    "currency": "INR",
    "status": "COMMITTED"
  },
  "contract_id": "uuid",
  "ack": "ACK"
}
```

`payment_terms.status` valid enum: DRAFT | COMMITTED | COMPLETE.

15.5.6 POST /confirm

Sends a Beckn /confirm action through onix-bap and awaits the `on_confirm` callback. On timeout returns `ack: "NACK"` with `order_id: null` so the orchestrator can build a mock response instead of propagating a 5xx.

Required fields: `transaction_id`, `contract_id`, `bpp_id`, `bpp_uri`.

Request body

```
{
  "transaction_id": "uuid",
  "contract_id": "uuid",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "items": [
    { "id": "item_001", "quantity": 300, "name": "Cat6 UTP cable",
      "price_value": "45.00", "price_currency": "INR" }
  ],
  "payment_terms": {
    "type": "ON_FULFILLMENT",
    "collected_by": "BPP",
    "currency": "INR",
    "status": "COMMITTED"
  }
}
```

Response 200 — success

```
{ "order_id": "order_uuid", "order_state": "ACTIVE", "ack": "ACK" }
```

Response 200 — callback timeout

```
{ "order_id": null, "order_state": "CREATED", "ack": "NACK" }
```

Contract.status.code valid enum: DRAFT | ACTIVE | CANCELLED | COMPLETE.CONFIRMED is **invalid** and rejected by the ONIX schema validator.

15.5.7 POST /status

Sends a Beckn /status action through onix-bap and awaits the on_status callback. Never returns 5xx — any failure returns a stub response so the orchestrator's polling loop is never interrupted.

Required fields: transaction_id, order_id, bpp_id, bpp_uri.

Request body

```
{
  "transaction_id": "uuid",
  "order_id": "order_uuid",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver",
  "items": [{ "id": "item_001", "quantity": 300 }]
}
```

Response 200 — on failure returns {"state": "CREATED", "fulfillment_eta": null, "tracking_url": null}.

```
{ "state": "SHIPPED", "fulfillment_eta": null, "tracking_url": null }
```

Beckn order state values reported by sim-bpp: ACCEPTED | PACKED | SHIPPED | OUT_FOR_DELIVERY | DELIVERED | CANCELLED

15.5.8 POST /on_discover

Dedicated webhook for the Beckn on_discover async callback from onix-bap. Dual publish behaviour:

- 1. Publishes the full payload to Redis channel `beckn_results:{transactionId}` — unblocks the MCP sidecar’s SUBSCRIBE (see [ADR-0001](#)).
- 2. Feeds the CallbackCollector so the orchestrator’s collect flow also unblocks.

If Redis is unavailable, logs a warning and continues via CallbackCollector only; the MCP sidecar will time out and return `found: false`.

ONIX routing note: the BAPReceiver routing YAML routes `on_discover` to the base URL `http://beckn-bap-client:8002`; ONIX appends `/on_discover`. Never include the action name in the routing target URL — ONIX appends it and including it doubles the path, producing a 404.

Response — immediate `{"message": {"ack": {"status": "ACK"}}}` returned before downstream processing to satisfy the Beckn protocol contract.

15.5.9 Beckn Lifecycle Sequence

15.5.10 becn-bap-client Environment Variables

Variable	Default	Notes
ONIX_URL	http://localhost:8081	Docker: http://onix-bap:8081
BAP_URI	http://localhost:8002	Must be reachable by onix-bap for callbacks
BAP_ID	bap.example.com	
DOMAIN	nic2004:52110	Docker: becn.one/testnet
CALLBACK_TIMEOUT	10.0	Seconds. Ceiling for on_discover / on_init/on_confirm wait
CATALOG_NORMALIZER_URL	http://localhost:8005	Docker: http://catalog-normalizer:8005

Variable	Default	Notes
REDIS_URL	redis://localhost:6379	Docker: redis://redis:6379

15.6 data-normalizer (:8006)

The **sole write path** into PostgreSQL for the procurement pipeline. Every persistence action in the orchestrator, becn-bap-client, sim-bpp, and the frontend goes through this service. It never calls any other service; it only reads from and writes to the database.

15.6.1 Pipeline Write Path

Method	Path	Purpose	Success
POST	/normalize/request	Create procurement_request from raw text	201 {request_id: uuid}
POST	/normalize/intent	Persist parsed_intent + becn_intent after Stage 1+2	201 {intent_id, becn_intent_id}
POST	/normalize/discovery	Persist discovery_query + seller_offerings	201 {query_id, offering_ids[]}
POST	/normalize/scoring	Persist scored_offers	201
POST	/normalize/order	Persist full FK chain → purchase_order	201 {po_id}
PATCH	/normalize/status	Update procure-ment_request.status	200
PATCH	/normalize/po_status	Update purchase_order.status by becn_confirm_ref	200

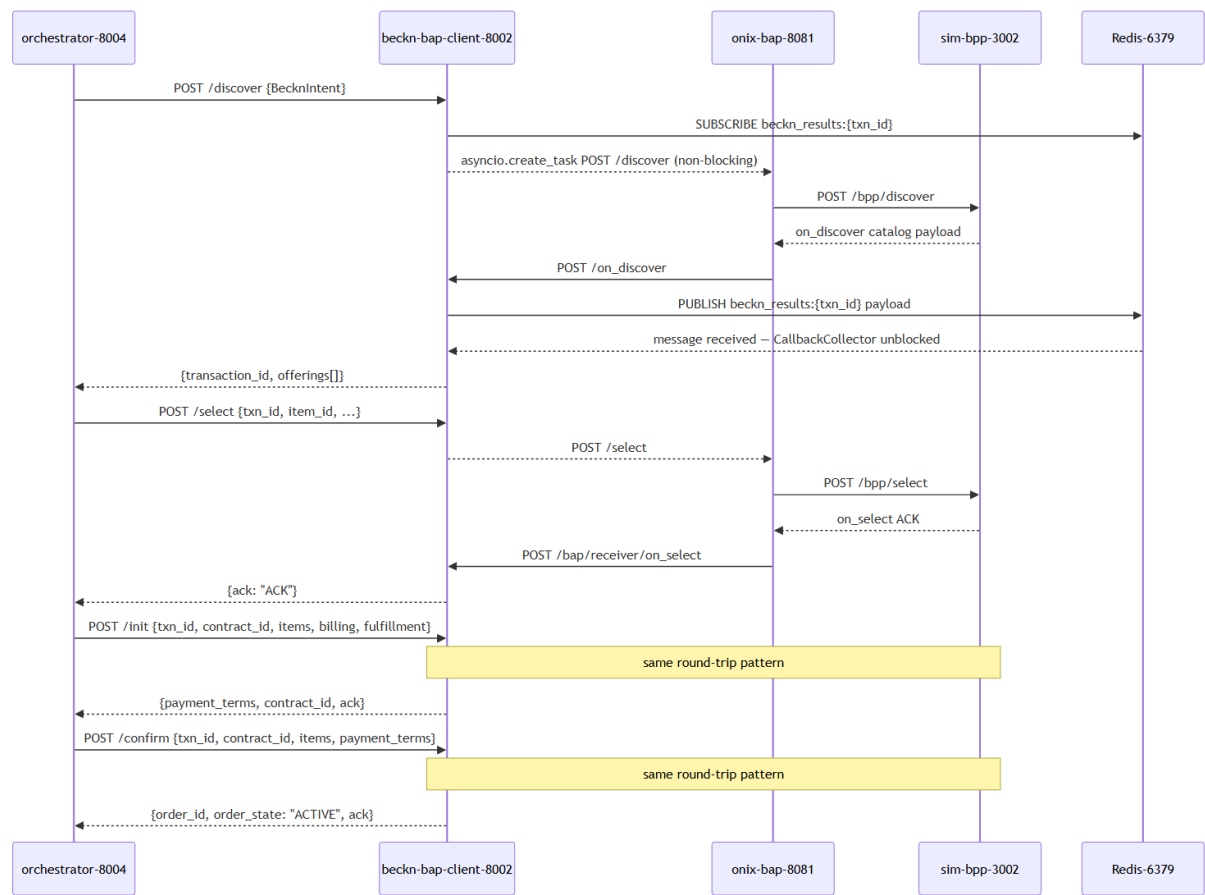


Figure 15.1: API_REFERENCE diagram 1

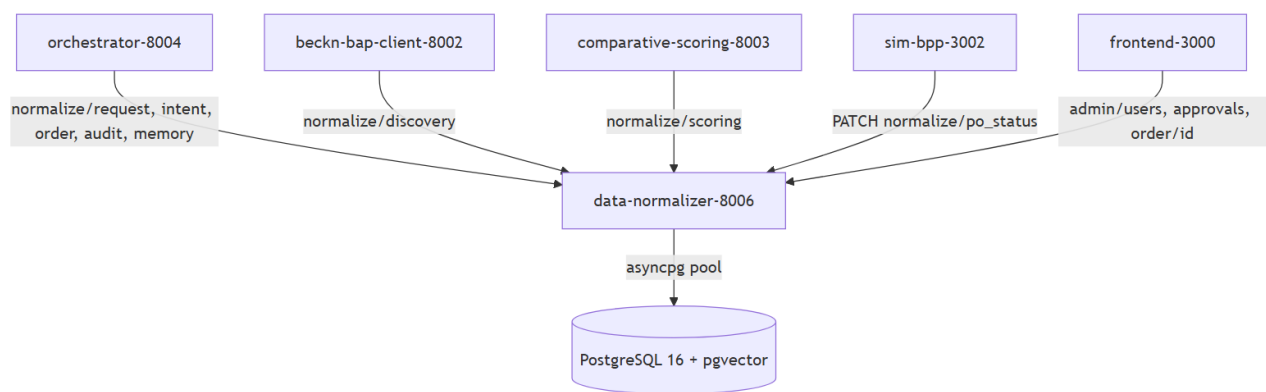


Figure 15.2: API_REFERENCE diagram 2

15.6.2 Order Retrieval

Method	Path	Purpose
GET	/order/{request_id}	Full order detail DTO. Returns 404 if not found

15.6.3 Audit Trail

Method	Path	Purpose
POST	/normalize/audit	Persist audit_trail_event. Returns {event_id: uuid} with status 201
GET	/normalize/audit?request_id	List all events for a request (max 500, chronological)
GET	/normalize/audit?po_id=X	List all events for a confirmed PO
GET	/normalize/audit/{event_id}	Single event with full reasoning_payload JSONB

retention_until is set to event_timestamp + 7 years on every insert.

15.6.4 Agent Memory

Method	Path	Purpose
POST	/normalize/memory/write	Embed and store a confirmed transaction in agent_memory_vectors (vector(384), HNSW cosine). Embedding failures are silently skipped — never returns an error for embedding failure alone.
POST	/normalize/memory/search	ANN cosine search. Returns top-k (default 3) past transactions with similarity >= 0.75

Embedding is performed by sentence-transformers all-MiniLM-L6-v2 (384 dimensions).

15.6.5 Admin and Approvals

Method	Path	Purpose	Notes
GET	/admin/users	List all users	
PATCH	/admin/users/{user_id}	Update approval_threshold or department	Role is not updatable here; Keycloak is source of truth
GET	/approvals	List requests in pending_approval status	
POST	/approvals/{request_id}	Approve or reject a pending request	

15.6.6 Liveness

GET /health → {"status": "ok", "service": "data-normalizer"}

15.6.7 Key Normalizations Applied on Write

Field	Input	Stored as	Rule
price_value	str	DECIMAL(15,2)	float(price_value)
fulfillment_hours	Optional[int]	INTEGER NOT NULL	None → 24
composite_score	float [0, 1]	FLOAT [0, 100]	× 100, clamped
channel	any string	text	Unrecognised channels coerced to "web"
confidence	float	float	Clamped to [0.0, 1.0]

15.6.8 FK Chain for purchase_order Creation

POST /normalize/order creates four rows in one transaction. If the amount exceeds the requester's approval_threshold, the chain pauses at approval_decisions with status="pending" and the orchestrator routes to the approvals queue.

```
negotiation_outcomes (strategy="skipped")
└─ approval_decisions (status="auto_approved")
    └─ purchase_orders
```

15.6.9 Error Handling

db_error_middleware maps asyncpg exceptions before they reach the caller:

asyncpg exception	HTTP status	Body
UniqueViolationError	409	{"error": "duplicate"}
ForeignKeyViolationError	409	{"error": "fk_violation"}
CheckViolationError	422	structured
NotNullViolationError	422	structured
Any other PostgresError	500	generic

15.6.10 Configuration

Variable	Default	Notes
DB_HOST	host.docker.internal	Override to container name if using containerised Postgres
DB_PORT	5432	
DB_NAME	procurement_agent	
DB_USER	postgres	
DB_PASSWORD	postgres123	
SYSTEM_USER_ID	00000000-0000-0000-0000-000000000001	UUID for automated/system-initiated requests when no requester_id is supplied. Not in docker-compose.yml; hardcoded default applies everywhere.

15.7 erp-adapter (:8007)

Vendor-neutral ERP integration microservice. Provides two synchronous interfaces consumed by the orchestrator (budget gate, policy evaluation) and one async outbox worker that pushes purchase orders to SAP S/4HANA or Oracle ERP Cloud. Receives inbound lifecycle events from ERP vendors via HMAC-authenticated webhooks.

15.7.1 All Endpoints

Method	Path	Auth	Purpose
GET	/healthz	none	Process liveness; always 200 while alive
GET	/readyz	none	DB + Redis + per-vendor health + outbox lag check
GET	/metrics	none	Prometheus exposition (9 metric series)
POST	/api/v1/budget/check	bearer	Synchronous budget gate; hard timeout 800 ms
POST	/api/v1/po/sync	bearer	Enqueue PO push; idempotent on transaction_id; returns 202
GET	/api/v1/po/sync/{sync_id}	bearer	Poll outbox row state
POST	/api/v1/admin/outbox/{sync_id}/replay	bearer	Reset dead-letter-queue row to pending for retry
POST	/api/v1/policy/evaluate	bearer	ERP policy gate; returns preferred suppliers, approval flags, constraints
POST	/api/v1/webhooks/sap/HMAC status	HMAC	Inbound from SAP S/4HANA
POST	/api/v1/webhooks/ora/HMAC status	HMAC	Inbound from Oracle ERP Cloud
POST	/api/v1/webhooks/mock/HMAC status	HMAC	Inbound from erp-mock (dev only; uses SAP_WEBHOOK_HMAC_SECRET)

15.7.2 Authentication

Bearer tokens — all `/api/v1/*` routes (except webhooks) require `Authorization: Bearer ${ERP_INTERNAL_TOKEN}`.

HMAC webhook verification — inbound vendor webhooks carry `X-{Vendor}-Signature: sha256=<hex>`. The service verifies against `{VENDOR}_WEBHOOK_HMAC_SECRET` (primary) and, on failure, against `{VENDOR}_WEBHOOK_HMAC_SECRET_NEXT` (rotation secret). This dual-secret pattern enables zero-downtime key rotation.

15.7.3 POST /api/v1/budget/check

Called synchronously by the orchestrator before `commit`. Blocks commit when `ERP_BUDGET_CHECK_REQUIRED=true` and the service is unreachable. Hard outer timeout: `BUDGET_CHECK_TOTAL_TIMEOUT_MS` (default 800 ms).

Request body

```
{
  "transaction_id": "uuid",
  "amount_total": 45000.00,
  "currency": "INR",
  "cost_center": "CC-IND-PROC-01",
  "requester_id": "uuid"
}
```

Response — allowed

```
{
  "allowed": true,
  "available_balance": "250000.00",
  "hold_id": "hold-<uuid12>"
}
```

Response — denied

```
{
  "allowed": false,
  "available_balance": "0.00",
  "reasons": ["INSUFFICIENT_FUNDS"]
}
```

15.7.4 POST /api/v1/po/sync

Enqueues the PO push to the outbox worker. Idempotent on `transaction_id`. Returns 202 immediately; the actual ERP call is asynchronous.

Request body

```
{
  "transaction_id": "uuid",
  "po_id": "uuid",
  "vendor_id": "bpp.example.com",
  "line_items": [{ "item_id": "...", "quantity": 10, "unit_price": 450.0 }],
  "total_amount": 4500.0,
  "currency": "INR"
}
```

Response 202

```
{ "sync_id": "uuid", "status": "pending" }
```

Poll status with GET `/api/v1/po/sync/{sync_id}`. Replay dead-letter rows with POST `/api/v1/admin/outbox/{sync_id}/replay`.

15.7.5 POST /api/v1/policy/evaluate

Returns ERP-level procurement policy constraints. Fail-open on transient errors.

Response

```
{
  "preferred_supplier_ids": ["preferred-bpp-001"],
  "approval_required": false,
  "auto_commit_allowed": true,
  "constraints": {}
}
```

15.7.6 Outbox Pattern

PO push uses FOR UPDATE SKIP LOCKED on `erp_sync_records`. Retry backoff is configured by `WORKER_BACKOFF_CSV` (default 5,30,120,600,3600 seconds). Rows that exhaust all retries enter failed (dead-letter) status and must be replayed manually.

15.7.7 Circuit Breakers

Per-vendor pybreaker circuit breakers. Trips to OPEN after `BREAKER_FAIL_MAX` (default 5) consecutive failures. Auto-probes after `BREAKER_RESET_TIMEOUT_SECS` (default 60 s). The budget check endpoint enforces a hard outer timeout of 800 ms regardless of breaker state.

15.7.8 erp-adapter Environment Variables

Variable	Default	Notes
DB_HOST	host.docker.internal	
REDIS_URL	redis://redis:6379	
ERP_INTERNAL_TOKEN	dev-internal-token-CHANGE_ME	Change before production
ERP_VENDORS	mock	Options: mock, sap, oracle, sap, oracle
ERP_BUDGET_CHECK_REQUIRED	true	Fail-closed; set false for dev fail-open
BUDGET_CHECK_TOTAL_TIMEOUT	800	Hard cap for /api/v1/budget/check
SAP_WEBHOOK_HMAC_SECRET	dev-sap-hmac-CHANGE_ME	Primary signing key

Variable	Default	Notes
SAP_WEBHOOK_HMAC_SECRET_NEX	""	Rotation key (empty = disabled)
ORACLE_WEBHOOK_HMAC_SECRET	dev-oracle-hmac-CHANGE_ME	
KAFKA_BOOTSTRAP	kafka:9092	Empty string = Kafka disabled
KAFKA_TOPIC	po.status.changed	

15.8 Remaining Services — Compact Reference

15.8.1 catalog-normalizer (:8005)

Converts raw `on_discover` payloads from any BPP catalog format into a uniform list of `DiscoverOffering` objects. No database dependency. LLM fallback path uses Ollama (`qwen3:1.7b`) via the OpenAI SDK as a compatibility shim — `OPENAI_API_KEY` is never read despite being mentioned in older README versions.

Method	Path	Purpose
GET	/health	<code>{"status": "ok", "service": "catalog-normalizer"}</code>
POST	/normalize	Convert raw <code>on_discover</code> payload → <code>{offerings[], format_variant}</code>

POST /normalize request body

```
{
  "payload": "<raw Beckn on_discover message object>",
  "bpp_id": "bpp.example.com",
  "bpp_uri": "http://onix-bpp:8082/bpp/receiver"
}
```

On UNKNOWN payloads with Ollama unreachable, `offerings` is `[]` — no error is raised.

Format detection order: ONDC (fingerprint: fulfillments[] + tags[]) → BECKN_V2_FLAT_RESOURCES (resources[] non-empty) → LEGACY_PROVIDERS_ITEMS → BPP_CATALOG_V1 → UNKNOWN (LLM fall-back).

Variable	Default
OLLAMA_URL	http://localhost:11434/v1
NORMALIZER_MODEL	qwen3:1.7b

15.8.2 mcp-sidecar (:3000, SSE transport)

An MCP server that lets IntentParser’s Stage 3 validation probe the live Beckn network without blocking the event loop. Stateless; every tool call is an independent probe. The service **never throws a JSON-RPC error** — all failure paths return {"found": false, ...}.

Method	Path	Purpose
GET	/sse	Opens an SSE stream; server sends "endpoint" event with the POST URL
POST	/messages/	JSON-RPC 2.0 tools/call dispatcher; response arrives as "message" SSE event

Tool: search_bpp_catalog

The only registered MCP tool. Fires a non-blocking Beckn discover probe, waits for on_discover callback via Redis Pub/Sub, and returns semantically ranked items.

Argument	Type	Required	Description
item_name	string	Yes	Blank value returns found: false immediately without a network call
descriptions	array[string]	Yes	Specification tokens; empty array [] is valid

Argument	Type	Required	Description
domain	string	Yes	Beckn domain, e.g. "procurement"
version	string	Yes	Beckn protocol version, e.g. "1.1.0"
location	string	No	"lat,lon" decimal string

Success response:

```
{
  "found": true,
  "items": [
    { "item_name": "Cat6 Cable UTP", "bpp_id": "bpp.example.com", "bpp_uri":
      ↪ "http://onix-bpp:8082/bpp/receiver" }
  ],
  "probe_latency_ms": 1420
}
```

All failure paths (timeout, unreachable BAP, blank item_name, zero matches, any internal exception):

```
{ "found": false, "items": [], "probe_latency_ms": 3000 }
```

Items are sorted by descending cosine similarity (all-MiniLM-L6-v2, 384 dims). Items below RANK-ING_MIN_SIMILARITY (default 0.30) are filtered.

The discover POST fires via `asyncio.create_task` and must **never be awaited** inside the sidecar. Awaiting it re-introduces the deadlock ADR-0001 was written to fix.

Variable	Default	Notes
BAP_API_KEY	(none)	Mandatory — service refuses to start without it; never commit to source control
BAP_CLIENT_URL	http://localhost:8002	
REDIS_URL	redis://localhost:6379	Must be an exported shell env var, not only in .env

Variable	Default	Notes
REDIS_RESULT_TIMEOUT	15	Must be an exported shell env var, not only in .env
RANKING_MIN_SIMILARITY	0.30	

15.8.3 sim-bpp (:3002)

Local Node.js Beckn Provider Platform simulator. Replaced fidedocker/sandbox-2.0. Supports all ten Beckn lifecycle actions, hot-reloads catalog.json without rebuild (bind-mounted), and optionally auto-advances order status.

Method	Path	Purpose
GET	/api/health	{"status": "ok", "service": "sim-bpp", "bpp_id": "bpp.example.com"}
POST	/api/webhook/{action}	Inbound Beckn action from onix-bpp. ACKs synchronously, fires on_{action} to onix-bpp:8082/bpp/caller asynchronously

Supported actions: discover, select, init, confirm, status, track, update, cancel, rate, support. Unknown actions return 404 NACK.

Discovery uses AND-token matching: all non-filler query tokens must appear in the item name + keywords. Returns matched items in Beckn v2 flat-resource wire format. Catalog: 9 providers, 31 items, 5 categories (Office Supplies, IT Equipment, IT Peripherals, Networking, Furniture).

Auto-advance (SIM_BPP_AUTO_ADVANCE=true, default): confirmed orders progress ACCEPTED → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED at SIM_BPP_ADVANCE_INTERVAL_SECS (default 5s) intervals. Each transition PATCHes data-normalizer and publishes to Kafka. Set SIM_BPP_AUTO_ADVANCE=false for demos requiring manual status control.

Variable	Default	Notes
ONIX_BPP_CALLER	http://onix-bpp:8082/bpp/caller	Where to send on_{action} callbacks
BPP_ID	bpp.example.com	
CATALOG_PATH	/app/catalog.json	Bind-mounted; edit without rebuild
SIM_BPP_AUTO_ADVANCE	true	Set false to stop lifecycle progression
KAFKA_BOOTSTRAP	kafka:9092	Empty string disables Kafka publishing
DATA_NORMALIZER_URL	http://data-normalizer:8006	

15.8.4 analytics (:8009)

Procurement reporting service. All dashboard data in the Next.js frontend is proxied through Next.js API routes at `/api/analytics/*` which call this service. Returns HTTP 503 (not mock data) when the database pool is unavailable.

Method	Path	Purpose
GET	/health	{"status": "ok", "service": "analytics", "db_connected": true}
GET	/analytics?period=30d\ 90d\	Full dashboard payload: KPIs, spend over time, request volume, cycle times, supplier metrics, recent requests
GET	/business-impact?period=...	Compact KPI set: {monthly_savings, requests_this_month, avg_cycle_time_hours}
GET	/benchmark?period=...	CPO benchmarking: contracted price vs. best market price per category

`period` defaults to `90d`. `mock.py` exists locally for development but is **not** auto-served.

15.8.5 comparative-scoring (:8003)

Thin ML scoring adapter. Forwards DiscoverOffering lists to prediction-api (RankNet/LambdaRank MLOps stack) as the primary path. Falls back to a min-price heuristic when the ML backend is unreachable.

Method	Path	Purpose
GET	/health	{ "status": "ok", "service": "comparative-scoring" }
POST	/score	Rank a list of DiscoverOfferings; returns top recommendation

POST /score request

```
{
  "offerings": [
    { "item_id": "item-001", "item_name": "A4 Paper 80gsm", "price_value":
      ↪ "168.00",
      "price_currency": "INR", "fulfillment_hours": 48, "rating": 4.2, "bpp_id":
      ↪ "bpp.example.com" }
  ]
}
```

Response — ML path active

```
{
  "selected": {},
  "scoring": {
    "engine": "ml",
    "model_version": "ProcurementRanker:v7 (Production)",
    "pipeline": "phase2_ranknet",
    "ranking": [{ "bpp_id": "bpp.example.com", "item_id": "item-001", "score":
      ↪ 0.87, "rank": 1 } ]
  }
}
```

Response — ML backend unreachable (fallback)

```
{ "selected": {}, "scoring": { "engine": "heuristic_min_price" } }
```

Empty list input returns { "selected": null } without raising.

The MLOps prediction-api runs in a separate Compose stack: `docker compose -f services/ComparativeAndS compose.mlops.yaml up prediction-api`.

Variable	Default
PREDICTION_API_URL	http://prediction-api:8004
PREDICTION_TIMEOUT_S	8.0
SCORING_FALLBACK_ENABLED	true

15.8.6 negotiation-engine (:8004 container / :18004 host)

Automated price negotiation using a LangGraph state machine. Accepts ranked DiscoverOffering objects, computes a guardrail-bounded counter-offer, exchanges rounds with the BPP via Beckn /select callbacks over Redis Pub/Sub.

Method	Path	Purpose
GET	/healthz	Always 200 while process alive
GET	/readyz	Readiness: graph compiled, Kafka/Postgres config checked
POST	/negotiate	Start negotiation; returns 202 with thread_id
GET	/negotiate/{transaction_id}	Inspect current LangGraph snapshot (HITL UI)

POST /negotiate request

```
{
  "transaction_id": "uuid",
  "category": "office_supplies",
  "ranked_offers": [],
```

```
"policy": { "max_discount_pct": 0.20, "approval_threshold_pct": 0.15 },
"max_rounds": 3
}
```

Response 202

```
{ "thread_id": "uuid", "paused_at": "wait_for_async_callback", "round": 0,
  "final_outcome": null }
```

The graph parks at `wait_for_async_callback` after issuing the counter-offer. Resumption is driven by `OnSelectListener` subscribing to Redis channel `beckn_on_select_results`.

Guardrail layers enforce a hard 20% maximum discount cap at three independent levels: Pydantic field constraint (`discount_pct` bounded `[0.0, 0.20]`), `validate_counter_offer()` (category+supplier caps), and ONIX schema validation of the Beckn wire format.

Variable	Default	Notes
REDIS_URL	redis://localhost:6379	
BECKN_ON_SELECT_CHANNEL	beckn_on_select_results	Must match frontend-demo-gateway REDIS_ON_SELECT_CHANNEL
NEGOTIATION_POSTGRES_DSN	""	Empty = ephemeral MemorySaver (state lost on restart)

15.8.7 discovery-engine (:8006) — Orphaned Service

Status: The `discovery_engine` service is absent from `docker-compose.yml` and is not called by any other service in the current stack. Its documented port (8006) conflicts with `data-normalizer`. The code at `services/discovery_engine/` is complete but has no deployment entry. It represents a planned future integration point for multi-network Beckn discovery.

When deployed, it fans out concurrent discovery across N configured Beckn networks, deduplicates results by `provider_id:item_id:currency`, and returns a merged `MultiSearchResult`. Always returns HTTP 200 regardless of individual network failures; degraded state is surfaced in the body.

Method	Path	Purpose
GET	/healthz	Always 200
GET	/readyz	Returns 503 if no networks are configured
POST	/search/multi-network	Concurrent fan-out discovery

POST /search/multi-network request — IntentPayload (same fields as BecknIntent).

Response — always HTTP 200. Check degraded and failed_networks[].

```
{
  "items": [],
  "total": 12,
  "degraded": false,
  "failed_networks": [],
  "sources": ["network_a", "network_b"]
}
```

15.8.8 erp-mock (:8008)

Local ERP stub simulating SAP S/4HANA and Oracle ERP Cloud surfaces. All state is in-memory and non-durable. Active scenario is set via MOCK_SCENARIO env var or the X-Mock-Scenario request header (per-request override).

Method	Path	Purpose
GET	/health	Liveness
POST	/mock/budget/check	Vendor-neutral budget gate
POST	/sap/budget/check	SAP-shaped budget gate
POST	/oracle/budget/check	Oracle-shaped budget gate
POST	/mock/policy/evaluate	ERP policy evaluation
POST	/mock/po/create	Vendor-neutral PO creation
POST	/sap/oauth2/token	SAP OAuth2 client_credentials token
GET	/sap/opu/odata/sap/API_PURC	SAP CSRF token fetch
POST	/sap/opu/odata/sap/API_PURC	SAP ORDER_CREATE_PROCESS_SRV/A_PurchaseOrder

Method	Path	Purpose
POST	/oauth2/v1/token	Oracle OAuth2 token
POST	/fscmRestApi/resources/11.1.3/applications/OraclePO/createPhaseOrders	Create PO
POST	/mock/scenario	Switch active scenario at runtime

Scenario	Budget result	PO failures	Webhook delay	Policy
happy (default)	allowed, balance 250,000	0	2 s	standard
budget_exhausted	denied, balance 0	—	—	—
po_create_fails	allowed	10 failures before success	standard	standard
webhook_delayed	allowed	0	30 s	standard
erp_approval_required	allowed	0	2 s	approval_required=True
erp_preferred_supplier	allowed	0	2 s	preferred_supplier_ids=["bpp-001"]

All PO create endpoints honour the Idempotency-Key request header. After a successful POST /mock/po/create, the service schedules a signed po-status webhook back to erp-adapter after WEBHOOK_DELAY_SECONDS.

15.8.9 frontend-demo-gateway (:8015)

Demo BFF bridging the Next.js frontend to the Phase 2 LTR PyTorch scoring model and the LangGraph negotiation engine. Uses real components (PyTorch, LangGraph) while mocking infrastructure. Also hosts the SupplierAgent, an LLM persona simulating a seller responding to counter-offers.

Port note: CLAUDE.md lists this service at :8005; the actual port is :8015. Set DEMO_GATEWAY_URL=http://localhost:8015 in the orchestrator.

Method	Path	Purpose
GET	/healthz	Kubernetes liveness

Method	Path	Purpose
GET	/readyz	Phase2Scorer + SupplierAgent loaded, Ollama reachable
POST	/api/demo/score	Run Phase 2 LTR PyTorch model on submitted suppliers
POST	/api/demo/negotiate	Start LangGraph negotiation; returns 202 with thread_id
GET	/api/demo/negotiate/{thread_id}	Get current negotiation state
POST	/api/demo/negotiate/{thread_id}/respond	Trigger LLM supplier response and resume buyer graph

The SupplierAgent LLM backend is the `claude_openai_proxy` service at `:8012` — not Ollama directly. `OLLAMA_BASE_URL` points to the proxy. Set `CLAUDE_PROXY_KEY` and ensure the proxy is running on the host before starting this service.

Variable	Default	Notes
NEGOTIATION_ENGINE_URL	http://localhost:8004	Docker: http://negotiation-engine:8004
REDIS_URL	redis://localhost:6379/0	
REDIS_ON_SELECT_CHANNEL	beckn_on_select_results	Must match negotiation_engine channel
OLLAMA_BASE_URL	http://host.docker.internal	Points to <code>claude_openai_proxy</code> , not Ollama
SUPPLIER_MODEL	claude-3-5-sonnet	Mapped by proxy
SUPPLIER_ACCEPTABLE_DISCOUNT	0.10	Minimum discount the SupplierAgent will accept

15.9 notification-dispatcher

Kafka consumer service. **Exposes no HTTP API surface** beyond a single liveness probe. Events are consumed from Kafka; there is no injection endpoint.

```
GET /health → {"status": "ok", "service": "notification-dispatcher"}
```

No host port is published in `docker-compose.yml`. The service is container-only.

15.9.1 Kafka Consumer

- Topic consumed: `po.status.changed`
- Consumer group: `notification-dispatcher`
- `auto_offset_reset`: `latest`
- `enable_auto_commit`: `True`
- No dead-letter queue — malformed messages are logged at WARNING level and permanently dropped.
- Consumer only starts when `KAFKA_BOOTSTRAP` is non-empty.

15.9.2 Routing Rules

The `po_status` field is checked first; `state` is the fallback. Matching is case-insensitive.

Order status	Slack	Teams	Email
confirmed	yes	yes	yes
shipped	yes	yes	no
delivered	yes	yes	yes
cancelled	yes	yes	no
any other	no	no	no

Each channel is independent — `asyncio.gather(return_exceptions=True)` ensures a Slack failure does not suppress a Teams notification.

15.9.3 Output Channel Formats

Channel	Format	Notes
Slack	Block Kit	Header block with status emoji; section block with <code>order_id</code> , <code>transaction_id</code> , <code>source</code> , <code>observed_at</code>

Channel	Format	Notes
Microsoft Teams	Adaptive Card v1.4	Color coding: confirmed/delivered = good, shipped = accent, cancelled = attention
Email	Jinja2 HTML + SMTP STARTTLS	Templates: email_confirmed.html, email_delivered.html, email_generic.html. Subject: "Order {STATE} – {order_id}"

The email recipient is resolved at runtime by joining from `purchase_orders.beckn_confirm_ref = order_id` through the FK chain to `users.email`. If the database is unavailable or the lookup returns no match, email is skipped for that event without affecting Slack or Teams.

15.9.4 Configuration

Variable	Default	Notes
KAFKA_BOOTSTRAP	kafka:9092	Consumer does not start if empty
KAFKA_TOPIC	po.status.changed	
KAFKA_GROUP_ID	notification-dispatcher	
SLACK_WEBHOOK_URL	set in docker-compose	Channel disabled if empty
TEAMS_WEBHOOK_URL	""	Channel disabled if empty
SMTP_HOST	""	Email disabled if empty
SMTP_PORT	587	
SMTP_FROM	noreply@procurement-agent.local	
DB_HOST	host.docker.internal	For email recipient lookup only

15.10 Authentication Summary

Service	Auth method	Mechanism	Dev default
orchestrator /web-hooks/seller/status	HMAC-SHA256	X-Signature: base64(HMAC-SHA256(body, secret))	dev-seller-hmac-CHANGE_ME
erp-adapter /api/v1/*	Bearer token	Authorization: Bearer \${ERP_INTERNAL_TOKEN}	dev-internal-token-CHANGE_ME
erp-adapter webhooks	HMAC-SHA256 dual-secret	X-{Vendor}-Signature: sha256=<hex> (primary + rotation key)	dev-sap-hmac-CHANGE_ME
mcp-sidecar	API key	BAP_API_KEY env var (validated at startup)	any non-empty value
claude-openai-proxy /v1/*	Bearer token	Authorization: Bearer \${CLAUDE_PROXY_KEY}	auth disabled if key is empty
All other services	none	—	No auth in dev

All *-CHANGE_ME secrets **must** be replaced with strong random values before deploying to any shared or production environment.

16 Glossary — Agentic AI Procurement Agent on Beckn Protocol

Alphabetically ordered reference for all domain-specific terms, architectural concepts, and database identifiers used in this project. Each entry gives a single-sentence definition followed by a cross-reference to the document where the concept is explained in depth.

For system context see [Architecture](#). For technology choices see [Components](#). For why each major decision was made see [System Design](#).

16.1 A

ACK The synchronous HTTP 200 response that the ONIX adapter returns immediately to any Beckn POST (e.g., `/discover`, `/select`) to acknowledge receipt; it contains no catalog or order data — the real payload arrives later via a separate `on_*` webhook callback. See [Architecture](#) §3 (Beckn Protocol Integration).

advisory mode An execution mode (set via `PROCUREMENT_EXECUTION_MODE=advisory`) in which the agent surfaces ranked supplier options to the buyer but takes no autonomous Beckn action; the buyer makes the final selection manually in the frontend. See [Architecture](#) §5 (Orchestrator Pipeline).

agent_memory_vector (table: `agent_memory_vectors`) A PostgreSQL table storing 384-dimensional HNSW-indexed cosine vectors that encode past procurement decisions, enabling the orchestrator to apply time-decayed supplier loyalty bonuses when scoring future offers; embeddings are produced by the BAAI/bge-small-en-v1.5 model via `fastembed` but the `embedding_model` column stores the proxy label `all-MiniLM-L6-v2` due to an ENUM constraint gap. See [Components](#) §4 (Embedding Models).

AND-token matching The catalog-matching algorithm in `sim-bpp` that tokenises a discovery query, discards tokens absent from the catalog vocabulary (e.g., quantities, city names), and then requires that **all** remaining tokens appear in a candidate item's name or keywords before the item is returned — preventing false-positive matches on over-broad queries. See [System Design](#) §AND-Token Catalog Matching.

ANN (Approximate Nearest Neighbor) A vector-search technique that returns the closest embeddings to a query vector in sub-linear time by trading exact precision for speed; this project uses pgvector's HNSW index for ANN over 384-dimensional cosine vectors in the `bpp_catalog_semantic_cache` and `agent_memory_vectors` tables. See [Components §4 \(Embedding Models\)](#).

asyncio.create_task The Python asyncio function used project-wide to schedule a coroutine on the running event loop without blocking the caller; all non-critical side-effect calls (persistence writes, the Beckn /discover POST inside the MCP sidecar) are wrapped in `asyncio.create_task` rather than `await`-ed to keep the critical latency path free. See [System Design §asyncio-First + Fire-and-Forget](#).

audit_trail_event (table: `audit_trail_events`) A PostgreSQL append-only table that records every procurement lifecycle event (nine event types including `request_received`, `intent_parsed`, `discovery_complete`, `order_confirmed`) with a `retention_until` timestamp, `kafka_offset` placeholder, `splunk_indexed` flag, and a `reasoning_payload` JSONB column for LLM trace data; designed for SOX 404 compliance with a 7-year retention target. See [Architecture §6 \(Main Happy-Path Sequence\)](#).

autonomous mode An execution mode (set via `PROCUREMENT_EXECUTION_MODE=autonomous`) in which the orchestrator commits a Beckn order automatically when the order value falls within the requester's pre-configured approval threshold and the ERP budget gate returns `allowed: true`. See [Architecture §5 \(Orchestrator Pipeline\)](#).

16.2 B

BAP (Beckn Application Platform) The buyer-side protocol participant in the Beckn network that initiates the procurement lifecycle (`discover` → `select` → `init` → `confirm` → `status`) and receives `on_*` callbacks from BPPs; in this project the BAP role is fulfilled by the `beckn-bap-client` service (:8002) routed through `onix-bap` (:8081). See [Architecture §3 \(ONIX Adapter Pair\)](#).

beckn_network The Docker Compose bridge network (`driver: bridge`) on which all 18 containerised services communicate using Docker DNS names (e.g., `http://data-normalizer:8006`); services reach the native-host PostgreSQL via `host.docker.internal:5432`. See [Architecture §2 \(Logical Layers\)](#).

Beckn Protocol An open, interoperable, transaction-layer protocol for decentralised commerce that defines a standard lifecycle (`discover` → `select` → `init` → `confirm` → `status` → `track`) between a BAP (buyer) and a BPP (seller) mediated by network-registered ONIX adapters; this project implements Beckn v2.0.0. See [Architecture §3 \(Beckn Protocol Integration\)](#).

BecknIntent The canonical Pydantic v2 model (`shared/models.BecknIntent`) produced by IntentParser Stage 2 that encodes a natural-language procurement request as a structured object with strictly typed fields: `item` (string), `descriptions[]` (list), `quantity` (int), `location_coordinates` ("lat,lon" decimal string), `delivery_timeline` (int hours), and `budget_constraints` (`BudgetConstraints`). See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

BPP (Beckn Provider Platform) The seller-side protocol participant that receives Beckn discovery and order requests, returns catalogs and order confirmations via `on_*` callbacks, and advances the fulfillment lifecycle; in development this role is played by `sim-bpp (:3002)` routed through `onix-bpp (:8082)`. See [Architecture §3](#) (ONIX Adapter Pair).

BudgetConstraints A typed Pydantic v2 sub-model (`{ "max": float, "min": float }`) that is a required field of `BecknIntent`; using a typed model (rather than a raw string) enforces that budget values are numeric and prevents downstream services from needing to parse free-form budget text. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

16.3 C

Circuit breaker A fault-isolation pattern (implemented via the `pybreaker` library in `erp-adapter`) that trips to OPEN after five consecutive vendor API failures, causing all subsequent calls to fail immediately rather than blocking for the 800 ms timeout; it resets to CLOSED automatically after 60 seconds, and its per-vendor state is exposed on `GET /readyz`. See [System Design §Per-Vendor Circuit Breakers](#).

conda environment The Conda virtual environment (conventionally named `infosys_project`) used to run the two unbundled local Python services — `IntentParser (:8001)` and `mcp-sidecar (:3000)` — outside of Docker; both services must be activated within this environment before running `uvicorn`. See [Architecture §2](#) (Logical Layers).

cosine similarity The similarity metric used for all vector comparisons in the project — computed as the dot product of two L2-normalised embedding vectors — with threshold bands of ≥ 0.85 (VALIDATED, cache hit), $0.45-0.85$ (AMBIGUOUS, MCP sidecar probe), and < 0.45 (CACHE_MISS, recovery flow) in IntentParser Stage 3. See [Components §4.2](#) (Similarity Thresholds).

16.4 D

DeDi registry The Decentralised Distributed registry used in a production Beckn network to resolve BPP endpoint URLs from provider identifiers; bypassed in this project by setting `targetType: url` in all four ONIX routing YAML files, which makes the full Beckn flow self-contained within the Docker bridge network. See [System Design §DeDi Registry Bypass](#).

DiscoverOffering The normalised representation of a single BPP catalog item produced by `catalog-normalizer (:8005)`; `beckn-bap-client` owns its own internal `DiscoverOffering` model distinct from the shared one — the `catalog-normalizer` converts raw `on_discover` catalog payloads into this typed structure before scoring. See [Architecture §6 \(Main Happy-Path Sequence\)](#).

16.5 E

ED25519 The elliptic-curve digital signature algorithm used by `onix-bap` to sign every outbound Beckn request via the `signer` .so plugin compiled into the `fidedocker/onix-adapter` Go image; the ONIX schema validator is pinned to commit `d43ec30d` because a later upstream commit introduced a `$ref` resolution bug that broke ED25519 signature validation. See [System Design §ONIX Schema Validator Pin](#).

erp_sync_record(s) (table: `erp_sync_records`) A PostgreSQL outbox table (migration `19b_erp_sync_records`) where the orchestrator writes a pending PO push record atomically with the `purchase_orders` insert; a background worker in `erp-adapter` polls `WHERE status = 'pending'` using `FOR UPDATE SKIP LOCKED`, calls the vendor ERP API, and updates the row status — effectively replacing a Kafka producer/consumer for the ERP event bus in Phases 1–3. See [System Design §Outbox Pattern for ERP PO Push](#).

ERPAdapter Protocol A `typing.Protocol` (structural subtype) in `erp-adapter` that defines six methods (`check_budget`, `push_po`, `get_po_status`, `evaluate_policy`, `get_health`, `get_vendor_name`) that every vendor adapter (SAP, Oracle, mock) must implement; the active adapter(s) are selected at startup via the `ERP_VENDORS` environment variable without requiring changes to the orchestrator or budget-gate logic. See [System Design §Vendor-Neutral ERPAdapter Protocol](#).

16.6 F

fastembed A Python library that runs ONNX-format embedding models without a PyTorch dependency; used by `data-normalizer` to generate 384-dimensional BAAI/bge-small-en-v1.5 embeddings for the `agent_memory_vectors` table, deliberately avoiding adding the ~1.5 GB PyTorch package to the `data-normalizer` container image. See [Components §4 \(Embedding Models\)](#).

fire-and-forget The project-wide pattern where non-critical side-effect coroutines (audit writes, memory writes, ERP sync records) are launched with `asyncio.create_task` and allowed to run concurrently without blocking the orchestrator's critical-path response; these tasks never raise exceptions to the caller and are given a 5-second timeout. See [System Design §asyncio-First + Fire-and-Forget](#).

16.7 H

HITL (Human-in-the-Loop) A general design pattern where an automated agent pauses and surfaces a decision to a human before proceeding; in this project it appears in two contexts: the `hitl` execution mode (human approves a Beckn commit) and as a LangGraph interrupt node in the negotiation engine when a counter-offer gap exceeds the policy threshold. See [Architecture §5.2 \(Negotiation Engine LangGraph\)](#).

HITL mode An execution mode (set via `PROCUREMENT_EXECUTION_MODE=hitl`) in which the orchestrator surfaces the top-ranked offer to the buyer for manual approval before issuing the Beckn `select → init → confirm` sequence; the pipeline pauses at `POST /approvals/{id}/decide`. See [Architecture §5 \(Orchestrator Pipeline\)](#).

HMAC HMAC-SHA256 webhook verification used by `erp-adapter` to authenticate inbound ERP vendor callbacks; each vendor maintains a primary secret (`{VENDOR}_WEBHOOK_HMAC_SECRET`) and a rotation secret (`{VENDOR}_WEBHOOK_HMAC_SECRET_NEXT`) so that secret rotation is zero-downtime. See [System Design §Dual-Secret HMAC Rotation](#).

HNSW (Hierarchical Navigable Small World) The graph-based ANN index type used for both pgvector tables (`bpp_catalog_semantic_cache` and `agent_memory_vectors`); the agent memory table uses `ef_search=100` for higher recall during similarity retrieval, while the catalog cache index uses pgvector defaults. See [Components §7.3 \(Database Schema Summary\)](#).

16.8 I

IntentParser The three-stage NLP pipeline (FastAPI service, :8001) that converts a natural-language purchase request into a validated BecknIntent; Stage 1 classifies intent via qwen3:8b, Stage 2 extracts the BecknIntent struct via qwen3 (complexity-routed), and Stage 3 validates the item against the pgvector semantic cache and MCP sidecar. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

16.9 K

kafka_offset A BIGINT placeholder column present in both `audit_trail_events` and `erp_sync_records` tables that stores NULL in Phases 1–3; it is reserved as a non-breaking migration path for Phase 4 when a real Kafka producer is wired and each record needs its corresponding Kafka topic offset for exactly-once semantics. See [Components §6](#) (Implementation vs Original Spec).

16.10 L

LangGraph An open-source library from LangChain that models agent workflows as directed StateGraph instances with typed state, conditional edges, and durable checkpointing; used in this project for the negotiation engine’s buyer/supplier negotiation graph and formerly prototyped in Bap-1 for the orchestrator. See [Architecture §5](#) (LangGraph State Machines).

16.11 M

MCP (Model Context Protocol) An open protocol that exposes tool functions to LLM agents over a transport layer (HTTP SSE in this project); IntentParser Stage 3 connects to the mcp-sidecar (:3000) as an MCP client, calling the `search_bpp_catalog` tool to validate procurement items against live BPP catalogs. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

MLflow An open-source ML lifecycle platform used by the `ComparativeAndScoring` MLOps sub-stack (`docker-compose.mlops.yaml`) for experiment tracking, model registry, and artifact storage; the

prediction-api retrieves the registered RankNet model from MLflow at startup. See [Components §1](#) (Full Technology Matrix).

16.12 N

NDCG@5 (Normalised Discounted Cumulative Gain at rank 5) The ranking quality metric used to evaluate the comparative-scoring service; it measures how well the model ranks the most relevant supplier offers in the top five positions, with a discount applied to lower-ranked positions — the Phase 2 ML target is NDCG@5 ≥ 0.85 . See [Components §3.1](#) (Models in Use).

16.13 O

on_confirm The Beckn asynchronous callback (POST `/bap/receiver/on_confirm`) that the BPP sends to the BAP after a `confirm` request is processed; it carries the final `order_id` and `Contract.status.code = "ACTIVE"` — the valid enum values are `DRAFT | ACTIVE | CANCELLED | COMPLETE`; ONIX rejects "CONFIRMED". See [Architecture §6](#) (Main Happy-Path Sequence).

on_discover The Beckn asynchronous callback (POST `/on_discover`) that delivers the BPP catalog payload to the BAP after a `discover` request; it is routed to a dedicated endpoint in `beckn-bap-client` (not the generic `/bap/receiver`) so the handler can both publish to the Redis `beckn_results:{transaction_id}` channel and notify the orchestrator's `CallbackCollector` simultaneously. See [System Design §on_discover](#) Split Routing.

on_init The Beckn asynchronous callback (POST `/bap/receiver/on_init`) sent by the BPP after an `init` request; it carries payment terms that the orchestrator must relay to the buyer before issuing the `confirm` step. See [Architecture §6](#) (Main Happy-Path Sequence).

on_select The Beckn asynchronous callback (POST `/bap/receiver/on_select`) sent by the BPP after a `select` request confirming item availability and updated pricing; in the negotiation engine, receiving `on_select` resumes the LangGraph buyer graph via `OnSelectListener` on the `beckn_on_select_results` Redis channel. See [Architecture §5.2](#) (Negotiation Engine LangGraph).

ONIX adapter The Go binary (`fidedocker/onix-adapter`) that acts as the Beckn protocol middleware between the BAP/BPP services and the network; it applies a middleware chain of routing, ED25519 signing,

and schema validation on every Beckn message, and is deployed as two instances — `onix-bap (:8081)` and `onix-bpp (:8082)` — pinned to commit `d43ec30d`. See [System Design §ONIX Schema Validator Pin](#).

orchestrator The FastAPI + aiohttp pipeline state machine (service :8004) that sequences the five procurement steps — intent parsing, discovery, scoring, ERP budget gate, and Beckn `select → init → confirm` — and exposes two flows: `POST /run` (end-to-end) and `POST /compare + POST /commit` (two-phase with human review). See [Architecture §5.1 \(Orchestrator Pipeline\)](#).

Outbox pattern The transactional messaging pattern used by `erp-adapter` to guarantee that every committed purchase order eventually triggers a PO push to the vendor ERP without requiring a Kafka broker; the orchestrator writes a row to `erp_sync_records` in the same PostgreSQL transaction as the `purchase_orders` insert, and a background worker picks it up via `FOR UPDATE SKIP LOCKED`. See [System Design §Outbox Pattern for ERP PO Push](#).

16.14 P

pgvector A PostgreSQL extension (version 0.7.0) that adds a native `vector (n)` column type and HNSW / IVFFlat indexes for approximate nearest-neighbour search directly inside PostgreSQL 16; used here to store 384-dimensional BPP catalog embeddings and agent memory vectors, eliminating the need for a separate Qdrant deployment. See [System Design §pgvector Instead of Qdrant](#).

Phase2Scorer The ML-based supplier ranking component in `comparative-scoring (:8003)` that calls the optional `prediction-api RankNet` model (from `docker-compose.mlops.yaml`) to rank `DiscoverOffering` items by predicted relevance; when the `prediction-api` is unavailable, `comparative-scoring` falls back to the Phase 1 minimum-price heuristic. See [Components §1 \(Full Technology Matrix\)](#).

procurement_request (table: `procurement_requests`) The first PostgreSQL record written at the start of every pipeline run, created by `data-normalizer`'s `POST /normalize/request` endpoint before intent parsing begins; all subsequent records (intent, discovery, scoring, order) reference this row's primary key, forming the audit trail root. See [Architecture §6 \(Main Happy-Path Sequence\)](#).

purchase_order (table: `purchase_orders`) The PostgreSQL record created by `data-normalizer` after a successful Beckn `confirm` callback, capturing the `order_id`, `order_state`, vendor details, and total amount; it is the anchor for `erp_sync_records` outbox rows and for audit trail events of type `order_confirmed`. See [Architecture §6 \(Main Happy-Path Sequence\)](#).

16.15 Q

query broadening The Stage 3 recovery technique that progressively simplifies a failed procurement query (strip modifiers, synonyms, drop constraints) to find any BPP catalog match; implemented in `IntentParser/recovery.py` via a regex-based simplifier first, then optionally Claude Sonnet 4.6 via `ANTHROPIC_API_KEY` as a last resort. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

16.16 R

RankNet A pairwise learning-to-rank neural network model (`PyTorch nn.Linear`) trained in the `ComparativeAndScoring` MLOps sub-stack to predict supplier offer preference from features such as price, delivery time, and rating; served by the optional `prediction-api` and consumed by the `Phase2Scorer` in `comparative-scoring`. See [Components §1](#) (Full Technology Matrix).

reasoning_payload A JSONB column in `audit_trail_events` that stores LLM trace data (model name, token counts, prompt/completion excerpts) for every LLM call made during a pipeline run; populated in Phases 1–3 as a local substitute for LangSmith tracing, which is deferred to Phase 4. See [Components §6](#) (Implementation vs Original Spec).

recovery flow The `IntentParser` sub-pipeline that activates when Stage 3 validation returns `not_found`; it calls `broaden_procurement_query`, retries Stage 3 once with the broadened query, and if still not found, invokes three stub functions (`log_unmet_demand`, `notify_buyer_no_stock`, `trigger_open_rfq_flow`) that are currently no-op logger calls pending Phase 4 implementation. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

Redis pub/sub The message-passing mechanism used to decouple the asynchronous `on_discover` Beckn callback from the synchronous MCP sidecar response path; the sidecar subscribes to `beckn_results:{transaction_id}` before firing the discover request, and `beckn-bap-client` publishes the catalog payload to the same channel when the callback arrives — eliminating the asyncio deadlock described in ADR-0001. See [System Design §ADR-0001](#).

retention_until A `TIMESTAMPTZ` column in `audit_trail_events` set to `NOW() + INTERVAL '7 years'` at insert time, marking when the record may be archived or deleted; no automated enforcement job exists in Phases 1–3 — the column is a placeholder for a Phase 4 nightly cleanup cron job required for SOX 404 compliance. See [Components §6](#) (Implementation vs Original Spec).

RFQ (Request for Quotation) A formal procurement document sent to suppliers when no matching item is found in the Beckn network; triggered in the recovery flow stub `trigger_open_rfq_flow` — currently

a no-op logger call that would in a production system generate and dispatch an RFQ to registered BPPs via the `Beckn init` action with open terms. See [Architecture §4](#) (IntentParser 3-Stage Pipeline).

16.17 S

sentence-transformers A Python library (PyTorch-backed) that provides pre-trained transformer-based text embedding models; used by IntentParser Stage 3 and the mcp-sidecar to encode procurement queries and catalog item descriptions into 384-dimensional `all-MiniLM-L6-v2` vectors for cosine similarity search in pgvector. See [Components §4](#) (Embedding Models).

sim-bpp The local Node.js BPP simulator (:3002) that replaced the `fidedocker/sandbox-2.0` image; it hot-reloads `catalog.json` at request time, implements all 10 Beckn actions, uses AND-token catalog matching to reduce false positives, and — when `SIM_BPP_AUTO_ADVANCE=true` — automatically advances confirmed orders through the fulfillment lifecycle publishing Kafka events at each transition. See [System Design §sim-bpp Replacing sandbox-2.0](#).

splunk_indexed A BOOLEAN placeholder column in `audit_trail_events` (default FALSE) that indicates whether a record has been exported to a Splunk SIEM; no Splunk exporter is wired in Phases 1–3 — the column is reserved for a Phase 4 Kafka-to-Splunk consumer. See [Components §6](#) (Implementation vs Original Spec).

SSE (Server-Sent Events) The HTTP streaming transport used by the MCP protocol in this project; the mcp-sidecar exposes its tool functions (including `search_bpp_catalog`) over an SSE endpoint at `http://localhost:3000/sse`, and IntentParser Stage 3 connects as an MCP SSE client. See [Components §1](#) (Full Technology Matrix).

16.18 T

targetType: url The ONIX routing configuration value set in all four routing YAML files (`config/generic-routing-BAPCaller.yaml`, etc.) that instructs the ONIX adapter to route Beckn messages directly to a configured URL rather than performing a DeDi registry lookup; enables a fully offline Beckn flow inside the Docker bridge network and can be reverted to `bap/bpp` with a single-line change for production network registration. See [System Design §DeDi Registry Bypass](#).